Supplementary material (Online Resource 1) for: One network for renders and video: vendor-neutral neural temporal super-resolution and low-latency frame generation trained on commodity cloud GPUs

Parth Mital

*School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, 632014, Tamil Nadu, India.

Corresponding author(s). E-mail(s): parth.mital2023@vitstudent.ac.in;

Abstract

Journal: *Journal of Real-Time Image Processing.* **Article:** One network for renders and video: vendor-neutral neural temporal super-resolution and low-latency frame generation trained on commodity cloud GPUs. **Corresponding author:** Parth Mital, School of Computer Science and Engineering, Vellore Institute of Technology, Vellore 632014, Tamil Nadu, India, parth.mital2023@vitstudent.ac.in.

This document is Online Resource 1 of the article. The article is limited to twelve pages, so it states the design and the main results only. This supplement explains every part of NeuralSS from first principles: the sampling, motion and image-quality concepts it relies on, the data pipeline and copy detection, the synthetic renderer, the temporal super-resolution and frame-generation networks with every equation and training objective, the evaluation protocol, and every table, plot and prediction that the three Kaggle notebooks saved, each with its interpretation. Every number is generated from the notebooks' metric files, and every finding carries a status label (confirmed, preliminary, observation or under revision), because the pipeline is an intermediate checkpoint that will be revised.

Keywords: Temporal super-resolution, Frame generation, Video frame interpolation, Real-time rendering, Latency, Efficient neural networks

S1 About this supplement

S1.1 Purpose and relation to the article

The article (Section 1) presents NeuralSS, a vendor-neutral system with one temporal super-resolution (TSR) network and one frame-generation (FG) network, each serving both game-engine renders with motion vectors and plain video without them. Its twelve-page limit allows only the design and the headline results. This supplement is the complete account. It assumes basic linear algebra, calculus and programming, and nothing about rendering, super-resolution or frame interpolation. It defines every concept before it is used, gives every equation with all symbols defined, lists every input, hyperparameter and loss weight of the code, and presents every table, plot and prediction that the three Kaggle notebooks saved, each with an interpretation.

Table S1 maps the article's sections to the sections of this supplement.

Table S1: Where each part of the article is explained in full

Article	Supplement
Concepts assumed by the article	Section S2
Section 3 Data and leakage control	Section S3
Section 4 Temporal super-resolution	Section S4 (method), Section S5 (results)
Section 5 Frame generation	Section S6 (method), Section S7 (results)
Section 6 Experimental setup	Sections S4 and S6 (protocols), Section S9 (configuration and runs)
Section 7 Results	Sections S5 and S7
Section 8 Discussion and threats to validity	Section S8

S1.2 An intermediate checkpoint

The three notebooks are not the final pipeline. They are a checkpoint of a project whose data, networks, training budgets and evaluation protocol will be revised, and this supplement will be revised with them. Two mechanisms keep it consistent with the latest runs.

1. *Generated numbers.* No number taken from a run is typed by hand. The script `tools/build_paper_results.py` reads the metric files that the notebooks saved and writes one named value per metric. The text, tables and the article refer to those names, so a new run updates every number in both documents, and the script lists the sentences whose numbers changed so that their interpretation can be checked.
2. *Saved outputs, not copies.* Figures are read directly from the notebooks' output folders, so a new run replaces them.

S1.3 Runs reported in this version

This version reports the data-pipeline run of 18 October 2026, the TSR run of 3 November 2026 to 4 November 2026, the FG run of 8 November 2026, and a Middlebury interpolation comparison computed afterwards on a CPU from the FG run's saved weights. Each notebook ran once on Kaggle with two NVIDIA T4 GPUs. Section S9 gives the hardware, software, budgets and configuration of each run and the procedure that regenerates every number from its saved outputs. Checking this supplement against the code found defects in the code of these runs, and Section S8.2 lists them with the results they affect.

S1.4 Status of findings

Every statement about a result carries one of four labels, so that a reader can tell settled results from provisional ones.

[Confirmed] The finding holds in every run available to us and its size is well above the run-to-run variation, which an earlier repeat of the whole TSR configuration put at about 0.3 dB of PSNR. It is not expected to change with the planned revisions.
[Preliminary] The finding comes from one run (one random seed, one training budget), or its margin is small, or it rests on few test samples. It is a measured result, but further runs may change it.
[Observation] The finding is qualitative: it comes from looking at predicted images or plots, or from a handful of samples, rather than from a test statistic.
[Under revision] The finding concerns a component or a protocol that the next version of the pipeline is expected to change, for example a threshold known not to transfer, or timings measured in eager PyTorch rather than a deployment runtime.

Section S8 collects all findings with their labels, the known limitations and the planned revisions.

S1.5 Conventions

Tiers are written in typewriter type (`igpu`, `low`, `mid`, `high`). "Engine mode" means the input carries engine data (motion vectors and, where available, depth), "plain mode" or "plain video" means colour only. PSNR values are in decibels (dB), and higher is better for PSNR and SSIM, lower is better for LPIPS, FLIP, the temporal errors and latencies. Cost is given in multiply-accumulate operations (MAC, Section S2.6) per output frame of 1920×1080 pixels. Figures, tables, sections and equations of this supplement are numbered with the prefix S, and those of the article without it.

S2 Background and fundamentals

This section defines every concept that the rest of the supplement uses, starting from how images are sampled and building up to the neural networks, the frame-time budget and the metrics. Each concept is defined before it is used, and each symbol is defined where it first appears. The code files named here are in `src/neuralss`. Where the code computes a quantity in a convention that differs from the textbook one, the

text states the code's convention, and the code is the reference for every definition in this section.

A frame at time index t is written I_t, a function from pixel positions to RGB values in $[0,1]^3$. A pixel position is $\mathbf{p} = (x, y)$ in pixel units, with x the column and y the row, and the code indexes integer positions in the same way. The scale factor s is the ratio of the output to the input linear size, so a low-resolution frame of $h \times w$ pixels is upscaled to $sh \times sw$ pixels.

S2.1 Digital images

A digital image is a finite array of samples. A frame of $H \times W$ pixels with three colour channels is stored as floating-point values in $[0,1]$, and 8-bit files are divided by 255. Three properties of such an array matter for everything that follows: how its values are encoded, how its luminance is measured and how many pixels it has.

Encoding and linear light.

Stored display values are gamma encoded, so they are not proportional to the light L that the display emits. The sRGB transfer curve, implemented by `srgb_to_linear` and `linear_to_srgb` in `nss_synth.py`, converts between an encoded value $E \in [0,1]$ and a linear light value $L \in [0,1]$:

$$L = \begin{cases} E/12.92, & E \le 0.04045, \\ \big((E + 0.055)/1.055\big)^{2.4}, & E > 0.04045, \end{cases} \qquad E = \begin{cases} 12.92\,L, & L \le 0.0031308, \\ 1.055\,L^{1/2.4} - 0.055, & L > 0.0031308. \end{cases} \tag{S1}$$

Light adds in linear values, so an average of light must be taken after decoding. The synthetic renderer decodes its shading, multiplies by the exposure and encodes again. Its four-sample supersampling, however, averages values that are already encoded, so the synthetic targets are not exact linear-light averages. All metrics in this supplement compare encoded values as stored, which is the usual convention for image quality measures and not a measurement of linear-light error.

Luma.

The code measures luminance as the studio-range luma of the encoded values,

$$Y = \frac{16 + 65.481\,R + 128.553\,G + 24.966\,B}{255} = \frac{16}{255} + \frac{219}{255}\big(0.299\,R + 0.587\,G + 0.114\,B\big), \tag{S2}$$

where $R, G, B \in [0,1]$ and the two forms are identical (`rgb_to_y`). The second form shows that Y is an affine function of the full-range luma $0.299R + 0.587G + 0.114B$. A white pixel gives $Y = 235/255 \approx 0.92$ and a black pixel gives $Y = 16/255 \approx 0.06$, so Y spans $[0.06, 0.92]$ rather than $[0,1]$. Every luma-based measure in the supplement uses Eq. (S2): PSNR-Y, SSIM on luma, the luma gradient mask of the detail region

and the cut detector. The defect test (`clip_defects`) and the census loss use the full-range form without the affine map. The weights 0.299, 0.587 and 0.114 are the ITU-R BT.601 luma weights, which are a standard rather than a journal article.[1]

YCoCg.

The analytic temporal upscaler clamps its history in the YCoCg space,

$$Y = \tfrac{1}{4}R + \tfrac{1}{2}G + \tfrac{1}{4}B, \qquad C_o = \tfrac{1}{2}R - \tfrac{1}{2}B, \qquad C_g = -\tfrac{1}{4}R + \tfrac{1}{2}G - \tfrac{1}{4}B, \tag{S3}$$

with the inverse $R = Y + C_o - C_g$, $G = Y + C_g$ and $B = Y - C_o - C_g$. The transform is invertible, so a round trip is exact up to rounding. The Y of Eq. (S3) is a green-weighted average of the channels and is a different quantity from the luma of Eq. (S2), which shares only its name. The code's functions are `rgb_to_ycocg` and `ycocg_to_rgb`.

Resolution and scale.

A frame of $H \times W$ pixels has HW pixels. In this supplement, 1080p means 1920×1080 pixels, which is 2,073,600 pixels with a 16:9 aspect ratio, unless a different size is stated. An upscaler at scale s maps $h \times w$ pixels to $sh \times sw$. Producing one 1080p output at scale s therefore requires $(1920/s) \times (1080/s)$ low-resolution pixels, which is $640 \times 360 = 230{,}400$ for $s = 3$ and $960 \times 540 = 518{,}400$ for $s = 2$. The notebooks time configurations from 640×360 to 1920×1080 at 3x, from 960×540 to 1920×1080 at 2x and from 1280×720 to 3840×2160 at 3x. The render resolution is the low-resolution size of the upscaler's input, and the output resolution is the display size.

S2.2 Sampling, aliasing and jitter

Sampling.

A renderer evaluates a continuous scene $f(\mathbf{q})$ at positions $\mathbf{q} \in \mathbb{R}^2$ and writes one value per pixel. The ideal value of pixel i is the average of the scene over the pixel's area A_i,

$$P_i = \frac{1}{|A_i|} \int_{A_i} f(\mathbf{q}) \, \mathrm{d}\mathbf{q}, \tag{S4}$$

which filters the scene with a box. A renderer that takes one point sample per pixel, $P_i \approx f(\mathbf{c}_i)$ at the pixel centre $\mathbf{c}_i$, does not filter. The synthetic renderer (`Scene.frame`) draws low-resolution frames with one bilinear sample of each texture per pixel and without mip filtering, so thin geometry is point-sampled and aliases. Real clips receive the same treatment when they become low-resolution inputs: `_sr_inputs` bilinearly samples the full-resolution frame at the sample positions, unless the clip supplies native low-resolution frames.

[1] Recommendation ITU-R BT.601-7, https://www.itu.int/rec/R-REC-BT.601, accessed 9 November 2026.

Nyquist and aliasing.

A signal sampled at f_s samples per unit length can represent frequencies up to the Nyquist frequency $f_s/2$, and no higher.[1] Content above the Nyquist frequency does not disappear. It is folded into the representable band, so a sinusoid of frequency f, sampled at multiples of the sample spacing, cannot be told apart from one of frequency

$$f_a = |f - k f_s|, \qquad k = \mathrm{round}(f/f_s), \tag{S5}$$

which lies in $[0, f_s/2]$. As a worked example, take one sample per pixel and stripes with a period of 1.5 pixels, which have $f = 2/3$ cycles per pixel. That is above the Nyquist frequency of $1/2$ cycle per pixel. At the pixel centres $n \in \mathbb{Z}$,

$$\begin{aligned}\cos\Big(2\pi \cdot \tfrac{2}{3}\, n\Big) &= \cos\Big(2\pi n - \tfrac{2\pi n}{3}\Big) \\ &= \cos\Big(\tfrac{2\pi n}{3}\Big),\end{aligned}$$

which is a sinusoid of frequency $f_a = |2/3 - 1| = 1/3$, a period of 3 pixels. The 1.5-pixel stripes therefore appear as 3-pixel stripes with a different phase. Thin geometry behaves the same way in time. A wire one texel wide that moves by a fraction of a pixel per frame is hit by the sample in some pixels and missed in others, and the set of hit pixels changes from frame to frame. The result is flicker, which a supersampled target for the same frame does not show.

Supersampling.

Averaging K point samples per pixel approximates Eq. (S4),

$$P_i \approx \frac{1}{K} \sum_{k=1}^{K} f(\mathbf{c}_i + \mathbf{o}_k), \tag{S6}$$

where $\mathbf{o}_k$ are sample offsets inside the pixel. The renderer's targets use $K = 4$ at output resolution with the offsets $(-0.375, -0.125)$, $(0.125, -0.375)$, $(0.375, 0.125)$ and $(-0.125, 0.375)$ in pixel units, which form a rotated grid (`SSAA_OFFSETS`). As noted above, these four values are averaged after encoding.

Sub-pixel jitter.

Rendering K samples per pixel costs K times as much as one. A cheaper alternative takes one sample per pixel per frame and moves the sample grid by a different sub-pixel offset $\mathbf{j}_t$ in each frame, so the samples of successive frames fall at different places inside each pixel. The offsets are measured in pixels of the low-resolution grid and lie in $[-0.5, 0.5)^2$. Figure S1 shows the sample positions of two consecutive frames.

Halton sequence.

The offsets come from the Halton $(2, 3)$ sequence, a low-discrepancy sequence whose points spread evenly over the unit square with fewer large gaps than random points.[2]

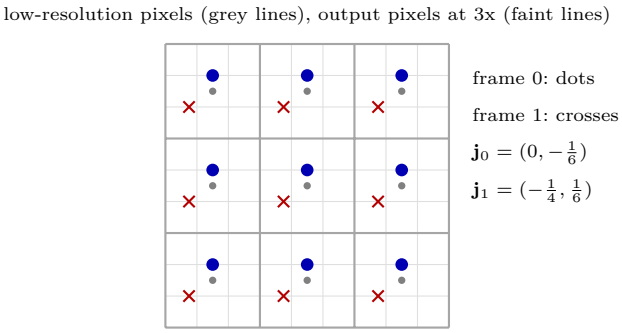


Fig. S1: Sample positions of two consecutive frames inside a 3×3 patch of low-resolution pixels, with rows increasing downwards as in the code. Grey dots mark the pixel centres. Blue dots are the samples of frame 0 and red crosses those of frame 1, shifted by the first two Halton offsets of Table S2. Faint lines mark the output pixels at 3x, which at this scale subdivide each low-resolution pixel into nine cells.

For an integer $n \geq 1$ with base-b digits $n = \sum_k d_k(n)\, b^k$, the radical inverse is

$$\varphi_b(n) = \sum_{k \geq 0} d_k(n)\, b^{-(k+1)}. \tag{S7}$$

For example, $3 = 11_2$ gives $\varphi_2(3) = \frac{1}{2} + \frac{1}{4} = \frac{3}{4}$, and $3 = 10_3$ gives $\varphi_3(3) = \frac{1}{9}$. The function `halton` computes this by repeated division by the base. Frame t uses the index $n_t = (t \bmod 16) + 1$ and the offset

$$\mathbf{j}_t = \big(\varphi_2(n_t) - \tfrac{1}{2},\ \varphi_3(n_t) - \tfrac{1}{2}\big), \tag{S8}$$

so the sequence repeats every 16 frames. The index starts at 1 because index 0 would give the corner $(-0.5, -0.5)$. Training gives each sample a random starting point in the cycle. Table S2 lists the first four offsets, which `jitter_sequence` produces.

Table S2: First four Halton jitter offsets. Frame t uses index $n = (t \bmod 16) + 1$, and the offset $\mathbf{j}_t$ of Eq. (S8) is in low-resolution pixels. The values were printed by `jitter_sequence` and are rounded to three decimals

Frame t	n	$\varphi_2(n)$	$\varphi_3(n)$	j_x	j_y
0	1	0.500	0.333	0.000	−0.167
1	2	0.250	0.667	−0.250	0.167
2	3	0.750	0.111	0.250	−0.389
3	4	0.125	0.444	−0.375	−0.056

Why jitter and accumulation recover resolution.

Take a static scene and one jittered sample per low-resolution pixel per frame. If the samples of earlier frames are placed at their true positions, using each frame's known offset, then after N frames each low-resolution pixel holds N samples at N different positions. The output needs s^2 pixels per low-resolution pixel, so the samples must cover the s^2 sub-cells, and the Halton positions do this within a few frames. Jitter-aware upsampling does the placement: it samples the low-resolution frame at the output pixel centre minus the jitter (`jitter_upsample`), so each sample lands at its true position.

Accumulation is a weighted average. Consider an exponential average $\hat{y} = \sum_{k\geq 0} w_k x_{t-k}$ with weights $w_k = \alpha(1-\alpha)^k$, $0 < \alpha \leq 1$, which sum to one. For samples x_{t-k} with independent noise of variance σ^2,

$$\operatorname{Var} \hat{y} = \sum_{k\geq 0} w_k^2 \sigma^2 = \frac{\alpha^2 \sigma^2}{1-(1-\alpha)^2} = \frac{\alpha}{2-\alpha}\,\sigma^2, \tag{S9}$$

so the average behaves like a mean of $N_{\text{eff}} = (2-\alpha)/\alpha$ independent samples. For $\alpha = 0.1$, the blend used by the analytic baseline of Section S2.5, $N_{\text{eff}} = 19$. This holds only when the history is aligned, so that a sample from an earlier frame belongs to the same surface point, and when the content is static. Motion and disocclusion break both conditions, which is why the next subsections need reprojection and rejection.

S2.3 Motion vectors and optical flow

Flow.

For frames t and s, the flow $F_{t\to s}$ is a field of displacements in pixels. For a pixel $\mathbf{p}$ of frame t, the point $\mathbf{p} + F_{t\to s}(\mathbf{p})$ is the position in frame s of the surface point that $\mathbf{p}$ shows in frame t. The code uses the backward convention for warping, in which the field is defined on the grid of the frame being produced, t, and points into the source frame s. Frame t is then predicted from frame s by

$$\hat{I}_t(\mathbf{p}) = I_s\big(\mathbf{p} + F_{t\to s}(\mathbf{p})\big), \tag{S10}$$

which is the backward warp of Section S2.4. A forward flow $F_{s\to t}$ is defined on the grid of the source frame and points forward. Datasets often store forward flow, and warping with it needs a scatter step that leaves holes and overlaps, so the code converts or reverses the flow instead. The code uses these conventions:

- Temporal super-resolution: the motion vector of frame t is $m_t = F_{t\to t-1}$, the displacement from the current frame to the previous one, at low-resolution pixels (`sr_batch`, `raft_mv_and_valid`).
- Frame generation: the engine motion vector of the newest real frame is $M_1 = F_{1\to 0}$, the displacement from I_1 to I_0 (`fg_batch`).
- Frame generation: the intermediate flows are $F_{\tau\to 0}$ and $F_{\tau\to 1}$, the outputs of `IFNet` (Section S2.7).

- Exact sources: the loader plays clips with forward flow backwards (`load_clip`). The stored flow from frame i to frame $i+1$ then becomes the displacement from the later frame to the earlier one, which is the convention of m_t, without any inversion.

Engine vectors and colour flow.

An engine writes a motion vector per pixel from the transforms of the geometry it draws. The vector is exact for opaque surfaces that the engine transforms, and particles and other effects without a transform get none, which is why engines also write a reactive mask. Research systems use such engine buffers, including motion vectors, as network inputs.[3] Colour-only flow estimates motion from the images. It works on any video, but it is an estimate that can be wrong at boundaries, and it cannot separate motion from a change of lighting or from an overlay that moves independently. The code uses torchvision's RAFT-Small [4] with 12 updates (`load_raft`, `raft_flow`), as the teacher for motion labels on real video and for tOF. The built-in motion estimator of the temporal super-resolution network is an IFNet [5].

Forward-backward consistency.

For two frames a and b with flows $F_{a\to b}$ and $F_{b\to a}$, a pixel passes the round-trip test when

$$\left\|F_{a\to b}(\mathbf{p}) + F_{b\to a}\big(\mathbf{p} + F_{a\to b}(\mathbf{p})\big)\right\| < 1 + 0.05\left\|F_{a\to b}(\mathbf{p})\right\|, \tag{S11}$$

which says that going to frame b and back returns to the start within one pixel plus 5 percent of the flow. The code's `raft_mv_and_valid` applies this test with a the current frame and marks the pixels that pass as valid. Pixels that fail lie at occlusions, at disocclusions and where the flow estimate is poor.

Occlusion and disocclusion.

A surface point is disoccluded in frame t when it is visible in t but absent from the previous frame s, and it is occluded when it is visible in s but covered in t. The warp of Eq. (S10) has no correct source for a disoccluded pixel, so history must not be used there. The code's validity mask v equals 1 where the pixel of frame t is visible in the previous frame, so $1 - v$ marks the disoccluded pixels (`load_valid`). The temporal super-resolution network also uses a depth cue, $|d_t - \mathcal{W}(d_{t-1})|/(|d_t| + 10^{-3})$, clipped to $[0, 1]$, as an input feature (`TSRNet.forward`).

Scene cuts.

A cut is a change of content between consecutive frames. No point corresponds across it, so any flow is meaningless, and no frame can be predicted from the one before. The code detects cuts with the distance between normalised 32-bin histograms of the luma (`cut_score`),

$$d_{\text{hist}}(I_0, I_1) = \tfrac{1}{2}\sum_{b=1}^{32}\big|h_0(b) - h_1(b)\big|, \tag{S12}$$

where h_0 and h_1 are the normalised histograms of the two frames. This distance lies in $[0, 1]$ and is the total-variation distance between the two histograms. When the motion vector M_1 is available, the score adds $2\,\overline{|I_1 - \mathcal{W}(I_0, M_1)|}$, twice the mean motion-compensated difference. A score above a threshold is a cut.

S2.4 Warping, reprojection and history accumulation

Backward warping.

Given a frame I and a flow F, the warp $\mathcal{W}(I, F)(\mathbf{p}) = I(\mathbf{p}+F(\mathbf{p}))$ must be evaluated at non-integer positions. The code uses bilinear interpolation. For $\mathbf{q} = \mathbf{p} + F(\mathbf{p})$ with integer part (x_0, y_0) and fractional parts $a = q_x - x_0$ and $b = q_y - y_0$,

$$\begin{aligned} I(\mathbf{q}) = (1-a)(1-b)\, I(x_0, y_0) + a(1-b)\, I(x_0+1, y_0) \\ + (1-a)b\, I(x_0, y_0+1) + ab\, I(x_0+1, y_0+1). \end{aligned} \tag{S13}$$

The four weights are non-negative and sum to one. Positions outside the frame are clamped to the border. Backward warping gathers each output pixel from one source location, so the output has no holes or overlaps, which forward splatting cannot guarantee. `backward_warp` implements Eq. (S13) with `grid_sample`, using the coordinate mapping of RIFE, in which pixel index i corresponds to the normalised coordinate $-1 + 2i/(W-1)$. Figure S2 shows one pixel.

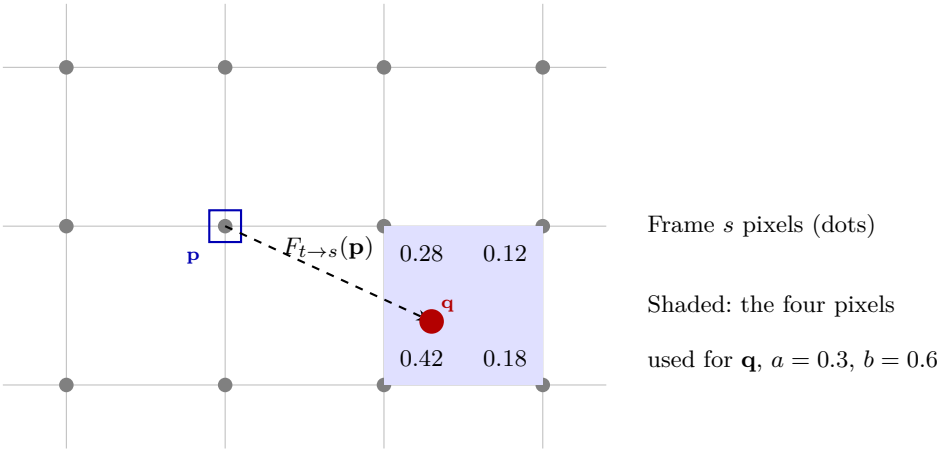


Fig. S2: Backward warp of one pixel. Pixel $\mathbf{p} = (1, 1)$ of frame t takes the value at $\mathbf{q} = \mathbf{p} + F_{t\to s}(\mathbf{p}) = (2.3, 1.6)$ in frame s. That value is the bilinear mixture of the four surrounding pixels of frame s, with the weights shown from Eq. (S13). The flow is illustrative.

Half-pixel attenuation.

Bilinear interpolation is a low-pass filter. For a shift of half a pixel the weights are $(\frac{1}{2}, \frac{1}{2})$, and the frequency response at angular frequency ω radians per pixel is $\frac{1}{2}(1 + e^{-i\omega})$, whose magnitude is $|\cos(\omega/2)|$. At $\omega = \pi/2$ this is $\cos(\pi/4) \approx 0.71$, and at the Nyquist frequency $\omega = \pi$ it is zero. Each warp at a fractional offset removes part of the high-frequency content. A recurrent network that warps its output every frame loses high frequencies at every step unless its residual path adds them back.

Reprojection and accumulation.

Reprojection moves a stored result to the current frame. A history image H from frame $t-1$ becomes $H_t(\mathbf{p}) = \mathcal{W}(H, m_t)(\mathbf{p})$ with $m_t = F_{t\to t-1}$, and accumulation then combines H_t with the new samples. The recurrent state of the temporal super-resolution network, its output and its hidden feature map, is reprojected in the same way with the low-resolution vectors. A pixel whose motion is wrong, or which is disoccluded, takes its history from the wrong surface, and that is the source of the ghosting described in Section S2.5.

S2.5 Temporal anti-aliasing and temporal upscaling

Temporal anti-aliasing (TAA) [6] combines jittered samples over time with a history buffer. For each frame it renders with a jittered sample grid, reprojects the history with motion vectors, clamps the reprojected history to the range of the current samples in a small neighbourhood, and blends the clamped history with the current samples. Temporal upscaling runs the same loop at a lower render resolution, with the accumulation buffer at output resolution.

Exponential blending.

Without reprojection and clamping, accumulation is $\hat{y}_t = \alpha x_t + (1-\alpha)\hat{y}_{t-1}$, where x_t is the current sample value and $0 < \alpha \leq 1$. Unrolling gives

$$\hat{y}_t = \sum_{k=0}^{t-1} \alpha(1-\alpha)^k \, x_{t-k} + (1-\alpha)^t \, \hat{y}_0, \tag{S14}$$

so the sample k frames old has weight $\alpha(1-\alpha)^k$, and the initial value keeps the weight $(1-\alpha)^t$. Figure S3 shows these weights for $\alpha = 0.1$. A sample 10 frames old has weight 0.035, and the initial value still has weight 0.12 after 20 frames.

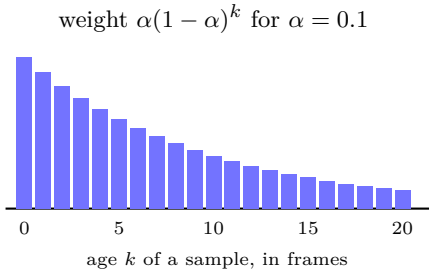


Fig. S3: Weight of a sample k frames old in the exponential average of Eq. (S14) with $\alpha = 0.1$. The bar height is the weight times 20, in centimetres of the drawing. The weights fall geometrically, so an old surface that stays in the history fades slowly, which is the ghosting trail that clamping limits.

Ghosting.

When a surface changes, because it is disoccluded, because its motion vector is wrong or because an object leaves a region, the history still holds the old surface. That surface keeps the weight $(1-\alpha)^k$ for k frames and appears as a trail behind moving objects. Clamping limits the trail. The reprojected value is confined to the range of the current neighbourhood, so an old colour far from the new samples is pulled back to the range boundary. Ghosting remains when the old colour lies inside that range, which happens when a surface and its background have similar colours. A larger α reduces ghosting but also reduces the averaging that removes aliasing, and that trade-off is the central design choice of TAA.

Analytic baseline.

The analytic upscaler of the evaluation (`AnalyticTAAU` in `nss_metrics.py`, run by `run_taau` in `nss_eval.py`) implements the loop with fixed rules and stands in for analytic TAA upscalers without their proprietary heuristics. For frame t, with low-resolution colour x_t, jitter $\mathbf{j}_t$ and low-resolution motion vector m_t, it performs the following steps.

1. Base image: $B_t(\mathbf{p})$ is the bilinear interpolation of x_t at the low-resolution position of the output pixel centre $\mathbf{p}$ minus $\mathbf{j}_t$, so each sample is placed at its true position.
2. If there is no history, the output is B_t.
3. History: $H_t = \mathcal{W}\big(\hat{y}_{t-1},\, s\,U(m_t)\big)$, where U is bilinear upsampling by s and the factor s converts low-resolution displacements into output pixels.
4. Clamp: with lo and hi the channel-wise minimum and maximum of $\mathrm{YCoCg}(B_t)$ over the 3×3 output neighbourhood of each pixel, the history is clamped in YCoCg to the box between lo and hi, and H'_t is the RGB value of the clamped history.
5. Blend: $\hat{y}_t = H'_t + 0.1\,(B_t - H'_t)$, which is 90 percent history and 10 percent current base image.

The displayed output is clamped to $[0,1]$, but the unclamped value becomes the next history. Step 5 gives

$$\hat{y}_t = 0.9\,H'_t + 0.1\,B_t. \tag{S15}$$

Two properties of this baseline matter for the comparison. First, its docstring says that it resets on invalid history, but the code has no validity test, so every frame reprojects and clamps the history and no disocclusion rejection takes place. Second, the motion vectors are sampled at the jittered sample positions (`_sr_inputs`), while the baseline interprets low-resolution vectors as if they were at the pixel centres. The resulting error is the jitter times the local gradient of the motion field, which vanishes for uniform motion and is largest at motion boundaries. The temporal super-resolution network receives the same low-resolution vectors and interprets them in the same way. The supplement describes these properties as they are in the code.

S2.6 Neural building blocks

The networks use a small set of operators. Each is defined here with the code's convention and with the reason for its use.

Convolution and multiply-accumulates.

A convolutional layer with C_{in} input channels, C_{out} output channels and a $k \times k$ kernel K computes, with stride 1 and padding $(k-1)/2$,

$$Y[o,i,j] = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K[o,c,u,v]\; X\Big[c,\; i+u-\tfrac{k-1}{2},\; j+v-\tfrac{k-1}{2}\Big], \tag{S16}$$

where X is the input, Y the output and b_o the bias of output channel o. The sum is a cross-correlation, which is PyTorch's convention. One multiply-accumulate (MAC) is one multiplication followed by one addition. Each output element of Eq. (S16) costs $C_{\text{in}}k^2$ MACs, so a layer with an $H \times W$ output costs

$$M = H\,W\,C_{\text{out}}\,C_{\text{in}}\,k^2 \qquad \text{MACs}, \tag{S17}$$

and grouped convolutions divide this by the number of groups. A transposed convolution from an $H_{\text{in}} \times W_{\text{in}}$ input costs $H_{\text{in}}W_{\text{in}}C_{\text{in}}C_{\text{out}}k^2$, because each input pixel scatters a $k \times k$ kernel into every output channel. Pixel shuffles, bilinear resizing and grid sampling perform no such multiply-accumulates in the count, so the warps are excluded from the cost figures, as in Table 1. For example, a 3×3 convolution with 24 input and 24 output channels costs $24 \cdot 24 \cdot 9 = 5{,}184$ MACs per output pixel.

Cost per 1080p frame.

The code counts MACs with forward hooks (`count_macs`) on the fused network, which is the deployed form in which every block is a plain convolution. For temporal super-resolution it runs one step with a history state on a 64×64 low-resolution input, including the built-in motion estimator, and divides the count by 64^2 to obtain MACs per low-resolution pixel (`tsr_macs_per_lr_pixel`). Each layer's cost is proportional to its pixel count, so the cost of one output frame is that per-pixel count times the number of low-resolution pixels of the 1080p output,

$$G_{1080p} = \frac{M_{\text{pix}}\,(1920/s)(1080/s)}{10^9} \quad \text{GMAC}. \tag{S18}$$

For frame generation the fused network runs on a 272×480 frame. The flow-resolution limit is scaled by $272/1080$ so that the flow runs at the scale that a 1080p frame would use, the MACs are counted, and the count is multiplied by $(1920 \cdot 1080)/(272 \cdot 480)$. This assumes that the cost is linear in the pixel count, which holds for convolutions and is approximate for the remaining operators. A MAC count is a cost measure and not a time, and the latency measurements are reported separately.

Pixel shuffle.

Pixel shuffle [7] rearranges a tensor with Cr^2 channels at $H \times W$ into C channels at $rH \times rW$,

$$Z[c,\; rh+a,\; rw+b] = X[c\,r^2 + a\,r + b,\; h,\; w], \qquad 0 \le a,b < r, \tag{S19}$$

without any multiplication. Its inverse, pixel unshuffle or space-to-depth, folds a tensor at $rH \times rW$ into r^2 times as many channels at $H \times W$. A convolution can therefore run at low resolution, where it is cheap, and the shuffle restores the output size. The temporal super-resolution network uses both directions. Its output convolution produces $4s^2$ channels at low resolution, which the shuffle turns into a residual and a history weight at output resolution, and it folds the reprojected output into $3s^2$ low-resolution channels before the trunk reads it.

Structural re-parameterisation.

A block can be trained as several parallel linear branches and deployed as one convolution. This works when the branches share their input and their spatial support, because convolution is linear in its kernel. The RepConv block of `nss_models.py` has four branches on the same input x, with C_{in} input and C_{out} output channels:

1. a 3×3 convolution with kernel W_3 and bias b_3.
2. a 1×1 convolution with kernel W_1 and bias b_1.
3. the identity, present when $C_{\text{in}} = C_{\text{out}}$.
4. fixed edge filters E_1 (Sobel x), E_2 (Sobel y) and E_3 (Laplacian), applied to every input channel, whose $3C_{\text{in}}$ responses are combined by a learned 1×1 projection $P \in \mathbb{R}^{C_{\text{out}} \times 3C_{\text{in}}}$ with bias b_e.

The block output is $\phi(Y)$, where ϕ is a PReLU activation and Y is the sum of the four branch outputs.

The four branches fold into one 3×3 convolution, exactly:

$$\begin{aligned} Y &= \sum_c K[\cdot, c] * x_c + b, \\ K[o, c] &= W_3[o, c] + \operatorname{pad}\big(W_1[o, c]\big) \\ &\quad + \sum_{k=1}^{3} \tilde{P}[o, c, k]\, E_k + \delta_{oc}\, \Delta, \\ b &= b_3 + b_1 + b_e, \end{aligned} \tag{S20}$$

where $*$ is convolution with stride 1 and padding 1, pad places a scalar at the centre tap of an otherwise zero 3×3 kernel, $\tilde{P}[o, c, k] = P[o,\, 3c + k]$ reorders the projection by input channel, δ_{oc} is the Kronecker delta and Δ is the 3×3 kernel with 1 at its centre and 0 elsewhere.

Proof. Each branch is linear in x and is a convolution with stride 1 and padding 1, so each can be written as a 3×3 convolution. (1) The 1×1 branch outputs $W_1[o, c]\, x_c(\mathbf{p})$. With padding 1, a 3×3 kernel whose only nonzero tap is the centre, with value $W_1[o, c]$, produces the same output, and that kernel is $\operatorname{pad}(W_1[o, c])$. (2) The identity outputs $x_o(\mathbf{p})$, which is the kernel Δ on the diagonal $c = o$. (3) The edge branch outputs $\sum_c \sum_k P[o, 3c + k]\, (E_k * x_c)(\mathbf{p}) + b_e[o]$. Convolution is linear in its kernel, so $\sum_k \tilde{P}[o, c, k]\, (E_k * x_c) = \big(\sum_k \tilde{P}[o, c, k] E_k\big) * x_c$, and the branch is a 3×3 convolution with kernel $\sum_k \tilde{P}[o, c, k] E_k$ and bias $b_e[o]$. (4) For a common stride and padding, $(A * x) + (B * x) = (A + B) * x$, so the four branch sums combine into

Eq. (S20). The activation is applied after the sum, so it is not folded, and the fold is exact. Up to floating-point rounding, the fused and unfused outputs are identical, and the code's fusion-parity test checks this numerically. □

Training uses the folded kernel directly. The code recomputes K from the branch parameters on every forward pass (online re-parameterisation), so a block costs one 3×3 convolution during training as well as in deployment. The function class is unchanged, because W_3 alone can represent any kernel that the block can reach. The extra branches only change which kernels gradient descent reaches. The 1×1 and edge-projection weights start at zero, so the block starts as a 3×3 convolution plus the identity. Only the weights are zeroed, so the small biases b_1 and b_e remain at their random initial values. Figure S4 summarises the fold.

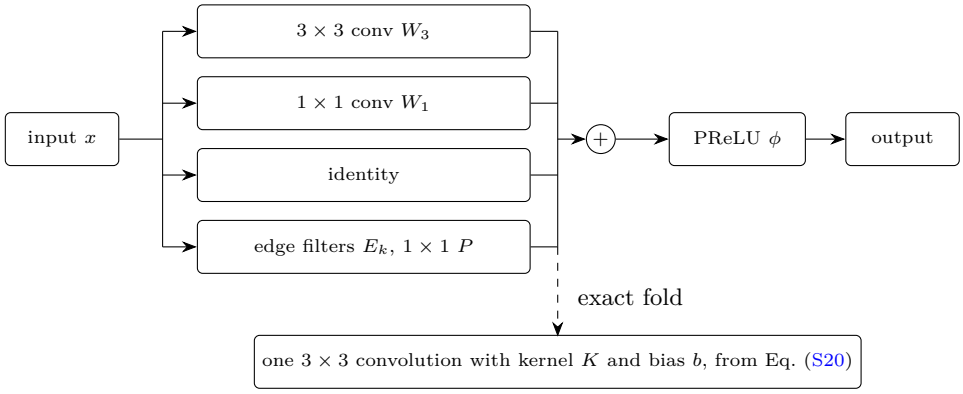


Fig. S4: Re-parameterisation of one RepConv block. The four linear branches share the input and sum before the activation, giving one kernel K and bias b of size 3×3.

U-Net levels.

The trunk of the temporal super-resolution network has an optional half-resolution level. It differs from the classic U-Net,[8] so it is described precisely here. A strided 3×3 convolution halves the resolution and doubles the channels. Residual blocks run at half resolution, a 1×1 convolution doubles the channels to $4w$, and a pixel shuffle returns to full resolution with w channels. The result is added to the features that entered the level, which is an additive skip, where the classic U-Net concatenates. A convolution with $2w$ channels at half resolution costs the same as one with w channels at full resolution, because $(2w)^2 \cdot \frac{1}{4} = w^2$, but each of its taps covers twice the distance in the original image.

Recurrent networks and hidden-state warping.

A recurrent network carries a state from frame to frame. The state of the temporal super-resolution network holds the previous output $\hat{y}_{t-1}$, a hidden feature map h_{t-1}, and the previous depth, colour and jitter. At each frame the network reads the current inputs together with the reprojected output and hidden state, and writes the new

output and a new hidden state $h_t = \tanh(\cdot)$ of a convolution. The hidden state is reprojected with the same motion as the output, $\mathcal{W}(h_{t-1}, m_t)$, because its channels describe the content at each pixel, and a feature warped with the wrong motion describes a different surface. Training unrolls the network over several frames, with a 16-frame unroll every fourth step, and backpropagates through the whole unroll, which is backpropagation through time. During the first quarter of training (`mv_warmup_frac` $= 0.25$), samples without motion vectors warp the history with the label motion instead of the estimated motion. The probability of doing so falls linearly from one to zero over that quarter. Feeding the label in place of the model's own estimate is teacher forcing, which this code applies as a schedule, and the schedule stops the history path from learning to ignore history while the estimator is still poor.

Zero initialisation.

A branch whose output layer has zero weights and a zero bias contributes exactly zero at initialisation, so the network starts as the network without that branch. Two consequences follow. The output layer receives a nonzero gradient, because its input features are nonzero, so it learns from the first step. The layers upstream of it receive a zero gradient at the first step, because they reach the loss only through the zero weights, and they start to learn once the output layer is nonzero. The bias must also be zero, because a nonzero bias gives a constant contribution at initialisation. The code uses three instances.

- The engine-hint input channels of the first convolution in each IFNet block have zero weights, so the hints do not change that layer's output at initialisation. This is exact.
- The frame generator's residual head has zero weights and a zero bias, so its colour residual r is zero at initialisation. Its UI logit has a bias of -4, so the UI weight is $u = \sigma(-4) \approx 0.018$ at initialisation. The output is therefore $u\, I_1 + (1-u)$ times the RIFE blend $m\mathcal{W}(I_0, F_{\tau\to 0}) + (1-m)\mathcal{W}(I_1, F_{\tau\to 1})$, and differs from that blend by $u\,(I_1 - \text{blend})$. The code's parity test compares the blend, not the final output, so a fresh model reproduces the blend of RIFE exactly and not its final output.
- The temporal super-resolution output layer has zero weights and a zero bias, so the residual is zero and the history weight is $\sigma(0+2) \approx 0.88$, since the code adds 2 to the logit before the sigmoid. At initialisation the output is therefore 0.88 of the history and 0.12 of the jitter-aware upsampling, an exponential blend of the kind described in Section S2.5.

Mixed precision and loss scaling.

In IEEE half precision (FP16) a value has one sign bit, five exponent bits and ten fraction bits. Its largest finite value is $(2 - 2^{-10})\,2^{15} = 65{,}504$, its smallest positive normal value is $2^{-14} \approx 6.1 \cdot 10^{-5}$, and its smallest positive subnormal value is $2^{-24} \approx 6.0 \cdot 10^{-8}$. Two failures follow. Activations above 65,504 become infinite, and gradients much smaller than $6 \cdot 10^{-8}$ round to zero. Mixed precision [9] runs the forward and backward passes in FP16 and keeps FP32 copies of the weights. Loss scaling multiplies the loss by a scale S before the backward pass, so that small gradients stay representable, and divides the gradients by S before the optimiser step. Dynamic loss

scaling lowers S when a step produces non-finite gradients and raises it again after a run of finite steps. The code uses PyTorch's `GradScaler` with autocast. It scales the loss, unscales the gradients, clips their norm to 1.0 (`grad_clip`) and then steps the optimiser. A run stops when the loss stays non-finite for 200 steps by default.

Exponential moving average.

The optimiser updates the weights w_k, and the code also keeps the average

$$\bar{w}_k = d_k\,\bar{w}_{k-1} + (1 - d_k)\,w_k, \qquad d_k = \min\Big(0.999,\ \frac{1+k}{10+k}\Big), \tag{S21}$$

which is used for validation and checkpoints, while the raw weights produce the gradients. The warm-up in d_k means that the decay reaches 0.999 only after about 9,000 steps, so early in training the average follows the weights closely. At a decay of 0.999 the average has a memory of about $1/(1 - 0.999) = 1{,}000$ steps.

AdamW.

AdamW [10] keeps running averages of the gradient and of its square, $m_k = \beta_1 m_{k-1} + (1 - \beta_1)g_k$ and $v_k = \beta_2 v_{k-1} + (1 - \beta_2)g_k^2$, corrects their start-up bias and applies

$$\theta_k = \theta_{k-1} - \eta\Big(\frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon} + \lambda\,\theta_{k-1}\Big), \qquad \hat{m}_k = \frac{m_k}{1 - \beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k}, \tag{S22}$$

where g_k is the gradient, η the learning rate and λ the weight decay. The decay term $\lambda\theta$ acts on the weights directly and is not added to the gradient, which is the difference from Adam with an L2 penalty. The code uses $(\beta_1, \beta_2) = (0.9, 0.99)$, PyTorch's default $\epsilon = 10^{-8}$ and $\lambda = 10^{-4}$.

Learning-rate schedule.

The schedule is driven by wall-clock time rather than by the step count. With progress $p \in [0, 1]$, the elapsed fraction of the run's time budget, a warm-up fraction $w = 0.03$, a peak rate η_0 and a floor $\eta_{\min} = 10^{-6}$,

$$\eta(p) = \begin{cases} \eta_0\ \max(p/w,\ 0.01), & p < w, \\ \eta_{\min} + \tfrac{1}{2}(\eta_0 - \eta_{\min})\big(1 + \cos \pi\tau\big), & \tau = \dfrac{p - w}{1 - w} \geq 0, \end{cases} \tag{S23}$$

which is `lr_at`. The learning rate follows a single cosine decay without restarts, the decay used within each cycle of SGDR [11]. The peak rates are $2 \cdot 10^{-3}$ for the temporal super-resolution student, 10^{-3} for its teacher, $4 \cdot 10^{-4}$ for the frame-generation students and 10^{-4} for the frame-generation tier fine-tuned from RIFE 4.26. Because the schedule follows wall-clock time, a slower GPU performs fewer updates in the same budget.

Charbonnier loss.

The Charbonnier penalty [12] is a smooth edge-preserving potential that approximates the absolute error,

$$\rho_\epsilon(e) = \sqrt{e^2 + \epsilon^2}, \qquad \epsilon = 10^{-3}, \tag{S24}$$

which is differentiable at zero and tends to $|e|$ for large errors. Its gradient $e/\rho_\epsilon(e)$ has magnitude at most one, so large errors do not dominate the gradient as they do under a squared error. The code averages ρ_ϵ over the elements (`charbonnier`), with an optional mask that normalises by the number of masked elements.

S2.7 Video frame interpolation and extrapolation

Given real frames I_0 at time 0 and I_1 at time 1, interpolation produces I_τ for $0 < \tau < 1$. Extrapolation produces I_τ for $\tau > 1$ from frames at or before time 1 only. Figure S5 shows the time axis of both modes. Multi-frame generation with a factor N inserts $N-1$ frames between real frames, so 3x generation uses $\tau = \frac{1}{3}$ and $\tau = \frac{2}{3}$. Generation raises the displayed frame rate without rendering the scene again, but it has to wait for the real frame it needs, as Section S2.8 explains.

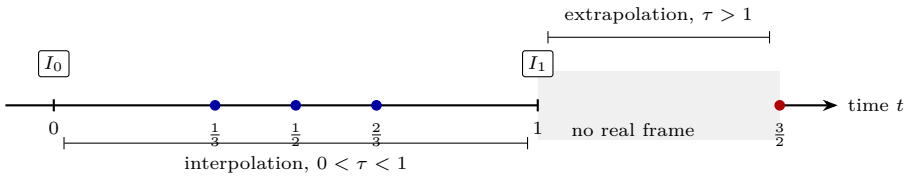


Fig. S5: Time axis of the two generation modes with real frames I_0 and I_1. Interpolation (blue) places generated frames between the real frames, at $\tau = \frac{1}{2}$ or at the two positions of 3x generation. Extrapolation (red) places a generated frame after I_1 at $\tau = \frac{3}{2}$ using only frames at or before time 1. The shaded interval has no real frame.

Intermediate flows.

The generator needs the motion from each real frame to the intermediate time τ, which is $F_{\tau\to0}$ and $F_{\tau\to1}$ as defined in Section S2.3. Under constant velocity, a surface point moves as $\mathbf{p}(\tau) = \mathbf{p}_0 + \tau\mathbf{v}$, where $\mathbf{v}$ is its displacement from time 0 to time 1, so that $\mathbf{p}_1 = \mathbf{p}_0 + \mathbf{v}$. Its displacement from I_1 to I_0 is $M_1 = -\mathbf{v}$, and therefore

$$F_{\tau\to0} = \tau M_1, \qquad F_{\tau\to1} = (\tau - 1)M_1, \tag{S25}$$

which are the linear-motion priors of the frame generator. To see this, note $F_{\tau\to0} = \mathbf{p}_0 - \mathbf{p}(\tau) = -\tau\mathbf{v} = \tau M_1$, and $F_{\tau\to1} = \mathbf{p}_1 - \mathbf{p}(\tau) = (1-\tau)\mathbf{v} = (\tau-1)M_1$. The code evaluates M_1 at the target pixel rather than at the moving point, which is exact when the motion field is locally constant.

Coarse-to-fine estimation.

The intermediate flows come from an IFNet with the design of RIFE 4.26 [5]. It is a chain of blocks, each working at a fraction $1/s_k$ of the input resolution, with scales $16, 8, 4, 2, 1$ for the five blocks of the full network. Each block predicts a blend logit and an update to the flow at its own resolution. The update is resampled to full resolution and multiplied by s_k, because a displacement in pixels scales with resolution, so

$$F^{(k)} = F^{(k-1)} + s_k \, \mathcal{U}_{s_k}\big(\tilde{\Delta}^{(k)}\big), \tag{S26}$$

where $\tilde{\Delta}^{(k)}$ is the raw update of block k and $\mathcal{U}_{s_k}$ is bilinear upsampling by s_k. The coarse blocks capture large motion at low cost, and the fine blocks add detail.

Blending.

The two warped frames are combined with a per-pixel blend $m \in [0, 1]$, which is the sigmoid of the network's logit,

$$\hat{I}_\tau = m \odot \mathcal{W}(I_0, F_{\tau\to 0}) + (1 - m) \odot \mathcal{W}(I_1, F_{\tau\to 1}), \tag{S27}$$

where $\odot$ is the element-wise product. Where a surface is visible in only one real frame, the correct blend selects that frame, and this is how interpolation handles occlusions.

Resolution rule and cuts.

Flow is costly, so the generator estimates it at a reduced scale for tall frames. The flow scale is the largest 2^{-k} with $H\,2^{-k} \le L$, where L is the flow limit of the tier, 576 or 1152 lines. At 1080p this gives 540 lines for $L = 576$ and full resolution for $L = 1152$. When two real frames come from different scenes, the flows and the blend describe no correspondence, so generation should not run. The code repeats I_1 in that case, as Section S2.3 describes. Interpolation also fails where the motion exceeds what the flow search can resolve, at disocclusions where the blend cannot choose, for particles and other motion without correspondence, and on overlays such as text and HUD elements that move differently from the scene.

S2.8 Frame budgets, pacing and latency

Frame budget.

A display that refreshes R times per second shows a new frame every $1000/R$ milliseconds. That is 16.7 ms at 60 Hz, 33.3 ms at 30 Hz and 8.3 ms at 120 Hz. A renderer that cannot finish a frame within the interval either presents it late or repeats the previous one, so every stage of the frame must fit the budget if the rate is to be sustained. The stages are a CPU update of c ms, a GPU render of r ms, the upscaling of s ms, the frame generation of g ms and the presentation of p ms. In a GPU-bound loop the period is set by the GPU, so the period of a generating loop is $P = \max(c,\, r + s + g)$, the rule that `simulate_latency` applies, and the CPU work starts just in time so that no frame waits in a queue.

Three quantities that differ.

Three measures are easily confused.

- The rendered frame rate is the number of real frames per second that the engine produces, which is $1000/P$ for a loop without generation.
- The displayed frame rate is the number of frames presented per second, including generated frames. With one generated frame per real frame it is about twice the rendered rate, provided that generation does not lengthen the period, and in a GPU-bound loop it does lengthen it.
- The input latency is the time from an input sample to the first photons of a frame that reflects it. The frame rate does not determine it.

Latency of interpolation and extrapolation.

Interpolation shows a generated frame between I_{k-1} and I_k, which needs I_k to exist. The real frame I_k is therefore held until the generated frame has been shown, and it is shown about half a period later, so the input latency of a real frame rises by $g + P/2$. Extrapolation generates from past frames only, so a real frame is presented as soon as it is rendered and its latency does not change. The generated frame follows the real frame, and in the simulator it is presented half a period after it. Figure S6 shows the presentation times that these rules give for one real frame in each mode.

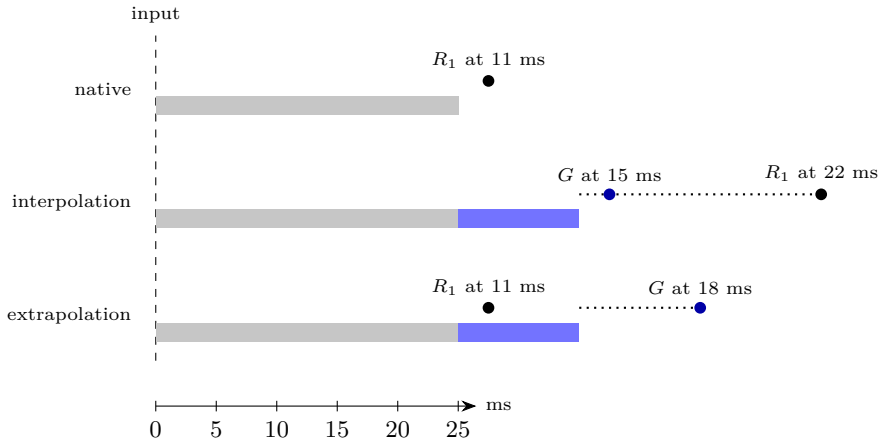


Fig. S6: Schematic presentation times for one real frame and one generated frame, from the rules of the frame-pacing simulator, with render 10 ms, generation 4 ms and presentation 1 ms, and with CPU work and upscaling omitted. Grey bars are rendering, blue bars are generation and dotted lines are frames held for presentation. Interpolation holds the real frame until the generated frame is shown, so the real frame arrives 11 ms after its native time. The values are illustrative and are not measurements.

The simulator's rules for the three modes are these. For a real frame, native latency is $c+r+s+p$, and interpolation adds $g+P/2$ to it. Extrapolation leaves it unchanged. For a generated frame, the camera latency, the time from input to the first displayed frame that reflects the camera, is $c+r+s+g+p$ for interpolation. For extrapolation it is $c+r+s+p+P/2$, or, with late latching, $g+p$.

Late latching.

Late latching samples the camera again as late as possible before a frame is generated, so that the generated frame moves with a newer camera. In the simulator this reduces the camera latency of an extrapolated frame to the generation time plus the presentation time, $g+p$. Late latching needs a camera input read just before generation, which the engine must provide.

Frame pacing and variable refresh.

Pacing means presenting frames at evenly spaced times. Uneven gaps are seen as stutter even when the mean rate is high, so the measure is the coefficient of variation of the gaps between presented frames, which the simulator reports as `frame_gap_cv`. A variable-refresh display presents each frame when it is ready within its supported range instead of at fixed intervals. The presentation time then equals the completion time, so irregular completion becomes irregular spacing unless the scheduler paces the frames. The simulator models such a display, which presents at the ready time.

S2.9 Metrics

Each metric compares an output frame o with a reference frame g, both encoded RGB, and the references are exact because the test data are synthetic or come with ground truth. Metrics are computed per frame and per sample and then averaged. Table S3 summarises the direction and the limits of each measure, and the paragraphs below give the definitions exactly as the code computes them.

PSNR.

With a mask $M(\mathbf{p}) \in \{0,1\}$, the masked mean squared error and the PSNR in decibels are

$$\mathrm{MSE}_M = \frac{\sum_{\mathbf{p}} \sum_c M(\mathbf{p}) \big(o_c(\mathbf{p}) - g_c(\mathbf{p})\big)^2}{\max\big(\sum_{\mathbf{p}} \sum_c M(\mathbf{p}),\, 1\big)}, \qquad \mathrm{PSNR} = 10 \log_{10} \frac{1}{\max(\mathrm{MSE}_M,\, 10^{-10})}, \tag{S28}$$

where c runs over the three channels and $M \equiv 1$ gives the plain PSNR. The clamp caps the PSNR at 100 dB. Higher is better.

PSNR-Y.

The same formula is applied to Y of Eq. (S2), after removing the s outermost pixels of the high-resolution frame on each side (`psnr` with `border` $= s$). Because Y is 219/255 times the full-range luma plus a constant, the PSNR-Y of an error is

Table S3: Metrics of the supplement: range, direction and what each measure misses. Ranges are those of the definitions, and the cap of PSNR is the code's clamp

Metric	Range	Better	What it misses
PSNR (RGB, frame generation)	up to 100 dB	higher	Only the per-pixel squared error, so a blurred frame can score well
PSNR-Y (temporal super-resolution)	up to 100 dB	higher	Luma only, so chroma errors are ignored
SSIM	-1 to 1	higher	Local structure in 11×11 windows, nothing over time
LPIPS	non-negative	lower	Learned features of one network, nothing over time
FLIP (mean)	0 to 1	lower	A perceptual model with fixed viewing assumptions
IE	0 to 255	lower	A global error, like PSNR on another scale
Temporal-change error	non-negative	lower	Static errors that do not change in time, and it needs motion labels
Warp error	non-negative	lower	Agreement with the reference, so a consistently blurred output can score well
tOF	pixels	lower	Errors of the flow estimator, computed on every fourth frame
UI ghosting rate	0 to 1	lower	Errors below 10 percent of full scale and UI pixels only
Region PSNR	up to 100 dB	higher	Pixels outside the region, and the quality of the region mask
Gated share	0 to 1	lower on cut-free clips	Cuts that the gate misses, since the clips contain none

$20 \log_{10}(255/219) \approx 1.32$ dB higher than the PSNR of the full-range luma with the same error. This offset is constant, so the differences between methods are unaffected.

SSIM.

The structural similarity index of [13] compares local means, variances and covariance under a Gaussian window,

$$\mathrm{SSIM}(\mathbf{p}) = \frac{(2\mu_o\mu_g + C_1)(2\sigma_{og} + C_2)}{(\mu_o^2 + \mu_g^2 + C_1)(\sigma_o^2 + \sigma_g^2 + C_2)}, \qquad C_1 = (0.01)^2, \quad C_2 = (0.03)^2, \tag{S29}$$

where μ, σ^2 and σ_{og} are the local mean, variance and covariance under an 11×11 Gaussian window with $\sigma = 1.5$ whose weights sum to one. The window is evaluated only where it lies inside the image (a valid convolution), so the map is 10 pixels smaller in each direction. The score is the mean over the map and over channels, lies in $[-1, 1]$ and is higher for a better match. The temporal super-resolution network scores Y, and the frame generator scores each RGB channel.

LPIPS.

The learned perceptual distance [14] is

$$d_{\mathrm{LPIPS}}(o, g) = \sum_{l} \frac{1}{H_l W_l} \sum_{h,w} \left\| \mathbf{w}_l \odot \left(\hat{\phi}^l_{hw}(o) - \hat{\phi}^l_{hw}(g) \right) \right\|_2^2, \tag{S30}$$

where $\hat{\phi}^l_{hw}$ is the unit-normalised feature vector of AlexNet layer l at position (h, w), $\mathbf{w}_l$ are learned per-channel weights and $H_l \times W_l$ is the layer's size. Lower is better. The code calls the `lpips` package with AlexNet and `normalize=True`. If that package cannot build the network, `LPIPSAlex` builds the same network from torchvision's ImageNet AlexNet weights and the version 0.1 linear weights of the package, and maps the inputs from $[0, 1]$ to $[-1, 1]$ with $2x - 1$ (`load_lpips`). The temporal super-resolution evaluation computes LPIPS on every fourth frame, and the frame generator computes it on every sample.

FLIP.

FLIP [15] is a perceptual difference evaluator that produces a per-pixel error map. The code calls `flip_evaluator.evaluate` with the LDR traits and takes element 1 of its result (`flip_error`). The official example of the package unpacks the result as the error map, the mean FLIP error and the parameters, so element 1 is the mean error. The traits string in the official example is `"HDR"`, so the LDR setting used by the code is not checked against an example here. The temporal super-resolution notebook calls the function on one frame pair, and FLIP is therefore a single-frame measure in this supplement. Lower is better.

Interpolation error (IE).

The code computes

$$\mathrm{IE} = 255 \sqrt{\frac{1}{3HW} \sum_{\mathbf{p}} \sum_{c} \left(o_c(\mathbf{p}) - g_c(\mathbf{p}) \right)^2}, \tag{S31}$$

which is the root-mean-square difference on the 0 to 255 scale (`interpolation_error`). Lower is better. The measure of Baker et al. [16] is the root-mean-square difference between the ground-truth and the interpolated image, taken over the RGB channels of every pixel for colour, and the 0 to 255 scale is this code's convention. `fg_metrics` computes IE for every frame-generation sample, and the Middlebury interpolation set [16] is one of the frame-generation suites, with the code naming the measure after that benchmark.

Temporal-change error.

This measure compares the frame-to-frame change of the output with that of the reference along the reference motion. Let m_t be the motion vector of frame t and v_t the validity mask. Define $D_o = o_t - \mathcal{W}(o_{t-1}, m_t)$ and $D_g = g_t - \mathcal{W}(g_{t-1}, m_t)$. The

error is

$$e_{\mathrm{tc}}(t) = \frac{\sum_{\mathbf{p}} v_t(\mathbf{p}) \, \frac{1}{3} \sum_c \left| D_o(\mathbf{p}, c) - D_g(\mathbf{p}, c) \right|}{\max\left(\sum_{\mathbf{p}} v_t(\mathbf{p}),\, 1 \right)}. \tag{S32}$$

It is computed for frames $t \geq 1$ of samples whose motion labels are exact or from RAFT (`temporal_change_error`). Lower is better. Flicker and ghosting appear as change where the reference has none, and the reference's own change is subtracted. A stable output with a constant error relative to the reference scores zero, so the measure does not see static errors. Its value also depends on the quality of the motion labels, which are less reliable for RAFT labels than for exact ones.

Warp error.

The warp error uses the same warp but compares the output with itself,

$$e_w(t) = \frac{\sum_{\mathbf{p}} v_t(\mathbf{p}) \, \frac{1}{3} \sum_c \left| o_t(\mathbf{p}, c) - \mathcal{W}(o_{t-1}, m_t)(\mathbf{p}, c) \right|}{\max\left(\sum_{\mathbf{p}} v_t(\mathbf{p}),\, 1 \right)}, \tag{S33}$$

and it is computed only for samples with exact motion labels (`warping_error`). Lower is better. It measures the consistency of the output along true motion and does not look at the reference, so a consistently blurred output can score well. It should be read together with the temporal-change error.

tOF.

The temporal optical-flow difference of [17] compares the RAFT-Small flow of the output with the flow of the reference,

$$\mathrm{tOF}(t) = \frac{1}{HW} \sum_{\mathbf{p}} \left\| F^{\mathrm{RAFT}}_{o_t \to o_{t-1}}(\mathbf{p}) - F^{\mathrm{RAFT}}_{g_t \to g_{t-1}}(\mathbf{p}) \right\|_2, \tag{S34}$$

in pixels (`tof`). It is computed for $t \geq 1$ with t divisible by 4 (the `lpips_every` setting). Lower is better. Flicker changes the estimated flow, so the measure detects it, but errors of RAFT itself enter both terms and cannot be separated from the output's errors. The file also defines `tlp`, the LPIPS analogue of tOF, but no evaluation function calls it, so it is not reported.

UI ghosting rate.

Let $U = \{\mathbf{p} : M_{\mathrm{UI}}(\mathbf{p}) > 0.5\}$, where $M_{\mathrm{UI}} = \max(\alpha_0, \alpha_1)$ combines the UI coverage of the two real frames. The ghosting rate is

$$R_{\mathrm{ghost}} = \frac{\left| \{\mathbf{p} \in U : \ \max_c |o_c(\mathbf{p}) - g_c(\mathbf{p})| > 0.1\} \right|}{\max(|U|, 1)}, \tag{S35}$$

which is the share of UI pixels whose error exceeds 10 percent of full scale in any channel (`fg_metrics`). It is defined only for samples that contain UI. Lower is better. It counts errors above the threshold and ignores their size.

Region PSNR.

Eq. (S28) is restricted to five masks. The disocclusion mask is $1 - v_t$ for $t \geq 1$. The detail mask is the top 15 percent of the reference luma gradient, where the central differences $\partial_x Y(\mathbf{p}) = Y(\mathbf{p}+\mathbf{e}_x) - Y(\mathbf{p}-\mathbf{e}_x)$ and $\partial_y Y$ give a magnitude $\sqrt{\partial_x Y^2 + \partial_y Y^2}$, thresholded at the 85th percentile of each frame (`hf_mask`). The particle mask is the reactive mask, upsampled to output resolution and thresholded at 0.1. The UI mask is U. The occluded mask of the frame generator is $1 - v_{\tau \to 1}$, the target pixels that are not visible in I_1. Each region PSNR is reported only when its mask covers more than 50 pixels. Higher is better. The measure ignores all pixels outside its mask, and the disocclusion mask is only as good as the validity labels. Where a sample has no motion labels, v is zero and the disocclusion mask covers the whole frame (`sr_frame_metrics`). When RAFT is available, every frame after the first of a real clip has labels, so the all-pixel case arises only without RAFT.

Gated share.

The cut gate skips generation and repeats the newest real frame where the score of Eq. (S12), with the motion term when a motion vector is available, exceeds its threshold (`cut_gate`). The test clips contain no cuts, so every gated real sample is a false alarm, and the gated share is the fraction of real samples on which the gate fires. Lower is better on cut-free clips. The gated share says nothing about cuts that the gate misses, because the clips contain none.

S3 Data, copy detection and leakage control

The data pipeline notebook builds the single manifest that both training notebooks read. It has three jobs. It gathers public stills and video sequences, it removes near copies so that no test content reaches training, and it records for every item its leakage group, its split and the locations of its frames, motion and depth. Section 3 of the article summarises the procedure. This section gives the complete method, every threshold, and every saved output of the notebook.

Every result here comes from one saved run of the pipeline. A result that rests on one run carries the [Preliminary] tag, because a run with other seeds or other caps can change the counts. Descriptions of what the code does carry no tag. Every count and threshold quoted in this section is a generated result key, except for design constants that are written in the code, and each of these is quoted from the code.

Four terms are used throughout. An *item* is one still image or one video sequence, and it is the unit of the manifest. A *keyframe* is a decoded frame of an item that is used for statistics and copy detection. A *leakage group* is a set of items that must never be split between training, validation and test, such as the frames of one video or one rendered scene. A *duplicate edge* joins two keyframes that show the same picture. Each item ends with one split: train, val or test for items that are used, drop for an item removed because it is a near copy of test content or shares a group with it, and filtered for an item removed by a quality rule.

The notebook runs eleven timed stages in this order: Kaggle discovery, Hugging Face materialisation, keyframe statistics, quality filters, perceptual hashes, DINOv2

embeddings, SIFT features, threshold calibration, candidate search and verification, the leakage-safe split with its audit, and the manifest with the dataset card. Table S4 gives the time of each stage.

Table S4: Stage times of the saved run, in minutes

Stage	Minutes
Kaggle discovery	1.659
Hugging Face materialisation	14.24
Keyframe statistics	11.95
Filters	0.2582
Perceptual hashes	0.1251
Embeddings	0.3602
Local features (SIFT)	5.086
Calibration	0.9402
Duplicate search and verification	1.379
Split and audit	1.341
Manifest	0.003482

S3.1 Source datasets

The pipeline reads two hosts. The Kaggle datasets are attached to the notebook and read from the read-only input mount, which the code locates by the dataset's owner and slug. The Hugging Face (HF) datasets are read over HTTPS. The code keeps one connection alive per thread, checks the length of every range read, and retries failed reads. A zip member is read through range requests without downloading the archive, and the GameIR tars are read as one sequential stream that stops once the clip cap is reached. The Vimeo-1080p step is different. It downloads each named parquet shard in full, reads the MP4 files inside it, and deletes the shard, so the notebook's statement that the pipeline makes no full archive downloads does not hold for that source. Each source is optional. A source that fails is logged and skipped, and the run continues.

Table S5 lists the eleven sources. The sizes are the means of the saved keyframe sizes. The licence column repeats the strings written in the code. The upstream licence pages were not checked for this section, so these are the code's statements and not checked legal facts. The code marks the game captures as needing a check for each use, and it gives the Vimeo-1080p licence as unspecified. The notebook's dataset card refers to `RESEARCH.md` for commercial-use restrictions. That file exists in the repository, but its contents were not checked for this section. The licences need checking against each upstream page before any release.

The roles are set by the code. The test items are the DIV2K validation images, the Vid4 clips, the official Vimeo-90K triplet test list, the Sintel `market` and `cave` scene families, the two TartanAir test environments, the GameIR test tar and the Vimeo-1080p validation shard. Every other item is a training candidate, and the split of Section S3.7 decides whether it ends in train or val. The sources are DIV2K [18], REDS [19], Vimeo-90K [20], MPI Sintel [21], TartanAir [22], GameIR [23], Vid4 [24]

Table S5: Source datasets, as the code reads them. Sizes are mean keyframe widths and heights in pixels from the saved keyframe table. Licence strings are the code's statements, not checked against upstream pages

Source	Host and identifier	Content and ground truth	Size (px)
DIV2K	Kaggle `soumikrakshit/div2k-high-resolution-images`	Photographs, still, no motion, and validation images pinned to test	1,972×1,437
Flickr2K	Kaggle `daehoyang/flickr2k`	Photographs, still, train only	1,934×1,450
Game stills	HF `ericphann/video-game-super-resolution`	1080p game captures, still, with `training-hr` as train and `test-hr` as test	1,920×1,080
REDS	Kaggle `amithkesavmrajagiri/reds-dataset`	720p video clips, no motion labels, train only	1,280×720
Vimeo-90K septuplets	Kaggle `wangsally/vimeo-90k-7`	Seven-frame video sequences, no motion labels, train only	448×256
Vimeo-90K triplets	Kaggle `chenshu123/vimeo-triplet`	Three-frame sequences from the official test list, test only	448×256
MPI Sintel	Kaggle `artemmmtry/mpi-sintel-dataset`	Rendered scenes with exact forward flow and occlusion masks, with `market` and `cave` as test	1,024×436
Vid4	Kaggle `uom200647r/vid4-dataset`	Video clips, no motion labels, test only	716×528
TartanAir	HF `theairlabcmu/tartanair`	Rendered scenes with exact forward flow, occlusion masks and depth, with two environments as test	640×480
GameIR	HF `LLLebin/GameIR`	CARLA and Unreal Engine 4 game renders at 1440p and 720p, with depth and no exact flow, and the test tar as test	1,920×1,440
Vimeo-1080p	HF `danjacobellis/vimeo1080p`	1080p MP4 videos, no motion labels, validation shard as test	1,920×1,080

and CARLA [25], which the GameIR renders use. The DIV2K training and validation counts follow the published split of 800 training and 100 validation images, which the catalogue reproduces.

Two sources carry ground truth beyond the frames themselves. Sintel and TartanAir give exact forward optical flow and occlusion masks, and TartanAir and GameIR give depth. These are the only sources the training uses as exact labels, and the rest rely on the frames, either as targets or through the RAFT teacher flow of Section S3.10.5.

S3.2 Discovery and catalogue

Discovery runs once per attached Kaggle dataset, and the seven Kaggle sources run in parallel threads. For each dataset the code lists files with `sampled_walk`. This function keeps a frontier of directories, expands one directory chosen at random at each step, and lists every file in that directory completely. A listed sequence folder is therefore never cut in half. Listing stops when the file cap is reached, which is 40,000 files by

default. The REDS and Vimeo-90K septuplet listings reached the cap before the walk finished, so their listings are incomplete, and the other sources listed completely.

A file is kept only if its path has no token that marks a degraded copy or a non-colour pass. The degraded tokens are `lr`, `lq`, `bicubic`, `blur`, `x2` to `x4` and similar, and the non-colour tokens include `flow`, `depth`, `occlusions`, `mask` and `albedo`. Tokens are the lower-case pieces of the directory and file names. For video, if any path carries a high-resolution token (`hr`, `gt`, `original` or `groundtruth`), only those paths are kept. Stills become one item each. Video frames become one item per folder, and the leakage group is the folder name for level 1 sources and the parent folder for level 2 sources. A folder with fewer than three frames is skipped. For Sintel, the group is the scene prefix (for example `market`), and each item carries the forward flow files and the occlusion masks that match its frames. The Vimeo-90K triplet clips are read directly from the official test list, and up to 1,000 of them are sampled with the seed.

Table S6 reports the discovery per source. The listed and kept counts are file counts, and the items are the sequences or stills that survived the path rules. Sintel lists 7,466 files and keeps 3,256 of them, and the difference comes from files that the path rules remove, such as non-colour passes and masks. **[Preliminary]**

Table S6: Kaggle discovery per source. Files are counted after the directory walk and after the path rules. The REDS and Vimeo-90K septuplet listings reached their cap, so their file counts are lower bounds

Source	Files listed	Files kept	Directories listed	Items	Test items
DIV2K	900	900	5	900	100
Flickr2K	2,650	2,650	2	2,650	0
REDS	40,000	22,100	404	221	0
Vimeo-90K septuplets	40,005	40,005	5,725	5,715	0
Vimeo-90K triplets	3,000	3,000	1,000	1,000	1,000
MPI Sintel	7,466	3,256	205	46	10
Vid4	513	171	16	4	4

The Hugging Face sources are materialised next, into the data root, and each one follows its own access rule. Game stills are a random sample of the listed PNG files in the training and test folders, with the seed. TartanAir is read per environment. For each environment the code reads the first trajectory of the Easy difficulty zips, takes a contiguous run of frames starting at one quarter of the way along the trajectory, and stores RGB, depth, forward flow and a validity mask. The validity mask is white where the TartanAir mask is zero, which means a valid correspondence. The run stops early if a flow or mask file is missing. Each run is cut into sequences of 24 frames, which is the TartanAir sequence length in the configuration. GameIR keeps the 1440p and 720p renders and the depth of every clip with at least three high-resolution frames and a matching low-resolution frame. The code records a frame stride of ten rendered frames for these clips, but it does not subsample them, so the frames are whatever the mini tar holds. Vimeo-1080p writes each MP4 to disk, counts its frames, and keeps videos with at least eight frames.

Table S7 is the catalogue, which counts items, frames, exact-motion items and depth items for each source and role. The counts are taken before the quality filters, so they describe what was materialised. **[Preliminary]**

Table S7: Catalogue before filtering, by source, kind and role. Exact motion counts items with exact forward flow and depth counts items with depth. Each row is one source and role that exists in the run

Source	Role	Items	Frames	Exact motion	Depth
DIV2K	train	800	800	0	0
DIV2K	test	100	100	0	0
Flickr2K	train	2,650	2,650	0	0
Game stills	train	1,200	1,200	0	0
Game stills	test	250	250	0	0
GameIR	train	40	240	0	40
GameIR	test	30	180	0	30
REDS	train	221	22,100	0	0
Vimeo-90K septuplets	train	5,715	40,005	0	0
Vimeo-90K triplets	test	1,000	3,000	0	0
MPI Sintel	train	36	1,612	36	0
MPI Sintel	test	10	470	10	0
TartanAir	train	60	1,440	60	60
TartanAir	test	10	240	10	10
Vid4	test	4	171	0	0
Vimeo-1080p	train	134	34,147	0	0
Vimeo-1080p	test	40	9,223	0	0

Figure S7 shows the same catalogue as plots. The left panel stacks the items of each source by role. The Vimeo-90K septuplets are the largest source by items, and the Vimeo-90K triplets, which are test only, form the largest block of test items. The right panel gives the frame totals on a logarithmic axis, which spans about three decades. The Vimeo-90K and Vimeo-1080p sources carry the most frames, and the sources with exact motion and depth are a small share of the frames. **[Observation]**

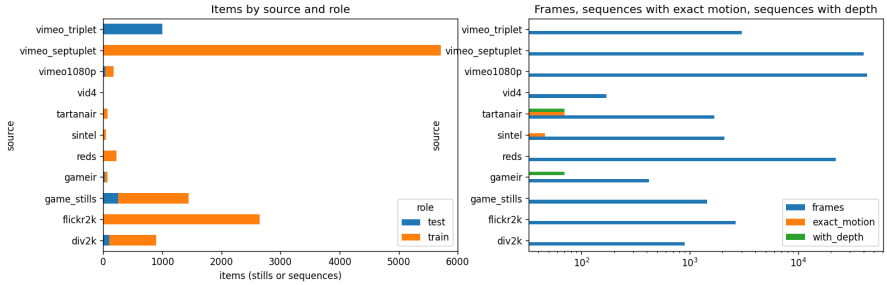


Fig. S7: Catalogue before filtering. Left: items per source, stacked by role. Right: frames on a logarithmic axis, and the sequences with exact motion and with depth. Both panels come from the notebook's catalogue cell

S3.3 Quality filtering

Quality is judged on statistics of the keyframes, which Section S3.4 defines. For each keyframe the code computes the grey mean and the grey standard deviation of the full-resolution frame, where grey is the mean of the red, green and blue values. It also computes the letterbox fraction λ. A row counts as dark when its mean grey value is below 8. The letterbox fraction is the number of dark rows at the top plus the number at the bottom, divided by the frame height. The sharpness is computed on the 256 by 256 thumbnail g. The code applies the four-neighbour Laplacian

$$L(x, y) = 4g(x, y) - g(x - 1, y) - g(x + 1, y) - g(x, y - 1) - g(x, y + 1), \tag{S36}$$

and takes the variance of L over the interior pixels as the sharpness. A flat image has a small variance of L, and a sharp image has a large one. The colourfulness measure of each keyframe uses the opponent channels $rg = R - G$ and $yb = \frac{1}{2}(R + G) - B$, and its value is

$$C = \sqrt{\sigma_{rg}^2 + \sigma_{yb}^2} + 0.3\sqrt{\mu_{rg}^2 + \mu_{yb}^2}, \tag{S37}$$

where σ and μ are the standard deviation and mean over the thumbnail. This is the colourfulness index of Hasler and Süsstrunk [26]. The colourfulness is recorded but no rule uses it.

An item is rejected if it has any of five reasons. The first is a decode error, which means a keyframe could not be read. The second is a side below 256 pixels, which means any keyframe with a smaller width or height than 256. The third is flatness, which means the median grey standard deviation over the item's keyframes is below 8. The fourth is letterboxing, which means the median letterbox fraction is above 0.35. The fifth is blur, which means the median sharpness is below 20 for stills and below 5 for sequences. Stills are judged on their one keyframe. Sequences are judged on the median of their keyframes, so one dark shot or one fade does not reject a whole video. A rejected item keeps its row in the manifest with split filtered and is left out of the duplicate search, so the decision can be audited. The manifest does not record which rule fired. The notebook printed these reasons, but the printed output was not saved, so this section gives no per-reason counts.

Table S8 gives the per-source totals. In this run 151 of 12,300 items were filtered. Filtering removed 117 Vimeo-90K septuplet items and 28 game-still items, and the remaining filtered items came from the triplet and Vimeo-1080p sources. The filtered items are a small share of all items. **[Preliminary]**

Figure S8 shows the distributions that the rules act on. The grey standard deviation of the septuplet keyframes forms a bell shape centred well above the threshold of 8, so the flatness rule touches only the lower tail. The sharpness panel is on a logarithmic axis, and its two dashed lines mark the still and sequence thresholds, which are both drawn for every source. The sequences sit well above the sequence threshold, the stills sit higher still, and the sequence threshold cuts only the far lower tail. Several keyframes have a sharpness of zero, which the figure shows at the left edge. The letterbox panel is concentrated at zero, with a smaller group near 0.25 and few keyframes beyond the threshold of 0.35. **[Observation]**

Table S8: Split per source. Drop means removed as a near copy of test content or as a member of a test group, and filtered means removed by a quality rule

Source	Train	Val	Test	Drop	Filtered
DIV2K	755	43	100	2	0
Flickr2K	2,497	153	0	0	0
Game stills	502	41	243	636	28
GameIR	40	0	30	0	0
REDS	209	12	0	0	0
MPI Sintel	36	0	10	0	0
TartanAir	60	0	10	0	0
Vid4	0	0	4	0	0
Vimeo-1080p	120	3	38	10	3
Vimeo-90K septuplets	4,655	835	0	108	117
Vimeo-90K triplets	0	0	997	0	3

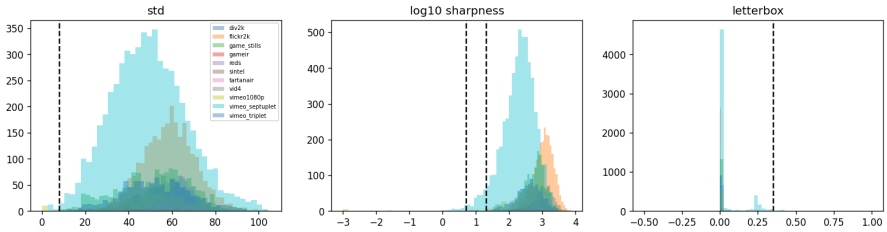


Fig. S8: Keyframe statistics that the filters use, by source: grey standard deviation (left), log10 sharpness (centre) and letterbox fraction (right). Dashed lines mark the thresholds. The sharpness panel draws both the still and the sequence thresholds for every source

Figure S9 shows the first sixteen rejected keyframes. Most are very dark. The flatness and letterbox rules reject dark frames, and the blurry examples are soft frames with little texture. The sharpness rule therefore rejects low-texture content as well as defocus, and the label blurry means low Laplacian variance. The figure is a selection, not a sample. The code takes the first two rejected keyframes of each source and reason and then keeps the first sixteen rows, so the grid shows only the Vimeo-90K septuplets, the Vimeo-90K triplets and the game stills. The notebook text describes examples from every source and reason, but the selection does not do that. **[Observation]**

S3.4 Keyframe selection and statistics

Each item is represented by keyframes, chosen from its frame list. A still and a sequence of one frame give one keyframe, which is the frame itself. A sequence of two to 23 frames gives its middle frame. A sequence of 24 or more frames gives three keyframes, at 10, 50 and 90 percent of its length, because the first and last frames of a long video are often fades or black. The triplets and septuplets are shorter than

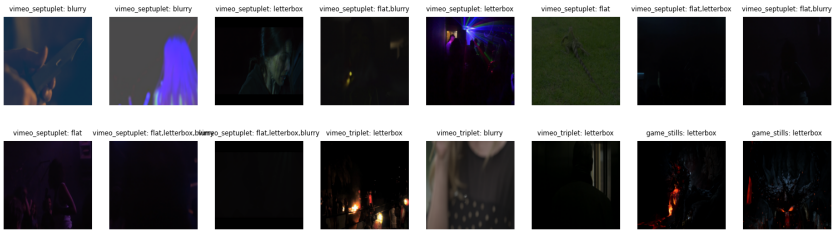


Fig. S9: The first sixteen rejected keyframes in keyframe order, with the reasons of each item. The grid does not cover every source or reason, because the code keeps the first sixteen rows of its selection

24 frames, and so they contribute one keyframe each, while the long REDS, Vimeo-1080p and TartanAir sequences contribute three each. The total is 13,322 keyframes for 12,300 items.

Each keyframe is decoded in a pool of spawned worker processes. The workers are spawned, not forked, so they do not inherit the CUDA context or the thread locks of the notebook kernel, and a dead worker raises an error instead of hanging. Each keyframe gives a 256 by 256 thumbnail, resized with bilinear filtering. The resize does not keep the aspect ratio, so a 16:9 frame is squashed horizontally and a 4:3 frame is squashed less. This is a property of the method and its effect on copy recall is not measured. The thumbnail is the input to the hashes, the embedding and the local features, and the full-resolution statistics above are computed before the resize. Table S9 gives the mean statistics per source. The sharpness mean is largest for Vid4 and smallest for the Vimeo-90K septuplets. Sharpness is measured on the 256 pixel thumbnail, so the value also depends on how much each source was reduced to that size, and it is not a measure of image quality across sources. **[Observation]**

Table S9: Keyframe counts and mean statistics per source. Width and height are in pixels, grey standard deviation is on the full-resolution frame, sharpness is the Laplacian variance of the 256 by 256 thumbnail, and letterbox is the largest fraction in the source

Source	Keyframes	Width	Height	Grey std	Sharpness	Letterbox max
DIV2K	900	1,972	1,437	57.91	1,304	0.2961
Flickr2K	2,650	1,934	1,450	58.6	1,333	0.2824
Game stills	1,450	1,920	1,080	50.99	677.3	0.9407
GameIR	70	1,920	1,440	56.11	559.3	0
REDS	663	1,280	720	63.25	1,138	0
MPI Sintel	130	1,024	436	45.95	500.7	0.01835
TartanAir	210	640	480	51.43	771.2	0.5729
Vid4	12	716	528	52.76	1,889	0.003472
Vimeo-1080p	522	1,920	1,080	53.59	623.3	1
Vimeo-90K septuplets	5,715	448	256	49.05	322.5	1
Vimeo-90K triplets	1,000	448	256	52.59	569.7	0.6133

S3.5 Copy detection

A copy is a keyframe that shows the same picture as another keyframe, after a change of scale, compression, colour, crop or mirror. Copy detection must separate copies from similar pictures. Two photographs of the same kind of scene, two frames of one game or two skylines can be close in a global description while showing different pictures. The pipeline therefore uses a two-stage rule. The first stage proposes candidate pairs with cheap global descriptions. The second stage confirms a pair with a geometric check, and two further tests accept pairs whose global scores lie beyond every hard negative.

Perceptual hashes.

A perceptual hash maps an image to a short bit string that changes little under compression and scaling [27]. The variant below, a DCT hash with a difference hash (dHash) alongside it, is this code's choice. The pHash of a keyframe takes its 32 by 32 grey image G and computes the two-dimensional discrete cosine transform

$$C = D\,G\,D^{\mathsf{T}}, \tag{S38}$$

where D is the 32 by 32 orthonormal DCT-II matrix. The 8 by 8 block of lowest frequencies is kept, and the direct-current term is left out of the median. Each of the 64 bits is one if its coefficient is above that median, and zero otherwise. The dHash takes a 9 by 8 grey image and sets each bit to one when the pixel to the right is brighter than the pixel on the left. The distance between two hashes is the Hamming distance, which counts the bits that differ. The code uses 64-bit hashes, so the distance ranges from 0 to 64. Hashes catch near-exact copies, including rescaled, recompressed and recoloured frames. They do not catch crops or mirror images, and the later stages cover those.

Global embeddings and cosine similarity.

The embedding uses DINOv2-small, a self-supervised vision transformer [28], and the code's descriptor for each image has 384 dimensions, as the embedding log of the data pipeline records. The code takes the pooled output of the model for each thumbnail after bilinear resizing to 224 by 224, with the usual channel means and standard deviations, and it normalises the output to unit length. For two unit vectors u and v, the cosine similarity is $u \cdot v$, and for general vectors it is

$$\cos(u, v) = \frac{u \cdot v}{\|u\|\,\|v\|}. \tag{S39}$$

A cosine near one means that the two descriptors point the same way. Two different pictures with similar content can also have a high cosine, which is why the embedding only proposes candidates. The model runs in half precision on both GPUs, in batches of 256 thumbnails.

Exhaustive neighbour search.

For every searched keyframe, the code finds its k neighbours with the largest dot products, where $k = 16$. A neighbour must belong to a different leakage group, so a keyframe never matches its own sequence. The search computes the full similarity matrix in chunks of 2,048 rows on the GPU and takes the top k of each row. Hash candidates are found in the same spirit. Each hash bit becomes $+1$ or -1, and two hashes have a Hamming distance h exactly when the dot product of their ± 1 vectors is $64 - 2h$. The code therefore finds all pairs whose dot product passes $64 - 2h$, in chunks of 4,096 rows, and it looks only at textured keyframes, which are those with a grey standard deviation of at least 15.

Local features and geometric verification.

The second stage looks for shared structure. The code runs the scale-invariant feature transform (SIFT) on each thumbnail and on its mirror image, keeps up to 400 keypoints per image, and stores their 128-dimensional descriptors [29]. For two keyframes A and B, a keypoint of A is matched to its nearest keypoint in B when the nearest descriptor distance is less than 0.8 times the second nearest distance. This is the ratio test of Lowe, which removes matches that are about as close to a second candidate. At least six matches are needed. The matches are then fitted with one affine map, which sends a point p of A to a point of B by

$$p' = Ap + t, \qquad A \in \mathbb{R}^{2\times 2},\ t \in \mathbb{R}^2, \tag{S40}$$

where A carries rotation, scale and shear and t carries the translation. The fit uses random sample consensus (RANSAC), which repeatedly fits the map to a random subset of matches and keeps the map with the most matches within 4.0 pixels of the prediction, for up to 1,000 iterations [30]. The map is rejected when its determinant is not positive or when its singular values fall outside the range 0.2 to 5. Those values would mean a collapsed or extreme scale. At least six inliers must remain.

The verification score is the grid coverage. The code divides the first image into an 8 by 8 grid of cells and counts the cells that contain at least one inlier. The score therefore ranges from 0 to 64. This rule is what makes the verification robust. A logo, a heads-up display or a caption can produce matches in one corner, but those matches cover only the cells of that corner. Two different frames then score low even when they share a small object. The code scores each pair as the larger of the scores against the second image and against its mirror, so a mirrored copy is also found.

The edge rule.

A pair becomes a duplicate edge when one of three tests passes. The exact-hash test passes when the pHash distance is at most h_{exact}, the dHash distance is at most $2h_{\text{exact}} + 4$, and both keyframes are textured. The strong-embedding test passes when the cosine is at least t_{strong}. The verification test passes when the grid score is at least t_{cells}. The labels are assigned in that order and a later test overwrites an earlier one, so a pair that passes several tests is labelled verified first. The thresholds come from the calibration of Section S3.6. A candidate pair is proposed when its cosine is at

least t_{ret}, or when its pHash distance is at most h_{cand} on textured keyframes. If the hash candidates exceed 20 per keyframe, the hash radius is tightened. In this run the candidate count was far below that limit, so the radius was not changed.

The search proposed 27,044 unique candidate pairs. The embedding stage proposed 45,213 pairs before merging and the hash stage proposed 849 pairs, so the embedding stage supplied almost all candidates. The verification confirmed 3,346 edges before the audit, of which 3,304 were verified, 37 were strong-embedding edges and 5 were exact-hash edges. [Preliminary]

Figure S10 shows ten edges, with the cross-source ones first. The top row shows the first keyframe of each pair and the bottom row the second, and the captions give the source, the rule, the cosine and the grid score. Every pair on the figure looks like one picture, so the shown edges are the kind that a reader would accept as copies. The Vimeo-90K septuplet and Vimeo-1080p pairs appear to show the same footage. The game-still pairs are within one source, and the drops below show that many training game stills are copies of test frames. The strong-embedding pairs have low grid scores although the pictures look identical. The grid score counts only the cells that SIFT matches cover, so a frame with few matches can score low, but this was not checked for these pairs. These readings come from the figure, and the strong-embedding edges were not checked one by one. [Observation]



Fig. S10: Ten duplicate edges, cross-source pairs first. Each column shows the two keyframes of one edge. The title above the top row gives the source of the first keyframe, the rule, the cosine and the grid score, and the title below the bottom row gives the source of the second keyframe

S3.6 Calibration on synthetic copies

The thresholds are calibrated on copies that the code makes itself, because the pipeline does not know which real pairs are copies. Positives are copies of random keyframes, with 1,500 copies in this run. Half are light copies and half are heavy copies. A light copy is rescaled by a factor between 0.4 and 1.0 and returned to 256 pixels, with a brightness and contrast change and a shift in half of the cases, and a Gaussian blur with a radius between 0.5 and 1.5 in 30 percent of the cases. Every copy is recompressed as a JPEG with a quality between 35 and 92. A heavy copy also takes a random crop covering 60 to 95 percent of the area, which is resized to 256 pixels, and then receives the same colour, blur and recompression steps as a light copy. It is mirrored with probability one half, rotated by up to four degrees with probability

0.3, and given the text “SAMPLE 1080p” with probability 0.3. Light copies are what a perceptual hash is meant to survive. Heavy copies add geometric changes that only the verification can handle.

Negatives come from two sets. Random negatives are pairs of keyframes drawn from different groups. Hard negatives are the nearest neighbour in another group for each keyframe of three sources whose groups are known to be distinct: DIV2K, Sintel and TartanAir. Distinct means that two different groups never show the same content, which is the premise of the hard set. The random set gives 2,930 pairs and the hard set gives 998 unique pairs in this run. Random negatives are sampled from the whole pool, so a random pair can be a genuine copy, which is why the code takes the 99.9th percentile of the random set rather than its maximum.

Each pair is scored with the four measures of Section S3.5, which are the cosine, the pHash Hamming distance, the dHash distance and the grid score. The recall of a rule is the share of positives it flags. The false positive count is the number of negatives it flags. The precision is the number of flagged positives divided by all flagged pairs. For the thresholds, let Q_q denote the q-quantile of a set of values, and clip to a range with clip. The code sets

$$t_{\text{ret}} = \text{clip}\big(Q_{0.02}(\cos_{\text{copy}}),\, 0.5,\, 0.9\big), \tag{S41}$$

$$t_{\text{strong}} = \text{clip}\big(N_{\cos} + 0.03,\, 0.92,\, 0.995\big), \tag{S42}$$

$$t_{\text{cells}} = \text{clip}\big(N_{\text{cells}} + 2,\, 10,\, 40\big), \tag{S43}$$

$$h_{\text{exact}} = \text{clip}\big(M_{\text{ham}} - 4,\, 0,\, 6\big), \qquad h_{\text{cand}} = \text{clip}\big(Q_{0.95}(\text{ham}_{\text{light}}),\, h_{\text{exact}},\, 12\big), \tag{S44}$$

where N is the most similar negative score, which is the larger of the 99.9th percentile of the random negatives and the most similar hard negative, M_{ham} is the smaller of the 0.1th percentile of the random negative Hamming distances and the smallest hard negative distance, and $\cos_{\text{copy}}$ and $\text{ham}_{\text{light}}$ are the cosine of all copies and the Hamming distances of the light copies. The first threshold retrieves 98 percent of copies, the second accepts pairs above every negative without verification, the third lies above the most similar negative’s grid score, the fourth lies four bits below the closest negative’s pHash distance, and the fifth covers 95 percent of light copies without going below the fourth. The threshold t_{ret} only proposes candidates, and an edge needs one of the three tests of the edge rule.

Table S10 gives the score distribution of each group, and Table S11 gives the thresholds and the recalls. The 99.9th percentile of the random negatives is set by the few largest values, so one unusual random pair can move t_{cells} or t_{strong}. The recall figures are over the calibration set and the precision is over its negatives, so they are not measured on real copies. The real share of copies in the data is unknown. **[Preliminary]**

The combined rule flags light copies almost completely, and its recall on heavy copies is lower, which is expected because a crop removes part of the picture and a rotation changes the geometry that the affine fit must absorb. The rule has 1 false positive among the negatives. That negative is the random pair with the largest grid score, which is 33 cells at a cosine of 0.854. The thumbnails are not saved, so the pair was not inspected, and it may be a genuine copy that random sampling drew. **[Observation]**

Table S10: Scores of the calibration pairs by group. Cosine is the DINOv2 cosine, Hamming the pHash distance and cells the grid score of the verification

Group	Pairs	Cosine mean	Cosine max	Hamming mean	Hamming min	Cells mean
Copies, light	750	0.8778	0.9921	0.4933	0	33.49
Copies, heavy	750	0.8419	0.9777	23.35	2	30.59
Hard negatives	998	0.5117	0.9348	30.83	18	0.05411
Random negatives	2,930	0.06324	0.854	31.35	8	0.02867

Table S11: Calibrated thresholds and the recalls and precision they give on the calibration set. Recall is the share of copies flagged, and the false positive count is over all negatives

Quantity	Meaning	Value
t_{ret}	candidate cosine	0.6075
Recall at t_{ret}	share of copies retrieved	0.980
t_{strong}	strong cosine	0.9648
Recall at t_{strong}	share of copies accepted without verification	0.038
t_{cells}	verification threshold in cells	10
Recall of verification	share of copies with a grid score of at least t_{cells}	0.949
h_{exact}	exact-hash distance	6
h_{cand}	candidate hash distance	6
Recall of light copies at h_{exact}	share of light copies	0.999
Combined recall	share of all copies flagged by the edge rule	0.967
Combined recall, light	share of light copies flagged	0.995
Combined recall, heavy	share of heavy copies flagged	0.940
False positives	negatives flagged by the edge rule	1
Precision	flagged copies over all flagged pairs	0.999

Figure S11 shows the four diagnostic plots. The cosine histogram separates the copies, which lie at the high end, from the random negatives near zero, and the hard negatives sit between them. The two thresholds lie at the edges of the copy distribution and above the hard negatives. The grid-score panel uses a logarithmic count axis. Copies spread over tens of cells, most negatives score in the low single digits, and the threshold lies above the random tail. The pHash panel shows light copies at distances near zero, heavy copies at intermediate distances, and the negatives far above the exact-hash threshold, which is why light copies fall under it. The receiver operating characteristic of the cosine alone reaches only part of the copies at the false positive rates where the rule keeps almost all of them. This is why the pipeline does not use the cosine alone. **[Observation]**

S3.7 Leakage groups and the split

The leakage group is the unit that the split must keep together. Its definition depends on the source. Each still is its own group, so a still never shares a group with another still. A video or rendered sequence takes its folder name as the group for level 1 sources (REDS and Vid4), and the parent folder for level 2 sources, which are the two Vimeo-90K sets. Vimeo-1080p uses each video as its own group. Sintel uses the scene prefix, TartanAir uses the environment, and GameIR uses the town. The group is written as

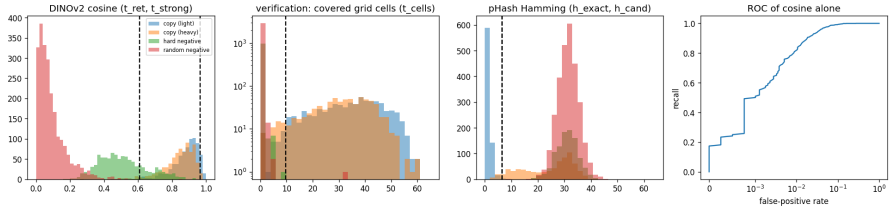


Fig. S11: Calibration diagnostics. From left: cosine histograms of the four groups with t_{ret} and t_{strong} dashed, grid scores on a log count axis with t_{cells}, pHash distances with h_{exact} and h_{cand}, and the receiver operating characteristic of the cosine alone

the source name followed by the group name, so groups from different sources never collide.

The split works on the duplicate edges and the groups together, in four steps. The first step is provenance. A group that holds an active test item is test-only, and every other item of that group is dropped with reason test group. The second step is content. Starting from the test items, the code walks the duplicate edges only, and never the shared groups, so every non-test item reached through an edge is dropped with reason test duplicate. A walk through the edges can pass through several items, so a chain of copies is also dropped. Walking only through edges stops one copied clip from taking its whole folder with it. The third step forms the remaining components. Two remaining items are in the same component when they share a group or when they are joined by an edge, and each component is found with a union-find structure. The fourth step assigns each component to validation or to training. The component goes to validation when a hash of the seed and the component's root item identifier, taken modulo 10,000, falls below 10,000× the validation fraction. The validation fraction is 0.05, so about one component in twenty is validation. The assignment is deterministic and depends only on the seed and the root.

Table S12 gives the split report and Table S8 gives the split per source. In this run 756 items were dropped as test duplicates and 0 as test groups. The zero count of test-group drops follows from the group definitions, because no training item of these sources shares a group with a test item. The largest duplicate cluster holds 50 items, and the remaining items form 3,956 components, of which the largest holds 2,936 items. **[Preliminary]**

Figure S12 shows the split per source as bars. The Vimeo-90K septuplets are the largest block of training and validation data, and the Vimeo-90K triplets are test only. The game stills account for most drops in this run, and every one of their drops is a training-role item linked to test frames. Table S8 gives that count in full, and the fraction of the game-still training items that were dropped is 0.53. **[Preliminary]** The septuplet drops are the second group. The cross-source edges link the septuplets mainly to the triplet test set and to Vimeo-1080p videos, and the septuplet drops are reached from test items through those edges. This is the leakage that the rule removes from the septuplet training data. **[Observation]**

Table S12: Split report of the saved run. Active items are the items not filtered, and the split counts are the final splits after the audit

Quantity	Value
Items	12,300
Active items (not filtered)	12,149
Duplicate edges, after audit repairs	3,370
Largest duplicate cluster	50
Train and validation components	3,956
Largest train and validation component	2,936
Dropped, test group	0
Dropped, test duplicate	756
Train	8,874
Validation	1,087
Test	1,432
Dropped	756
Filtered	151
Audit rounds	2
Cross-split violations after the audit	0

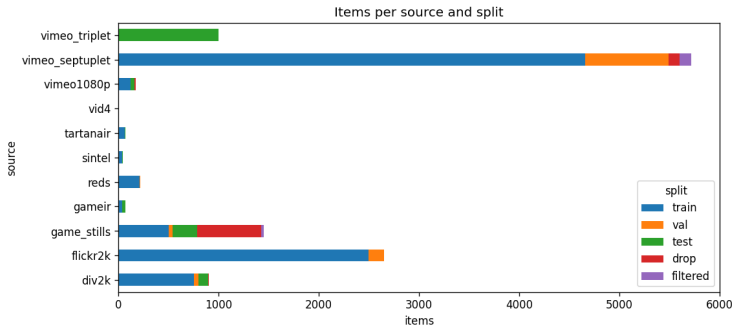


Fig. S12: Items per source and split, after the audit. Drop is a near copy of test content or a member of a test group, and filtered is a quality rejection

S3.8 Leakage audit

The split is checked after it is made. The audit compares each validation keyframe with the training keyframes, and each test keyframe with the training and validation keyframes. For each query it takes the 5 nearest references by cosine and the nearest reference by pHash. Every such pair is scored with the same rule as the search. Any pair that passes the rule is a violation, which is a duplicate across two splits. Each violation is added as an edge, and the split is made again from the start. The loop repeats until a round finds no violation, up to 3 rounds. The notebook asserts that no violation remains.

The first round of this run found 24 cross-split duplicates, and the verified edges rose by the same number from the search to the final split. The second round found none, so the audit stopped after 2 rounds with 0 violations left. The violations of the

first round were not saved, so their rule and cosines are not recorded in the outputs, and the verified label is inferred from the change in rule counts. [Preliminary]

Table S13 gives how close the validation and test keyframes come to their nearest references. The worst values are the largest over all keyframes and the median is the typical value. The cosine of the worst validation keyframe is 0.9614, and that of the worst test keyframe is 0.957. Both lie below the strong-embedding threshold of 0.9648. The largest grid score is 9 cells, below the threshold of 10, and the smallest pHash distance is 0, which is inside the exact-hash range, so some test keyframe has an identical hash to a training keyframe. Because the audit found no violation, the edge rule must reject that pair, which can only happen if the pair is not textured or its dHash distance exceeds the bound of the edge rule. The pair itself is not in the outputs, so this is an inference from the rule and not an observation. [Preliminary]

Table S13: Audit of the final split. Each row is a comparison of one split with its references. Worst is the largest value over the keyframes for cosine and grid score, and the smallest for the pHash distance. Median is the typical value over the keyframes

Split and references	Metric	Worst	Median	Violations
Validation against training	Maximum cosine	0.9614	0.5757	
	Maximum grid score	9	0	0
	Minimum pHash distance	8	16	
Test against training and validation	Maximum cosine	0.957	0.5745	
	Maximum grid score	9	0	0
	Minimum pHash distance	0	16	

Figure S13 shows the audit as histograms. The cosine panel places most test keyframes at a nearest-reference cosine in the middle of the range, with a secondary group at high cosines, and no keyframe reaches the strong-embedding line. The grid-score panel is on a logarithmic count axis, and most keyframes score in the low single digits, far below the threshold line. The pHash panel places most nearest distances in the middle teens, above the exact-hash line. These histograms show that the audit's nearest references are similar pictures and not copies. The secondary group at high cosines is the population that the audit must not confuse with copies, and the grid score is what separates it. [Observation]

The audit is a check and not a proof. It compares each keyframe with its five nearest cosine neighbours and with its nearest hash, so a copy that is neither among the five cosine neighbours nor the nearest hash would pass unseen. The candidate search used the same rule with a wider reach, which lowers that risk, but the audit has no independent test of its own coverage. The notebook text does not discuss this limit.

S3.9 Final split, dataset card and stage times

The manifest is written with one row per item, with its source, kind, role, frame URIs, motion and depth URIs, leakage group, component, split and drop reason. The

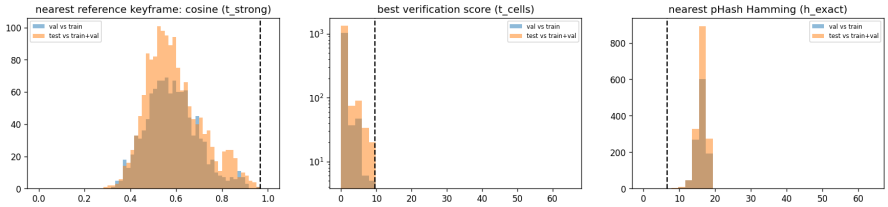


Fig. S13: Audit histograms for validation against training and test against training and validation. From left: the nearest-reference cosine with the strong-embedding line, the best grid score with the verification line, and the nearest pHash distance with the exact-hash line

dataset card repeats the source, host, licence string, content type, item count and whether any item has exact motion or depth. Table S14 gives the card with the item counts from the saved output and the licence strings of Table S5. The card records exact motion for Sintel and TartanAir and depth for TartanAir and GameIR, which matches the catalogue of Table S7.

Table S14: Dataset card. Items are the materialised and discovered items before the split. Exact motion and depth are yes where at least one item of the source carries them, as recorded in the saved card

Source	Host	Items	Exact motion, depth	Content
DIV2K	Kaggle	900	no, no	photo
Flickr2K	Kaggle	2,650	no, no	photo
Game stills	HF	1,450	no, no	game
GameIR	HF	70	no, yes	game
REDS	Kaggle	221	no, no	video
MPI Sintel	Kaggle	46	yes, no	rendered
TartanAir	HF	70	yes, yes	rendered
Vid4	Kaggle	4	no, no	video
Vimeo-1080p	HF	174	no, no	video
Vimeo-90K septuplets	Kaggle	5,715	no, no	video
Vimeo-90K triplets	Kaggle	1,000	no, no	video

Table S4 gives the time of each stage. The Hugging Face transfer and the keyframe decoding take most of the run, and the copy detection, calibration and audit together take a few minutes. The run used 2 Tesla T4 GPUs with 14.56 GB of memory each, 4 CPU cores and 31.3 GB of RAM, with Python 3.10.12, PyTorch 2.0.0 and CUDA 11.8. The total run time was 0.63 hours. **[Preliminary]**

Figure S14 plots the stage times on a logarithmic axis. The bars differ by more than three decades, from the manifest write to the Hugging Face transfer, and the figure makes clear that the data transfer and decoding dominate the time, while the search and the audit are small. **[Observation]**

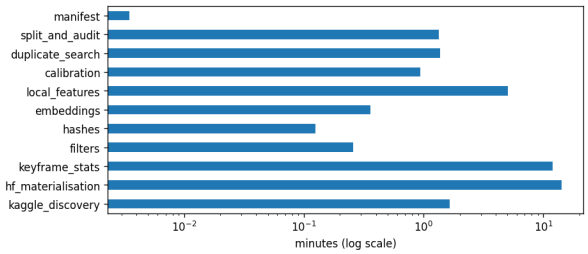


Fig. S14: Stage times of the saved run on a logarithmic axis, in minutes

S3.10 Synthetic game-like sequences

Only Sintel and TartanAir carry exact labels among the real sources, and the low-resolution inputs of real sequences are built in one of two ways described below. The synthetic renderer supplies the cases that real data does not: exact motion, exact depth, exact disocclusions, thin geometry and particles without motion vectors, HUD overlays, menus and scene cuts. The renderer is a PyTorch module, `nss_synth.py`, that runs on the GPU. It takes a batch of cached still textures and returns a batch of frames with every label derived from the scene, so the labels are exact by construction. Its outputs go to the temporal super-resolution (TSR) and frame-generation (FG) networks described in Section 4 and Section 5. The TSR and FG notebooks give the textures and draw from the renderer in fixed proportions, which Section S3.10.7 lists.

The renderer relies on definitions from Section S2, in particular the sampling and motion concepts of Section S2.2 and Section S2.3. The rest of this section describes the code in order.

S3.10.1 Layered scene

A scene is a two-and-a-half-dimensional stack of two layers over a background. The background layer is a texture, which the notebooks take as 512 by 512 still crops. The foreground layer is another texture from the same batch, obtained by rolling the batch by one position, so that the foreground of each sample is the background of a different sample. With probability 0.25 the foreground is a flat colour instead, drawn with each channel uniform on $[0, 1]$ and multiplied by a per-pixel factor clipped to the range $[0.85, 1]$. The foreground carries an alpha mask a that decides where it covers the background. The mask has three parts. The first is a superellipse blob, whose boundary is the set where

$$\left|\frac{u'}{r_x}\right|^p + \left|\frac{v'}{r_y}\right|^p = 1, \qquad p \in [1.5, 4],\ r_x, r_y \in [0.12, 0.42], \tag{S45}$$

with (u', v') the screen position relative to the blob centre after a random rotation. The second part is up to four thin wires, each present with probability 0.6. A wire is a band between 0.6 and 2 texels wide, and it covers the points within half its width of a random line. The third part is a fence with probability 0.35, made of periodic bars with a period of 6 to 20 texels, a bar width of 1 to 2 texels and a random orientation. The

fence fills a square region of side 1.2 around the blob. The union of the blob, the wires and the fence is the mask, and the foreground is present in a sample with probability 0.85. An absent foreground has a zero mask. The thin parts are deliberately between 0.6 and 2 texels wide, which is below what the low render resolution can resolve, so the renderer aliases them as a game does.

For TSR samples, a world-space text tile is stamped into the background of a sample with probability 0.3, at a random integer position of the still. The notebooks render a text bank once with the Python imaging library, which holds 64 tiles of 128 by 24 pixels for TSR and 256 tiles of 160 by 28 pixels for FG. A tile is a word or phrase in a font, a translucent subtitle plate, a bar or a crosshair. The fonts are the font set that the runtime finds on the system, so the tile images depend on the machine. The FG batches do not stamp world text, because the FG renderer creates its scenes without a text bank.

S3.10.2 Motion and speed classes

Each layer moves along a continuous affine path. Screen positions are normalised to $[-1, 1]$, texture positions are normalised to $[-1, 1]$, and the layer maps screen to texture by

$$M(t) = \begin{pmatrix} s(t)\cos\theta(t) & -s(t)\sin\theta(t) & o_x(t) \\ s(t)\sin\theta(t) & s(t)\cos\theta(t) & o_y(t) \end{pmatrix}, \tag{S46}$$

where $s(t) = s_0 e^{\zeta t}$ is the scale, $\theta(t) = \theta_0 + \omega t$ is the rotation, and $o(t) = o_0 + ut + \frac{1}{2}at^2 - 2u\,\max(0, t - t_{\text{flip}})f$ is the offset, with $f \in \{0, 1\}$ the reversal flag. The zoom rate ζ and the rotation rate ω are drawn from normal distributions with standard deviations 0.006 and 0.008, and they are zero for static layers. The acceleration a is normal with a standard deviation of 0.1 times the texture speed. The reversal flag makes the offset change direction at t_{flip}, so the layer pans back along its path. In the FG batches the camera layer reverses at $t = 1$ with the reversal probability of Section S3.10.7.

Each layer draws a speed class from a mixture, and the class sets the screen speed in output pixels per frame. Table S15 lists the classes. The camera layer and the object layer draw their classes independently. The texture speed is the screen speed scaled by the texels per output pixel, and the path is limited so that the view stays inside the texture for the whole clip. The room available to a path is $\max(0,\, 1 - 1.42 s_0 - 0.02)$, and the displacement of the path over the clip is limited to that room, which keeps the view inside the texture. A path that would leave the texture is slowed to fit. The object layer is placed so that a random texel of its texture lands at a random screen position at $t = 0$, and its rotation range is twice that of the camera.

The speed classes are the ones the TSR evaluation reports, so their boundaries are part of that protocol. The renderer draws a speed uniformly inside the class range. A static camera or object has no rotation and zoom, so its only motion is its offset. The batch records the camera's class, and the TSR evaluation reports each class separately.

S3.10.3 Particles and the reactive mask

Each sample has six particles. Each particle is on with probability 0.5, has a centre in $[-1, 1]^2$, moves with a velocity drawn in $[-0.06, 0.06]^2$ per unit time, and has a

Table S15: Speed classes of the layer trajectories. Speeds are in output pixels per frame, and the probabilities are the mixture weights of `MOTION_PROFILE` in the renderer

Class	Speed range (px per frame)	Probability	Rotation and zoom
Static	0	0.15	none
Slow	0.3 to 3	0.45	drawn
Medium	3 to 10	0.25	drawn
Fast	10 to 28	0.15	drawn

radius of $2 + 10U$ pixels for a uniform U on $[0, 1]$. Its colour is drawn in $[0.6, 1]^3$ and its amplitude in $[0.2, 0.7]$. The weight of a particle at a pixel x is a Gaussian in the distance to its centre,

$$g_k(x,t) = \mathbb{1}_{\text{on}}\, A_k \exp\left(-\frac{\|x - c_k(t)\|^2}{2r_k^2}\right), \tag{S47}$$

where $c_k(t)$ is the moving centre, A_k the amplitude and r_k the radius. The particles are blended into the colour one after the other, each by $col \leftarrow col(1-g_k)+p_k g_k$, where p_k is the particle colour. The reactive mask is the combined coverage $1 - \prod_k (1 - g_k)$, which is one where particles are dense and zero where none is present. The particles write no motion vectors, as in real engines. Their flow is taken from the layers only, so a particle moves the colour without moving the labels, and the reactive mask is meant to mark the pixels where the history should be trusted less.

S3.10.4 Sampling, jitter and exact motion

The colour of a layer at a screen position is sampled from its texture with bilinear interpolation. The renderer does not build a mip pyramid for the texture, so a sample at a low resolution reads the full-resolution texture at one point and fine detail aliases. A low-resolution frame is rendered at $h \times h$ pixels with one sample per pixel. The sample sits at a sub-pixel position given by a Halton sequence [2] in bases 2 and 3, with offsets in $[-0.5, 0.5)$ of a low-resolution pixel. The sequence has a period of 16 frames, and each sample starts at a random phase in that period, so neighbouring frames of one sample are offset differently. The high-resolution target is rendered at $P \times P$ pixels with four samples per pixel. The four offsets lie on a rotated grid, $(-0.375, -0.125)$, $(0.125, -0.375)$, $(0.375, 0.125)$ and $(-0.125, 0.375)$ in pixels, and the target is their mean. The four-sample target is an anti-aliased reference of the same scene, and the single-sample input is what a game produces at a low render resolution. The four-sample rule is named 4x SSAA in the figures, which means four samples per pixel and not a four-times scale.

Depth is stored per layer and per pixel, and each pixel takes the depth of the foreground if its alpha exceeds one half, and otherwise the depth of the background. The background depth is $d_{\text{bg}}(1 + 0.3y_{\text{tex}})$ for a per-sample constant d_{bg} between 0.6 and 1.0, and the foreground depth is a constant d_{fg} between 0.15 and 0.5. The network receives inverse depth, $1/(1+d/\tilde{d})$, where $\tilde{d}$ is the median depth of the sample.

Exposure is a per-sample gain 2^V with V uniform on $[-0.8, 0.8]$, applied in linear light after the sRGB decoding and before the encoding. The composited colour is

$$c = a\, c_{\mathrm{fg}} + (1 - a)\, c_{\mathrm{bg}}, \tag{S48}$$

where c_{fg} and c_{bg} are the sampled colours of the two layers and a is the foreground alpha at the same point. The background is sampled with reflection padding at its edges and the foreground with zero padding, so the background is continuous at the texture edge and the foreground vanishes there.

Exact motion is computed from the layer matrices, not from the pixels. For every pixel of a frame at time t, the renderer finds the surface point of the visible layer, maps it back to time $t - 1$ and returns the displacement in pixels. The visibility test asks whether the point at $t - 1$ is covered by the foreground. A pixel is valid when its point stays inside the view and is visible at $t - 1$, and invalid when it was covered or left the view. The motion and the validity are both in the frame's own grid, and the renderer uses the jittered grid for the low-resolution motion, so the motion refers to the sample position that the network sees. The validity map marks the disoccluded pixels, which have no valid history.

S3.10.5 Temporal super-resolution batches

The TSR batch function `sr_batch` renders T frames of n samples at scale s, with targets of $P \times P$ pixels, and returns the low-resolution inputs, the targets, the motion, the validity, the depth, the reactive mask, the jitter offsets and the exposure. The notebook uses $s = 3$, $T = 6$ frames and $P = 192$ pixels. The batch records four flags, which are the availability of depth, the quality of the motion labels, the quality of the other labels and the kind of input. The motion quality is one for exact motion, one half for estimated motion and zero for none. The kind is one for a render input and zero for ordinary video. Synthetic renders have quality one and kind one, and a later step can change them.

Some samples are turned into ordinary video. `video_degrade` selects a share p of the samples and replaces their low-resolution input by `video_lr`. That function blurs the high-resolution frame with a five-tap Gaussian whose standard deviation is up to 0.6 high-resolution pixels, and downscales it with one of three filters chosen per sample (an anti-aliased bicubic filter [31], an anti-aliased bilinear filter, or an area average), adds Gaussian noise with a standard deviation up to 2/255, and quantises to eight bits. Without the blur, only the bicubic filter is used, which is the standard bicubic degradation of video benchmarks. For a video sample the depth and reactive inputs are zeroed, the jitter is zeroed, and the depth flag is cleared. The motion vectors are recomputed as the block average of the full-resolution motion divided by the scale, and a sample is valid only where every pixel of its block is valid. Of the video samples, a share p_{mv} keeps its motion with quality at most one half, which stands for motion a decoder could supply. The rest have motion quality zero, so the network must estimate their motion. The docstring of the function says that the labels are re-sampled at pixel centres, but the code takes the block average. The docstring also gives the blur in

output pixels, while the code applies it to the high-resolution frame, so the docstring is out of date.

The other source of TSR batches is cached real sequences. `sr_batch_from_frames` reads cached frames, flips each sample horizontally and vertically with probability one half each, negates the flow components that a flip reverses, applies an exposure gain, and samples the low-resolution input as one jittered read of the full-resolution frame. Its motion comes from the cache: the exact motion of Sintel and TartanAir, which is played backwards so that its forward flow points from the current frame to the previous one, or the RAFT teacher motion of every other sequence, with the forward-backward check of the RAFT estimate as its validity [4]. Real samples carry the input kind one, so an unchanged real sample looks like a render sample to the network, unless the video step changes it. GameIR clips are part of the TSR sequences, and their frames lie ten rendered frames apart, which the notebook's FG section names as the reason for leaving them out of FG. The TSR batches read the consecutive manifest frames as adjacent frames. If the frames are ten rendered frames apart, as the manifest records, each GameIR clip is one window whose motion is estimated over a ten-frame gap, and the stored frame stride is never read. The two notebooks disagree on this point, and the TSR use of the GameIR windows needs a decision. **[Under revision]**

After a batch is built, `withhold_engine_data` withholds the motion and depth of a share of every batch, so that the network also works without them. The withheld motion sets the quality to zero, and the withheld depth clears its flag. The docstring says that only render samples lose their motion, but the function applies to every sample in the batch, so the docstring is out of date. The function runs only during training and not during validation.

S3.10.6 Frame-generation batches

The FG batch function `fg_batch` renders two real frames I_0 at time 0 and I_1 at time 1, and a target at time τ. The target time is $\tau = 0.5$ for interpolation, a uniform time in $[0.15, 0.85]$ for a share p_τ of interpolation samples, and $\tau = 1.5$ for extrapolation. Extrapolation is chosen with probability p_{extrap}, and then the target lies after the newest real frame, which is the low-latency mode. The FG frames are rendered at $P \times P$ pixels with four samples per pixel, so they are anti-aliased. The synthetic FG frames therefore have no low-resolution aliasing, unlike the TSR inputs. The batch also returns the engine motion vector of I_1, the depth of I_1, the exact flows from the target to each real frame, the validity of each flow, and a camera prior. The camera prior maps every target pixel through the camera layer only to time 1, and it is the flow that a camera-only engine would know. In an extrapolation sample the camera prior is available to the network with probability 0.8, as set in `set_availability`.

Four events are added to the batch. A camera reversal reverses the camera path after $t = 1$, which produces a rapid change of direction. A scene cut replaces I_1, its motion and depth, and the target by those of another sample, zeroes every validity mask, and makes the correct output the repeated frame I_1. A heads-up display overlay and subtitle changes are composited in screen space after rendering, with a UI layer of four slots. Each slot shows a tile with probability 0.7 at a random position, and with probability 0.3 the tile changes between $t = 0$ and $t = 1$, which is the subtitle

change. The UI layer is composited as $I_k \leftarrow I_k(1-\alpha_k)+c_k$, where c_k and α_k are the premultiplied colour and the alpha of the UI at time k. The target receives the UI of I_1, not an interpolated UI, so a generated frame shows the newest overlay. A menu is a full-screen panel with alpha 0.6, a dark grey colour and a probability set by the FG configuration. The UI mask records the union of the two UI alphas, and the network learns the mask through a cross-entropy term that is outside this section.

The FG batch from real quads, `fg_batch_from_frames`, takes four consecutive frames k to $k+3$ of a real sequence. The teacher flows are $2 \to 0$, $1 \to 0$, $1 \to 2$ and $3 \to 2$, computed by RAFT at half resolution and scaled back to the full grid [4]. An interpolation sample uses $I_0 = k$, $I_1 = k+2$ and the target $k+1$. An extrapolation sample uses the same two inputs and the target $k+3$. A multi-frame sample, with share p_{mfg} of the interpolation samples, uses $I_0 = k$, $I_1 = k+3$ and a target at $k+1$ or $k+2$, which corresponds to the times $1/3$ and $2/3$ of 3x generation. The cache has no flows for these pairs, so their motion and flow labels are marked unavailable. The real quads use the same flip and UI rules, and they carry the availability flags set by `set_availability`.

S3.10.7 Probabilities

Table S16 lists every probability that governs the synthetic and real samples. The first block is the configuration of the TSR and FG notebooks, which the generator reads from the saved configuration files. The second block is fixed in the renderer code, and the notebooks do not change it.

Table S16: Probabilities of the synthetic and real batches. Configuration values come from the saved configuration of each notebook. Renderer constants are fixed in the code. Availability is the probability that the auxiliary input is withheld

Quantity	Value	Meaning
TSR, real share p_{real}	0.4	share of each batch drawn from cached real sequences
TSR, video share, synthetic $p_{\text{video,synth}}$	0.3	share of synthetic samples turned into ordinary video
TSR, video share, real $p_{\text{video,real}}$	0.6	share of real samples turned into ordinary video
TSR, video motion $p_{\text{video,mv}}$	0.15	share of video samples that keep estimated motion
TSR, withheld motion $p_{\text{drop,mv}}$	0.25	share of samples whose motion is withheld in training
TSR, withheld depth $p_{\text{drop,depth}}$	0.3	share of samples whose depth is withheld in training
TSR, long unroll interval	4	one step in this many is a long unroll of 16 frames at 96 pixels
TSR, teacher forcing	0.25	share of each run with teacher-forced motion for the estimator
FG, real share p_{real}	0.4	share of each batch drawn from real quads
FG, extrapolation p_{extrap}	0.4	share of samples with $\tau = 1.5$
FG, random time p_{τ}	0.5	share of synthetic interpolation samples with a uniform τ
FG, multi-frame p_{mfg}	0.25	share of real interpolation samples taken as 3x generation
FG, withheld motion, synthetic $p_{\text{drop,mv}}$	0.3	share of synthetic samples whose motion is withheld
FG, withheld motion, real $p_{\text{drop,mv,real}}$	0.5	share of real samples whose motion is withheld
FG, UI p_{ui}	0.5	share of samples with a HUD or subtitle layer
FG, menu p_{menu}	0.03	share of samples with a full-screen menu
FG, scene cut p_{cut}	0.03	share of samples with a scene cut
FG, camera reversal p_{flip}	0.2	share of samples whose camera reverses after $t = 1$
Renderer, foreground present	0.85	share of samples with a foreground mask
Renderer, flat foreground	0.25	share of samples whose foreground is a flat colour
Renderer, world text	0.3	share of TSR samples with a text tile stamped into the background
Renderer, particle on	0.5	chance that each of the six particles is on
Renderer, wire present	0.6	chance that each of the four wires is present
Renderer, fence present	0.35	chance that the fence is present
Renderer, UI slot shown	0.7	chance that each of the four UI slots is shown
Renderer, UI change	0.3	chance that a shown UI slot changes at $t = 1$
Calibration, JPEG quality	35 to 92	JPEG quality of the copies made for calibration
Availability, depth, FG	0.3	chance that the depth of a synthetic sample is withheld
Availability, camera prior, FG	0.2	chance that the camera prior of an extrapolation sample is withheld

The saved configuration files record the values of the TSR and FG runs. The renderer constants are fixed in the code, and no saved ablation varies them, so their effect on the results is not measured.

Figure S15 shows one synthetic TSR sample at three times. The first column is the low-resolution jittered render, enlarged with nearest-neighbour interpolation so the pixels are visible. The second column is the same frame as ordinary video, which is smoother because of the blur and the filter. The third column is the four-sample target. The fourth column is the motion vector field, with the direction and size coded by colour, and the fifth column is the validity map, in which white marks pixels with a valid history. The sixth column is the depth, and the last column is the reactive mask.

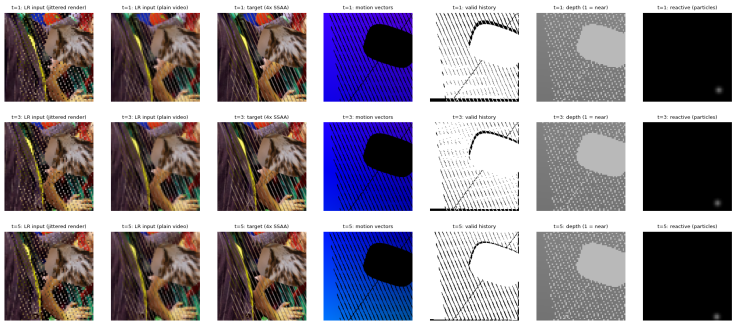


Fig. S15: One synthetic TSR sample at three times, from the TSR notebook. Columns: jittered render input, the same frame as plain video, the four-sample target, motion vectors, valid history, depth and reactive mask. 4x SSAA means four samples per pixel

The rendered sample shows the intended behaviour. The jittered render has coarse, shifting aliasing on the fence bars and wires, and its colour is blocky. The plain video is smoother, but it keeps the same content. The target has the thin fence and wire lines that the render loses. The motion field changes smoothly across the background, which follows one affine path, and the black blob marks the foreground, which moves on its own path. The validity map has white over most of the frame and dark diagonal streaks that follow the fence bars, where the background behind a moving bar is not visible in the previous frame. The depth shows a lighter blob for the foreground and a speckled background, which appears to follow the wire and fence pattern of the render. The reactive mask shows a single soft spot in the lower right, which is one particle, and it moves slightly between the three times. These readings are from the figure and not from a numerical test. **[Observation]**

Figure S16 shows the FG preview, one row for each event type. Each row shows the older real frame I_0, the newer real frame I_1, the target, the UI mask and the flow from the target to I_1. The interpolation row has a small motion, with nearly identical I_0 and I_1 and a target between them. The UI mask of that row is empty. The extrapolation row, with a camera reversal, shows a green blob that moves further between I_1 and the target, and the flow shows the blob and the background moving in different directions. The HUD row shows a crosshair and the text “XP 487” that appear in I_0, I_1 and the target, and the UI mask holds both. The menu row dims the scene, because the menu panel covers the full frame at alpha 0.6, and the text “Wave Health” appears in the mask. The scene cut row shows that I_1 and the target are the same picture from a different scene, so the target repeats I_1 as designed. These readings come from the preview and not from a test. **[Observation]**

S4 Temporal super-resolution: method

This section specifies the temporal super-resolution (TSR) network completely: its inputs and their encodings, the built-in motion estimator, the recurrent state, the

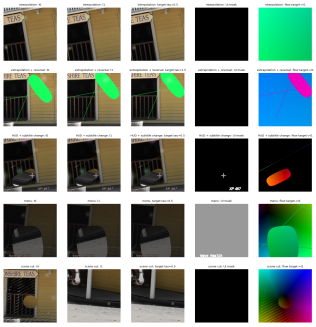


Fig. S16: Synthetic FG samples, one row per event. Columns: the older real frame I_0, the newer real frame I_1, the target, the UI mask and the flow from the target to I_1. Rows: interpolation, extrapolation with a camera reversal, a HUD and subtitle change, a menu, and a scene cut

reconstruction trunk and output rule, the model tiers, the training samples and losses, the optimisation and checkpoint selection, the ablation configurations, the export path and the test protocol. The results are in Section S5. The design summary is in Section 4 and the measured tier costs are in Table 1. Warping, bilinear sampling, pixel shuffle, temporal anti-aliasing and the image metrics are explained in Section S2, and the synthetic renderer and the real frame windows are described in Section S3. Every number in this section is either a value set in the code or the training configuration of the run, or a count computed from the code, and each one names its source.

S4.1 Problem and notation

TSR turns a stream of low-resolution (LR) frames into a stream of high-resolution (HR) frames, one output for each input, and each output may use the earlier outputs. The LR resolution is $h \times w$ and the output resolution is $H \times W$, with $H = sh$ and $W = sw$ for an integer scale s. The primary model uses $s = 3$ (360p to 1080p), and a second student uses $s = 2$ (540p to 1080p).

At frame t the network receives the colour frame $\mathbf{x}_t \in [0,1]^{3\times h\times w}$ in sRGB, the sub-pixel jitter $\mathbf{j}_t$ of its samples, and whatever side information the caller supplies. It returns $\hat{\mathbf{y}}_t \in [0,1]^{3\times H\times W}$, an estimate of the HR target $\mathbf{y}_t$. For synthetic sequences the target is the four-sample anti-aliased render at output resolution (Section S3.10), and for real windows it is the cached HR frame (Section S3). A pixel is written $\mathbf{p} = (X, Y)$, with X along the width and Y along the height. A discrete image is read at continuous positions by bilinear interpolation. Sample i of a row sits at continuous coordinate $i + \frac{1}{2}$, so pixel i covers the interval $[i, i + 1)$.

The problem has two regimes. In *engine mode* the caller is a renderer that also supplies motion vectors, depth, a reactive mask and exposure, and its input samples are jittered. In *plain mode* the caller is ordinary video, which is an anti-aliased bicubic downscale without jitter and without side information. One set of weights serves both regimes. The network uses the side information when it is present, estimates the

motion when it is absent, and is told which regime applies through the input-kind and availability flags of Section S4.3.

The symbols are listed in Table S17. Two conventions matter throughout. A motion field $\mathbf{m}$ gives, for every pixel of frame t, the displacement to the same surface point in frame $t-1$, so the history at pixel $\mathbf{p}$ is read at $\mathbf{p}+\mathbf{m}(\mathbf{p})$. Motion is stored in LR pixels at render resolution, and an upsampled copy is stored in HR pixels.

Table S17: Symbols of the TSR method. Shapes give the channels, then the height and width, for one frame of one sample

Symbol	Meaning	Symbol	Meaning
s	integer scale, $H = sh$, $W = sw$	$\mathbf{m}_t$	motion used by the network, $2 \times h \times w$, LR pixels, frame t to $t-1$
t	frame index, $t = 0$ is the first frame and resets the state	$\mathbf{m}_t^{\uparrow}$	the same motion in HR pixels (Eq. (S56))
$\mathbf{x}_t$	colour input, $3 \times h \times w$, sRGB in $[0, 1]$	$\hat{\mathbf{m}}_t$	motion of the built-in estimator (Eq. (S52))
$\mathbf{j}_t$	jitter of the input samples, in LR pixels, $[-\frac{1}{2}, \frac{1}{2})$	q_t, κ_t, η_t	motion-quality flag, input kind, depth availability (Table S18)
$\mathbf{d}_t$	normalised depth, $1 \times h \times w$ (Eq. (S49))	$\bar{\mathbf{m}}_t$, $\bar{\mathbf{M}}_t$	label motion at LR and HR, used for supervision and metrics only
$\mathbf{u}_t$, e_t	reactive mask, $1 \times h \times w$, and exposure as $\log_2$ gain	$\bar{\mathbf{v}}_t$, $\bar{\mathbf{V}}_t$, λ_t	label validity at LR and HR, and label quality (1 exact, $\frac{1}{2}$ RAFT, 0 none)
$\mathcal{W}(I, \mathbf{m})$	backward warp of I by $\mathbf{m}$ (Eq. (S57))	$\mathcal{U}(\mathbf{x})$	jitter-aware bilinear upsampling (Eq. (S63))
$\mathbf{h}_t$	hidden state, $C_h \times h \times w$	a_t	history weight, $1 \times H \times W$, in $(0, 1)$
$\mathbf{r}_t$	output residual, $3 \times H \times W$	$\hat{\mathbf{y}}_t$, $\mathbf{y}_t$	output and target, $3 \times H \times W$
C, B, U, R	trunk width, trunk blocks, U-Net blocks, refinement width (Table S19)	C_h	width of the hidden state

Table S18: Channels of the TSR input stack with their resolution, where LR is the render resolution and scalar is one value per sample, broadcast over the LR grid. Engine mode is the jittered render with its side information, and plain mode is ordinary video. Normalisation is applied before the stack is built

Input	Ch., size	Engine mode	Plain mode	Encoding
Colour $\mathbf{x}_t$	3, LR	One jittered sample per LR pixel, read from the render (synthetic) or the cached HR frame (real windows) at $\mathbf{p} + \frac{1}{2} + \mathbf{j}_t$	Anti-aliased bicubic downscale of the target without jitter (test). Training uses one of three filters with blur and noise (Section S4.10)	sRGB in $[0, 1]$, unchanged
Depth $\mathbf{d}_t$	1, LR	Renderer depth (synthetic), decoded depth (TartanAir, GameIR), or zero when the source has none	Zero	$1/(1 + d/\operatorname{med}(d))$, Eq. (S49), where 1 is near
Normals $\mathbf{n}_t$	3, LR	Central differences of $\mathbf{d}_t$	Zero	Eq. (S50), times η_t

continued on next page

Input	Ch., size	Engine mode	Plain mode	Encoding
Motion $\mathbf{m}_t$	2, LR	Engine motion (exact, RAFT, or zero when withheld), or the estimate $\hat{\mathbf{m}}_t$	Estimate $\hat{\mathbf{m}}_t$	Multiplied by 1/8 after the selection, so 8 LR pixels give 1
Disocclusion cue δ_t	1, LR	From depth and the warped previous depth, Eq. (S51)	Zero	In $[0, 1]$
Reactive mask $\mathbf{u}_t$	1, LR	Particle coverage $1 - \prod_k (1 - g_k)$ for synthetic samples, zero for real windows and test clips	Zero	In $[0, 1)$
Exposure e_t	1, scalar	$\log_2$ of the gain (synthetic scene gain, or real-window gain)	Unchanged	Not scaled, range $[-0.8, 0.8]$ in training
Jitter $\mathbf{j}_t$	2, scalar	Sub-pixel offset of the samples	Zero	In $[-\frac{1}{2}, \frac{1}{2})$
Depth availability η_t	1, scalar	1 when the sample has depth	Zero	Flag
Motion quality q_t	1, scalar	1 exact, $\frac{1}{2}$ RAFT or capped video motion, $\frac{1}{4}$ estimate (zero is replaced by $\frac{1}{4}$ when the estimator runs)	$\frac{1}{4}$ estimate	Flag
Input kind κ_t	1, scalar	1	0	Flag
Folded history $\downarrow_s \mathbf{H}_t$	$3s^2$, LR	Previous output warped by $\mathbf{m}_t^{\uparrow}$, pixel unshuffled	Same	Zero at reset
History disagreement $\mathbf{g}_t$	3, LR	$\lvert\tilde{\mathbf{H}}_t - \mathbf{x}_t\rvert$, where $\tilde{\mathbf{H}}_t$ is the $s \times s$ block mean of $\mathbf{H}_t$	Same	Zero at reset
Hidden state, warped by the motion	C_h, LR	Previous hidden state $\mathbf{h}_{t-1}$ warped at LR by $\mathbf{m}_t$	Same	In $(-1, 1)$ from the tanh output

Table S19: TSR tiers at scale 3, with the 2x cost of each tier. Parameters are fused counts and costs are in GMAC per 1080p output frame, counted in the notebook's model tiers step. Estimator widths are per level, and n is the number of residual blocks. Only `teacher`, `igpu` at 3x and `low` at 3x and 2x were trained in the TSR run (Section S4.9)

Tier	C	B	U	C_h	Estimator widths, n	R	Fused params	Estimator params	GMAC 3x	GMAC 2x
`igpu`	16	3	0	8	(32, 24, 16), 2	–	137,844	116,504	6.2	11.3
`low`	24	4	2	8	(48, 32, 24), 3	8	328,159	228,256	16.1	30.7
`mid`	32	4	3	16	(64, 48, 32), 4	8	653,343	444,464	28.1	56.3
`high`	48	6	4	16	(96, 64, 48), 4	12	1,405,803	837,152	69.0	145.0
`teacher`	64	8	6	32	(128, 96, 64), 6	16	3,395,095	2,019,680	156.1	337.4

S4.2 Architecture overview

One frame passes through the stages of Fig. S17. The estimator supplies motion when a sample has none, the previous output and the hidden state are warped and folded, the inputs are stacked, a re-parameterisable trunk maps the stack to C channels, two heads predict a history weight and a residual, and Eq. (S62) with a refinement gives the output. The coarser view of the article is Fig. 1.

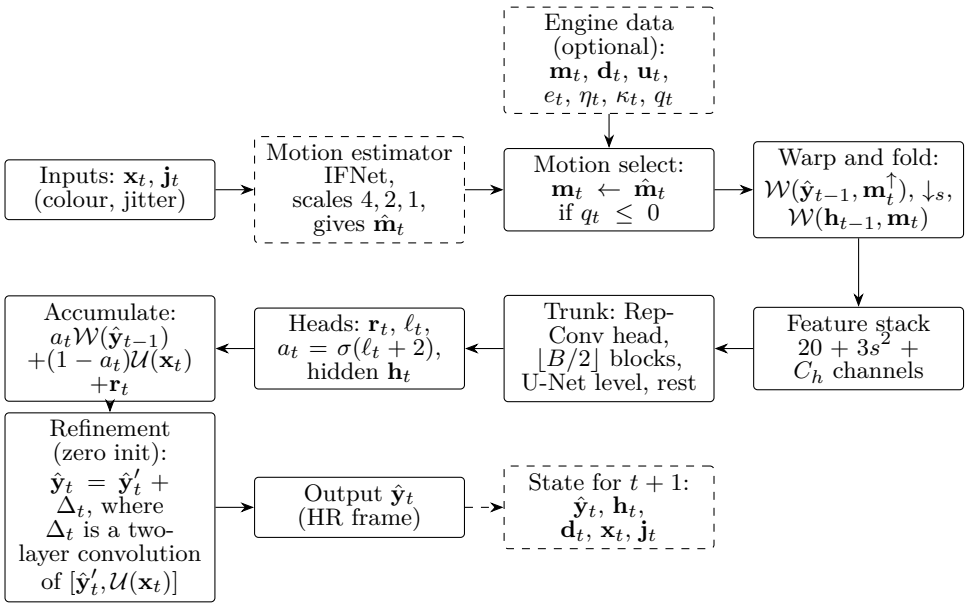


Fig. S17: Data flow of one TSR time step at scale s. Dashed boxes are optional: the engine data are used when the caller supplies them, and the estimator runs in training for every frame and at inference only when a sample has no motion. The state of frame t holds the history, hidden state, depth, colour and jitter that the warp and disocclusion cue of frame $t + 1$ read (Section S4.5)

S4.3 Inputs

The trunk receives a stack of channels at render resolution. Table S18 lists them in the order of the code. Seventeen channels come from the frame and its side information, $3s^2$ from the folded history, three from the history disagreement and C_h from the hidden state, so the stack has $20 + 3s^2 + C_h$ channels. For $s = 3$ and $C_h = 8$ this is 55, which the code confirms for the `low` tier. The table gives the value that the network sees in each mode. Two replacements happen before the stack is built. A sample without motion receives the estimate $\hat{\mathbf{m}}_t$ and the flag 0.25 (Section S4.4). A sample with withheld depth receives zero depth and $\eta_t = 0$.

The engine motion of the table is the motion in the input, which is exact (synthetic, TartanAir and Sintel), RAFT (real video and GameIR), or absent. The value $\frac{1}{2}$ marks RAFT motion and video samples that keep their motion labels, which are capped at $\frac{1}{2}$ by the degradation of Section S4.10.

The depth normalisation of Table S18 is

$$\mathbf{d}_t = \frac{1}{1 + \mathbf{d}_t^{\text{raw}} / \operatorname{med}(\mathbf{d}^{\text{raw}})}, \tag{S49}$$

where $\mathbf{d}^{\text{raw}}$ is the depth in the unit of its source, and med is the median over the whole sequence for synthetic samples, and over one frame for real windows and test clips.

Non-positive and non-finite depth values are set to infinity before the normalisation, so they map to values near zero.

The normals are derived from the normalised depth with edge replication at the border, using the central differences $g_t^x(X,Y) = 8\,[\mathbf{d}_t(X+1,Y) - \mathbf{d}_t(X-1,Y)]$ and g_t^y defined in the same way along Y:

$$\mathbf{n}_t = \eta_t \, \frac{(-g_t^x,\, -g_t^y,\, 1)}{\sqrt{(g_t^x)^2 + (g_t^y)^2 + 1}}. \tag{S50}$$

The factor 8 is a fixed constant of the code that scales the depth gradient, and the vector is normalised to unit length.

The disocclusion cue compares the current depth with the depth of the previous frame warped to the current frame, using the motion of the network:

$$\delta_t = \eta_t \, \min\left\{1,\; \frac{|\mathbf{d}_t - \mathcal{W}(\mathbf{d}_{t-1}, \mathbf{m}_t)|}{|\mathbf{d}_t| + 10^{-3}}\right\}. \tag{S51}$$

A large value marks a depth that differs from the warped previous depth, which is typical of a disocclusion. It is a heuristic that the trunk refines, not a visibility test.

S4.4 Motion estimator

The estimator is the IFNet of RIFE 4.26 [5] in a narrower form, a coarse-to-fine chain of flow blocks. It is built by `IFNet(widths, n_res, n_hint=3)` in `nss_models.py`. Its widths and residual counts are set per tier in the constant `TSR_TIERS`: $(32, 24, 16)$ with two residual blocks for `igpu`, $(48, 32, 24)$ with three for `low`, $(64, 48, 32)$ with four for `mid`, $(96, 64, 48)$ with four for `high`, and $(128, 96, 64)$ with six for `teacher`. The three levels run at scales $\sigma_1 = 4$, $\sigma_2 = 2$ and $\sigma_3 = 1$ relative to the LR frame, so the coarsest block sees a frame of a quarter of the LR size in each direction.

The estimator takes the previous colour frame $\mathbf{x}_{t-1}$ and the current frame $\mathbf{x}_t$, both padded at the right and bottom by edge replication to multiples of 16. Its timestep is the constant 1, so its target frame is the current frame and its reference is the previous one. Its final flow, the first two of its four channels, is the motion of Section S4.1:

$$\hat{\mathbf{m}}_t = \mathbf{F}_0^{(3)}, \qquad \mathbf{F}^{(1)} = \Delta^{(1)}, \qquad \mathbf{F}^{(k)} = \mathbf{F}^{(k-1)} + \Delta^{(k)} \;\; (k = 2, 3), \tag{S52}$$

where $\Delta^{(k)}$ is the four-channel flow update of block k and $\mathbf{F}^{(k)} = (\mathbf{F}_0^{(k)}, \mathbf{F}_1^{(k)})$ holds the flows from the target to the previous and to the current frame. The flow outputs are multiplied by the block scale, so the flows stay in LR pixels.

The first block receives $\mathbf{x}_{t-1}$, $\mathbf{x}_t$, the per-frame features of both frames and the constant timestep, which is 15 channels. Each later block receives the colour of both frames warped by the current flow, the features of both frames warped by the current flow, the timestep, the blend mask and the features of the previous block, and the current flow itself, which is 28 channels. Three hint channels follow for the TSR estimator: the jitter difference $\mathbf{j}_t - \mathbf{j}_{t-1}$ (two channels) and the input kind κ_t (one channel). The per-frame feature encoder has a stride-two 3×3 convolution and two

3×3 convolutions, each followed by LeakyReLU, and a transposed convolution that returns to full resolution with four channels. Inside each block, two stride-two 3×3 convolutions reduce the resolution by four, a set of residual 3×3 convolutions with a learned per-channel scale follows, and a transposed convolution with pixel shuffle returns to the block resolution with 4 flow channels, one mask logit and 8 features.

The estimator is initialised from the RIFE 4.26 weights wherever the names match, and its narrower layers take the leading channel slices of the RIFE tensors. The weight loader `load_ifnet_weights` copies the matching tensors whole and the narrower ones as slices, and the hint channels keep their zero weights. The training log of the `low` run records the result as 11 whole tensors and 42 sliced tensors with no missing tensor. Without the RIFE weights, the code falls back to random initialisation and logs the fact.

The estimate replaces the motion only where the caller gave none. The rule is the one line of `TSRNet.forward`:

$$\mathbf{m}_t = \begin{cases} \bar{\mathbf{m}}_t \text{ (teacher-forced label)} & \text{if } q_t \le 0 \text{ and the sample is forced,} \\ \hat{\mathbf{m}}_t & \text{if } q_t \le 0 \text{ otherwise,} \\ \text{given motion} & \text{if } q_t > 0, \end{cases} \tag{S53}$$

and the quality flag of every sample with $q_t \le 0$ becomes $\frac{1}{4}$ (the constant `EST_MV_QUALITY`). The flag is therefore the same for estimated and forced samples, and the network cannot tell them apart. At inference the estimator runs only when some sample of the batch has $q_t \le 0$. In training it runs for every frame of every sample, because the motion loss of Section S4.12 supervises every sample that has a label, including those with engine motion.

Teacher forcing protects the trunk while the estimator is still poor. A sample without motion is forced to warp its history with the label motion $\bar{\mathbf{m}}_t$ with probability $\pi(\tau)$, drawn once per sample and used for all its frames, where τ is the elapsed training time and T_b the training budget in seconds:

$$\pi(\tau) = \max\left(0,\ 1 - \frac{\tau}{\rho\, T_b}\right), \qquad \rho = 0.25. \tag{S54}$$

The forcing applies only to samples that have a label, so $\lambda > 0$. Its probability falls linearly from one to zero over the first quarter of the budget, after which no forced mask is drawn. The clock is the wall-clock time of each process from its first step, so the two DDP ranks draw their masks on their own clocks. Teacher forcing is the name for feeding a recurrent predictor a target in place of its own output during training. Here it is this code's schedule, which replaces the estimate by the label for part of the early training only.

S4.5 Warping, folding and the recurrent state

The state carried from frame $t-1$ to frame t is

$$\mathcal{S}_{t-1} = \left(\hat{\mathbf{y}}_{t-1},\ \mathbf{h}_{t-1},\ \mathbf{d}_{t-1},\ \mathbf{x}_{t-1},\ \mathbf{j}_{t-1}\right). \tag{S55}$$

The depth, colour and jitter are kept for the disocclusion cue and for the estimator of frame t. The ONNX export passes them as explicit inputs (Section S4.17).

The previous output is warped to the HR grid with the upsampled motion. The LR motion is interpolated bilinearly to HR and multiplied by the scale, because a displacement of d LR pixels is sd HR pixels:

$$\mathbf{m}_t^{\uparrow} = s \cdot \mathrm{Bil}^{\times s}[\mathbf{m}_t], \qquad \mathbf{H}_t = \mathcal{W}\big(\hat{\mathbf{y}}_{t-1}, \mathbf{m}_t^{\uparrow}\big), \tag{S56}$$

where $\mathrm{Bil}^{\times s}$ is bilinear upsampling by s with align-corners set to false. The folded history is $\downarrow_s \mathbf{H}_t \in \mathbb{R}^{3s^2 \times h \times w}$, the space-to-depth rearrangement of PyTorch's pixel unshuffle, which places every $s \times s$ block of HR pixels in the channels of one LR pixel. The history disagreement uses the mean of each block, $\bar{\mathbf{H}}_t = \mathrm{avgpool}_s(\mathbf{H}_t)$, and its absolute difference to the LR colour, $\mathbf{g}_t = |\bar{\mathbf{H}}_t - \mathbf{x}_t|$.

The hidden state and the depth are warped at LR with the LR motion $\mathbf{m}_t$, which is the motion of the network. The hidden state after frame t is produced by the trunk, as Section S4.7 gives. Warping uses the backward warp of Eq. (S57):

$$\mathcal{W}(I, \mathbf{m})(\mathbf{p}) = I\big(\mathbf{p} + \mathbf{m}(\mathbf{p})\big), \tag{S57}$$

where I is read bilinearly and clamped to its border. The implementation uses grid sampling with align-corners set to true, the convention of RIFE, so that the pixel centres of the first and last rows and columns lie on the image border.

On the first frame of a sequence, or in every frame of the no-history ablation, the state is empty. The history $\mathbf{H}_t$, the folded history, the history disagreement and the warped hidden state are all multiplied by zero, the disocclusion cue is computed against the current depth so that it is zero, and the history weight is set to zero exactly, not learned, by the rule

$$a_t = 0 \quad \text{when the state is empty.} \tag{S58}$$

The network has no other reset. It has no scene-cut detector, so the history weight must reject a cut by itself. The synthetic renderer produces no cuts, and the cache builders contain no cut filter, so a real window may contain a cut, and the training does not measure how often that happens.

S4.6 Reconstruction trunk

The trunk is a chain of re-parameterisable blocks, called RepConv here, in the sense of RepVGG [32] and ECBSR [33]. A RepConv block is trained as a sum of linear branches and deployed as one 3×3 convolution. A training forward pass folds the branches into one kernel on every call, so each block costs one convolution in training and one in deployment. The folded kernel for an output channel o and an input channel i is

$$\mathbf{W}_{oi} = \mathbf{W}_{oi}^{(3)} + \mathrm{pad}_{3\times 3}\left(\mathbf{W}_{oi}^{(1)}\right) + \sum_{k=1}^{3} \mathbf{P}_{o,i,k}\, \mathbf{E}_k + \delta_{oi}\, \mathbf{C}, \tag{S59}$$

with the bias $b_o = b_o^{(3)} + b_o^{(1)} + b_o^{(e)}$. The symbols are as follows. $\mathbf{W}^{(3)}$ is a learned 3×3 kernel and $\mathbf{W}^{(1)}$ a learned 1×1 kernel, padded with zeros to 3×3. The matrices $\mathbf{E}_1$, $\mathbf{E}_2$ and $\mathbf{E}_3$ are the fixed Sobel-x, Sobel-y and Laplacian kernels, $[[-1,0,1],[-2,0,2],[-1,0,1]]$, $[[-1,-2,-1],[0,0,0],[1,2,1]]$ and $[[0,1,0],[1,-4,1],[0,1,0]]$. The coefficients $\mathbf{P}_{o,i,k}$ form a learned 1×1 convolution from $3C_i$ inputs to C_o outputs, reshaped so that each input channel i has its three edge responses k. The matrix $\mathbf{C}$ has a one at its centre and zeros elsewhere, and the term with δ_{oi} is present only when $C_i = C_o$. Because the folding is linear, the folded convolution computes exactly the same function as the training branches.

Each RepConv is followed by a PReLU with one learned slope per output channel, initialised to PyTorch's default of 0.25. The side branches start at zero, in the following sense. The weights of the 1×1 branch and of the edge branch are set to zero, so the block starts as a 3×3 convolution plus identity. Their biases keep PyTorch's default uniform initialisation, so the side branches do not start exactly at zero. The code records that default initialisation grew the activations by about 1.9 times per block and gave a non-finite teacher loss from step 10 in an earlier run **[Observation]**.

The trunk of Eq. (S60) is built from the stack $\mathbf{z}_t$, which has $20+3s^2+C_h$ channels. Let $\mathrm{RC}_{\mathrm{head}}$ denote the head RepConv block, $\mathcal{A}$ the first $\lfloor B/2 \rfloor$ blocks, $\mathcal{B}$ the remaining $B - \lfloor B/2 \rfloor$ blocks, and $\mathcal{V}$ the U-Net level, which exists when $U > 0$:

$$\begin{aligned} \mathbf{z}_0 &= \mathrm{RC}_{\mathrm{head}}(\mathbf{z}_t) \\ \mathbf{z}_1 &= \mathcal{A}(\mathbf{z}_0) \\ \mathbf{z}_2 &= \mathbf{z}_1 + \mathcal{V}(\mathbf{z}_1) \\ \mathbf{y}_t &= \mathcal{B}(\mathbf{z}_2). \end{aligned} \tag{S60}$$

The head is one RepConv from $20+3s^2+C_h$ channels to C channels. The U-Net level, when present, is a 3×3 convolution with stride two from C to $2C$ channels followed by PReLU, then U RepConv blocks at $2C$ channels, then a 1×1 convolution from $2C$ to $4C$ channels and a pixel shuffle with factor two, which returns C channels at LR resolution. The U-Net level therefore works at half the LR resolution and is added to the trunk as a residual. The LR input is padded to an even size by edge replication before the trunk, which the stride-two convolution needs. The numbers of blocks per tier are in Table S19.

S4.7 Output head and accumulation

The output head is one 3×3 convolution from C to $4s^2$ channels, with zero weights and zero bias, applied to $\mathbf{y}_t$. A pixel shuffle with factor s [7] then gives four channels at HR. The first three are the residual $\mathbf{r}_t$ and the fourth is a logit ℓ_t, and the history weight is

$$a_t = \sigma(\ell_t + 2), \qquad \sigma(z) = \frac{1}{1+e^{-z}}. \tag{S61}$$

At initialisation the zero weights give $\ell_t = 0$, so $a_t = \sigma(2) \approx 0.881$ and $\mathbf{r}_t = 0$, and the network starts as a blend of the warped history and the upsampled colour. The hidden state is produced by a 3×3 convolution from C to C_h channels followed by tanh, $\mathbf{h}_t = \tanh(\mathrm{Conv}_{3\times3}(\mathbf{y}_t))$, and it is warped at the next frame by Eq. (S57).

The output is the accumulation rule

$$\hat{\mathbf{y}}'_t = a_t \odot \mathcal{W}\big(\hat{\mathbf{y}}_{t-1}, \mathbf{m}_t^{\uparrow}\big) + (1 - a_t) \odot \mathcal{U}(\mathbf{x}_t) + \mathbf{r}_t, \tag{S62}$$

where $\odot$ is the element-wise product, $\mathcal{U}$ is the jitter-aware bilinear upsampling

$$\mathcal{U}(\mathbf{x}_t)(X, Y) = \mathrm{Bil}[\mathbf{x}_t]\Big(\frac{X + \frac{1}{2}}{s} - \mathbf{j}_t\Big), \tag{S63}$$

and $\mathrm{Bil}[\mathbf{x}]$ is the bilinear interpolant of the LR samples, sample i at coordinate $i + \frac{1}{2}$, with clamped borders. The term $\frac{X+1/2}{s}$ is the continuous LR coordinate of output pixel (X, Y), and subtracting $\mathbf{j}_t$ reads each LR sample at its jittered position. The output is then refined as Section S4.8 gives. On the first frame, $a_t = 0$ by Eq. (S58), so $\hat{\mathbf{y}}_0 = \mathcal{U}(\mathbf{x}_0) + \mathbf{r}_0$.

Why this is a learned temporal accumulation.

Ignore the residual and the motion for a moment, and let $a_t = a$ be constant, with $\mathbf{m}_t = 0$ on a static scene. Then Eq. (S62) unrolls to

$$\hat{\mathbf{y}}_t = a^t\, \mathcal{U}(\mathbf{x}_0) + \sum_{k=1}^{t} (1 - a)\, a^{t-k}\, \mathcal{U}(\mathbf{x}_k). \tag{S64}$$

The weights add to one, because $a^t + (1 - a) \sum_{j=0}^{t-1} a^j = a^t + (1 - a^t) = 1$. The sample k frames old has weight $(1 - a)a^k$, which is an exponential moving average over the frames of the sequence, with mean age $a/(1 - a)$ frames. Each $\mathcal{U}(\mathbf{x}_k)$ reads the colour at a different sub-pixel jitter, so the average gathers the samples of several sub-pixel positions. This is the principle of temporal accumulation for supersampling [6] (Section S2.5). At the initial history weight $\sigma(2)$ the mean age is about 7.4 frames.

Classical TAA has the same update with a fixed or heuristic blend weight, and it rectifies the reprojected history by clamping it to the colour neighbourhood of the current frame. Equation (S62) keeps the update and changes the rectification. The weight a_t is a learned per-pixel gate, and the network has the disagreement $\mathbf{g}_t$ as input and a receptive field of several convolutions, so the gate can learn to go to zero where the history is invalid, which happens at disocclusions and at particle effects. The residual $\mathbf{r}_t$ corrects the result additively. There is no explicit clamp anywhere in the TSR network, so the residual can move a pixel outside the neighbourhood of the current frame. The analytic baseline of Section S4.18 does clamp, and the two rectifications are different.

S4.8 Refinement and the reset rule

Every tier except `igpu` refines the accumulated output at HR resolution. The refinement is

$$\hat{\mathbf{y}}_t = \hat{\mathbf{y}}'_t + \mathrm{Conv}_{3\times3}\Big(\mathrm{PReLU}\big(\mathrm{Conv}_{3\times3}([\hat{\mathbf{y}}'_t, \mathcal{U}(\mathbf{x}_t)])\big)\Big), \tag{S65}$$

where $[\cdot,\cdot]$ concatenates six channels. The first convolution has R output channels and the second has three. The second convolution has zero weights and zero bias, so the refinement starts at the identity. Its purpose is to remove the pixel-shuffle pattern of the LR head, as the code comment states. The width R is 8 for `low` and `mid`, 12 for `high` and 16 for the teacher, and the `igpu` tier has no refinement.

The reset rule is Eq. (S58). It applies to the first frame of each training window and each test clip, and in every frame of the no-history ablation (Section S4.16). The ONNX step of Section S4.17 has no reset input.

S4.9 Model tiers

The tiers are the entries of `TSR_TIERS` in `nss_models.py`. Table S19 gives their widths, block counts, U-Net blocks, hidden widths, estimator, refinement width, parameter counts and costs. The parameter counts are those of the fused network, with the estimator included, and the estimator parameters are the difference from the same network built without the estimator. The costs count multiply-accumulates of every convolution, including the estimator, for one steady-state frame with history, per 1080p output frame, and exclude the warps and the pixel shuffles, which have no multiply-accumulates. The counting is done with forward hooks on the fused network in `tsr_macs_per_lr_pixel`, and the notebook scales it by the LR pixel count.

The fused parameter counts and the 3x and 2x costs are the values that `model_tiers.csv` records for the run. They were also recomputed from `src/neuralss/nss_models.py` on the current code, and both agree. The estimator column is the difference between the training parameter count with and without the estimator, which is also computed from the code. The `mid` and `high` tiers are defined and costed, but the TSR notebook never trains them. Its runs use the teacher at 3x, `low` at 3x and 2x, `igpu` at 3x, and the six ablation runs at 3x, which all use `low`. The tier table of the article (Table 1) therefore gives no latency for these two tiers.

The estimator holds a large share of the training parameters of every tier. It is about 81 percent for `igpu`, 63 percent for `low` and 51 percent for the teacher, and it runs on every frame without engine motion.

S4.10 Training samples and degradations

Each training step draws from two streams, and both produce sequences of T frames with HR targets, LR inputs, motion labels and validity masks. The synthetic stream renders layered game-like scenes from cached texture crops with the renderer of Section S3.10. Its scene parameters, which are used here, are a background layer and a foreground layer with procedural alpha (a superellipse, four thin wires each present with probability 0.6, and a fence present with probability 0.35), moving on continuous affine paths. The foreground texture is flat with probability 0.25 and is otherwise the still of the neighbouring sample in the batch, the foreground layer is present with probability 0.85, world-space text tiles are stamped into the background with probability 0.3, and six particles are drawn, each active with probability 0.5. The motion class is static, slow, medium or fast with probabilities 0.15, 0.45, 0.25 and 0.15, which give speeds of 0, 0.3 to 3, 3 to 10 and 10 to 28 output pixels per frame. Each sample

has an exposure 2^u with u uniform on $[-0.8, 0.8]$, and a sub-pixel jitter phase drawn uniformly from the sixteen Halton offsets. The LR frames are single-sample jittered reads of the renderer, and the HR targets are four-sample renders.

The real stream reads cached windows of T frames at $P = 192$ output pixels from the sequence sources of Section S3. The window is drawn from a source-specific downscale range before cropping. A real window is flipped horizontally with probability $\frac{1}{2}$ and vertically with probability $\frac{1}{2}$, and its motion vectors are negated in the flipped components. It is scaled in linear light by an exposure gain 2^u with u uniform on $[-0.8, 0.8]$. Its LR frames are jittered bilinear reads of the HR frame, so they are aliased like a render. The labels are the cached motion and validity, with the label quality of the window. Exact motion, from TartanAir and Sintel, has label quality 1. Other windows get RAFT motion when RAFT is available, with label quality $\frac{1}{2}$, and otherwise zero motion with label quality 0. Table S20 lists the probabilities that the TSR configuration or the degradation functions set.

Table S20: Probabilities of the TSR training data that are set by the TSR configuration or by the degradation functions. The renderer's own probabilities are given in Section S4.10 and Section S3.10

Quantity	Value	Code location
Real windows per batch	0.4, so 19 of 48	`p_real`, `make_loader`
Synthetic samples turned into plain video	0.3	`p_video_synth`, `video_degrade`
Real samples turned into plain video	0.6	`p_video_real`, `video_degrade`
Plain-video samples that keep their motion labels	0.15	`p_video_mv`, `video_degrade`
Samples that lose their motion vectors (all samples of the batch)	0.25	`p_drop_mv`, `withhold_engine_data`
Samples that lose their depth (all samples of the batch)	0.3	`p_drop_depth`, `withhold_engine_data`
Long steps, every fourth step from step 0	1/4	`long_every` = 4, `train_tsr.step_fn`
Teacher forcing of motion	$\pi(\tau)$ in Eq. (S54)	`mv_warmup_frac` = 0.25
Horizontal and vertical flip of real windows	1/2 each	`sr_batch_from_frames`
Gain of real windows and scene exposure	2^u, $u \sim U[-0.8, 0.8]$	`sr_batch_from_frames`, `Scene`
Downscale filter of plain video (bicubic, bilinear, box)	1/3 each	`video_lr`
Blur σ of plain video	$U[0, 0.6]$ output pixels	`video_lr`
Noise σ of plain video	$U[0, 2/255]$	`video_lr`

The plain-video degradation turns a sample into ordinary video. A plain-video LR frame is the target frame $\mathbf{y}$ after a Gaussian blur, an anti-aliased downscale, additive noise and 8-bit quantisation. The blur is a five-tap separable Gaussian on the output grid with σ uniform on $[0, 0.6]$. The downscale D is bicubic (cubic convolution [31]),

bilinear or box averaging, each chosen with probability 1/3. The noise is added after the downscale, with σ_n uniform on $[0, 2/255]$, and the frame is clamped and rounded to multiples of $1/255$:

$$\mathbf{x}_t^{\mathrm{vid}} = \mathrm{round}_{1/255}\Big(\mathrm{clip}_{[0,1]}\big(D\big(G_\sigma * \mathbf{y}_t\big) + \mathbf{n}\big)\Big), \qquad \mathbf{n} \sim \mathcal{N}(0, \sigma_n^2 I). \tag{S66}$$

A plain-video sample loses its jitter, its depth, its reactive mask and its depth availability, and its input kind becomes $\kappa_t = 0$. Its motion is removed, except for the share $p_{\mathrm{mv}} = 0.15$ that keeps the label motion with flag $\min(q, \frac{1}{2})$. The label motion of a plain-video sample is the block mean of the HR motion, divided by s, and its validity is one only where all $s \times s$ HR pixels are valid. The labels keep their label quality, so a plain-video sample still supervises the estimator.

The withholding rule is applied to the whole batch after both streams are built. Its two masks are drawn once per sample and apply to every sample, including the plain-video samples that kept their motion. A sample loses its motion with probability 0.25, and it loses its depth with probability 0.3. A plain-video sample that kept its motion therefore loses it with probability 0.25.

Most real windows come from Vimeo-90K [20], and only a small share has exact motion, as Section 8 reports **[Under revision]**. The configuration gives the cache capacities of 6,500 still crops and 3,600 windows.

S4.11 Unroll lengths and patch sizes

Each training window has $T = 6$ frames of $P = 192$ output pixels, which is 64 LR pixels at $s = 3$ and 96 at $s = 2$. The network runs through the frames of a window in order, the state starts empty, and the loss of every frame is summed before one backward pass. The gradient flows through every frame of the window, and it stops at the window boundary, which is truncated back-propagation through time [34]. The notebook gives six frames per window, as truncated back-propagation through the recurrence, and 192-pixel patches as its defaults. It sets 512-pixel texture crops so that fast camera paths have room inside the texture.

Every fourth step is a long step with $T = 16$ frames of $P = 96$ pixels, which is $16 \times 96^2 = 147{,}456$ output pixels per window, two thirds of the $6 \times 192^2 = 221{,}184$ of a normal window. The notebook gives the reason for the long steps as the decline of the earlier students on static scenes after frames 6 to 10, which the notebook measured in one earlier run **[Preliminary]**. The long steps use only synthetic windows, so each GPU has $48 - 19 = 29$ of them in a long step, against 48 windows in a normal step.

The batch is 48 windows per GPU for the teacher and for the students. The batch probe of the notebook tried sizes 4, 8, 12, 16, 24, 32 and 48, and it kept the largest size that stayed below 75 percent of GPU memory. Both models passed at 48 with memory use well below that limit, so the list, not the memory, set the batch. The main stages run on both GPUs with distributed data parallel, which gives 96 windows per step, and the paired runs and ablations run on one GPU each, with 48 windows per step.

S4.12 Loss

The loss is computed for every window and then averaged over the batch. For the frames $t = 0, \dots, T-1$ of a window, let $\hat{\mathbf{y}}_t$ be the output and $\mathbf{y}_t$ the target. The pixel term is a Charbonnier penalty, with $\varepsilon = 10^{-3}$, averaged over the frames with weight $\frac{1}{4}$ on the first frame, which has no history:

$$\mathcal{L}_{\text{pix}} = \frac{\sum_t \omega_t \, \mathbb{E}\sqrt{(\hat{\mathbf{y}}_t - \mathbf{y}_t)^2 + \varepsilon^2}}{\sum_t \omega_t}, \qquad \omega_0 = \tfrac{1}{4}, \;\; \omega_t = 1 \;\; (t \geq 1). \tag{S67}$$

The expectation $\mathbb{E}$ is the mean over the channels and pixels of the batch. The Charbonnier penalty [12] is a smooth edge-preserving potential that approximates the absolute difference.

The gradient term compares the finite differences of the output and the target over a distance of two pixels in each direction. With $D_x\mathbf{z}(X, Y) = \mathbf{z}(X+2, Y) - \mathbf{z}(X, Y)$ and D_y defined in the same way along Y,

$$\mathcal{L}_{\text{grad}} = \mathbb{E}_t\big|D_x\hat{\mathbf{y}}_t - D_x\mathbf{y}_t\big| + \mathbb{E}_t\big|D_y\hat{\mathbf{y}}_t - D_y\mathbf{y}_t\big|, \tag{S68}$$

where $\mathbb{E}_t$ is the mean over all frames of the window, the channels and the pixels.

The Fourier term compares the magnitudes of the two-dimensional spectra of the frames, computed with the orthonormal real FFT, so that texture energy at every frequency counts:

$$\mathcal{L}_{\text{fft}} = \mathbb{E}_t\Big|\,\big|\mathcal{F}\hat{\mathbf{y}}_t\big| - \big|\mathcal{F}\mathbf{y}_t\big|\,\Big|, \tag{S69}$$

where $\mathcal{F}$ is the orthonormal real two-dimensional FFT of each channel and $|\cdot|$ is the complex magnitude.

The temporal term matches the change of the output to the change of the target along the label motion, at HR, over the pixels that are visible in both frames:

$$\mathcal{L}_{\text{tmp}} = \frac{1}{T-1}\sum_{t=1}^{T-1} \frac{\sum_{\mathbf{p}} \bar{\mathbf{V}}_t(\mathbf{p})\,\lambda_t \sum_c \big|\Delta^{\text{out}}_{t,c}(\mathbf{p}) - \Delta^{\text{gt}}_{t,c}(\mathbf{p})\big|}{3\sum_{\mathbf{p}} \bar{\mathbf{V}}_t(\mathbf{p})\,\lambda_t + 10^{-6}}, \tag{S70}$$

where $\Delta^{\text{out}}_t = \hat{\mathbf{y}}_t - \mathcal{W}(\hat{\mathbf{y}}_{t-1}, \bar{\mathbf{M}}_t)$ and $\Delta^{\text{gt}}_t = \mathbf{y}_t - \mathcal{W}(\mathbf{y}_{t-1}, \bar{\mathbf{M}}_t)$ are the temporal changes of the output and of the target along the label motion, $\bar{\mathbf{M}}_t$ is the HR label motion, $\bar{\mathbf{V}}_t$ its validity and λ_t its label quality. The term is zero for windows without labels.

The motion term supervises the estimator on every frame after the first, with the LR label motion $\bar{\mathbf{m}}_t$ and its validity $\bar{\mathbf{v}}_t$:

$$\mathcal{L}_{\text{flow}} = \frac{\sum_{t=1}^{T-1}\sum_{\mathbf{p}} \lambda_t\,\bar{\mathbf{v}}_t(\mathbf{p})\,\big\|\hat{\mathbf{m}}_t(\mathbf{p}) - \bar{\mathbf{m}}_t(\mathbf{p})\big\|_1}{\sum_{t=1}^{T-1}\sum_{\mathbf{p}} \lambda_t\,\bar{\mathbf{v}}_t(\mathbf{p}) + 1}, \tag{S71}$$

where $\|\cdot\|_1$ sums the absolute differences of the two components, so the error is measured in LR pixels. The $+1$ in the denominator avoids a division by zero when no window has a valid label.

The perceptual term is LPIPS [14] with its AlexNet backbone on the last frame of the window, computed on the output clamped to $[0, 1]$:

$$\mathcal{L}_{\mathrm{lp}} = \mathbb{E}_{\mathrm{batch}}\,\mathrm{LPIPS}\big(\mathrm{clip}_{[0,1]}(\hat{\mathbf{y}}_{T-1}), \mathbf{y}_{T-1}\big). \tag{S72}$$

The distillation term $\mathcal{L}_{\mathrm{kd}}$ is the mean absolute difference between the output and the output of a frozen teacher on the same inputs. It is inactive (Section S4.15). The total loss is

$$\mathcal{L} = \mathcal{L}_{\mathrm{pix}} + 0.1\,\mathcal{L}_{\mathrm{grad}} + 0.05\,\mathcal{L}_{\mathrm{fft}} + 0.5\,\mathcal{L}_{\mathrm{tmp}} + 0.1\,\mathcal{L}_{\mathrm{flow}} + 0.05\,\mathcal{L}_{\mathrm{lp}} + 0 \cdot \mathcal{L}_{\mathrm{kd}}. \tag{S73}$$

The weights are the configuration values `w_grad`, `w_fft`, `w_temporal`, `w_flow`, `w_lpips` and `w_distill` of the run. The frames that each term uses are stated with its equation, and the validation metric is not part of the loss.

S4.13 Optimisation, schedule, precision, moving average and checkpoints

The optimiser is AdamW [10] with betas (0.9, 0.99), PyTorch's default $\epsilon = 10^{-8}$ and weight decay 0.0001, applied to every parameter. The peak learning rate is 0.002 for the students and 0.001 for the teacher. The learning rate follows the wall-clock budget. Let $\varrho = \tau / T_b$ be the fraction of the budget used, with $\varrho_w = 0.03$, α_0 the peak learning rate and $\alpha_{\min} = 0.000001$:

$$\alpha(\varrho) = \begin{cases} \alpha_0 \,\max\big(\varrho/\varrho_w,\ 0.01\big), & \varrho < \varrho_w, \\ \alpha_{\min} + \tfrac{1}{2}(\alpha_0 - \alpha_{\min})\Big(1 + \cos\Big(\pi\,\dfrac{\varrho - \varrho_w}{1 - \varrho_w}\Big)\Big), & \varrho \geq \varrho_w. \end{cases} \tag{S74}$$

The first 3 percent of the budget is a linear warm-up from $0.01\,\alpha_0$, and the rest is a single cosine decay without restarts, the decay used within each cycle of SGDR [11]. The rate is updated every 10 steps, together with the decision to stop and the validation check, because the processes agree on the elapsed time only at those points.

The gradients are unscaled and their global L_2 norm is clipped to 1 before the optimiser step. The forward pass of the network and of the teacher, when there is one, runs under FP16 autocast, and the losses are computed in FP32 [9]. A dynamic gradient scaler with PyTorch's default initial scale multiplies the loss before the backward pass, and it skips the step when a gradient is non-finite. A run whose loss stays non-finite for 200 steps stops with an error, so that a diverged run does not use its whole budget.

The moving average of the weights is the recursion below, which is this work's choice.

$$\theta_k^{\mathrm{EMA}} = d_k\,\theta_{k-1}^{\mathrm{EMA}} + (1 - d_k)\,\theta_k, \qquad d_k = \min\Big(0.999,\ \frac{1+k}{10+k}\Big), \tag{S75}$$

where $k \geq 1$ counts the updates of the average on rank 0. The warm-up rule $\min\big((1+k)/(10+k),\ \mathrm{decay}\big)$ is this code's choice, and it makes the averaging window $1/(1-$

$d_k) = (10 + k)/9$ updates at update k, until the cap 0.999 is reached at $k \approx 8{,}990$ updates. The average is therefore short for most of a run and reaches its full window only late in the run, and the validation and the checkpoints of Section S4.18 use the averaged weights.

Validation runs every $\min(8\text{ min},\ \max(T_b/6,\ 60\text{ s}))$ and once more at the end of the budget. It uses a fixed validation set: the first 32 still crops and the first 32 windows of the validation caches, in 4 batches of 8 still crops and 8 windows, each evaluated twice. The render copy keeps the engine data, and the plain-video copy degrades every sample with the probability 1 of Section S4.10. The score is the PSNR on luma (Eq. (S77)) over the frames 1 to $T-1$ of each window, with the s-pixel border removed, averaged over the windows of each copy, and the selection score is the mean of the two copies.

Each validation round saves the raw and the averaged weights. The averaged weights of the first round with the highest score are kept separately, and evaluation loads them.

The stage budgets are 170 minutes for the teacher, 190 minutes for the `low` student at 3×, 110 minutes for each of the paired `igpu` at 3× and `low` at 2× runs, and 30 minutes for each ablation. The seeds are the configuration seed 11 plus the rank for the model, and 100×11 plus the rank for the generator of the synthetic renderer, which keeps every rank's random streams apart. The data loader draws its windows from memory-mapped caches without replacement inside a batch, and each batch index selects its own random generator seed.

S4.14 Per-frame training loop

The training loop of `train_loop` and `train_tsr.step_fn` runs the following steps for every iteration on each rank.

1. Draw one batch from the memory-mapped caches: 19 real windows and 29 synthetic still crops on each GPU in a normal step. The indices are sorted and drawn without replacement within the batch.
2. Decide the step type. A step whose index is a multiple of four is a long step, with 29 synthetic windows of 16 frames at 96 pixels, and the teacher is not run. Otherwise the batch is 6-frame windows, with the real windows in the batch.
3. Build the synthetic windows with the renderer (`sr_batch`): motion class, jitter phase, exposure, particles, text, the LR jittered samples, the HR four-sample targets, the exact motion and validity at LR and HR, depth, and the reactive masks.
4. Build the real windows (`sr_batch_from_frames`): random flips with the motion components negated, the exposure gain in linear light, jittered bilinear reads of the HR frames, and the cached motion, validity and depth.
5. Degrade the plain-video shares: a synthetic window with probability 0.3, a real window with probability 0.6, with the filter, blur and noise of Eq. (S66). The degraded samples keep their motion labels with probability 0.15.
6. Withhold the engine data of the batch: motion with probability 0.25 and depth with probability 0.3, drawn once per sample.

7. Compute the teacher-forcing probability $\pi(\tau)$ of Eq. (S54) from the elapsed time, draw the forced mask for the samples with no motion and a label, and add it to the batch when $\pi > 0$.
8. Run the recurrent unroll under FP16 autocast. For each frame t in order: form the inputs of Table S18, run the estimator for every frame in training, select the motion of Eq. (S53), warp and fold the state, run the trunk and the heads, accumulate with Eq. (S62), refine with Eq. (S65), and keep the state for the next frame. The first frame starts from the reset state.
9. Compute the losses of Eqs. (S67) to (S72) in FP32 and their weighted sum Eq. (S73).
10. Scale the loss, run the backward pass through the whole unroll, unscale the gradients, clip their global norm to 1, and take one AdamW step. A non-finite gradient skips the step.
11. Update the moving average of the weights on rank 0 with Eq. (S75).
12. Every 10 steps, all-reduce the elapsed time, the stop flag and the memory counters. Set the learning rate by Eq. (S74), count the consecutive non-finite checks, and run the validation of Section S4.13 when the interval has passed or the budget has ended. Save the checkpoints and, when the validation score is a new best, the best averaged weights.
13. Stop when the budget is used, write the history of the rounds, and report the best round.

S4.15 Distillation

The training code contains a distillation path. A frozen teacher is unrolled on the same inputs without gradients, and its output enters the loss through Eq. (S73) with weight w_{distill}. No TSR run used it, because each run sets the teacher checkpoint to none and the weight to zero. The teacher of the article is therefore trained directly, as the quality ceiling, and not distilled. The earlier runs of 20 to 21 and 24 October 2026 recorded no measurable gain from distillation, which the notebook gives as the reason for switching it off **[Preliminary]**.

S4.16 Ablation configurations

Every ablation trains the `low` tier at 3× for 30 minutes, two at a time on the two GPUs, against a reference run of the same recipe and budget. Table S21 lists what each one changes and how the change is implemented. The reference is the full recipe at the same budget.

Each ablation validates on batches built with its own recipe, so validation scores are not comparable between ablations. The test protocol of Section S4.18 evaluates them all on the same suites. Its engine-mode evaluation of `abl_no_motion_net` is an exception. The evaluation sets the colour-only flag only for `abl_no_gbuffers` and the no-history flag only for `abl_no_history`. The engine-mode rows of `abl_no_motion_net` therefore receive depth and motion that the model was never trained with **[Under revision]**. The plain-mode rows of this ablation are not affected, because plain inputs have no engine data, and Table 3 uses only those rows.

Table S21: TSR ablation configurations. Each row changes one part of the `low` recipe at 3× and 30 minutes, against the reference

Name	Change	Implementation
`abl_reference`	None	Full recipe with the same 30-minute budget
`abl_no_history`	Single-frame upscaling: no history, no hidden state	`use_history=False`: every frame resets (Eq. (S58)), and unused parameters are allowed in DDP
`abl_no_gbuffers`	Colour-only render input: the estimator supplies the motion	`ablate_gbuffers`: depth, motion, depth availability and motion quality are zero in every frame
`abl_no_motion_net`	No motion estimator, colour-only input, zero motion	`motion_net=False` and `ablate_gbuffers`: no estimator and no motion loss
`abl_no_video_training`	Render-like samples only	Both video shares set to zero, the `p_video` keys of the configuration, and the withholding of Section S4.10 still applies
`abl_no_long_unroll`	Six-frame windows only	`long_every`= 0

S4.17 Fusion, precision and export

Fusion replaces every RepConv by a plain 3×3 convolution with the folded kernel and bias of Eq. (S59), followed by a copy of its PReLU. The rest of the network is unchanged. The motion estimator is built from ordinary convolutions and is not folded. The fused and unfused networks compute the same function up to floating-point error, and the notebook checks the difference on a synthetic batch of each scale and records the maximum absolute difference in FP32.

The evaluation and the latency measurements use the fused networks converted to FP16 weights, and the evaluation runs them under autocast. The latency is the mean over 100 timed forward passes of one fused FP16 step at batch 1, after 10 warm-up steps, with CUDA events and cuDNN autotuning. It is measured at 1080p and 2160p outputs for 3× and at 1080p for 2×. The arithmetic estimate in the notebook is a parameter, not a measurement. All latencies are eager PyTorch on one T4, and in-engine latency on target GPUs is not measured **[Under revision]**.

The ONNX export wraps one fused step with explicit state tensors. Its inputs are the colour, depth, motion, reactive mask, exposure, jitter, a flag vector of depth availability, motion quality and input kind, and the previous output, hidden state, depth, colour and jitter. Its outputs are the output and the new hidden state. The export uses opset 17 with the TorchScript exporter, static shapes for one LR size (the notebook uses 36×64 and the scale), and it is checked with ONNX Runtime on the CPU against PyTorch in FP32. A deployment must export one graph per render resolution.

The exported step has no reset input. The caller supplies the previous output for the first frame, and the graph then blends that output with the learned weight

a_t, instead of the zero weight of Eq. (S58). The notebook's export test runs a single frame and does not exercise the first frame of a sequence. An engine integration must therefore supply a valid previous output on the first frame, which the export does not check [Under revision].

S4.18 Test protocol

The test suites are built from the test split of the manifest, which the notebook loads before the evaluation and never uses for training. Table S22 lists them. A clip is the first T frames of an item, starting at frame 0, cropped at the centre to the crop size of its suite. The first $4n$ items of a source with at least T frames are tried, and a clip is skipped when it cannot be decoded or when one of its frames is blank, meaning a luma standard deviation below 2 on the 0 to 255 scale. A blank frame would be reconstructed exactly by every method and would score the PSNR cap of 100 dB. Clips with frozen frames or scene cuts are not filtered.

Table S22: TSR test suites. Motion labels are exact (renderer or exact flow) or RAFT. Each suite runs in both modes except GameIR, which runs in engine mode only

Suite	Source and frame size	Motion	T	Scales
`synthetic-game`	Game stills and DIV2K [18] test items, 1024-pixel texture crops, rendered at 384 output pixels, six sequences per motion class	Exact	16	3×, 2×
`tartanair`	TartanAir [22] test environments, 480×576	Exact (forward flow played backwards)	16	3×, 2×
`sintel`	MPI Sintel [21] test scene families, 432×1008	Exact (with occlusion masks)	16	3×, 2×
`vid4`	Vid4 [24] test sequences, 480×696	RAFT	16	3×, 2×
`vimeo1080p`	Pinned validation shards of Vimeo-1080p, which the manifest assigns to the test split, shorter side at most 540, crop 540×960	RAFT	16	3×, 2×
`gameir-native`	GameIR [23] test town, 720p native input and 1440×1920 target	RAFT, engine mode only	6	2×

In engine mode the network receives the jittered single-sample input of each suite, with the motion labels as motion vectors and the depth where the source has it. On the real video suites and GameIR the motion is RAFT-small [4] with its default weights, run for 12 iterations on the clean full-resolution frames, and it is kept where the forward-backward error is below one pixel plus five percent of the flow magnitude. RAFT sees the target frames rather than the jittered input, so engine mode on these suites is an oracle for motion, as the notebook notes [Under revision]. In plain mode the input is the anti-aliased bicubic downscale of the target, and the exact or RAFT

motion serves only as a metric label. The synthetic suite uses the same text-tile bank as training, generated with the same seed.

The baselines are jitter-aware bicubic and bilinear upsampling in both modes, the bilinear one being $\mathcal{U}$ of Eq. (S63), and the analytic TAA upscaler in engine mode only. TAA reprojects its history with the engine motion, clamps the history to the 3×3 YCoCg neighbourhood of the current bilinear upsample, and blends in 10 percent of the new frame. The networks are the teacher, `low` and `igpu` at $3\times$, the `low` student at $2\times$, and the six ablations at $3\times$, in both modes.

The static-convergence test is a separate run. It renders a camera at rest with sub-pixel jitter on the synthetic static class, with the first six test stills, sixteen frames and the seed 4242, in engine mode. It covers the bilinear and TAA baselines, the `low`, `igpu` and teacher networks, and the reference, no-long-unroll and no-history ablations. Its per-frame PSNR is averaged over the six sequences, and the FLIP error [15] of the last frame is reported.

S4.19 Metrics and aggregation

The metrics are computed for every frame $t \geq 1$ of every clip, because the first frame has no history. The luma of an RGB image $\mathbf{c} \in [0,1]^3$ is the limited-range value

$$Y(\mathbf{c}) = \frac{16 + 65.481\, c_R + 128.553\, c_G + 24.966\, c_B}{255}, \tag{S76}$$

which lies in $[16/255, 235/255]$. The peak-signal-to-noise ratio of luma is then

$$\mathrm{PSNR}_Y = 10 \log_{10} \frac{1}{\mathrm{MSE}}, \qquad \mathrm{MSE} = \frac{1}{|\Omega|} \sum_{\mathbf{p} \in \Omega} \big(Y(\hat{\mathbf{y}}_t)(\mathbf{p}) - Y(\mathbf{y}_t)(\mathbf{p}) \big)^2, \tag{S77}$$

where Ω is the set of output pixels with an s-pixel border removed on each side. The structural similarity (SSIM) [13] uses the same luma and border with an 11×11 Gaussian window of standard deviation 1.5. The detail PSNR uses the top 15 percent of the gradient magnitude of the target luma, which contains thin geometry, foliage and text. The disoccluded PSNR and the particle PSNR use the pixels with $\bar{\mathbf{V}}_t = 0$ and the pixels where the reactive mask, upsampled by nearest neighbour, exceeds 0.1, each when the batch holds more than 50 such pixels. This rule is tested over the whole batch and not per sample, so in a batch of several synthetic samples a sample without such pixels receives the cap of 100 dB **[Under revision]**.

The warping error measures the output alone along the exact label motion, and it is defined for $\lambda_t = 1$:

$$E_{\mathrm{w}}(t) = \frac{\sum_{\mathbf{p}} \bar{\mathbf{V}}_t(\mathbf{p})\, \frac{1}{3} \sum_c \big| \hat{\mathbf{y}}_t - \mathcal{W}(\hat{\mathbf{y}}_{t-1}, \bar{\mathbf{M}}_t) \big|_c (\mathbf{p})}{\sum_{\mathbf{p}} \bar{\mathbf{V}}_t(\mathbf{p})}. \tag{S78}$$

The temporal-change error compares the change of the output with the change of the target along the label motion, and it is defined for every label with $\lambda_t > 0$:

$$E_{\mathrm{tc}}(t) = \frac{\sum_{\mathbf{p}} \bar{\mathbf{V}}_t(\mathbf{p}) \, \frac{1}{3} \sum_c \left| \left(\hat{\mathbf{y}}_t - \mathcal{W}(\hat{\mathbf{y}}_{t-1}, \bar{\mathbf{M}}_t)\right) - \left(\mathbf{y}_t - \mathcal{W}(\mathbf{y}_{t-1}, \bar{\mathbf{M}}_t)\right) \right|_c (\mathbf{p})}{\sum_{\mathbf{p}} \bar{\mathbf{V}}_t(\mathbf{p})}. \quad \text{(S79)}$$

LPIPS [14] and the tOF of TecoGAN [17] are computed on frames $t \equiv 0 \pmod 4$ only, to limit the cost. The tOF is the mean norm, in pixels, of the difference between the RAFT flow from the output frame to its predecessor and the RAFT flow from the target to its predecessor.

Suite results average each metric over the rows of a suite, mode, scale and method where the metric is defined. Optional metrics therefore weigh each clip by its number of defined rows, and comparisons on single clips average the rows of each clip first.

S5 Temporal super-resolution: results

This section reports every output of the temporal super-resolution (TSR) notebook, `notebooks/neural-supersampling.py`, from its saved metric files, plots, predictions and logs. For each output it states what is measured, what it shows, the mechanism that the method (Section S4) suggests, and what it does not show. The notebook trained one set of networks in one run, from 3 November 2026 to 4 November 2026. Every number here is generated from the notebook's files by `tools/build_paper_results.py`, and every difference is computed by the same script. Each claim carries a status label as defined in Section S1.4. Because the saved outputs come from one run, a result that depends on training or evaluation noise is tagged **[Preliminary]**. Only deterministic results, such as cost counts, are tagged **[Confirmed]**.

S5.1 Protocol and reading rules

Test suites. The test suites are built from the test split only. Every suite is evaluated at 3x, and the 2x student is evaluated on the suites of Section S5.4.

Synthetic-game Layered scenes rendered from held-out game stills and DIV2K validation images [18] at 384×384 output, with six stills per motion class. The four classes are static, slow (0.3 to 3 output pixels per frame), medium (3 to 10) and fast (10 to 28), as defined in the notebook. Motion, disocclusion, thin geometry and particles are exact.
TartanAir Two held-out environments with exact optical flow and depth, at 576×480 [22].
Sintel The `market` and `cave` scene families, with exact flow, at 1008×432 [21].
Vid4 and Vimeo-1080p Real clips with RAFT motion [4], computed from the clean frames. Vid4 [24] has four clips. The Vimeo-1080p clips are held-out test clips of 1080p Vimeo videos[2] taken from the dataset's validation shard, which the pipeline assigns to the test split. Two blank clips were skipped.

[2]Hugging Face dataset `danjacobellis/vimeo1080p`, https://huggingface.co/datasets/danjacobellis/vimeo1080p, accessed 9 November 2026. No licence is stated.

GameIR native Held-out town 05 of the GameIR dataset [23], whose clips the notebook describes as CARLA and Unreal Engine 4 renders [25]. The 720p native render is the input and the 1440p render the target, so this is the only suite whose input is a real renderer's low-resolution image. Its clips have six frames, so the metrics use five frames per clip, and LPIPS and tOF use one frame per clip (frame 4).

Modes. In *engine* mode the input is a jittered single-sample render with motion vectors, and with depth where the suite has it. On the synthetic, TartanAir and Sintel suites the motion vectors are the exact motion of the scene. On Vid4, Vimeo-1080p and GameIR they come from RAFT on the clean full-resolution frames. This makes the motion of the video suites an oracle in engine mode, and the analytic baseline gets the same motion. In *plain* mode the input is a bicubic downscale with no jitter, depth or motion vectors, and the networks use their built-in motion estimator. The analytic TAA baseline needs motion vectors, so it is not run in plain mode.

Metrics. The definitions of the image and temporal metrics are in Section S2.9. The code computes the following for every frame. The tables average them over frames 1 to 15 of each 16-frame clip, so the first frame, which has no history, is skipped.

PSNR-Y and SSIM-Y Peak signal-to-noise ratio and structural similarity [13] on luma, with a border of one scale factor cropped on every side.
LPIPS The learned perceptual distance [14], computed on every fourth frame (frames 4, 8 and 12 of a 16-frame clip).
Disocc., Detail and Part. PSNR-Y inside the disoccluded pixels (where the history is invalid), inside the top 15 per cent of ground-truth gradient magnitude (thin geometry, foliage and text), and inside the particle mask. The particle mask exists only for the synthetic scenes, so the real suites show n/a.
Temp. The temporal-change error. It is the mean absolute difference between the change of the output and the change of the ground truth between consecutive frames, after warping by the ground-truth motion, over the pixels visible in both frames.
Warp The warping error. It is the mean absolute difference between each output and the previous output warped by the ground-truth motion. It is computed only where the motion is exact, so the RAFT-motion suites show n/a.
tOF The temporal optical-flow error [17]: the mean length, in pixels, of the difference between the RAFT flow of the output and the RAFT flow of the ground truth, on every fourth frame.

Higher is better for PSNR and SSIM, and lower is better for LPIPS, Temp., Warp and tOF.

Two properties of the metrics limit their reading. Temp. and Warp reward blur, because a smooth output changes little from frame to frame, so a stability metric is read together with PSNR-Y. The real suites average over four or eight clips, so differences of a few hundredths of a decibel, or a few thousandths of LPIPS, are not resolved.

S5.2 Training runs

Budgets and stage times. Table S23 compares the planned budgets in the configuration with the measured stage times. The planned budgets were not scaled down, so every stage received its planned time. The `igpu` 3x and `low` 2x students shared one stage, and the three ablation pairs ran one after another. [Confirmed] The run stayed inside its 10.8-hour ceiling. The notebook logs the total run time as 10.09 hours at the end of evaluation, and its summary cell records 10.13 hours, because the summary cell runs after the ONNX export.

Table S23: Planned and measured stage times of the TSR run, in minutes. The planned values come from the notebook configuration and the measured values from its stage timer. The ablations are three pairs of runs with a 30-minute budget each, and the caches stage had a 30-minute decoding budget

Stage	Planned	Measured
Teacher, both GPUs	170	170.5
`low` 3x student, both GPUs	190	190.3
`igpu` 3x and `low` 2x, in parallel	110	110.3
Ablations, three pairs	30 each	90.88
Training caches	30 (budget)	19.97
Batch probe	n/a	0.6127
Test suites and evaluation	n/a	2.682 and 19.21

Hardware and batch. The run had Tesla T4 GPUs with 14.56 GB each, 4 CPU cores, 31.3 GB of RAM, PyTorch 2.11.0+cu130 and CUDA 13.0. The batch probe ran six-frame training steps with a Charbonnier loss only, on GPU 0, for batch sizes from 4 to 48 sequences. It therefore omits the gradient, Fourier, temporal, motion and perceptual terms of the real loss, and the notebook counts this as the margin. Every tested size fitted. The teacher used 45.1 per cent of VRAM at batch 48, and the student 19.5 per cent. The teacher's throughput levelled off early, at 29.7 sequences per second at batch 32 and 29.9 at batch 48. The chosen batch of 48 is the largest size in the list, not a memory limit, because the 75 per cent VRAM rule never bound. [Observation] A larger batch would probably have fitted, but the probe did not test one.

Validation and failed runs. Table S24 gives the best validation round of every run. The checkpoint metric is the mean of the render and plain-video validation scores, which is the mean of the two middle columns. The validation batches are built with each run's own recipe, and the tiers differ in scale, so the validation values are comparable only within a group of the same scale and recipe. No run failed, and the failed-run record is empty.

Figure S18 shows the validation curves. The validation PSNR-Y of the teacher and of the `low` 3x student keeps rising over the budget, and the curves flatten at the end, where the cosine schedule ends. The 2x student has the highest validation score, as expected for the easier 2x task, so the scales are not compared directly. The temporal-change error of the students falls and then levels off, and the teacher's training loss

Table S24: Best validation round of every training run. PSNR-Y is the checkpoint metric (mean of the render and video copies). Last frame is the PSNR-Y of the last validation frame at the same round. Temp. err. is the validation temporal-change error. Samples/s and Peak GB are the mean throughput and the peak memory of the run. Values are comparable only within a scale and recipe

Run	Round	PSNR-Y	Render	Video	Last	Temp.	Samples/s	GB
Teacher 3x	20	33.47	33.43	33.51	33.63	0.0137	56.4	7.92
`low` 3x	23	32.49	32.28	32.70	32.58	0.0147	79.02	4.19
`igpu` 3x	13	31.62	31.23	32.02	31.72	0.0158	47.36	3.24
`low` 2x	13	36.39	36.44	36.34	36.46	0.0109	34.23	5.73
Reference	5	31.81	31.44	32.18	31.88	0.0157	40.24	4.18
No history	5	31.28	30.47	32.09	31.21	0.0233	47.37	3.34
No G-buf.	5	31.47	30.79	32.15	31.39	0.0194	40.18	4.19
No motion	5	31.33	30.68	31.98	31.23	0.0197	45.65	3.81
No video	5	31.06	31.69	30.44	31.17	0.0156	40.75	4.18
No unroll	5	31.88	31.51	32.25	31.85	0.0157	44.7	4.18

ends lowest. [Observation] The schedules ended while the models were still improving, so these numbers measure the budget as much as the architecture. The 30-minute ablations stop earlier, so the ablation results describe short-budget training.

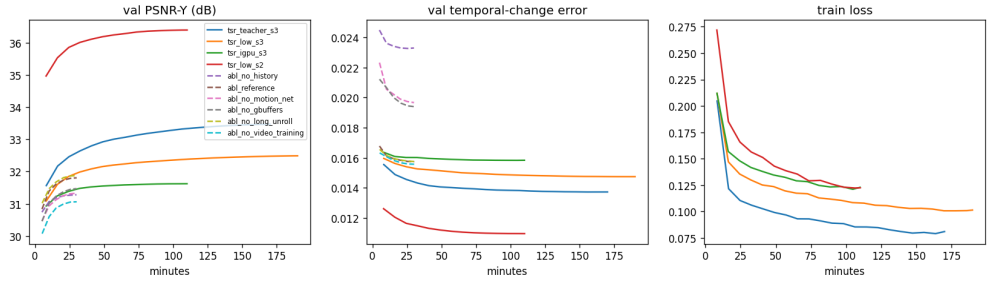


Fig. S18: Validation PSNR-Y (left), validation temporal-change error (middle) and training loss (right) against wall-clock minutes for the teacher, the main students, the 2x student and the six 30-minute ablation runs (dashed in the first two panels). The training loss is shown for the main stages only

S5.3 Main results at 3x

Tables S25 and S26 give every metric at 3x for each suite and mode. The rows are the analytic TAA baseline (engine mode only), the bicubic and bilinear upsamplers, the trained tiers and the teacher. The ablations are in Section S5.8, and Section S5.3.1 compares the suites.

Synthetic-game. In engine mode the `low` student reaches 34.27 dB PSNR-Y, which is 4.76 dB above bicubic and 4.91 dB above the analytic TAA baseline. The gain is largest in the hard regions. Disocclusion PSNR is 3.98 dB above TAA, and particle

Table S25: Synthetic-game suite at 3x. Means over per-frame rows (frames 1 to 15 of each clip). Engine mode: 360 rows per method. Plain mode: 360 rows per method. Disocc., Detail and Part.: PSNR-Y inside disoccluded, fine-detail and particle pixels. Temp.: temporal-change error. Warp: warping error. tOF: temporal optical-flow error in pixels. n/a: not computed for this suite.

Method	PSNR-Y	SSIM-Y	LPIPS	Disocc.	Detail	Part.	Temp.	tOF	Warp
Engine mode									
Bicubic	29.50	0.832	0.288	34.08	23.53	33.01	0.0321	0.8444	0.0338
Bilinear	29.80	0.833	0.273	34.47	23.87	33.13	0.0253	0.7819	0.0268
Analytic TAA	29.35	0.817	0.261	31.51	23.61	29.64	0.0110	0.7898	0.0092
`igpu`	32.72	0.907	0.097	34.25	26.64	33.94	0.0125	0.6064	0.0129
`low` student	34.27	0.931	0.058	35.48	28.17	35.19	0.0109	0.5295	0.0122
Teacher	35.55	0.947	0.041	36.41	29.43	37.17	0.0092	0.4031	0.0110
Plain mode									
Bicubic	31.23	0.850	0.260	35.95	25.60	n/a	0.0090	0.5882	0.0105
Bilinear	30.36	0.826	0.256	35.22	24.79	n/a	0.0090	0.5734	0.0100
`igpu`	32.65	0.888	0.101	36.92	27.07	n/a	0.0098	0.3874	0.0120
`low` student	33.42	0.904	0.082	37.72	27.85	n/a	0.0087	0.396	0.0111
Teacher	34.17	0.918	0.064	38.50	28.61	n/a	0.0077	0.3232	0.0104

Table S26: TartanAir suite at 3x. Means over per-frame rows (frames 1 to 15 of each clip). Engine mode: 120 rows per method. Plain mode: 120 rows per method. Disocc., Detail and Part.: PSNR-Y inside disoccluded, fine-detail and particle pixels. Temp.: temporal-change error. Warp: warping error. tOF: temporal optical-flow error in pixels. n/a: not computed for this suite.

Method	PSNR-Y	SSIM-Y	LPIPS	Disocc.	Detail	Part.	Temp.	tOF	Warp
Engine mode									
Bicubic	31.83	0.896	0.216	31.18	24.92	n/a	0.0149	0.4859	0.0138
Bilinear	31.52	0.890	0.216	30.85	24.64	n/a	0.0132	0.4852	0.0113
Analytic TAA	30.64	0.875	0.243	27.92	23.85	n/a	0.0104	0.415	0.0037
`igpu`	33.00	0.920	0.103	29.66	26.12	n/a	0.0105	0.3924	0.0052
`low` student	33.43	0.928	0.090	30.34	26.56	n/a	0.0103	0.3822	0.0051
Teacher	33.81	0.934	0.071	30.63	26.97	n/a	0.0101	0.3341	0.0050
Plain mode									
Bicubic	32.34	0.897	0.227	31.77	25.49	n/a	0.0103	0.3808	0.0073
Bilinear	31.55	0.884	0.227	31.02	24.78	n/a	0.0107	0.3787	0.0072
`igpu`	33.55	0.917	0.105	32.57	26.76	n/a	0.0106	0.4384	0.0080
`low` student	34.03	0.925	0.087	32.92	27.25	n/a	0.0102	0.3505	0.0075
Teacher	34.34	0.930	0.068	33.17	27.59	n/a	0.0100	0.3062	0.0076

PSNR is 5.55 dB above TAA. Fine-detail PSNR is 4.64 dB above bicubic. **[Preliminary]** The LPIPS of the student is 0.058 against 0.288 for bicubic, so its output is also perceptually closer to the target. The temporal-change error of the student is tied with TAA (0.0109 against 0.0110) and far below bicubic (0.0321). A plausible mechanism is that TAA keeps its output stable by blending a fixed share of history, which also blurs it, while bicubic has no history and reproduces the sub-pixel jitter of each frame

as flicker. The learned history weight keeps detail and stability together. [Observation] Figure S19 plots this combination for every method.

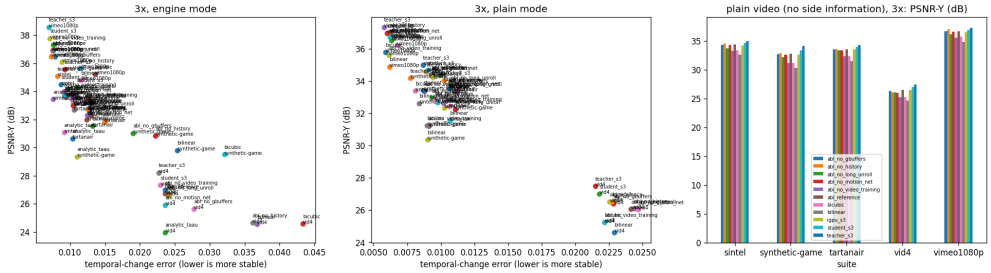


Fig. S19: Quality against stability at 3x. Left: engine mode, PSNR-Y (dB) against temporal-change error, one point per method and suite. Middle: the same for plain mode. Right: plain-video PSNR-Y of each method per suite. The scatter panels include the ablations, and their labels overlap

In plain mode the student reaches 33.42 dB, 2.19 dB above bicubic, and its LPIPS is 0.082 against 0.260. Its temporal-change error is slightly below bicubic's (0.0087 against 0.0090), but its warping error is slightly above it (0.0111 against 0.0105). [Observation] Without jitter the plain-video student gains little stability, and its warping error shows no gain.

TartanAir. The student reaches 33.43 dB in engine mode, 2.79 dB above TAA and 1.61 dB above bicubic. Its temporal-change error is 0.0103 against 0.0104 for TAA. In plain mode the student reaches 34.03 dB against 32.34 dB for bicubic, and its LPIPS is 0.087 against 0.227. [Preliminary] The gains are of the same order on TartanAir as on the synthetic scenes in plain mode, and the visual comparisons of Section S5.12 show where they occur on this suite.

Table S27 gives the Sintel suite in the same layout.

Table S28 gives the Vid4 suite in the same layout.

Table S27: Sintel suite at 3x. Means over per-frame rows (frames 1 to 15 of each clip). Engine mode: 120 rows per method. Plain mode: 120 rows per method. Disocc., Detail and Part.: PSNR-Y inside disoccluded, fine-detail and particle pixels. Temp.: temporal-change error. Warp: warping error. tOF: temporal optical-flow error in pixels. n/a: not computed for this suite.

Method	PSNR-Y	SSIM-Y	LPIPS	Disocc.	Detail	Part.	Temp.	tOF	Warp
Engine mode									
Bicubic	32.82	0.881	0.275	30.84	26.86	n/a	0.0148	1.564	0.0184
Bilinear	32.55	0.874	0.269	30.49	26.61	n/a	0.0127	1.703	0.0159
Analytic TAA	31.07	0.848	0.271	27.58	25.19	n/a	0.0092	1.519	0.0079
`igpu`	33.70	0.909	0.136	29.83	27.72	n/a	0.0093	1.463	0.0103
`low` student	34.48	0.921	0.107	30.76	28.52	n/a	0.0087	1.371	0.0100
Teacher	35.08	0.928	0.092	31.35	29.16	n/a	0.0082	1.343	0.0095
Plain mode									
Bicubic	33.38	0.880	0.281	31.37	27.55	n/a	0.0081	1.354	0.0109
Bilinear	32.61	0.865	0.274	30.55	26.82	n/a	0.0084	1.316	0.0107
`igpu`	34.23	0.898	0.125	32.30	28.62	n/a	0.0094	1.19	0.0124
`low` student	34.64	0.906	0.105	32.67	29.06	n/a	0.0090	1.112	0.0121
Teacher	34.99	0.912	0.084	32.98	29.46	n/a	0.0087	0.9462	0.0120

Table S28: Vid4 suite at 3x. Means over per-frame rows (frames 1 to 15 of each clip). Engine mode: 60 rows per method. Plain mode: 60 rows per method. Disocc., Detail and Part.: PSNR-Y inside disoccluded, fine-detail and particle pixels. Temp.: temporal-change error. Warp: warping error. tOF: temporal optical-flow error in pixels. n/a: not computed for this suite.

Method	PSNR-Y	SSIM-Y	LPIPS	Disocc.	Detail	Part.	Temp.	tOF	Warp
Engine mode									
Bicubic	24.58	0.738	0.391	22.33	19.60	n/a	0.0433	0.3981	n/a
Bilinear	24.55	0.722	0.362	22.32	19.71	n/a	0.0368	0.397	n/a
Analytic TAA	23.96	0.679	0.382	20.53	19.29	n/a	0.0236	0.1831	n/a
`igpu`	26.75	0.842	0.154	22.15	21.58	n/a	0.0236	0.1555	n/a
`low` student	27.32	0.863	0.118	22.56	22.06	n/a	0.0229	0.1479	n/a
Teacher	28.20	0.888	0.085	23.21	22.91	n/a	0.0227	0.1443	n/a
Plain mode									
Bicubic	25.24	0.728	0.400	23.10	20.58	n/a	0.0222	0.1455	n/a
Bilinear	24.62	0.695	0.362	22.43	20.05	n/a	0.0229	0.1512	n/a
`igpu`	26.52	0.806	0.166	23.94	21.82	n/a	0.0228	0.1349	n/a
`low` student	27.00	0.827	0.147	24.15	22.30	n/a	0.0218	0.1294	n/a
Teacher	27.48	0.847	0.119	24.57	22.83	n/a	0.0215	0.1214	n/a

Table S29 gives the Vimeo-1080p suite in the same layout.

Sintel. In engine mode the student reaches 34.48 dB, 3.41 dB above TAA. In plain mode it reaches 34.64 dB, 1.26 dB above bicubic. **[Preliminary]** The stability measures disagree on this suite. The plain-video temporal-change error of the student is higher than bicubic's (0.0090 against 0.0081), but its tOF is lower (1.112 against 1.354). The two measures sample the motion differently, so this suite does not support a claim that the student is more stable in plain mode.

Table S29: Vimeo-1080p suite at 3x. Means over per-frame rows (frames 1 to 15 of each clip). Engine mode: 120 rows per method. Plain mode: 120 rows per method. Disocc., Detail and Part.: PSNR-Y inside disoccluded, fine-detail and particle pixels. Temp.: temporal-change error. Warp: warping error. tOF: temporal optical-flow error in pixels. n/a: not computed for this suite.

Method	PSNR-Y	SSIM-Y	LPIPS	Disocc.	Detail	Part.	Temp.	tOF	Warp
Engine mode									
Bicubic	35.21	0.902	0.191	41.50	28.67	n/a	0.0136	4.077	n/a
Bilinear	34.80	0.899	0.189	40.65	28.19	n/a	0.0116	4.212	n/a
Analytic TAA	33.43	0.888	0.203	36.98	26.79	n/a	0.0076	6.128	n/a
`igpu`	36.88	0.938	0.074	40.64	30.43	n/a	0.0076	2.996	n/a
`low` student	37.77	0.945	0.052	41.98	31.35	n/a	0.0071	3.786	n/a
Teacher	38.57	0.952	0.035	43.04	32.21	n/a	0.0069	3.244	n/a
Plain mode									
Bicubic	35.77	0.902	0.205	41.87	29.28	n/a	0.0059	3.652	n/a
Bilinear	34.88	0.893	0.200	40.53	28.36	n/a	0.0062	3.409	n/a
`igpu`	36.50	0.918	0.090	42.43	30.16	n/a	0.0063	3.512	n/a
`low` student	36.96	0.923	0.075	43.21	30.73	n/a	0.0060	3.473	n/a
Teacher	37.33	0.927	0.062	43.79	31.20	n/a	0.0058	2.826	n/a

Vid4. In engine mode, where the motion comes from RAFT on the clean frames, the student reaches 27.32 dB, 2.73 dB above bicubic. In plain mode it reaches 27.00 dB against 25.24 dB, and its LPIPS is 0.147 against 0.400, the largest perceptual gain of the plain suites. [Observation] Figure S20 shows that much of the difference is fine texture, and that the texture the student adds on the cobblestones is a regular pattern, which the ground truth does not share exactly.

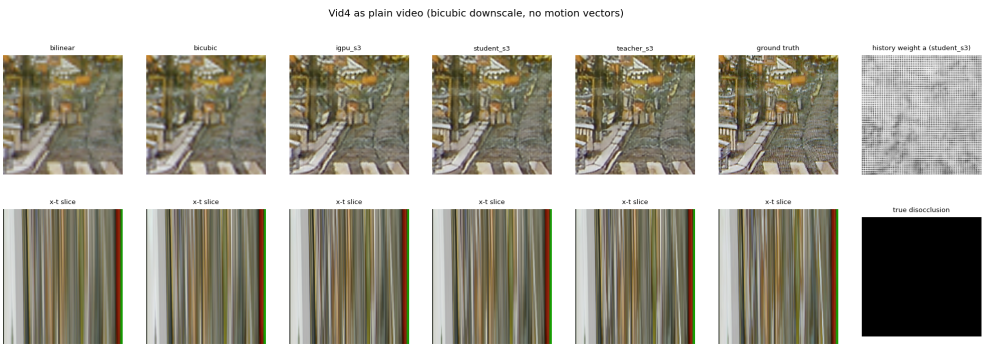


Fig. S20: Vid4 as plain video at 3x (bicubic downscale, no motion vectors), last-frame crops, history weight and x-t slices, in the layout of Fig. S21

Vimeo-1080p. In engine mode the student reaches 37.77 dB, 2.55 dB above bicubic, with RAFT motion as on Vid4. In plain mode the gain over bicubic is the smallest of all suites, 1.19 dB, while LPIPS falls from 0.205 to 0.075. [Preliminary] The plain-video temporal-change error of the student is marginally higher than bicubic's (0.0060

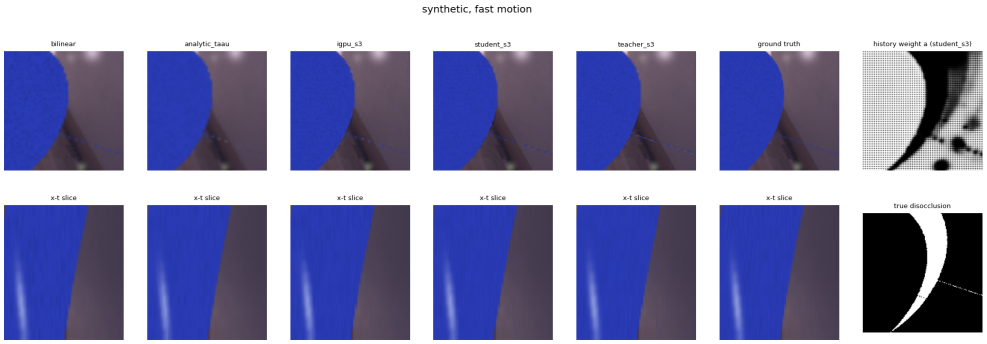


Fig. S21: Synthetic fast motion at 3x, last-frame crops. Top: bilinear, analytic TAA, `igpu`, `low` student, teacher and ground truth, then the history weight of the student. Bottom: x-t slices of each method and the true disocclusion mask

against 0.0059). Bicubic has the highest plain-mode PSNR-Y of the five suites, 35.77 dB, which leaves the least room for a learned upscaler. This is a plausible reason for the small gain, and this run does not test it.

S5.3.1 Cross-suite comparisons and the teacher

Plain-video gain over bicubic. The student beats bicubic in plain mode on all five 3x suites. The mean PSNR-Y gain ranges from 1.19 dB on Vimeo-1080p to 2.19 dB on the synthetic scenes. [Preliminary] The per-clip counts of the real suites, which are not written as keys, show that the student's clip mean is above bicubic's on every real clip except one, a Vimeo-1080p clip with the highest bicubic score of its suite.

Engine-mode gain over the analytic TAA baseline. The student beats TAA on each of the five 3x engine suites. The gap ranges from 2.79 dB on TartanAir to 4.91 dB on the synthetic scenes. TAA is below bicubic on each of the five engine suites, for example by 1.78 dB on Vimeo-1080p. [Observation] The baseline is therefore a weak reference, and the engine-mode margins are margins over this reference, not over a production temporal upscaler. A comparison with an engine's own TAA would be needed before these margins are used as a claim about deployed TAA.

Teacher as the quality ceiling. The teacher has the same architecture at 64 channels. It costs 156.1 GMAC per 1080p frame, 9.68 times the `low` student, and it is the best 3x model on every suite and mode. In engine mode it exceeds the student by 1.29 dB on the synthetic scenes and by 0.38 dB on TartanAir. In plain mode the margins are 0.74 dB and 0.31 dB. [Preliminary] The teacher is the quality ceiling of this architecture under this budget. Its margin over the student is smaller on the plain suites than on the engine suites, at about ten times the cost, so the teacher is not a deployment candidate.

The integrated-GPU tier. The `igpu` tier costs 0.38 times the `low` student's MAC count. On the synthetic scenes in plain mode it is 0.77 dB below the student and 1.42 dB above bicubic. [Preliminary] It keeps part of the student's gain at that cost. On

Vimeo-1080p in plain mode it scores 36.50 dB, which is 0.73 dB above bicubic and 0.46 dB below the student.

S5.3.2 Per-frame spread

Table S30 gives the spread of the per-frame PSNR-Y of the student and of bicubic. A clip mean hides the frames where a method fails, and a single bad frame is visible in a real-time renderer. [Preliminary] The worst frame of the student is above the worst frame of bicubic on every suite and mode in the table. The best frame is a different matter. On Vimeo-1080p the best frame of bicubic is above the best frame of the student in both modes, so the Vimeo gain is not in the sharpest frames. The spread between the best and worst frame is wide for both methods on every suite, because it follows the content of each clip, so the table supports a claim about the worst frames of this run, not about any one clip.

Table S30: Per-frame PSNR-Y (dB) at 3x: minimum, mean and maximum over the per-frame rows of each suite and mode (frames 1 to 15 of each clip), for the `low` student and bicubic. The minimum is the worst frame.

		`low` student			Bicubic		
Suite	Mode	Min	Mean	Max	Min	Mean	Max
Synthetic-game	Engine	24.03	34.27	52.93	21.67	29.50	51.25
Synthetic-game	Plain	24.78	33.42	52.31	24.14	31.23	51.20
TartanAir	Engine	28.41	33.43	37.99	25.95	31.83	36.32
TartanAir	Plain	28.28	34.03	38.39	27.11	32.34	36.66
Sintel	Engine	27.63	34.48	39.84	25.25	32.82	39.06
Sintel	Plain	28.09	34.64	40.63	26.37	33.38	39.37
Vid4	Engine	22.89	27.32	30.22	20.26	24.58	28.27
Vid4	Plain	22.62	27.00	30.28	21.32	25.24	28.43
Vimeo-1080p	Engine	24.43	37.77	53.25	24.06	35.21	54.16
Vimeo-1080p	Plain	25.70	36.96	53.70	25.47	35.77	54.07

S5.4 Results at 2x and on native renders

Table S31: Results at 2x: the `low` 2x student against bicubic, bilinear and analytic TAA (TAA in engine mode only). Means over per-frame rows. Disocc., Detail and Part.: PSNR-Y inside disoccluded, fine-detail and particle pixels. Temp.: temporal-change error. Warp: warping error. tOF: temporal optical-flow error in pixels. n/a: not computed for this suite.

Method	PSNR-Y	SSIM-Y	LPIPS	Disocc.	Detail	Part.	Temp.	tOF	Warp
Synthetic-game, engine mode									
Bicubic	33.41	0.919	0.126	37.61	27.28	37.06	0.0214	0.4659	0.0242
Bilinear	33.35	0.916	0.108	37.79	27.22	36.90	0.0172	0.43	0.0196
Analytic TAA	30.85	0.871	0.193	31.70	24.93	30.22	0.0083	0.6313	0.0060
`low` 2x student	38.43	0.970	0.024	39.20	32.21	39.47	0.0081	0.283	0.0108

Continued on next page

Table S31, continued

Method	PSNR-Y	SSIM-Y	LPIPS	Disocc.	Detail	Part.	Temp.	tOF	Warp
Synthetic-game, plain mode									
Bicubic	34.59	0.928	0.140	39.03	28.71	n/a	0.0071	0.2953	0.0097
Bilinear	32.95	0.900	0.168	37.72	27.09	n/a	0.0066	0.4008	0.0084
`low` 2x student	37.26	0.961	0.035	41.37	31.57	n/a	0.0064	0.2108	0.0098
TartanAir, engine mode									
Bicubic	34.71	0.939	0.115	33.92	27.75	n/a	0.0107	0.2445	0.0108
Bilinear	33.98	0.931	0.121	33.27	26.99	n/a	0.0100	0.2734	0.0091
Analytic TAA	31.44	0.896	0.212	28.04	24.54	n/a	0.0099	0.3471	0.0029
`low` 2x student	35.51	0.950	0.063	33.58	28.61	n/a	0.0091	0.3339	0.0051
TartanAir, plain mode									
Bicubic	34.86	0.939	0.136	34.22	27.91	n/a	0.0088	0.2303	0.0078
Bilinear	33.66	0.923	0.166	33.07	26.70	n/a	0.0088	0.2772	0.0065
`low` 2x student	36.65	0.956	0.047	35.65	29.94	n/a	0.0086	0.2385	0.0082
Sintel, engine mode									
Bicubic	36.37	0.941	0.139	34.36	30.38	n/a	0.0090	0.8291	0.0138
Bilinear	35.45	0.929	0.142	33.31	29.39	n/a	0.0083	0.8966	0.0125
Analytic TAA	31.70	0.875	0.220	27.73	25.59	n/a	0.0082	1.277	0.0067
`low` 2x student	37.62	0.957	0.068	34.43	31.72	n/a	0.0064	0.9437	0.0092
Sintel, plain mode									
Bicubic	36.39	0.938	0.158	34.38	30.44	n/a	0.0066	0.799	0.0108
Bilinear	35.01	0.918	0.188	32.88	28.98	n/a	0.0064	0.9686	0.0098
`low` 2x student	38.07	0.956	0.050	36.29	32.54	n/a	0.0069	0.7227	0.0117
Vid4, engine mode									
Bicubic	28.98	0.886	0.202	27.46	24.07	n/a	0.0253	0.15	n/a
Bilinear	27.78	0.854	0.191	25.90	22.90	n/a	0.0238	0.1614	n/a
Analytic TAA	25.00	0.752	0.347	21.01	20.17	n/a	0.0218	0.1382	n/a
`low` 2x student	30.76	0.934	0.058	27.00	25.42	n/a	0.0187	0.1119	n/a
Vid4, plain mode									
Bicubic	28.59	0.871	0.242	26.84	23.69	n/a	0.0191	0.0986	n/a
Bilinear	27.07	0.821	0.261	25.11	22.21	n/a	0.0193	0.1173	n/a
`low` 2x student	31.05	0.932	0.063	28.29	26.12	n/a	0.0181	0.0891	n/a
Vimeo-1080p, engine mode									
Bicubic	38.79	0.944	0.103	45.58	32.32	n/a	0.0093	1.937	n/a
Bilinear	37.78	0.937	0.106	44.09	31.15	n/a	0.0082	1.894	n/a
Analytic TAA	34.47	0.912	0.173	37.65	27.72	n/a	0.0067	2.766	n/a
`low` 2x student	41.59	0.972	0.022	46.28	35.22	n/a	0.0054	1.893	n/a
Vimeo-1080p, plain mode									
Bicubic	38.68	0.942	0.123	45.53	32.24	n/a	0.0047	1.835	n/a
Bilinear	37.32	0.928	0.144	43.78	30.70	n/a	0.0048	3.263	n/a
`low` 2x student	40.25	0.959	0.040	46.89	34.09	n/a	0.0046	1.914	n/a
GameIR native, engine mode									
Bicubic	32.33	0.912	0.169	34.69	25.27	n/a	0.0169	4.136	n/a
Bilinear	32.30	0.907	0.190	34.57	25.29	n/a	0.0163	4.377	n/a
Analytic TAA	30.83	0.879	0.227	32.39	23.97	n/a	0.0168	2.325	n/a
`low` 2x student	32.60	0.914	0.142	34.92	25.57	n/a	0.0158	4.344	n/a

Table S31 gives the 2x results. The `low` 2x student is compared with bicubic, bilinear and TAA, and the GameIR native suite is run in engine mode only. In plain mode the 2x student beats bicubic on every suite, by 1.57 dB on Vimeo-1080p to 2.67 dB on the synthetic scenes. **[Preliminary]** The plain-mode gain at 2x is larger than at 3x on every suite. A 2x input keeps more of the target's detail, which is a plausible reason, but the notebook does not test it.

GameIR native. The student reaches 32.60 dB on the native renders, against 32.33 dB for bicubic and 30.83 dB for TAA. The gain over bicubic is 0.27 dB, which is small. Its LPIPS is lower (0.142 against 0.169), but its tOF is higher (4.344 against 4.136). **[Preliminary]** This is the only real-renderer test, and it is the test closest to the intended input. It rests on four clips, one LPIPS frame per clip and RAFT motion from the clean frames. It supports the claim that the 2x student does not lose to bicubic on native renders, but it does not establish the size of the gain or any claim about stability.

S5.5 Quality against stability

Figure S19 plots each method's PSNR-Y against its temporal-change error for 3x engine mode (left) and plain mode (middle), and gives the plain-video PSNR-Y of each suite (right). A better method lies towards the top left, because it is sharper and more stable. The points of one suite lie close together, so the scatter

panels compare methods mostly within a suite. The labels overlap in the scatter panels, so the tables of Section S5.3 are the reference for any single point. **[Observation]** In engine mode the trained tiers and the teacher form a cluster at the top left. Bicubic and bilinear lie towards the right, which means less stable, and the analytic TAA lies low on the PSNR axis, which means stable but blurred. In plain mode the methods differ little in temporal-change error within a suite, and the trained models separate from the baselines mainly in PSNR.

S5.6 Motion regions and hard regions

Figure S22 splits the synthetic engine results by motion class (left), gives the PSNR-Y in the three hard regions (middle), and gives LPIPS and tOF (right). The hard-region values for the main methods are in Table S25. In every hard region the student is ahead of TAA and bicubic, and the teacher is ahead of the student. **[Preliminary]** The class means come from the per-frame file, which the generator does not yet aggregate by class, so the left panel is read qualitatively. **[Observation]** The class means are not a function of speed alone, because each class uses its own test textures. The fast class has the highest PSNR-Y for every method in the left panel, so the class differences mix speed and content. Within each class the student and the teacher are above the baselines. The gap to the baselines is largest in the static class in engine mode, where the history accumulates without the disturbance of motion.

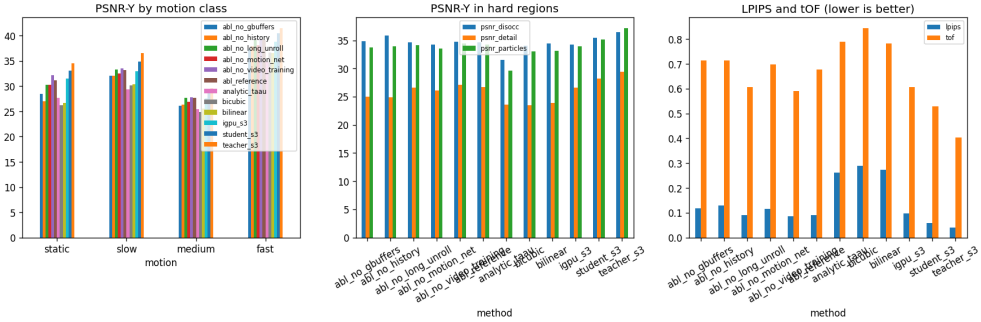


Fig. S22: Synthetic-game at 3x in engine mode. Left: PSNR-Y by motion class. Middle: PSNR-Y in disoccluded, fine-detail and particle regions. Right: LPIPS and tOF. The bars are means over the test stills, and lower is better for LPIPS and tOF

Disocclusions. A disoccluded pixel has no valid history, so a temporal network must reject the history there. The student's disocclusion PSNR is 1.40 dB above bicubic, a smaller gap than its 4.64 dB gap in the fine-detail region. **[Observation]** The history weight in Fig. S21 shows the mechanism, because it drops to black along the leading edge of the moving object, where the true disocclusion lies.

S5.7 Static-scene convergence

With a static camera and sub-pixel jitter, each frame adds new sub-pixel samples, so a temporal network can in principle reconstruct the native image as the sequence continues. Figure S23 and Table S32 show PSNR-Y against frame index for the static-camera methods. Single-frame and fixed-rule methods stay flat. **[Preliminary]** The student rises over the 16 frames, and the teacher rises faster and to a higher level. The 30-minute reference rises to about frame 6 and then declines slightly. Without the 16-frame unrolls the curve peaks earlier and falls well below its peak by frame 15, and its last-frame value is the lowest of the recurrent models. The single-frame ablation stays near the level of TAA, which means that the gain over frame 0 comes from the history and not from the network's single-frame upscaling. This test uses engine input only, so plain-video convergence was not measured.

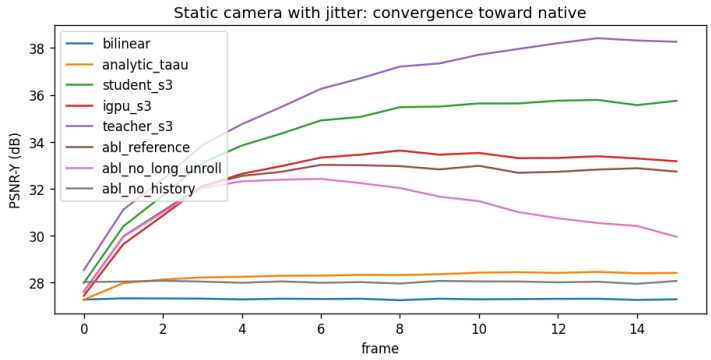


Fig. S23: Static camera with sub-pixel jitter, synthetic-game, engine mode at 3x: PSNR-Y against frame index for the baselines, the trained tiers and three 30-minute ablations

Table S32: Static camera with sub-pixel jitter at 3x (synthetic-game, engine mode): PSNR-Y and FLIP of the last frame (index 15) of the convergence sequence. Lower FLIP is better

Method	PSNR-Y (dB)	FLIP
Bilinear	27.29	0.068
Analytic TAA	28.42	0.062
`igpu`	33.17	0.033
`low` student	35.75	0.025
Teacher	38.26	0.020
Ablation reference	32.73	0.036
Ablation: no long unrolls	29.96	0.046
Ablation: no history	28.08	0.055

Interpretation. On the last frame of the convergence sequence the student is 7.33 dB above TAA, and its FLIP error [15] is 0.025 against 0.062 for TAA. **[Observation]** The FLIP ordering agrees with the PSNR ordering. A single static sequence of six stills is not enough to establish a convergence rate. The decline of the model without unrolls after its peak is the motivation for the 16-frame unrolls of training, and the reference with unrolls declines much less. Both results come from one run, so the size of each difference is **[Preliminary]**.

S5.8 Ablations

Each ablation changes one part of the reference recipe and trains for 30 minutes, the same budget as the reference. The removed components are the history (single-frame upscaling), the G-buffers (depth, motion vectors and the availability flags of the render input, so that only colour remains), the built-in motion estimator, the video samples of training and the 16-frame unrolls. Table S33 gives the PSNR-Y of every run on every suite and mode at 3x, and Table S34 gives the differences to the reference. Figure S24 plots the differences. The ablations are short runs, so they are read against the reference and not against the main student, and differences of a few tenths of a decibel need the noise estimate given below.

Table S33: Ablations at 3x: PSNR-Y (dB) of every run on every suite and mode. Each run is a 30-minute run of the `low` recipe. E: engine mode, P: plain mode. Syn: synthetic-game, Tar: TartanAir, Sin: Sintel, Vid4, Vim: Vimeo-1080p. Means over per-frame rows

Run	Syn E	Syn P	Tar E	Tar P	Sin E	Sin P	Vid4 E	Vid4 P	Vim E	Vim P
Reference	32.78	32.79	33.02	33.59	33.71	34.38	26.69	26.50	36.92	36.72
No history	30.84	32.92	32.31	33.59	33.27	34.56	24.65	26.08	35.65	37.02
No G-buffers	31.03	32.74	32.29	33.57	33.16	34.35	25.63	26.39	36.09	36.67
No motion net	31.98	32.63	32.67	33.38	33.59	34.36	25.92	26.04	36.49	36.66
No video training	33.23	31.21	33.18	32.41	33.97	33.34	26.98	25.25	37.30	35.59
No long unrolls	32.60	32.23	32.99	33.36	33.70	33.75	26.61	26.07	36.48	36.21

Table S34: Ablations at 3x: PSNR-Y change (dB) of each run against the reference on every suite and mode, from the same runs as the previous table. Negative values mean that removing the component costs quality. E: engine mode, P: plain mode. Syn: synthetic-game, Tar: TartanAir, Sin: Sintel, Vid4, Vim: Vimeo-1080p.

Run	Syn E	Syn P	Tar E	Tar P	Sin E	Sin P	Vid4 E	Vid4 P	Vim E	Vim P
No history	−1.94	+0.12	−0.71	−0.01	−0.43	+0.17	−2.04	−0.43	−1.27	+0.30
No G-buffers	−1.75	−0.05	−0.72	−0.03	−0.55	−0.04	−1.06	−0.12	−0.82	−0.05
No motion net	−0.80	−0.16	−0.35	−0.21	−0.11	−0.03	−0.77	−0.46	−0.43	−0.06
No video training	+0.45	−1.58	+0.17	−1.18	+0.26	−1.04	+0.29	−1.26	+0.38	−1.12
No long unrolls	−0.18	−0.56	−0.02	−0.23	−0.01	−0.64	−0.08	−0.44	−0.44	−0.51

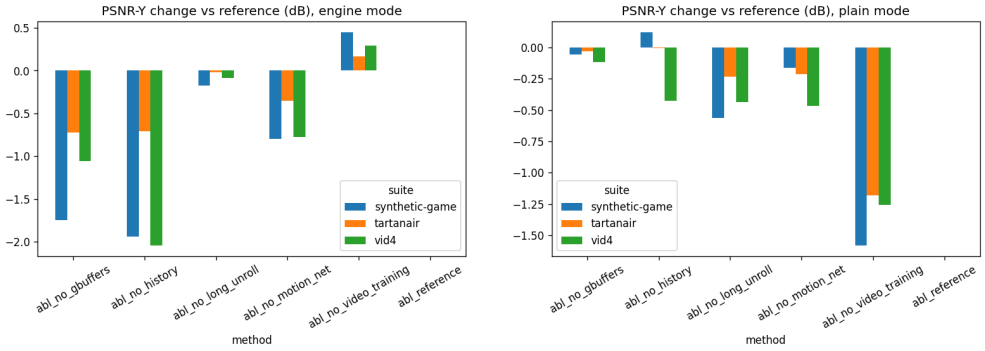


Fig. S24: PSNR-Y change of each ablation against the 30-minute reference (dB), per suite, for engine mode (left) and plain mode (right)

History. Removing the history costs the most in engine mode, −1.94 dB on the synthetic scenes and −2.04 dB on Vid4. Its temporal-change error on the synthetic scenes rises from 0.0126 to 0.0222. **[Preliminary]** In plain mode the same removal changes little on the synthetic scenes (+0.12 dB). Plain frames carry no new sub-pixel samples for the history to accumulate, so this is the expected result. **[Observation]** The history helps when the input carries sub-pixel jitter, which is the mechanism of Section S5.7.

G-buffers. In engine mode the colour-only input costs −1.75 dB on the synthetic scenes, the second largest engine effect there after the history. Plain inputs have no G-buffers, so the plain-mode differences of the same ablation come only from training. They are 0.05 dB on the synthetic scenes and 0.12 dB on Vid4, and below 0.03 dB on TartanAir. **[Observation]** These differences are a rough estimate of the spread between two runs of this recipe. They are smaller than the 0.3 dB run-to-run spread quoted in Section S1.4, but one pair of runs cannot give a reliable estimate.

Motion estimator. In plain mode, removing the estimator costs −0.16 dB on the synthetic scenes and −0.46 dB on Vid4. Without the estimator the plain input carries no motion, so the history is warped with the zero motion vectors of the plain input, which is consistent with the loss. **[Preliminary]** **[Under revision]** The engine-mode rows of this ablation are not a clean test. Training removes the G-buffers for this run (the `ABLATIONS` entry sets `ablate_gbuffers`), but the notebook's evaluation sets that flag only for the G-buffer ablation. The estimator run is therefore evaluated with depth and motion vectors that its training never showed, and its engine-mode differences mix the two changes. The flag must be set for this run in evaluation before its engine-mode value is used.

Video training. Training without the video samples costs −1.58 dB on the synthetic scenes and −1.26 dB on Vid4 in plain mode, the largest plain-mode effect of the study. In engine mode the render-only model is 0.45 dB higher on the synthetic scenes. **[Preliminary]** The video samples teach the network how ordinary video behaves, and in this run they cost render-input quality at this budget. The article's trade-off is therefore visible here, and a larger video share or a longer budget could change it.

Long unrolls. Removing the 16-frame unrolls costs −0.56 dB on the synthetic scenes in plain mode and −0.18 dB in engine mode. The static-scene decline of Fig. S23 is the effect the unrolls target. **[Preliminary]** The plain-mode loss is larger than the engine-mode loss on the synthetic scenes, and the engine-mode effect is small on every suite.

S5.9 Model tiers, cost and latency

Table S35: Network tiers. Fused parameters at 3x, multiply-accumulates (GMAC) per 1080p output frame at 3x and 2x (motion estimator running), and the largest difference between the multi-branch and the fused network on random input. Trained: whether the tier was trained in this run. The parity values are reported as produced and are not established (see the text)

Tier	Trained	Fused params (3x)	GMAC 3x	GMAC 2x	Parity (3x)
`igpu`	Yes	138K	6.2	11.3	0
`low`	Yes	328K	16.1	30.7	0
`mid`	No	653K	28.1	56.3	0
`high`	No	1.41M	69.0	145.0	0
Teacher	Yes	3.40M	156.1	337.4	0

Table S35 lists the tiers. The parameter counts are the trainable and the fused networks. The multiply-accumulate counts (MAC, Section S2.6) are per steady-state frame, with the motion estimator running, at 1080p output for 2x and 3x. **[Confirmed]** These counts are properties of the code, so they do not vary between runs. The `mid` and `high` tiers were not trained in this run, so they have costs but no quality or latency. The parity column is the largest difference between the multi-branch and the fused networks on random input. **[Under revision]** It is exactly zero for every tier, which is not expected for a fold of convolutions in FP32. The notebook's fusion function copies the network before it swaps the re-parameterised blocks, so the two sides are different graphs, but the cause of the exact zero is not established, and fusion parity is therefore not claimed here. Figure S25 plots the cost of each tier at both scales.

Table S36 gives the measured T4 times of the trained networks, with and without engine motion vectors, next to an arithmetic estimate. The timed step includes the warps, the jitter-aware upsampling and the final blend, at batch 1, in FP16 with cuDNN autotuning and CUDA events after warm-up. **[Under revision]** These are eager PyTorch times on a T4. They rank the networks, but they are not the latency of a deployment runtime on a target GPU, which needs the ONNX or DirectML path. The arithmetic estimate

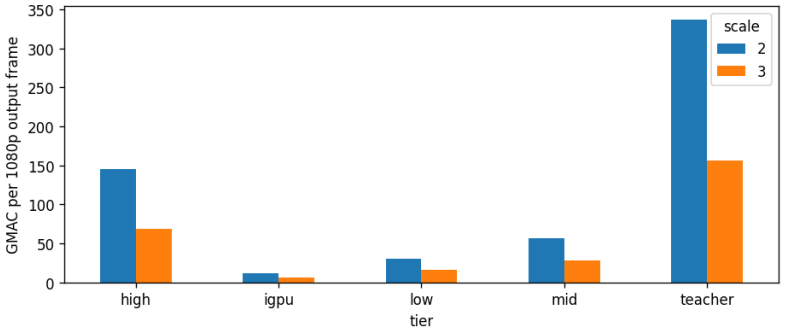


Fig. S25: Multiply-accumulate operations per 1080p output frame of each tier at 2x and 3x

converts each MAC count to milliseconds at an assumed sustained throughput of 4 or 8 TFLOPS. It is a parameter, not a measurement, and it is not a bound. The teacher at 1080p runs in 58.7 ms, faster than its estimate at 4 TFLOPS of 78.0 ms. The `igpu` tier runs in 16.3 ms, far slower than its estimate of 3.1 ms. Small networks are limited by per-operation overhead and memory traffic in eager mode, while the teacher's larger convolutions use the GPU better.

Table S36: Measured T4 time of the fused FP16 networks per output frame (eager PyTorch, batch 1, mean and 95th percentile of 100 steps) with the motion estimator running (plain video) and with engine motion vectors (estimator skipped). Mean, MV is the mean time with engine motion vectors. Est. 4 and Est. 8 are arithmetic estimates in milliseconds at an assumed sustained throughput of 4 and 8 TFLOPS. They are parameters, not measurements

Model	Output	Mean	P95	Mean, MV	GMAC	Est. 4	Est. 8
`igpu` 3x	1080p	16.3	16.7	10.0	6.2	3.1	1.6
`igpu` 3x	4K	61.5	62.7	40.5	24.8	12.4	6.2
`low` 3x	1080p	27.7	28.5	21.0	16.1	8.1	4.0
`low` 3x	4K	110.6	112.3	83.4	64.5	32.3	16.1
`low` 2x	1080p	42.6	43.0	29.8	30.7	15.3	7.7
Teacher 3x	1080p	58.7	59.5	50.1	156.1	78.0	39.0
Teacher 3x	4K	240.9	246.4	220.2	624.4	312.2	156.1

Figure S26 shows the measured times. With engine motion vectors the estimator is skipped, and the time falls from 16.3 ms to 10.0 ms on the `igpu` tier, and from 27.7 ms to 21.0 ms for the `low` student. The 4K time of the `igpu` tier is 3.78 times its 1080p time, against four times the pixels.

Table S37 compares the trained tiers and the teacher on cost and quality, on the same test sets. **[Preliminary]** The `low` student costs 2.60 times the MAC count of the `igpu` tier, and 27.7 ms against 16.3 ms. Its gain over the `igpu` tier is +0.77 dB on the synthetic scenes in plain mode and +0.48 dB on Vid4. The teacher costs 9.68 times the MAC count of the student for +0.74 dB on the synthetic scenes, and for +0.37 dB on Vimeo-1080p. **[Observation]** The `igpu` tier is the cheapest trained tier, and its gain over bicubic on the real plain suites is smaller than the student's, so choosing it saves cost at a visible loss of quality.

ONNX export. The `low` 3x student and the `igpu` tier were exported as one fused recurrent step, with explicit state tensors, static shapes and opset 17, and were checked with ONNX Runtime on the CPU against PyTorch in FP32. Table S38 gives the results. **[Confirmed]** The maximum difference is 0.00002792 for the student and 0.00001484 for the `igpu` tier, so the exported graphs compute the PyTorch function on CPU to within floating-point rounding. The export was checked only on CPU in FP32, so the parity of FP16 kernels in a GPU runtime is not verified here.

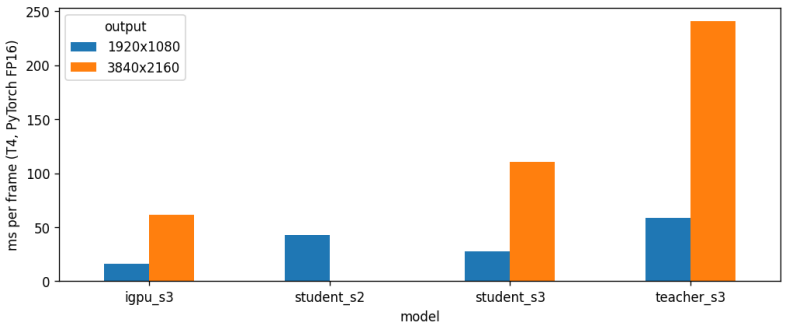


Fig. S26: Measured T4 time of the fused FP16 networks per output frame at 1080p and 4K, with the built-in motion estimator running (plain video)

Table S37: Cost against quality at 3x. MAC per 1080p frame and measured T4 time (plain video, 1080p) against PSNR-Y (dB) on four suites and modes. The bicubic row has no network and therefore no cost

Method	GMAC	T4 ms	Synthetic, engine	Synthetic, plain	Vid4, plain	Vimeo-1080p, plain
Bicubic	n/a	n/a	29.50	31.23	25.24	35.77
`igpu`	6.2	16.3	32.72	32.65	26.52	36.50
`low` student	16.1	27.7	34.27	33.42	27.00	36.96
Teacher	156.1	58.7	35.55	34.17	27.48	37.33

Table S38: ONNX export of the fused recurrent step: file size and the largest difference between ONNX Runtime (CPU) and PyTorch (FP32) on the export input

Model	File size (MB)	Max difference
`low` 3x student	1.475	0.00002792
`igpu` 3x	0.706	0.00001484

S5.10 Training caches and data

The notebook decoded the training data once into memory-mapped caches, so that both training processes read the same files through the page cache. Table S39 gives the counts. The still caches hold crops of high-resolution textures for the synthetic renderer, and the window caches hold six-frame windows with motion, validity and depth at half resolution. The caches took 19.97 minutes and occupy 9.3 GB, which the notebook deletes before packaging the output. Most training windows come from Vimeo-90K septuplets [20], 3,267 of 3,600 windows, and the other sources are small by comparison. The notebook also prints that almost all training windows carry estimated motion from RAFT, with a small share of exact motion. That share is not written as a key, so it is not generated here.

Table S39: Training and validation caches: crops and windows, and the windows by source. Windows of the video sources carry RAFT motion. n/a: the source has no validation windows

Item	Training	Validation
Still crops (512 pixels)	6,500	256
Windows (6 frames at 192 pixels)	3,600	192
Vimeo-90K septuplets windows	3,267	188
REDS windows	149	4
Vimeo-1080p windows	91	n/a
GameIR windows	26	n/a
TartanAir windows	39	n/a
Sintel windows	28	n/a
Cache size on disk (GB)		9.3

Figure S27 shows random still crops, the textures of the synthetic renderer, and Fig. S28 shows four windows of the sequence cache. Each window panel gives the first and last frame, the motion vectors of the last frame in the notebook's colour wheel (hue is direction, brightness is magnitude) and the validity of the last frame.

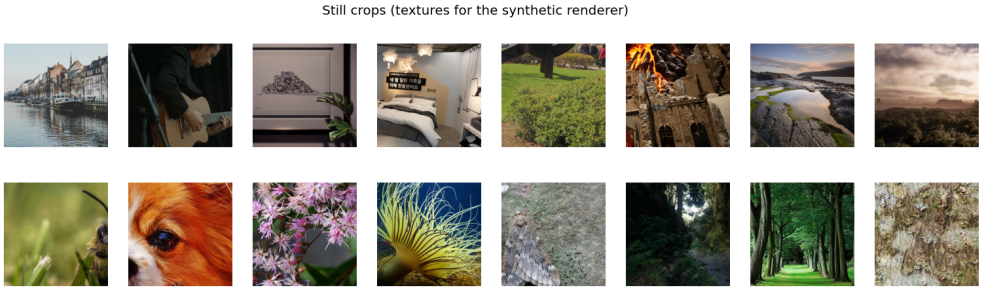


Fig. S27: Random still crops of the training cache, the textures of the synthetic renderer

Figure S27 shows textures with a wide range of detail, from photographs and paintings to rendered scenes. In Fig. S28, the motion fields of three windows are close to uniform, and the third window shows a boundary between two motions. **[Observation]** The validity maps of the first two windows are black over the whole frame, which the notebook's legend reads as disoccluded or out of view, so no pixel of those windows is trusted. The temporal-change loss of training is weighted by the same validity map (`train_tsr.py`), and the warping and temporal-change metrics are normalised by the valid area, so such a window contributes nothing to them. The third and fourth windows are valid except for small patches. The share of fully invalid windows is not in the saved outputs. Whether this is a true failure of the forward-backward check on these clips or a fault in the cache is not decided here, and it must be checked on the cache before the next run.

S5.11 Synthetic renderer preview

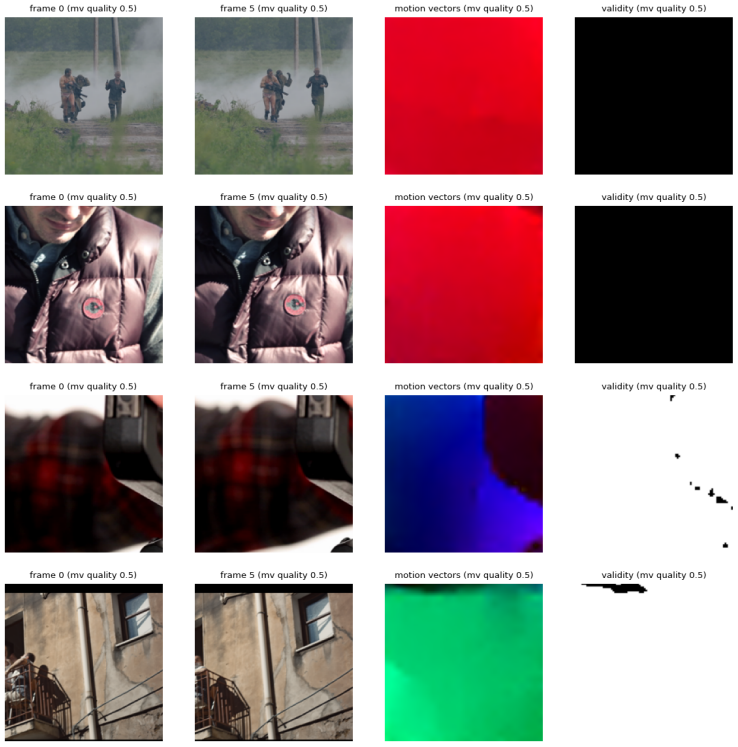


Fig. S28: Four windows of the sequence cache: first and last frame, motion vectors of the last frame (hue is direction, brightness is magnitude) and validity of the last frame (black is invalid)

Figure S29 shows one synthetic training sequence at three frames. The jittered render input, enlarged by nearest neighbour, shows the aliasing of thin geometry, which the renderer's sub-pixel jitter is meant to let the network resolve. The plain-video input is the same frame downscaled with anti-aliasing, blur and noise, and it has no such aliasing. The target is four-sample anti-aliased. The motion-vector panel uses the colour wheel. The valid-history panel is black along the wire and fence lines and around the moving object. The depth panel separates the near and far layers, and the reactive panel shows the glow of a particle, which writes no motion vector. **[Observation]** The preview confirms that the disocclusions, the thin geometry and the particles of Section S5.6 are present in the training data.

S5.12 Visual comparisons

Each visual comparison shows the last frame of each method as a crop, the history weight of the `low` student, and an x-t slice, which is one image row over time. The x-t slice shows temporal flicker and ghosting as streaks. The notebook chose these clips, so they show chosen content and not a sample of the test set.

Figure S21 shows a fast-motion synthetic sequence. The blue object on the left moves across the frame, and its edge is where the valid history is rejected. Bilinear and TAA blur the object's edge and the thin dashed wire in the lower right. The `igpu` tier, the student and the teacher keep the edge sharp. The teacher draws the wire most continuously, and the student nearly as well. The history-weight panel is white over the static background and black in the crescent along the leading edge of the object, which matches the true disocclusion panel. A regular dot grid is visible in the weight map. It comes from the pixel-shuffle unfolding of the per-pixel weights (Section S4), and the output refinement removes it from the image. **[Observation]** The x-t slices of all methods look alike, with no obvious extra streaks in the learned outputs, which agrees with the low temporal-change error of the student on this clip.

Figure S30 shows a TartanAir environment in engine mode. The scene is dark, with a red sign at the top right and lit windows and lamps. Bilinear and TAA are soft throughout. The learned outputs keep

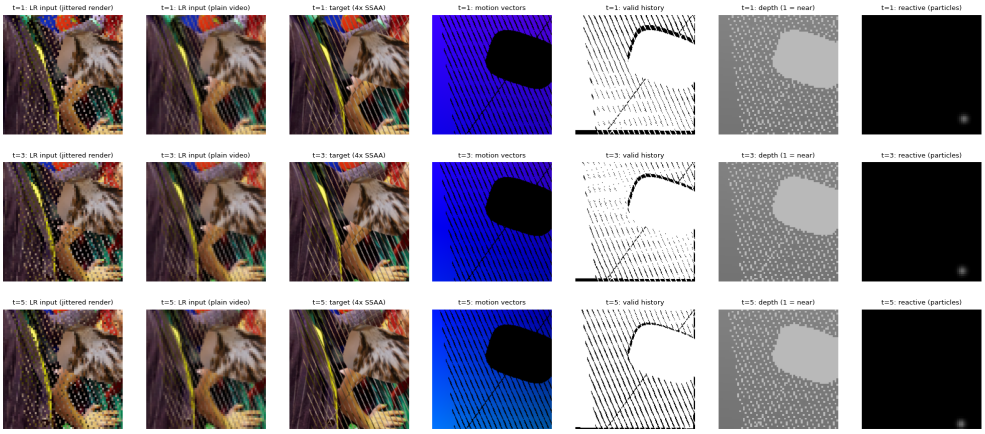


Fig. S29: One synthetic training sequence at three frames (rows): render input (jittered, nearest-neighbour enlarged), plain-video input, target, motion vectors, valid history, depth and reactive mask

the window frames and the edges of the lit sign more sharply. The history weight is dark along the strong building edges, with sparse dark dots elsewhere, and is white over most of the frame. **[Observation]** The x-t slice shows the vertical structure of the sign and the lamps, with no visible trail that the history might have smeared.

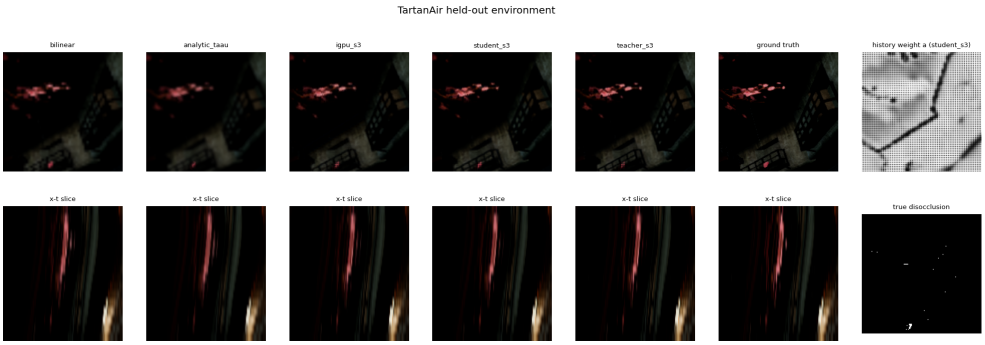
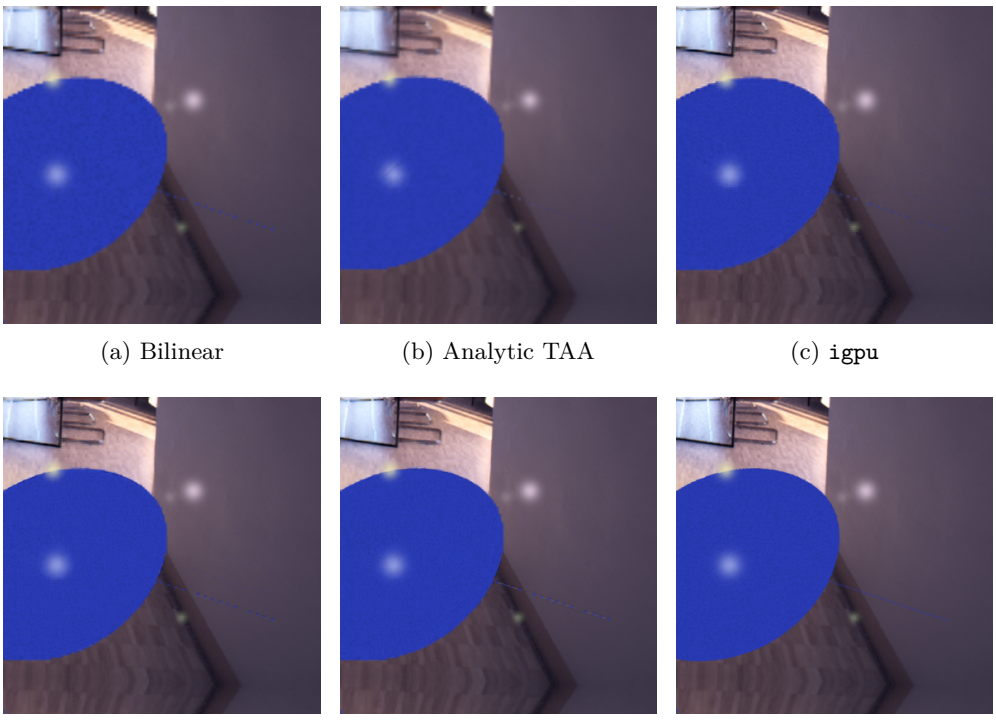


Fig. S30: TartanAir held-out environment at 3x in engine mode, last-frame crops, history weight and x-t slices, in the layout of Fig. S21

Figure S20 shows Vid4 as plain video. Bicubic is soft on the signs and the building edges. The learned outputs sharpen the sign lettering and the awning stripes, as in Fig. 3 of the article. On the cobblestones the `igpu` tier, the student and the teacher add a fine texture that bicubic lacks. It is a regular pattern, not a copy of the ground truth, whose texture is irregular. The history weight shows a dot texture rather than a disocclusion structure, and the true disocclusion panel is empty for this clip. The x-t slices of the methods are close to one another, with no visible flicker. **[Observation]** The cobblestone texture is the clearest difference between the methods on this clip.

The full predicted frames of each scene are shown below, one panel per method, with the ground truth last. Figure S31 shows the synthetic frame. The largest visible differences lie along the blue object's edge and along the dashed wire in the lower right, and the background is similar across the methods. The learned outputs keep the wire more clearly than bilinear and TAA.

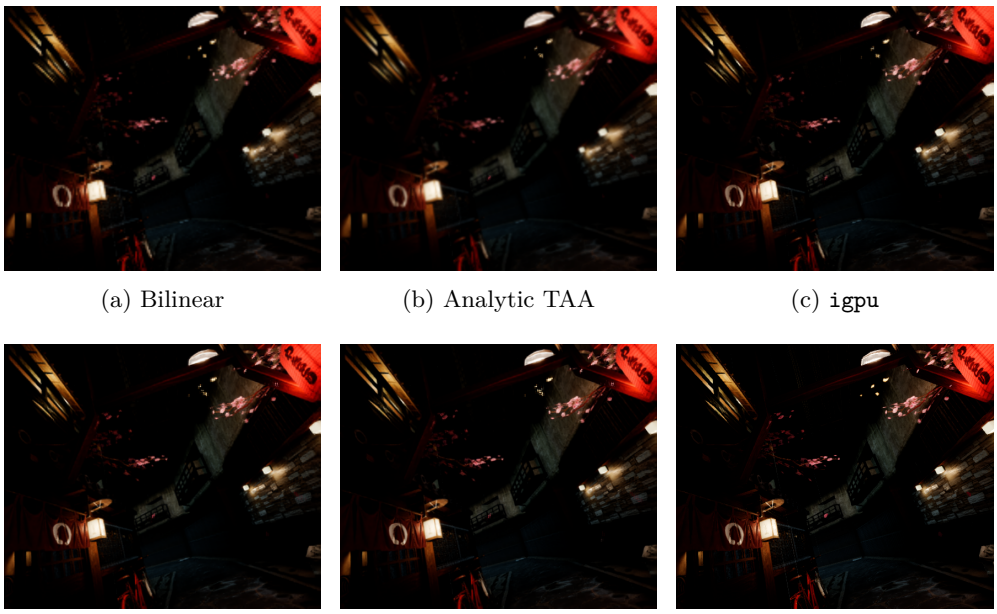


(a) Bilinear (b) Analytic TAA (c) `igpu`

(d) `low` student (e) Teacher (f) Ground truth

Fig. S31: Full predicted frames of the synthetic fast-motion scene at 3x (last frame), one panel per method

Figure S32 gives the TartanAir frames in the same layout. The learned outputs look closest to the ground truth on the red sign and the lit windows, and bilinear and TAA are visibly softer on the lit surfaces.

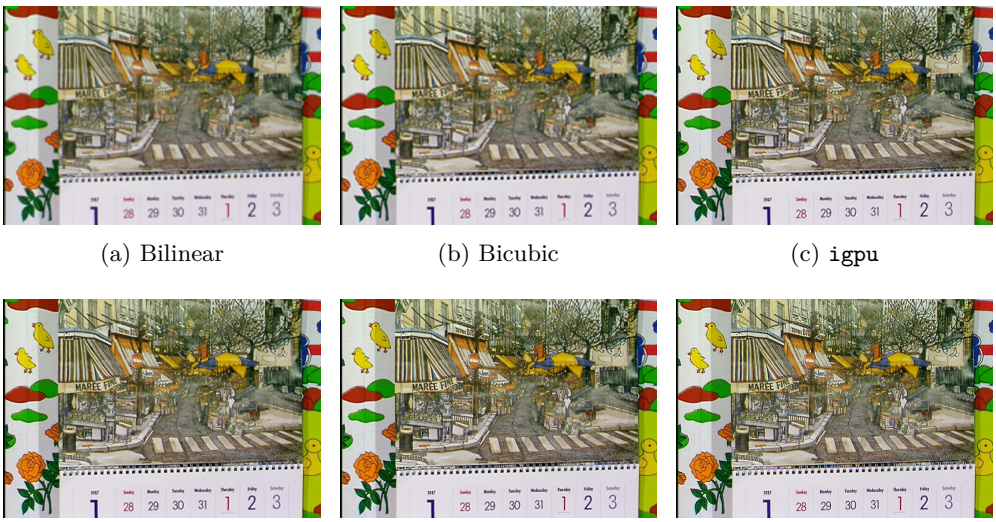


(a) Bilinear (b) Analytic TAA (c) `igpu`

(d) `low` student (e) Teacher (f) Ground truth

Fig. S32: Full predicted frames of the TartanAir environment at 3x in engine mode (last frame), one panel per method

Figure S33 gives the Vid4 frames as plain video. The learned outputs sharpen the signs, and the cobblestone texture of each learned output differs from the ground truth, as described above.



(a) Bilinear (b) Bicubic (c) `igpu`

(d) `low` student (e) Teacher (f) Ground truth

Fig. S33: Full predicted frames of the Vid4 calendar clip as plain video at 3x (last frame), one panel per method

S5.13 Status summary and open checks

The main findings of this section are listed below, each with the tag that its evidence supports.

- [Preliminary] In engine mode the `low` student is ahead of TAA and bicubic on every 3x suite, by the largest margin on the synthetic scenes.
- [Preliminary] In plain mode the `low` student is ahead of bicubic on every 3x suite, by the smallest margin on Vimeo-1080p.
- [Preliminary] The teacher is the best 3x model on every suite and mode, at about ten times the MAC count of the student.
- [Preliminary] The history, the G-buffers and the plain-video training are the largest ablation effects, and the 16-frame unrolls matter for static convergence.
- [Confirmed] The MAC counts and parameter counts of the tiers are properties of the code.
- [Under revision] The latencies are eager PyTorch times on one T4, not the latency of a deployment runtime.
- [Under revision] The fusion parity of the tiers is exactly zero and is not established.
- [Under revision] The engine-mode rows of the motion-estimator ablation are confounded by the G-buffer flag.
- [Observation] Two displayed training windows have a fully invalid validity map, and the cause is open.

The results need these checks before the next run. The engine mode of the video suites uses RAFT motion from the clean frames, so it should be reported with the plain mode alongside it, or replaced. The analytic baseline should be compared with a stronger temporal baseline. The estimator ablation needs its flag set in evaluation. The validity of the training windows must be measured on the cache. The GameIR suite needs more clips, so that its LPIPS and tOF rest on more than one frame. The budgets must be long

enough for the validation curves to flatten before the stage comparisons are read. The limits of the whole study and the planned revisions are in Section S8.

S6 Frame generation: method

This section specifies the frame generator completely enough to re-implement it. It defines the task and the time parameter, then the network block by block, the engine hints and their zero initialisation, the output and UI compositing, the scene-cut gate, the latency model, the training samples, losses and schedule, the ablations, the export, and the test protocol. Measured results are in Section S7. Every symbol is defined where it is first used and collected in Table S40. Where a design element is a known weakness of the current pipeline, it carries the **[Under revision]** tag.

S6.1 Problem statement

A frame generator produces an image between or after real frames that a renderer or a video decoder has already produced. The main article (Section 5) gives the aim: one compact network that serves engine renders with motion vectors and ordinary video without them, at a cost that fits integrated and entry GPUs. Its inputs are the two most recent real colour frames, a target time, and optional engine data. Its output is one generated colour frame, or the newest real frame when the scene-cut gate of Section S6.11 fires:

$$\hat{I}_\tau = G_\theta(I_0,\, I_1,\, \tau,\, \mathbf{h})\,, \tag{S80}$$

where θ are the network weights and $\mathbf{h}$ is the stack of engine hints of Section S6.8, which is empty for plain video.

Video frame interpolation networks such as RIFE [5], IFRNet [35], FILM [36] and EMA-VFI [37] estimate intermediate motion from the frames alone, and logos and UI break them [38]. For rendered content, interpolation holds a frame back [39], whereas extrapolation avoids that delay [40–43].

The mode matters for latency. In interpolation the target lies between I_0 and I_1, so the generated frame needs the newest real frame I_1, and I_1 is held back until the generated frame has been shown. In extrapolation the target lies after I_1, so nothing is held back, but the generator must predict motion that no real frame shows yet. Section S6.12 turns this into a latency model.

S6.2 Time parameter and generation modes

The time parameter τ measures the target position in units of the interval between the two real frames. Frame I_0 is at $\tau = 0$ and I_1 at $\tau = 1$, so the target is shown at $t_0 + \tau\,\Delta t$, where t_0 is the time of I_0 and Δt is the frame interval between I_0 and I_1. The generator receives τ as a scalar for each sample, and the first block sees it as a constant input plane (Section S6.4).

Interpolation uses $0 < \tau < 1$. Two-times generation uses $\tau = 1/2$, and three-times multi-frame generation (3x) uses $\tau \in \{1/3, 2/3\}$. Training draws τ uniformly from $[0.15, 0.85]$ for a share of the synthetic interpolation samples, so that one model covers the in-between times of 2x to 6x sequences.

Extrapolation uses $\tau = 1.5$. The target lies half an interval beyond I_1, so the frame is extrapolated from the motion between I_0 and I_1 and is shown after I_1. Extrapolation is the only mode in which no real frame is held back.

Multi-frame generation produces two frames between a real pair, both from the same two real frames. The training quads realise the three modes with frame indices k to $k + 3$, as given in Section S6.13.

S6.3 Notation

S6.4 IFNet: motion from colour

The motion estimator is an intermediate-flow network (IFNet) with the design of RIFE 4.26 [5]. It is a coarse-to-fine estimator of intermediate flows (Section S2.7) that works on the colour frames alone. It estimates the two flows of the target grid, $F_{\tau\to 0}$ and $F_{\tau\to 1}$, and it also produces a blend weight and features for the head. The backward warp $\mathcal{W}$ of Section S2.4 is written here for the code convention:

$$\mathcal{W}(I, F)(x) = I\big(x + F(x)\big), \tag{S81}$$

where $I(\cdot)$ is evaluated by bilinear interpolation, coordinates outside the image are clamped to the border, and the sampling grid is normalised so that the first and last pixel centres sit at -1 and $+1$ (the align-corners convention). The warp is the same for every tier, and the flows are in pixels, so $F(x)$ is a displacement of the sampling position.

Encoder. A per-frame convolutional encoder (Section S2.6) maps each frame to four feature channels at full resolution:

$$f_i = E(I_i), \qquad i \in \{0, 1\}, \tag{S82}$$

Table S40: Symbols of the frame generator. Images are $3 \times H \times W$ with values in $[0, 1]$ unless stated, and flows are in pixels of the full-resolution grid

Symbol	Shape	Meaning
I_0, I_1	$3 \times H \times W$	real frames at $\tau = 0$ and $\tau = 1$, with I_1 the newest real frame
I_τ, $\hat{I}_\tau$	$3 \times H \times W$	ground-truth and generated target frame
τ	scalar	target time
$\mathbf{h}$	$10 \times H \times W$	engine hint stack (Table S41)
M_1	$2 \times H \times W$	engine motion vector of I_1, the displacement from I_1 to I_0
C	$2 \times H \times W$	camera-only flow from the target grid to I_1
$F_{\tau\to0}$, $F_{\tau\to1}$	$2 \times H \times W$	flows from target pixels to I_0 and to I_1
$\mathcal{W}(I, F)$	$\cdot \times H \times W$	backward warp of I by F (Eq. (S81))
a_{mv}, a_{cam}, a_{dep}	$\{0, 1\}$	availability of motion vectors, camera prior and depth
f_0, f_1	$4 \times H \times W$	per-frame encoder features of I_0 and I_1
s_k, c_k	scalars	scale and width of IFNet block $k = 0, \ldots, 4$
Δ_k, μ_k, φ_k	4, 1, 8 channels	flow update, mask logit and features of block k
$F_k = [F_k^0, F_k^1]$	$4 \times H \times W$	accumulated flows after block k, to I_0 and to I_1
m, B	$1 \times H \times W$, $3 \times H \times W$	blend weight $\sigma(\mu_4)$ and blend (Eq. (S87))
X, r, ℓ, u	21, 3, 1, 1 channels	head input, residual colour, UI logit and UI weight $u = \sigma(\ell)$
σ	function	logistic function $\sigma(x) = 1/(1 + e^{-x})$
s, L	scalar, lines	flow scale of a frame and the flow line limit of a tier

Table S41: Engine hint channels $\mathbf{h}$ (10 channels). The first six are flows in pixels, divided by the block scale like the flow input (Eq. (S84)). The availability flags a_{mv}, a_{cam} and a_{dep} are scalars per sample, and channels 7, 9 and 10 are constant planes of the flags

Channels	Content	Definition	Source in training
1–2	motion prior τM_1	$\tau\, a_{\text{mv}} M_1$	exact (synthetic), RAFT (real)
3–4	motion prior $(\tau - 1)M_1$	$(\tau - 1)\, a_{\text{mv}} M_1$	as channels 1–2
5–6	camera-only flow C	$a_{\text{cam}} C$	synthetic extrapolation only
7	motion availability	a_{mv} (constant plane)	all samples
8	depth of I_1	$a_{\text{dep}}\, d_1$	synthetic only
9	depth availability	a_{dep} (constant plane)	all samples
10	camera availability	a_{cam} (constant plane)	all samples

where E applies a 3×3 convolution with stride 2 (3 to 16 channels) and LeakyReLU with slope 0.2, two more 3×3 convolutions (16 to 16 channels, each followed by LeakyReLU), and a 4×4 transposed convolution with stride 2 and padding 1 (16 to 4 channels). E depends on its frame only, which is what allows the features to be reused (Section S6.10).

Blocks. IFNet has five blocks $k = 0, \ldots, 4$ with scales $(s_0, \ldots, s_4) = (16, 8, 4, 2, 1)$. Block k works on the frames downsampled by s_k, and its width is c_k. The `high` tier has $(c_0, \ldots, c_4) = (192, 128, 96, 64, 32)$. The first block receives the frames, the features and the target time:

$$x_0 = \big[I_0,\ I_1,\ f_0,\ f_1,\ \tau\mathbf{1}\big] \qquad (3 + 3 + 4 + 4 + 1 = 15 \text{ channels}), \tag{S83}$$

where $\mathbf{1}$ is a constant plane of ones, so $\tau\mathbf{1}$ is a plane with value τ. Every later block receives the previous mask and features, the previous flow, and the warped inputs under the current flow:

$$x_k = \big[\mathcal{W}(I_0, F_{k-1}^0),\ \mathcal{W}(I_1, F_{k-1}^1),\ \mathcal{W}(f_0, F_{k-1}^0),\ \mathcal{W}(f_1, F_{k-1}^1),\ \tau\mathbf{1},\ \mu_{k-1},\ \varphi_{k-1},\ F_{k-1}\big], \tag{S84}$$

which has $3+3+4+4+1+1+8+4=28$ channels. Here F_{k-1}^{0} are the first two channels of F_{k-1}, the flow to I_0, and F_{k-1}^{1} are the last two, the flow to I_1. Each block outputs

$$\big(\Delta_k,\ \mu_k,\ \varphi_k\big) = B_k(x_k, \mathbf{h}), \tag{S85}$$

with a four-channel flow update Δ_k, a one-channel mask logit μ_k and an eight-channel feature map φ_k. The flow accumulates, with $F_0 = \Delta_0$ and $F_k = F_{k-1} + \Delta_k$ for $k \geq 1$.

Inside a block, RIFE's layout is kept (Fig. S34). The block input is bilinearly downsampled to H/s_k, and its flow channels, and the flow hints, are divided by s_k, because a displacement in pixels shrinks with the grid. Two stride-2 3×3 convolutions with LeakyReLU bring the input to a quarter of the block resolution in each direction, with $c_k/2$ and then c_k channels. Then n residual blocks follow, each computing $\mathrm{LeakyReLU}(\beta \odot \mathrm{conv}_{3\times3}(x) + x)$ with a learned per-channel scale β that starts at one. A 4×4 transposed convolution with stride 2 (to 52 channels) and a pixel shuffle with factor 2 [7] restore the block resolution H/s_k and give 13 output channels. These are upsampled by s_k to the full frame, and the flow is multiplied by s_k to return to pixel units:

$$\Delta_k = s_k\,\mathrm{up}_{s_k}\big(\mathrm{out}_{0:4}\big), \qquad \mu_k = \mathrm{up}_{s_k}\big(\mathrm{out}_{4}\big), \qquad \varphi_k = \mathrm{up}_{s_k}\big(\mathrm{out}_{5:13}\big), \tag{S86}$$

where up_s is bilinear upsampling by s and out is the 13-channel output of the block. The convolutions of block k therefore run at $H/(4s_k)$, which is $H/64$ for block 0 and $H/4$ for block 4.

Warps and blend. After block k the frames are warped by the accumulated flow, $\mathcal{W}(I_0, F_k^0)$ and $\mathcal{W}(I_1, F_k^1)$, and these warps are the inputs of block $k+1$. After block 4 the blend weight and the blend are

$$m = \sigma(\mu_4), \qquad B = m \odot \mathcal{W}(I_0, F_4^0) + (1-m) \odot \mathcal{W}(I_1, F_4^1). \tag{S87}$$

The blend B is the output of RIFE 4.26, and the head of Section S6.9 adds the generative part.

S6.5 RIFE 4.26 weights and the reproduction check

The weights are RIFE 4.26's archive, published in the author's Hugging Face repository. The downloader checks the SHA-256 hash of the archive and then of the `flownet.pkl` file inside it, and it loads the state dictionary with `weights_only=True`, so the file cannot run code. The parameter names and shapes of the `high` tier equal RIFE's, so every tensor is loaded by name. Where a tensor of a tier is smaller than its RIFE counterpart, the loader copies the leading slice along every dimension, the minimum of the two sizes. Where RIFE has more residual blocks than the tier, the first n blocks are copied and the rest are ignored. Tensors that have no RIFE counterpart keep their initialisation, and the hint input columns keep their zero weights (Section S6.8). For the `high` tier the loader reports 153 tensors copied exactly and 5 copied with a slice. The five sliced tensors are the first convolutions of the blocks, which are widened by the 10 hint channels, and no tensor is missing.

Reproduction check. The check compares the `high` tier with RIFE 4.26's own network code, the `IFNet_HDv3` module of the archive, on the same inputs and with no engine data. RIFE's warp function is not in the archive, so the check recreates it with the grid convention of Eq. (S81). The inputs are a random 8×12 image upsampled bicubically to 128×192 as I_0, and I_0 rolled by $(2, 5)$ pixels as I_1, at $\tau = 1/2$. The repository's check accepts a difference below 10^{-4} between the blend B and RIFE's merged output in float32 on the CPU, and the agreement it measures is below 10^{-5}. It also checks that the residual head and the hints add nothing at initialisation (Sections S6.8 and S6.9). The check tests the network code against RIFE's network code. It does not test the warp against RIFE's own kernel, and it does not test the delivered output of the generator, which differs from the blend (Section S6.9).

S6.6 Tiers and channel-slice inheritance

Table S42 lists the four tiers. The `high` tier has RIFE's widths, residual depth and block layout, and it is the one that starts from RIFE 4.26's network. The smaller tiers are narrower, with fewer residual blocks per block and a narrower head. Each tier is the generator of Eq. (S80) with its own widths, residual depth, head width and flow line limit.

Channel-slice inheritance. A smaller tier starts from RIFE's weights through the copy rule of Section S6.5. The narrower tensors take the leading channels of RIFE's tensor along each channel dimension, so the output channels of a block and the input channels of the next layer keep the same ordering. For the `igpu` tier the loader reports 13 tensors with exact shapes and 70 sliced, for the `low` tier 13 and 85, and for the `mid` tier 13 and 115, with no missing tensor. The head has no RIFE counterpart, so it starts from its initialisation in every tier. The inheritance is a heuristic. Nothing forces RIFE's leading channels to be the best subset for a narrower network, and the ablation without inheritance (Section S6.16) is the test of how much it matters.

S6.7 Flow-resolution caps and their cost

Estimating flow at full resolution is the most expensive part of the network, so every tier caps the number of image lines at which it estimates flow. A frame of height H lines is halved until it fits under the tier's

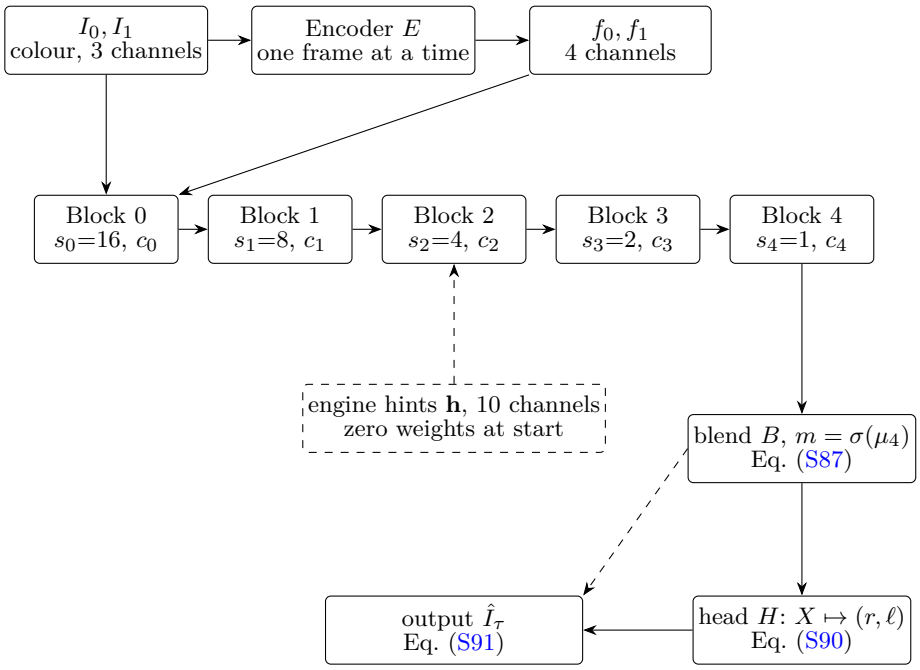


Fig. S34: IFNet with the head. The encoder runs once per frame. Block k takes the warped frames and features under the current flow, the previous mask and features, and τ (Eqs. (S83) and (S84)), and it outputs Δ_k, μ_k and φ_k (Eq. (S85)). Engine hints enter every block through zero-initialised weights (Section S6.8). Block 4 gives the blend B, and the half-resolution head gives the residual r and the UI logit, which form the output (Eqs. (S90) and (S91)). The block scales and widths shown are those of the `high` tier

Table S42: Frame-generation tiers. Widths are the channel counts of blocks 0 to 4, n the residual blocks per block, c_h the head width, L the flow line limit of Eq. (S88), and GMAC the multiply-accumulates per 1080p output frame (every convolution, warps excluded), with flow at the scale the tier uses at 1080p

Tier	Widths $c_0, \ldots, c_4$	n	c_h	L	Params	GMAC
`igpu`	64, 48, 32, 24, 16	3	12	576	454K	9.4
`low`	96, 64, 48, 32, 16	4	16	576	953K	12.7
`mid`	128, 96, 64, 48, 24	6	24	1152	2.29M	44.3
`high`	192, 128, 96, 64, 32	8	32	1152	5.71M	76.7

limit L, as RIFE's own scale argument allows:

$$s(H) = 2^{-j^*}, \qquad j^* = \min\{\, j \in \mathbb{N}_0 : H\,2^{-j} \le L \,\}, \tag{S88}$$

with $L = 576$ lines for the `igpu` and `low` tiers and $L = 1152$ lines for the `mid` and `high` tiers. At 1080 lines the `igpu` and `low` tiers therefore estimate flow at $s = 1/2$ (540 lines), and the `mid` and `high` tiers at $s = 1$. The 256-pixel training crops and video up to 576 lines keep full-resolution flow in every tier. The frames are downsampled by s before the encoder and the blocks, the flow hints are multiplied by s, and the flows that come out are upsampled and multiplied by $1/s$ to return to pixels. The head, the warps and the output stay at full resolution.

The convolution cost of the encoder and the blocks scales with the pixel count of the frames they see, so it scales with s^2. The head does not change. Table S43 gives the consequence at 1080p, counted with the same hooks as Table S42. The cap removes more than half of the flow-path cost of the `igpu` and `low` tiers. For the `low` tier the head is about a third of the capped cost, so the head matters most for the smallest tiers.

Table S43: Cost per 1080p generated frame of the tiers with the flow cap of Eq. (S88) and with full-resolution flow (GMAC, warps excluded). The encoder column counts both real frames, as the repository's tier table does, so one encoder pass is half of it. The head is the residual head of Section S6.9

Tier	Flow scale	Capped total	Full-flow total	Encoder (two frames)	Head
`igpu`	0.5	9.4	20.4	1.6	2.9
`low`	0.5	12.7	25.5	1.6	4.5
`mid`	1	44.3	44.3	6.3	8.5
`high`	1	76.7	76.7	6.3	13.8

S6.8 Engine hints and their zero initialisation

When an engine provides data, the generator receives them as extra input channels of every IFNet block. Table S41 lists the ten channels in the order of the code. The signal channels are multiplied by their availability, so an absent signal contributes zero, and the flags tell the network which case it is in.

Motion priors. If a point moves with constant velocity between I_0 and I_1, its displacement from I_1 to I_0 is M_1, and at time τ its displacement to I_0 is τM_1 and to I_1 is $(\tau - 1)M_1$. The priors are exact only for linear motion, so the network has to learn when to trust them. The zero initialisation below lets it start from a working point.

Camera-only flow and late latching. The camera flow C is the displacement that the camera motion alone induces on the static background, from the target pixel to I_1. Let A_t be the background affine map from screen to texture at time t (Section S3.10), with normalised coordinates $x \in [-1, 1]^2$. Then

$$C(x) = \tfrac{P}{2}\big(A_1^{-1}(A_\tau(x)) - x\big), \tag{S89}$$

where P is the frame size in pixels, so that $P/2$ converts normalised displacements to pixels for a $P \times P$ frame. Object motion is absent from C. At display time an engine knows the camera pose of the frame it is about to show, so the pose can be latched just before generation and supplied as this prior. In the synthetic training the prior uses the exact background trajectory at τ. For real video the camera availability is zero, so the prior is never present.

Depth. The depth channel is the inverse depth of I_1 normalised by its median, $d_1 = 1/(1+d/\operatorname{median}(d))$, so that a pixel at the median depth has the value one half. It is defined only for the synthetic renderer, where the depth is exact.

Availability. Each availability flag is set per sample, and the signal channels are multiplied by the flags. Withholding a signal in training is therefore the same as an engine that does not provide it. The withholding rates are in Table S44.

Zero-initialisation argument.

Let $W_k^{\mathbf{h}}$ be the weights of the first convolution of block k that multiply the hint channels, and let every other weight of the `high` tier equal RIFE's. The claim is that with $W_k^{\mathbf{h}} = 0$ for every k, the blend B does not depend on $\mathbf{h}$ and equals RIFE 4.26's blend. The argument is by induction over the blocks. Block 0

Table S44: Sampling probabilities of the frame-generation training data, with the configuration names used in the saved files, or the constant in the code where no configuration name exists

Quantity	Value	Configuration or constant
Real share of each GPU batch	0.4 (19 of 48)	`p_real`
Extrapolation, both sample kinds	0.4	`p_extrap`
Random τ for synthetic interpolation	0.5	`p_tau`, $\tau \sim U[0.15, 0.85]$
Multi-frame share of real interpolation	0.25	`p_mfg`
Camera reversal	0.2	`p_flip`
Scene cut	0.03	`p_cut`
UI overlay	0.5	`p_ui`
Menu, given a UI overlay	0.03	`p_menu`
Withhold motion vector (synthetic)	0.3	`p_drop_mv`
Withhold motion vector (real)	0.5	`p_drop_mv_real`
Withhold depth (synthetic)	0.30	default argument in `set_availability`
Withhold camera prior (extrapolation)	0.20	constant in `set_availability`
UI slot filled / tile changed at $t = 1$	0.70 / 0.30	`ui_layer`

receives the same input as in RIFE, so its first convolution computes $z = W_x * x + W_0^{\mathbf{h}} * \mathbf{h} + b$ with $W_0^{\mathbf{h}} = 0$, which equals RIFE's pre-activation $W_x * x + b$. The later layers of the block see only z, so the block outputs $(\Delta_0, \mu_0, \varphi_0)$ equal RIFE's. The induction step is the same for block k, because the warps and features that enter block k are functions of the earlier outputs only. The blend of Eq. (S87) is then RIFE's blend. The head adds nothing at initialisation, because its last layer has zero weights and bias $(0, 0, 0, -4)$ (Section S6.9). The gradient with respect to the hint columns is $\partial L / \partial W_k^{\mathbf{h}} = \sum_p \delta_k(p)\, \mathbf{h}(p)^\top$, summed over the sampled positions p, where δ_k is the gradient at the first-layer output. This is not zero when the hints are non-zero and the loss is sensitive to block k, so the hint columns leave zero at the first update. The fresh model therefore reproduces RIFE's blend, and then learns from the engine data.

S6.9 Head, UI weight and output

The head turns the warped frames into a residual colour and a UI logit. Its input is the full-resolution stack of the two real frames, the two warps under the final flow, the blend weight and the last features:

$$X = \big[I_0,\ I_1,\ \mathcal{W}(I_0, F_4^0),\ \mathcal{W}(I_1, F_4^1),\ m,\ \varphi_4\big] \qquad \text{(21 channels)}. \tag{S90}$$

A stride-2 3×3 convolution with c_h channels and LeakyReLU, two residual blocks at half resolution with the design of Section S6.4, and a 4×4 transposed convolution with stride 2 give four output channels, $(r, \ell) = H(X)$. Here r is the residual colour and ℓ the UI logit. The head starts with a zero transposed-convolution weight and bias $(0, 0, 0, -4)$, so $r = 0$ and $\ell = -4$ at initialisation, and the UI weight $u = \sigma(\ell)$ is about 0.018. The output is

$$\hat{I}_\tau = u \odot I_1 + (1 - u) \odot \big(B + r\big), \qquad u = \sigma(\ell), \tag{S91}$$

with the blend B of Eq. (S87). This is Eq. (2) of the article.

The UI weight is a soft mask in $[0, 1]$, not a hard classification. Training pushes u towards one on HUD, subtitle and menu pixels, where the UI mask of Eq. (S103) is one, and towards zero elsewhere, so that those pixels come from the newest real frame I_1. The starting value means that a fresh model already mixes a small fraction of I_1 into every pixel. The delivered output of a fresh model therefore differs from the blend by $u \odot (I_1 - B)$, which is small but not zero. This is why the reproduction check of Section S6.5 compares the blend and not the output.

Evaluation clamps the output to $[0, 1]$ before computing metrics. Training does not clamp, so the loss sees the unclamped output.

UI layer compositing. When an engine renders the HUD separately from the scene, it can supply HUD-less real frames and the UI layer for the newest state, as premultiplied colour c and alpha α. The generator then runs on the HUD-less frames, and the displayed frame is $\hat{I}_\tau(1 - \alpha) + c$. In the design this is applied to every displayed frame of the layer's sequence. The evaluation applies it to the generated frame only (Section S6.18).

S6.10 Per-frame encoder reuse

The encoder of Eq. (S82) depends on one frame, so a running sequence can encode each real frame once and reuse its features. The IFNet forward pass takes optional precomputed features for this, but no caller in the repository passes them. Training, evaluation and the exported graph all encode both frames (Section S6.17).

The cost accounting follows from this. The encoder column of Table S43 counts two frames, so one pass per real frame is half the column. With one generated frame per real frame, reuse replaces two encoder passes per generated frame by one.

S6.11 Scene-cut gate

[Under revision] The gate decides when the generator should not generate. When two real frames look as if a scene cut lies between them, it outputs the newest real frame I_1 instead of $\hat{I}_\tau$. The repository marks this stage as a known weakness. Its thresholds are calibrated on synthetic adjacent pairs only, so the calibration does not cover real content or wider frame gaps.

The score combines a colour-distribution distance, which needs no motion, with a motion-compensated difference, which is used when M_1 is available. Let $Y(x) = (16{+}65.481R(x){+}128.553G(x){+}24.966B(x))/255$ be the limited-range luma of an RGB pixel with values in $[0, 1]$, and let $h(I) \in \mathbb{R}^{32}$ be the fraction of pixels whose luma falls in each of 32 equal bins of $[0, 1]$. The histogram distance and the motion-compensated difference are

$$d_{\mathrm{h}}(I_0, I_1) = \tfrac{1}{2}\left\|h(I_0) - h(I_1)\right\|_1 \in [0, 1], \tag{S92}$$

$$d_{\mathrm{m}}(I_0, I_1, M_1) = \frac{1}{3HW}\sum_{x,c}\left|I_1(x, c) - \mathcal{W}(I_0, M_1)(x, c)\right|. \tag{S93}$$

The two scores are

$$S_{\mathrm{mv}} = d_{\mathrm{h}} + 2\, d_{\mathrm{m}}, \qquad S_0 = d_{\mathrm{h}}, \tag{S94}$$

and the gate is

$$g = \begin{cases} \mathbf{1}[S_{\mathrm{mv}} > \theta_{\mathrm{mv}}] & \text{if } a_{\mathrm{mv}} = 1, \\ \mathbf{1}[S_0 > \theta_0] & \text{otherwise,} \end{cases} \tag{S95}$$

where $\mathbf{1}[\cdot]$ is one when its condition holds and zero otherwise. Without motion vectors the histogram distance alone is used, because a raw frame difference would read ordinary fast motion as a cut. The gate lives in the evaluation code, and it is not part of the exported graph.

Calibration. Each threshold is set separately on synthetic pairs, so that continuous motion is a false alarm on about one pair in a hundred.

1. Draw four batches of up to 64 validation stills. For each batch, generate interpolation pairs at $\tau = 1/2$ with motion from all four speed classes, no camera reversal and no UI, and with a cut probability of one half. Each pair is labelled cut or continuous. This gives 256 pairs, about half of them cuts.
2. Compute S_{mv} with the exact motion vector and S_0 for every pair.
3. For each score, sort the scores of the continuous pairs in ascending order and take the element at 0-based index $\lfloor 0.99(n-1) \rfloor$, where n is the number of continuous pairs. This element is the threshold θ.
4. Report the detection rate, which is the share of cut pairs whose score exceeds θ.

The calibrated values are part of the results (Section S7). When the gate fires, the output is the newest real frame,

$$\hat{I}_\tau \quad \leftarrow \quad I_1 \qquad \text{if } g = 1. \tag{S96}$$

S6.12 Latency model and pacing

[Under revision] The latency numbers are timed on one data-centre GPU in eager PyTorch, and the pacing model is a simulation with assumed scheduling. The latency terms are those of Section S2.8. The repository marks in-engine latency on target hardware as not measured. The model therefore answers what frame generation does to the displayed rate and to two latencies, under the stated assumptions.

The model is a discrete-event simulation of a GPU-bound loop with just-in-time CPU scheduling, in which the CPU starts each frame's work so that the GPU never queues frames, on a variable-refresh display. Its parameters are in Table S45. The modes are *native* (no generation), *interpolation* (a generated frame between two real frames, with the newer real frame held), *extrapolation* (a generated frame after the newest real frame, with nothing held) and *extrapolation with late latching* (extrapolation whose camera is sampled just before generation).

The simulation runs as follows.

1. For each real frame $j = 1, \ldots, N$, draw the render time $R_j = R_0 \exp(c_v z_j)$, where z_j is standard normal.

Table S45: Parameters of the pacing model, as set in the simulation code and the notebook. The generation time T_{gen} is the measured one-generated-frame time of the `low` tier at 1080p (Section S7). The temporal super-resolution time T_{sr} is zero, because this pacing run has no such stage

Symbol	Value	Meaning
R_0	33.3, 25.0, 16.7 ms	base render time per real frame (30, 40 and 60 fps renders)
C_{cpu}	4.0 ms	CPU time per frame before rendering starts
c_v	0.08	log-normal spread of the render time
T_{pres}	1.0 ms	present time
T_{sr}	0	temporal super-resolution time
T_{gen}	measured	frame generation time per generated frame
N	2000	simulated real frames per run
seed	0	random seed of the render times

2. Set the GPU time $g_j = R_j + T_{\text{sr}} + T_{\text{gen}}\,\mathbf{1}[\text{mode} \neq \text{native}]$ and the frame period $\pi_j = \max(C_{\text{cpu}}, g_j)$.
3. Let t_j be the time of the input sample, which is the current simulation time.
4. Set the time at which the rendered real frame is ready, $a_j = t_j + C_{\text{cpu}} + R_j + T_{\text{sr}}$.
5. In the *native* mode, display the real frame at $p_j^{\text{r}} = a_j + T_{\text{pres}}$.
6. In the *interpolation* mode, display the generated frame at $p_j^{\text{g}} = a_j + T_{\text{gen}} + T_{\text{pres}}$, and hold the real frame until $p_j^{\text{r}} = p_j^{\text{g}} + \pi_j/2$.
7. In the *extrapolation* mode, display the real frame at $p_j^{\text{r}} = a_j + T_{\text{pres}}$, and the generated frame at $p_j^{\text{g}} = p_j^{\text{r}} + \pi_j/2$.
8. In the *extrapolation with late latching* mode, proceed as in step 7, and sample the camera of the generated frame at $t_j^{\text{cam}} = p_j^{\text{g}} - T_{\text{gen}} - T_{\text{pres}}$, the moment generation starts.
9. Record the displayed frames and the latency samples below, and advance the simulation time by $t \leftarrow t + \pi_j$.

The rates are computed on the displayed times after the first tenth of the displayed sequence, so that start-up does not count. With Δ_i the gaps between consecutive displayed times, the displayed frame rate and the gap variation are $\text{fps} = 1000/\overline{\Delta}$ and $\text{CV} = \text{sd}(\Delta)/\overline{\Delta}$, in milliseconds for Δ.

Three latencies are reported. The game input latency is the time from an input sample to the display of the real frame that reflects it, averaged over the real frames of the run,

$$L_{\text{game}} = \text{mean}_j\left(p_j^{\text{r}} - t_j\right). \tag{S97}$$

The camera latency is the same kind of time, measured to each displayed frame that counts as reflecting the camera. In the code a native frame counts with its real display. An interpolated frame counts with its generated display only, on the assumption that the generated frame already moves half way toward the camera of the newer frame. An extrapolated frame counts with both its real and its generated display, and with late latching the generated display is measured from t_j^{cam}:

$$L_{\text{cam}} = \begin{cases} \text{mean}_j\left(p_j^{\text{r}} - t_j\right) & \text{native,} \\ \text{mean}_j\left(p_j^{\text{g}} - t_j\right) & \text{interpolation,} \\ \frac{1}{2}\Big(\text{mean}_j\left(p_j^{\text{r}} - t_j\right) + \text{mean}_j\left(p_j^{\text{g}} - t_j\right)\Big) & \text{extrapolation,} \\ \frac{1}{2}\Big(\text{mean}_j\left(p_j^{\text{r}} - t_j\right) + \text{mean}_j\left(p_j^{\text{g}} - t_j^{\text{cam}}\right)\Big) & \text{late latching.} \end{cases} \tag{S98}$$

The content age averages the time from the input sample to every displayed frame, real or generated,

$$A = \text{mean}_p\left(p - t\right) \quad \text{over all displayed frames } p. \tag{S99}$$

Each latency is also reported at its 95th percentile over the samples.

The arithmetic estimate of a network's time at a sustained throughput F in TFLOPS, is $T_{\text{est}} = 2G/F$ milliseconds, for a frame that costs G GMAC, because one multiply-accumulate is two floating-point

operations. The repository uses $F = 4$ TFLOPS for an assumed integrated GPU. The estimate scales the GMAC with pixel count, so at 1440p it ignores the flow cap of Eq. (S88).

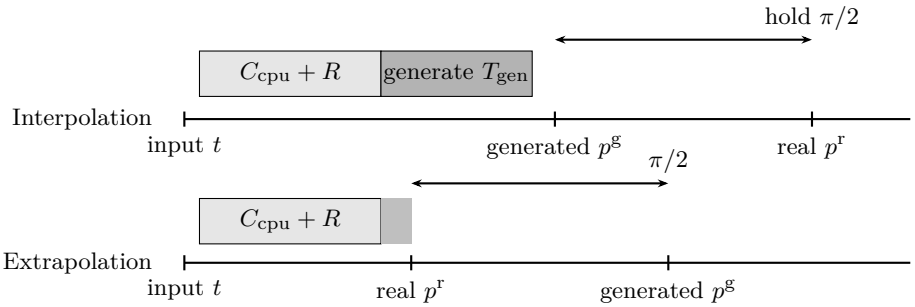


Fig. S35: Timeline of one input sample in the two generation modes, schematic and not to scale. In interpolation the generated frame needs the newest real frame, and the real frame is held for half a period, so it is shown after the generated frame. In extrapolation the real frame is shown as soon as it is rendered, and the generated frame follows half a period later, with the generation time T_{gen} inside that period. The grey bar at T_{pres} in the extrapolation row is the present time. The labels are the symbols of Table S45

Fig. S35 shows the two modes. Interpolation adds T_{gen} to the time of the generated frame and delays the real frame by half a period. Extrapolation adds no delay to the real frame and adds half a period to the generated frame, which is shown after the real frame.

S6.13 Training samples

In the main stages each of the two GPUs takes its own batch of 48 samples per step, under distributed data parallel training. A batch holds 19 real quads and 29 synthetic samples, since the real share 0.4 of 48 rounds to 19, drawn at random without replacement from the caches. Still crops are 512×512 pixels cut from the textures of Section S3.10, and quads are four frames of 256×256 pixels from the sequence sources of Section S3, both after the random downscale of Section S3. The cache sizes are in Section S7.

Synthetic samples.

Each synthetic sample is one scene of the renderer of Section S3.10, rendered at 256×256 pixels with four anti-aliasing samples per pixel for I_0, I_1 and the target. The background is one still. The foreground is the still of the neighbouring sample in the batch, visible through a procedural alpha mask in 85 percent of samples, and a flat colour in 25 percent of samples. The construction of one sample is as follows.

1. Draw the camera reversal with probability 0.2. A reversal reverses the camera velocity after $t = 1$.
2. Draw extrapolation with probability 0.4. If it is extrapolation, set $\tau = 1.5$. Otherwise set $\tau = 1/2$, or with probability 0.5 draw τ uniformly from $[0.15, 0.85]$.
3. Render I_0 at $t = 0$, I_1 at $t = 1$ and the target at $t = \tau$.
4. Compute the exact motion vector M_1, which is the flow from I_1 to I_0, the exact flows $F_{\tau\to 0}$ and $F_{\tau\to 1}$ with their validity masks (visible and inside the view), the camera flow C and the inverse depth d_1.
5. Draw a scene cut with probability 0.03. A cut replaces I_1, M_1 and d_1 by those of the neighbouring sample, sets the target to the replaced I_1, and sets the validity masks to zero.

6. Draw UI overlays with probability 0.5. Four UI slots are each filled with probability 0.7 by a tile from a bank of 256 text, bar and crosshair tiles of 160×28 pixels, and 30 percent of the filled slots change their tile at $t = 1$. A full-screen menu panel of alpha 0.6 and colour 0.05 is drawn with probability 0.03 given a UI overlay, which is 1.5 percent of all samples. The tiles are alpha-composited into I_0 with the state at $t = 0$, into I_1 with the state at $t = 1$, and into the target with the state at $t = 1$. The UI mask is the maximum of the two alphas.
7. Withhold the signals according to Table S44, and set the camera availability to zero for every interpolation sample.

Real quads.

Each real sample is a window of four consecutive frames from one of the sequence sources of Section S3, decoded and cut to 256×256 pixels. The window is flipped horizontally and vertically with probability one half each, and its flow fields are flipped with it, with the vector components negated along each flipped axis. The sample then takes one of three forms.

1. With probability 0.4, extrapolation: $I_0 = k$, $I_1 = k+2$ and target $k+3$, at $\tau = 1.5$.
2. Otherwise, with probability 0.25, multi-frame generation: $I_0 = k$, $I_1 = k+3$, and the target is $k+1$ or $k+2$ with equal probability, at $\tau = 1/3$ or $2/3$.
3. Otherwise interpolation: $I_0 = k$, $I_1 = k+2$ and target $k+1$, at $\tau = 1/2$.

The flows come from the RAFT optical-flow network [4], in the small variant that torchvision ships with its weights. The cache computes four flows per quad, $k+2 \to k$, $k+1 \to k$, $k+1 \to k+2$ and $k+3 \to k+2$, at the 256×256 crop size, and stores them at half resolution, in pixels of the full grid, by two-by-two average pooling. Every quad carries these four flows, including the TartanAir and Sintel quads that have exact flows in the frame decoder, because the quad cache does not use those flows. The motion vector M_1 is the flow $k+2 \to k$. The flows $F_{\tau\to 0}$ and $F_{\tau\to 1}$ are $k+1 \to k$ and $k+1 \to k+2$ for interpolation, and $k+3 \to k+2$ for extrapolation. An extrapolated sample has no label from the target to I_0, so its $F_{\tau\to 0}$ validity is zero. A multi-frame sample has no flow label, so its M_1 and both flow validities are zero. A real sample has no depth and no camera prior, so its depth and camera availabilities are zero. Every pixel of a real flow is valid, because the cache computes no validity mask.

Availability and the plain-video case.

Table S44 gives the withholding rates. Synthetic samples withhold the motion vector with probability 0.3 and the depth with probability 0.3, and they keep the camera prior with probability 0.8 on extrapolation samples only. Real samples withhold the motion vector with probability 0.5 when a flow label exists, and they never have depth or a camera prior. The validation batches are fixed, with 16 stills and 16 quads in each of 4 batches, drawn with a seed of 54321. Each is evaluated twice, once with its availabilities and once with all of them set to zero, which is plain video.

S6.14 Training objective

The objective is a weighted sum of six terms, with a unit weight on the first,

$$\mathcal{L} = \mathcal{L}_{\text{pix}} + 0.5\,\mathcal{L}_{\text{cen}} + 0.1\,\mathcal{L}_{\text{lp}} + 0.01\,\mathcal{L}_{\text{flow}} + 0.2\,\mathcal{L}_{\text{ui}} + 0.5\,\mathcal{L}_{\text{kd}}, \tag{S100}$$

where each term is a mean over the batch. Table S46 gives the definitions, and the paragraphs below give the formulas.

Charbonnier. With the target I_τ and the output $\hat{I}_\tau$, $\mathcal{L}_{\text{pix}} = \text{mean}\sqrt{(\hat{I}_\tau - I_\tau)^2 + \varepsilon^2}$, where the mean runs over every colour value. This is a smooth form of the L1 penalty. It is a smooth edge-preserving potential of the Charbonnier family [12].

Census. For a grey image $g = 0.299R + 0.587G + 0.114B$, the census vector at pixel x has one soft-ternary entry per offset δ in the 7×7 window D, which has 49 offsets including zero,

$$c(x,\delta) = \frac{g(x+\delta) - g(x)}{\sqrt{0.81 + \big(g(x+\delta) - g(x)\big)^2}}. \tag{S101}$$

The census distance at a pixel is $\rho(x) = \text{mean}_{\delta\in D}\; d/(0.1+d)$, with $d = \big(c_{\hat{I}}(x,\delta) - c_I(x,\delta)\big)^2$. The loss $\mathcal{L}_{\text{cen}}$ is the mean of ρ over the image with a border of three pixels removed on each side. Census distances

Table S46: Training losses of the frame generator, with weights and the samples to which each term applies

Term	Quantity	Weight	Applies to
$\mathcal{L}_{\text{pix}}$	Charbonnier penalty, $\varepsilon = 10^{-3}$	1	every sample
$\mathcal{L}_{\text{cen}}$	soft census distance, 7×7 window	0.5	every sample
$\mathcal{L}_{\text{lp}}$	LPIPS with AlexNet, on the clipped output	0.1	every sample, when the package loads
$\mathcal{L}_{\text{flow}}$	flow L1 on valid pixels, weight 1 (synthetic) or 1/4 (real)	0.01	every sample with flow labels
$\mathcal{L}_{\text{ui}}$	binary cross-entropy on the UI mask, positive weight 4	0.2	every sample
$\mathcal{L}_{\text{kd}}$	distillation from the `high` teacher	0.5	students only

are insensitive to brightness offsets, the property for which the census loss was introduced for unsupervised flow [44]. The soft-ternary normalisation and the robust distance are the repository's own choices.

LPIPS. $\mathcal{L}_{\text{lp}}$ is the LPIPS distance with the AlexNet network and version 0.1 of its calibration weights [14], applied to the output clipped to $[0, 1]$ and to the target, and averaged over the batch. The weight is zero when the package does not load, and the saved configuration records that.

Flow. The flow term is the weighted L1 error of both flows over the valid pixels,

$$\mathcal{L}_{\text{flow}} = \frac{\sum_x \sum_c \left|\hat{F}_{0,c}(x) - F_{0,c}(x)\right| v_0(x)}{2 \sum_x v_0(x) + 1} + \frac{\sum_x \sum_c \left|\hat{F}_{1,c}(x) - F_{1,c}(x)\right| v_1(x)}{2 \sum_x v_1(x) + 1}, \tag{S102}$$

where $c \in \{x, y\}$ runs over the two flow components. where $v_i = w\,\text{valid}_i$ with $w = 1$ for synthetic samples and $w = 1/4$ for real samples. The denominator counts the valid pixels and not their weights, so a real sample contributes a quarter of the error of a synthetic one at the same pixel error.

UI. With $y(x) = \mathbf{1}[\text{ui}(x) > 0.5]$ for the UI mask $\text{ui} = \max(\alpha_0, \alpha_1)$ of Section S6.13,

$$\mathcal{L}_{\text{ui}} = -\,\text{mean}\left[4\, y \log \sigma(\ell) + (1 - y) \log\left(1 - \sigma(\ell)\right)\right], \tag{S103}$$

which is the binary cross-entropy on the UI logit ℓ, with the positive term weighted by four to up-weight the rarer UI pixels.

Distillation. When a teacher checkpoint is given, the teacher is the `high` tier with the same inputs and availabilities, run without gradients. With $\hat{I}^{\text{T}}$ and $\hat{F}_i^{\text{T}}$ the teacher output and flows,

$$\mathcal{L}_{\text{kd}} = \text{mean}\left|\hat{I}_\tau - \hat{I}^{\text{T}}\right| + 0.01\Big(\text{mean}\left|\hat{F}_0 - \hat{F}_0^{\text{T}}\right| + \text{mean}\left|\hat{F}_1 - \hat{F}_1^{\text{T}}\right|\Big). \tag{S104}$$

Training a student to match a teacher's outputs in this way is distillation [45]. The 0.01 weight on the flows is a constant of the code, not a configuration value.

S6.15 Optimisation and schedule

The optimiser is AdamW [10] with betas $(0.9, 0.99)$ and weight decay 0.0001 on all parameters. The gradient norm is clipped to 1 after unscaling. The forward pass runs under fp16 autocast with dynamic loss scaling, and the losses are computed in float32 [9]. The learning rate follows the fraction $p = t/T$ of the wall-clock budget T already used. It rises linearly over the first 0.03 of the budget and then decays with a cosine to a floor of $\text{lr}_{\min} = 10^{-6}$:

$$\text{lr}(p) = \begin{cases} \text{lr}_0\ \max(p/0.03,\ 0.01) & p < 0.03, \\ \text{lr}_{\min} + \tfrac{1}{2}(\text{lr}_0 - \text{lr}_{\min})\left(1 + \cos \pi q\right) & p \geq 0.03, \end{cases} \qquad q = \frac{p - 0.03}{0.97}. \tag{S105}$$

A moving average of the weights, the exponential moving average (EMA), is updated after every step on the rank-0 model with decay $d_s = \min(0.999, (1 + s)/(10 + s))$ at step s, so the decay starts near zero and reaches 0.999. Validation runs every $\min(8\,\text{min}, \max(T/6, 60\,\text{s}))$ and at the end of the budget. Each validation saves the latest weights, and the best weights by the selection metric, which is the mean of the PSNR with and without engine data (Section S6.18). The run stops with an error after 200 consecutive steps with a non-finite loss. If a run exhausts GPU memory, it is retried once with half the batch for the rest of its budget.

Training follows two stages, listed in Table S47. First the `high` tier is fine-tuned from RIFE 4.26 on both GPUs at a learning rate of 0.0001 and with no teacher, since it is the tier that starts from RIFE's network. Then the smaller tiers start from RIFE's leading slices and are trained with the full objective, including distillation from the EMA weights of the fine-tuned `high` run, at a learning rate of 0.0004. The `low` tier, the main student, uses both GPUs. The `igpu` and `mid` tiers train side by side, one GPU each. The nominal budgets are those of the table, and the run scales them by one common factor below one, so that the whole pipeline fits the session. The saved configurations show the scaled budgets.

Table S47: Training runs of the frame generator, in stage order. Budgets are the nominal wall-clock times, and batches are per GPU

Run	Tier	Initialisation	Distillation	Learning rate	Budget (min)	Batch
`fg_teacher`	`high`	RIFE 4.26	none	0.0001	150	48
`fg_low`	`low`	RIFE slices	`high`	0.0004	200	48
`fg_igpu`	`igpu`	RIFE slices	`high`	0.0004	120	48
`fg_mid`	`mid`	RIFE slices	`high`	0.0004	120	24
six ablations	`low`	see Table S48	see Table S48	0.0004	30	48

S6.16 Ablations

Each ablation is a `low` run with the same 30-minute budget as the reference. Each changes one setting of the configuration (Table S48), and the ablation runs go in pairs, one run per GPU. All of them use the same wall-clock budget, so the run without distillation takes more steps, because it skips the teacher's forward pass.

Table S48: Ablations of the `low` tier, with the configuration change of each relative to the reference (the saved `cfg_abl_*.json` files)

Run	Change from the reference
`abl_reference`	none, so distillation from the `high` run and UI training are on
`abl_no_ui_training`	no HUD, subtitle or menu overlays in training, and UI loss weight zero
`abl_no_motion_prior`	motion vector and camera availability forced to zero in training (depth kept)
`abl_extrap_only`	every training sample is an extrapolation ($p_{\text{extrap}} = 1$)
`abl_no_distillation`	no teacher, so $\mathcal{L}_{\text{kd}}$ is absent
`abl_no_inheritance`	no RIFE weights, so the IFNet starts from the default initialisation

The `abl_no_motion_prior` model is evaluated without any engine data in both modes, because its training never supplied the motion vector or the camera prior. Validation scores are not compared across ablations, which share only the test suites (Section S6.18).

S6.17 Fusion and ONNX export

The re-parameterisation fusion of the temporal super-resolution network does not apply to the generator, since it has no such blocks. Fusion here is a copy in evaluation mode, and the check that fused and unfused outputs agree holds by construction. The fused weights are stored, and their fp16 copies are used for timing.

The export wrapper takes the two real frames, the motion vector, the depth, the camera prior, the time and the three availability flags, and returns the output frame,

$$\hat{I}_\tau = \mathrm{FG}\big(I_0, I_1, M_1, d_1, C, \tau, a_{\text{mv}}, a_{\text{cam}}, a_{\text{dep}}\big). \tag{S106}$$

The exporter is TorchScript based, with opset 17, and the graph is static. The example input is 128×128 pixels for both exported models, the `low` and `igpu` tiers. The check runs ONNX Runtime on the CPU and compares it with PyTorch in float32 on the same example, and the difference is below 10^{-5} for both models. Two limits follow from the export. First, the flow scale of Eq. (S88) is decided from the frame height when the graph is traced, and a 128-line example has $s = 1$ for every tier. The exported graph therefore computes full-resolution flow, and a graph for a 1080-line output must be exported at that size. Second, the exported graph encodes both real frames, so the reuse of Section S6.10 is not in it.

S6.18 Test protocol

All evaluation uses the test split only. Every suite is evaluated in an *engine* mode, where the batch carries the engine data that the suite has, and in a *plain* mode, where the batch carries none, so that the generator sees ordinary video. The 3x multi-frame suites are plain only. Table S49 lists the suites. The synthetic suites use 48 textures of 512×512 pixels from held-out game frames and DIV2K validation images, each rendered at 256×256 pixels. The real suites take up to 12 clips each, four times that for the Vimeo-90K triplets, and 6 at native resolution.

Table S49: Test suites of the frame generator. Modes are interpolation at $\tau = 1/2$ (interp), extrapolation at $\tau = 1.5$ (extrap) and 3x multi-frame generation (mfg3x, plain input only). The engine motion vector of real suites is exact for TartanAir and Sintel and estimated by RAFT otherwise

Suite	Source	Crop (lines × width)	Modes	Notes
`synthetic`	held-out textures	256×256	interp, extrap	the four speed classes, separately
`synthetic-ui`	held-out textures	256×256	interp, extrap	HUD and subtitle changes on every sample
`synthetic-menu`	held-out textures	256×256	interp, extrap	menu on every sample
`synthetic-cut`	held-out textures	256×256	interp, extrap	every sample is a cut
`synthetic-reversal`	held-out textures	256×256	extrap	every camera reverses, in engine mode with and without the camera prior
`vimeo_triplet`	Vimeo-90K	256×448	interp	three frames per clip
`tartanair`	TartanAir	480×640	interp, extrap, mfg3x	exact motion vectors
`sintel`	MPI Sintel	432×1024	interp, extrap, mfg3x	exact motion vectors
`vid4`	Vid4	480×704	interp, extrap, mfg3x	RAFT motion
`vimeo1080p`	Vimeo 1080p	544×960	interp, extrap, mfg3x	frames scaled to 544 lines
`vimeo1080p_native`	Vimeo 1080p	1072×1904	interp, extrap, mfg3x	native resolution

Clip selection. A real clip is the first four frames of a candidate item (three for triplets), centre-cropped to the size of Table S49, after the downscale of the 544-line suite. Candidates are read in order, and a candidate is skipped when it cannot be decoded or when it has a defect. The defects are a blank frame, with a luma standard deviation below 2 on the 0 to 255 scale, a frozen pair, with a mean absolute luma difference below 0.1 on the same scale, and a cut, with half the L1 distance between the normalised 32-bin luma histograms of consecutive frames above 0.5. By this rule the test clips contain no cuts, which makes the gated share of Section S6.11 a false-alarm rate. The sequences of TartanAir and Sintel are played backwards, so that their exact forward flow becomes the motion from the newest frame to the older one. The engine motion vector of I_1 is composed from two consecutive exact motion vectors, $M_1 = m_{21} + \mathcal{W}(m_{10}, m_{21})$, where m_{ij} is the exact motion from frame i to frame j.

On the other real suites the motion is RAFT-small from I_1 to I_0, estimated at full resolution up to a longer side of 1024 pixels. Beyond that the RAFT input is resized to a longer side of 1024 pixels, and its flow is scaled back, so the native suite's motion is estimated at reduced size. For real suites the occlusion mask of the target is estimated by RAFT with a forward-backward consistency test [44], and it is used by the metrics only. For synthetic suites the mask is exact.

Methods. Table S50 lists the methods, how each is gated and what inputs each receives.

Table S50: Methods of the frame-generation test protocol and their inputs. The gate of Eq. (S95) replaces the output of a gated method by I_1. Baselines are never gated, and the frame repeat is I_1 itself, so gating it changes nothing

Method	Definition	Gated	Inputs
frame repeat	I_1	not applicable	both
frame average	$\frac{1}{2}(I_0 + I_1)$	no	both
motion-vector reprojection	Eq. (S107) with τM_1 and $(\tau - 1)M_1$	no	engine only
RIFE 4.26 unchanged	blend B of Eq. (S87)	yes	colour only in both modes
`high`, `low`, `igpu`, `mid`	output $\hat{I}_\tau$	yes	both
ablations	output $\hat{I}_\tau$ of each run	yes	both
UI layer (UI suites only)	$\hat{I}_\tau(1 - \alpha) + c$ on HUD-less frames	yes	engine only

The reprojection baseline is an analytic warp-and-blend that uses linear motion. The two frames are warped by the two priors, $W_0 = \mathcal{W}(I_0, \tau M_1)$ and $W_1 = \mathcal{W}(I_1, (\tau - 1)M_1)$, and they are blended per pixel with a weight that prefers the warp agreeing with the other,

$$\mathrm{Rep}(I_0, I_1, M_1) = \mu\, \tfrac{1}{2}(W_0 + W_1) + (1 - \mu)\, W_1, \qquad \text{(S107)}$$
$$\mu = \sigma\big(40\,(0.1 - \bar{\epsilon})\big), \qquad \bar{\epsilon} = \mathrm{mean}_c\, \big|W_0 - W_1\big|.$$

Where the two warps disagree by more than about 0.1, the weight μ falls to zero and the result is W_1. This treats disagreeing pixels as occluded and gives them the newer frame, which is a heuristic. The baseline needs motion vectors, so it is absent in the plain mode. The networks use no engine data in the plain mode, and the RIFE reference is colour only in both modes, as its design requires.

Metrics. Each sample gives the following values, which follow Section S2.9, with the generated frame clipped to $[0, 1]$ before each metric.

- PSNR on RGB, $10 \log_{10}(1/\mathrm{MSE})$, with the mean squared error over all colour values, and also over the UI pixels or over the occluded pixels when those exist. The occluded pixels are those not visible in I_1 by the target's occlusion mask, and a sample counts only if it has more than 50 of them.
- SSIM with an 11×11 Gaussian window of standard deviation 1.5, computed per colour channel with $C_1 = 0.01^2$ and $C_2 = 0.03^2$, and averaged [13].
- LPIPS with AlexNet [14].
- Interpolation error, the root-mean-square difference on the 0 to 255 scale [16].
- UI ghosting rate, the share of UI pixels whose largest colour error exceeds 0.1.
- Motion magnitude, the mean length of M_1 in pixels, recorded for each sample.

The temporal consistency of the displayed sequence is measured on the interpolation suites in the plain mode, with RAFT-small flows [4]. For each clip, the flow from I_0 to the output and the flow from the output to I_1 are compared with the flow from I_0 to the true target and from the target to I_1. The per-clip value is the mean of the two mean norms of the flow differences, in pixels. It is a sequence version of the tOF measure of TecoGAN [17], not the function in the library, and it is reported per clip, because a few large-motion clips can dominate a suite mean.

The Middlebury comparison uses the 12 sequences of the Middlebury "other" interpolation set [16]. It takes frames 10 and 11 as inputs and the middle frame 10i11 as the ground truth. It runs on the CPU with

colour input only, after training, with the saved checkpoints. Frames of 380 to 480 lines are below the flow line limit of every tier, so this comparison estimates flow at full resolution for every tier.

Gated samples and the reporting rule. A gated output equals I_1 exactly, so a sample is gated when the gate of Eq. (S95) fires and the PSNR of the RIFE output equals that of the frame repeat. The notebook does not build these tables. They are made after the run from the per-sample file. The gate is shared by every method on the same batch. The as-saved table includes gated samples, and the not-gated table excludes them. The test clips contain no cuts by construction, so every gated real sample is a false alarm. Including those samples would measure the gate's false-alarm rate, not how well each generator interpolates, so the not-gated table is the one that compares generators. The gated share is reported for each suite, mode and input mode.

Latency protocol. The per-frame latency of each fused fp16 generator is the mean of 100 timed forward passes at batch size 1, after 10 warm-up passes, with CUDA events and the cuDNN autotuner on. The inputs contain every engine signal. The timing runs at 1920×1080 and at 2560×1440, and the 1920×1080 time of the `low` tier feeds the pacing model of Section S6.12.

S7 Frame generation and real-time cost: results

This section reports every saved output of the frame-generation notebook and of the Middlebury comparison that was run after it. The notebook trains the frame generator, calibrates its scene-cut detector, evaluates every network on the test split and writes the metric files, the plots and the predictions that the tables and figures below show. The method is described in Section S6, the metrics in Section S2.9 and the latency concepts in Section S2.8. For each result we state what is measured, what the table or figure shows, the mechanism where the design explains it, what the result does not show and the number of samples when that number is small.

Two conventions apply throughout. The `high` tier is the notebook's teacher, a network with RIFE 4.26's widths that starts from the pre-trained RIFE weights. The notebook calls the `low` tier the student, and the tables use the article's tier names. Each test result also has two views. The all-sample view averages every sample, including those where the cut gate replaced the network output with the newest real frame. The not-gated view averages only the samples the gate left alone, and it is the view the article uses for plain video. Section S7.2.1 explains why.

[Preliminary] All frame-generation results come from one run. It started on 8 November 2026 on two Tesla T4 GPUs with 4 CPU cores and PyTorch 2.11.0+cu130. It took 10.02 hours to the end of the evaluation, and the summary cell recorded 10.07 hours. The Middlebury comparison in Section S7.2.5 was run afterwards on a CPU with the same checkpoints. Because there is one run, no finding below has a repeat that would show its run-to-run variation.

S7.1 Training runs

The notebook trained the `high` tier for 150 minutes, the `low` tier for 200 minutes, and the `igpu` and `mid` tiers in parallel for 120 minutes each. Before training it built the caches and probed the batch size. After the `low` tier it ran six ablations of 30 minutes each, in three parallel pairs. Figure S36 gives the stage times. The caches took 31.41 minutes against a build budget of 30 minutes, and the test suites and the evaluation took 2.283 and 8.442 minutes.

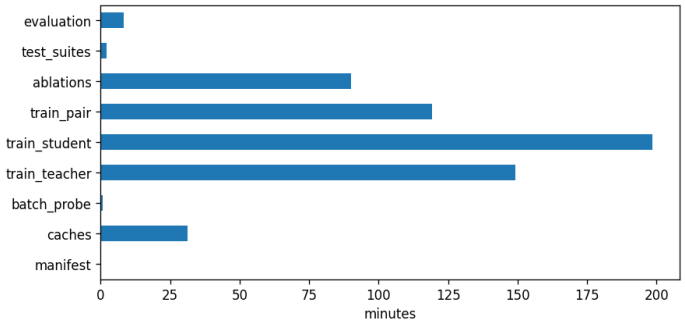


Fig. S36: Wall-clock minutes per stage of the frame-generation run, from the notebook's stage timer

Every validation round scored the checkpoint on held-out batches with engine data, without it, and on the interpolation, extrapolation and UI regions. The checkpoint metric is the mean of the engine and plain scores, so each best value is an average over the two input modes. Table S51 lists the best round of every run. **[Preliminary]** The `high` tier reached 30.99 dB, the `mid` tier 30.47 dB, the `low` tier 29.29 dB and the `igpu` tier 27.35 dB. For the `high` tier the engine and plain scores differ by 0.03 dB, so the engine dropout during training leaves the two input modes close on the validation batches.

Table S51: Training runs (the `high` row is the notebook's teacher): validation PSNR (dB) of the best checkpoint by the checkpoint metric (mean of the engine and plain validation PSNR), with the round and the minutes of its run. The rounds and minutes columns give the validation rounds and the elapsed training time

Run	Rounds	Min.	Best round	Val. PSNR	Engine	Plain	Interp.	Extrap.	UI
`high`	19	148.9	18	30.99	31.01	30.97	32.25	29.30	31.89
`low`	25	198.3	24	29.29	29.68	28.89	30.58	28.44	32.05
`mid`	15	119	14	30.47	30.52	30.42	31.52	29.15	32.95
`igpu`	15	119.2	14	27.35	28.26	26.44	28.98	27.27	31.25
Reference	6	29.96	5	26.89	27.65	26.13	28.19	26.91	29.89
No UI	6	29.92	5	27.75	28.63	26.87	30.66	26.19	n/a
No prior	6	29.88	5	26.41	26.44	26.39	27.19	25.40	31.94
Extrap. only	6	29.93	5	25.69	26.40	24.98	n/a	26.40	30.36
No distill.	6	29.77	5	26.75	27.50	25.99	28.03	26.78	29.70
No inherit.	6	29.86	5	25.99	26.91	25.06	26.88	26.95	28.82

[Preliminary] Every run's best round was its last validation round, so no run had converged when its budget ended. The `low` tier had 25 validation rounds and was still improving at the last one, after 198.3 minutes. The tier ranking therefore partly reflects the budgets, and a longer run would change the gaps between the tiers by an amount this run cannot measure.

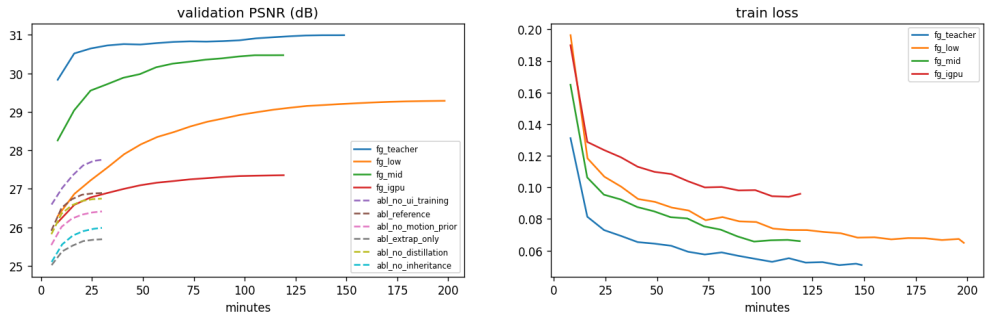


Fig. S37: Validation PSNR (dB, left) against elapsed minutes for every run, with the ablations dashed, and the training loss of the four main runs (right). Training losses of the students include the distillation term and are not comparable with the teacher's loss

[Observation] Figure S37 shows the same picture as the table. The `high` tier rises quickly and flattens late in its budget, the `mid` tier follows, and the `igpu` tier stays lowest. The ablations run for 30 minutes and their curves stop there, with the reference reaching 26.89 dB. The training losses fall in every run. They

are plotted for the shape of each curve, and their levels do not compare the tiers, because the students add a distillation term that the teacher does not have.

The batch probe (Table S52) chose the largest batch whose peak memory stayed below 75 percent of the GPU memory, a margin the notebook keeps for the losses and data that the probe leaves out. For the `high` tier alone the largest such batch was 48 per GPU, at 64.5 percent peak memory, and a batch of 64 reached 85.3 percent and was rejected. With the teacher alongside, the `low` student also used 48, at 59.1 percent. The `mid` tier used half the student's batch, with a minimum of 8, by the notebook's rule. **[Observation]** Throughput levelled off once the batch reached 16 samples, so the larger batches bought little speed.

Table S52: Batch-size probe on GPU 0: peak allocated memory as a share of the GPU memory and training samples per second for the `high` teacher and for the `low` student with the teacher alongside. The probe picks the largest batch with at most 75 percent peak memory

Batch per GPU	`high` peak memory (%)	`high` samples/s	`low` peak memory (%)	`low` samples/s
4	7.3	19.9	6.6	22.6
8	12.4	28	11.2	26
12	17.7	29.7	16.1	26.8
16	22.8	30.2	20.8	27.7
24	33.2	31.1	30.4	28.5
32	43.6	31.3	39.9	28.9
48	64.5	31.9	59.1	29.7
64	85.3	31.3	78.2	30

No run failed and no run wrote an out-of-memory log, so every run trained at the batch the probe chose.

S7.2 Interpolation

Interpolation generates the frame at $\tau = 0.5$ between two real frames. The test suites are the Vimeo-90K triplets, the Vimeo-1080p clips at 544 lines and at their native 1072 lines, Vid4, TartanAir, Sintel, the synthetic suite of game-like scenes and, after the run, the twelve Middlebury sequences. Each suite is run on plain input, with engine data where the suite has it, and the baselines are frame repetition, frame averaging and, with engine data, motion-vector reprojection.

S7.2.1 Why the not-gated view is reported

[Preliminary] The test clips contain no scene cuts, so every real sample that the cut gate replaced is a false alarm, and the replaced output is the newest real frame rather than an interpolation. Including those outputs measures the gate and not the generator. Table S53 shows the effect. On TartanAir one gated sample out of ten lowers the all-sample mean of RIFE 4.26 by 1.55 dB. The not-gated view therefore reports the generators on the samples where generation actually runs. The gate is evaluated separately in Section S7.7.

[Under revision] The gate also fires on samples without a cut. It replaced 18 of 192 plain synthetic samples, a share of 0.094, while its threshold was calibrated for one percent false alarms on continuous pairs. The gap between calibration and test is the reason the not-gated view is needed, and a gate that false-alarms at this rate is not deployable in its present form.

S7.2.2 Plain video

Table S54 gives the not-gated PSNR of every tier and baseline.

[Preliminary] The `high` tier is the best tier on every suite. It beats RIFE 4.26 by 0.46 dB on the Vimeo-90K triplets, by 0.21 dB on Vimeo-1080p at 544 lines, by 0.25 dB at native resolution and by 0.84 dB on Vid4. RIFE 4.26 stays ahead on TartanAir, by 0.06 dB, and on Sintel, by 0.50 dB. The Sintel gap rests on 8 kept samples, and the Vid4 margin rests on 4 samples, which makes it the least certain of the positive margins. On the synthetic suite RIFE 4.26 is also ahead, by 0.91 dB, although the `high` tier has the lower interpolation error there.

[Observation] The two suites where RIFE 4.26 leads are the two with the fewest real training quads, which are Sintel with 28 and TartanAir with 45 quads, against 3,483 Vimeo-90K quads in the cache. This is

Table S53: Effect of the cut gate on plain interpolation (PSNR, dB): all samples against the samples the gate left alone. Gated samples carry the newest real frame, so they lower the all-sample mean of every method that the gate covers

Suite	Gated / samples	RIFE, all	RIFE, kept	`high`, all	`high`, kept
Synthetic	18 of 192	39.62	41.97	38.79	41.06
Vimeo-90K triplet	3 of 48	33.88	34.87	34.31	35.33
Vimeo-1080p, 544 lines	3 of 12	33.25	36.93	33.40	37.14
Vimeo-1080p, native	1 of 6	32.64	33.00	32.85	33.24
Vid4	0 of 4	28.21	28.21	29.05	29.05
TartanAir	1 of 10	28.62	30.18	28.57	30.11
Sintel	2 of 10	25.10	26.88	24.70	26.38

Table S54: Frame interpolation on plain video (no engine data), PSNR (dB), not-gated samples (kept of all samples in brackets). Bold marks the best deployable tier per row. Repeat is the newest real frame, average is the mean of the two real frames and RIFE 4.26 is the pre-trained network unchanged

Suite (kept of samples)	Repeat	Average	RIFE 4.26	`igpu`	`low`	`mid`	`high`
Synthetic (174 of 192)	30.71	36.57	41.97	34.32	37.10	39.26	**41.06**
Vimeo-90K triplet (45 of 48)	23.34	26.36	34.87	29.69	32.94	34.38	**35.33**
Vimeo-1080p, 544 lines (9 of 12)	29.33	32.47	36.93	32.89	34.93	36.52	**37.14**
Vimeo-1080p, native (5 of 6)	27.21	29.37	33.00	29.58	31.21	33.01	**33.24**
Vid4 (4 of 4)	20.69	24.09	28.21	26.23	26.72	28.66	**29.05**
TartanAir (9 of 10)	19.31	21.29	30.18	22.08	25.23	28.64	**30.11**
Sintel (8 of 10)	21.66	23.60	26.88	23.75	24.48	25.97	**26.38**

a hypothesis about the training data mix. The run did not test it, and a run with those suites up-weighted would be needed to confirm it.

[Preliminary] The `mid` tier, at 44.3 GMAC per 1080p frame, is within 0.5 dB of RIFE 4.26 on five of the seven suites, counting Middlebury. The `low` tier, at 12.7 GMAC and with half of RIFE's channel widths, beats frame averaging on every suite and trails RIFE 4.26 by between 1.49 dB on Vid4 and 4.95 dB on TartanAir. The `igpu` tier is barely above averaging on the two 1080p suites, by 0.42 dB at 544 lines and by 0.20 dB at native resolution. **[Observation]** The `low` tier has half the channel widths of RIFE 4.26 (Section S6), and its gap is consistent with that narrower network. The run did not test the cause.

[Preliminary] Table S55 gives the other metrics on the same samples. RIFE 4.26 has the lowest LPIPS on every real suite, and the `high` tier is second on six of the seven suites, with the `mid` tier second on the native suite. Interpolation error ranks the `high` tier first on four suites and RIFE 4.26 first on the others, so the perceptual and error measures disagree about the best network on this set. Occluded-region PSNR orders the networks differently from full-frame PSNR on several suites, most clearly on the triplets, where RIFE 4.26 leads on occluded pixels while `high` leads on the full frame. The occluded pixels are where a blend of the two frames cannot be right, so the full-frame PSNR does not describe them.

S7.2.3 Engine data

Table S56 gives the not-gated interpolation results with engine data. **[Preliminary]** Motion-vector reprojection is below RIFE 4.26 on every real suite. On all samples, the gap is 0.65 dB on Sintel and 2.97 dB on the Vimeo-90K triplets. The comparison is made within the engine mode, so both networks see the same samples, the same gate and the same engine data.

[Observation] On TartanAir the motion vectors are exact, and reprojection is still 1.81 dB below RIFE 4.26, so exact vectors alone do not make reprojection competitive with a learned blend. The effect of engine hints on the `high` tier is given in Section S7.4, where the comparison is paired on the same samples.

Table S55: Frame interpolation on plain video, the metrics other than PSNR, not-gated samples. Synth. is the synthetic suite and 1080p the Vimeo-1080p clips at 544 lines

Method	Synth.	Triplet	1080p	Native	Vid4	TartanAir	Sintel
SSIM (higher is better)							
Repeat	0.800	0.728	0.859	0.861	0.677	0.605	0.600
Average	0.833	0.769	0.881	0.884	0.772	0.622	0.643
RIFE 4.26	0.970	0.955	0.975	0.964	0.909	0.908	0.816
`igpu`	0.882	0.859	0.904	0.900	0.876	0.687	0.674
`low`	0.938	0.924	0.948	0.936	0.894	0.791	0.704
`mid`	0.967	0.949	0.973	0.964	0.923	0.884	0.780
`high`	0.975	0.959	0.976	0.964	0.926	0.912	0.801
LPIPS (lower is better)							
Repeat	0.042	0.062	0.043	0.076	0.068	0.158	0.185
Average	0.067	0.095	0.062	0.087	0.154	0.167	0.180
RIFE 4.26	0.011	0.017	0.013	0.029	0.035	0.037	0.089
`igpu`	0.067	0.083	0.061	0.085	0.104	0.185	0.193
`low`	0.041	0.046	0.037	0.066	0.068	0.137	0.178
`mid`	0.021	0.028	0.019	0.032	0.044	0.069	0.126
`high`	0.017	0.023	0.017	0.033	0.040	0.052	0.113
Interpolation error, root-mean-square on the 0 to 255 scale (lower is better)							
Repeat	13.53	19.4	12.86	14.91	23.91	29	21.45
Average	9.47	14.11	9.47	10.96	16.17	22.98	17.15
RIFE 4.26	3.791	5.349	4.743	6.668	10.94	8.494	11.76
`igpu`	7.765	10.1	8.388	10.05	12.68	21.16	16.71
`low`	5.335	6.863	6.134	8.091	12.22	15.27	15.33
`mid`	4.117	5.753	5.074	6.849	9.864	10.15	13.01
`high`	3.5	5.184	4.804	6.819	9.598	8.426	12.41
PSNR over occluded pixels (dB, higher is better)							
Repeat	18.74	19.35	31.45	30.89	11.91	23.05	17.92
Average	24.18	22.58	32.32	34.92	15.66	25.77	19.81
RIFE 4.26	29.63	29.10	36.98	37.47	19.97	33.72	20.99
`igpu`	25.97	25.44	34.45	35.52	15.30	27.43	19.16
`low`	27.66	27.23	36.06	35.35	16.48	29.34	19.71
`mid`	29.07	27.65	36.05	36.69	19.57	31.39	20.38
`high`	30.71	28.61	37.11	36.69	20.95	33.52	20.65

Table S56: Frame interpolation with engine data (motion vectors, depth and camera where the suite has them), PSNR (dB), not-gated samples. The gate depends on the input mode, so the kept samples differ from the plain table

Suite (kept of samples)	RIFE 4.26	MV reprojection	`igpu`	`low`	`mid`	`high`
Synthetic (187 of 192)	41.19	36.48	36.54	37.93	38.57	**40.02**
Vimeo-90K triplet (48 of 48)	34.80	31.83	33.34	34.08	34.47	**35.03**
Vimeo-1080p, 544 lines (11 of 12)	35.63	34.66	34.31	34.90	35.38	**35.98**
Vimeo-1080p, native (6 of 6)	32.67	31.81	31.98	31.34	32.68	**32.89**
Vid4 (4 of 4)	28.21	27.35	28.05	28.33	28.86	**28.94**
TartanAir (10 of 10)	29.79	27.98	29.29	29.58	29.47	**29.98**
Sintel (8 of 10)	26.88	25.17	25.75	25.54	**26.21**	26.10

S7.2.4 Three-frame generation

Three-frame generation produces two frames between real frames k and $k+3$, at $\tau = 1/3$ and $\tau = 2/3$, with plain input only. Table S57 gives the not-gated results.

Table S57: 3x multi-frame generation on plain real clips (frames 0 and 3 give targets at tau = 1/3 and 2/3), PSNR (dB), not-gated samples. Bold marks the best deployable tier per row

Suite (kept of samples)	Repeat	Average	RIFE 4.26	`igpu`	`low`	`mid`	`high`
Vimeo-1080p, 544 lines (16 of 24)	29.87	32.27	37.01	32.86	34.54	36.24	**37.07**
Vimeo-1080p, native (8 of 12)	29.51	31.03	34.76	30.38	32.18	34.66	**35.20**
Vid4 (8 of 8)	19.56	21.42	26.03	24.45	25.44	26.52	**26.82**
TartanAir (14 of 20)	18.71	20.61	27.77	21.02	23.47	26.63	**27.91**
Sintel (10 of 20)	21.66	23.50	26.14	23.11	23.53	25.22	**25.71**

[Preliminary] The `high` tier matches or beats RIFE 4.26 on four of the five suites. It is ahead on Vid4 by 0.80 dB, on native Vimeo-1080p by 0.44 dB, on Vimeo-1080p at 544 lines by 0.06 dB and on TartanAir by 0.14 dB. It is behind on Sintel by 0.43 dB.

[Under revision] The gate fires more often on these samples, because the real frames are three apart and the histogram threshold was calibrated on adjacent pairs. It fired on 10 of 20 Sintel samples, 6 of 20 TartanAir samples, 8 of 24 Vimeo-1080p samples and 4 of 12 native samples, and on none of the 8 Vid4 samples. Three-frame results on Sintel and TartanAir therefore rest on heavily gated sets, and the cut gate needs recalibration for wider frame gaps before these rows can be read as generator quality.

S7.2.5 Middlebury

Each Middlebury other-set sequence gives two input frames and a ground-truth middle frame, so an interpolated frame is scored directly against it. The comparison was run after the notebook on a CPU, with the RIFE 4.26 weights and the checkpoints of the run, and with colour input only. The frames are about 380 to 480 lines high, below the 576-line limit of the `igpu` and `low` flows and the 1152-line limit of the `mid` and `high` flows, so every tier estimates flow at full resolution, as it does in training. The comparison also includes RIFE 4.26 with its flow at half and at quarter resolution, which shows how much the flow scale matters on this set. Table S58 gives the PSNR of every sequence.

[Preliminary] On the twelve sequences the `high` tier averages 34.23 dB against 33.34 dB for RIFE 4.26, a margin of 0.89 dB. The `mid` tier averages 33.83 dB, the `low` tier 31.56 dB, the `igpu` tier 30.30 dB and frame averaging 28.02 dB. The mean hides large differences between sequences. The `high` tier beats RIFE 4.26 on nine of the twelve sequences, by 4.34 dB on Dimetrodon at most, and it trails by 3.25 dB on Beanbags, 1.29 dB on Urban2 and 1.89 dB on Urban3. The `low` tier beats RIFE 4.26 on three sequences, and the `igpu` tier beats it on two, Rubberwhale and Walking, with a tie on Dimetrodon. With twelve scenes and one run, the margin of the `high` tier is a measured result on this set and not a general ranking.

[Observation] The flow scale matters on this set. RIFE 4.26 with flow at half resolution averages 30.90 dB, which is 2.44 dB below the full-flow network, and at quarter resolution it averages 27.67 dB. Halving the flow saves cost, from 76.7 to 38.5 GMAC per 1080p frame, but on nine sequences it loses quality. The flow scale is not monotone per sequence, because the half-resolution flow is the better of the two on three sequences, so no single flow scale suits every scene.

S7.3 Extrapolation

Extrapolation generates a frame after the newest real frame, at $\tau = 1.5$, from the two real frames before it. No later frame is available to hold, so the output must predict motion. Frame repetition is the cheapest baseline and it has no motion error at all when the scene is static. Table S60 gives the plain results and Table S59 the results with engine data.

S7.3.1 Plain video

[Preliminary] The `high` tier beats frame repetition on every suite, by 1.86 dB at the least, on native Vimeo-1080p, and by 5.57 dB at the most, on TartanAir. The `low` tier beats repetition on every suite too, by

Table S58: Middlebury other-set interpolation (frames 10 and 11 to frame 10 and 11 interpolated, colour only, no engine data), PSNR (dB) per sequence, measured on CPU after the run with the saved checkpoints. Bold marks the best deployable tier per row

Sequence	Avg.	RIFE	RIFE half	RIFE quarter	igpu	low	mid	high
Beanbags	26.64	32.64	30.72	26.59	26.84	27.62	27.82	**29.38**
Dimetrodon	32.53	36.63	31.88	30.51	36.63	37.26	40.16	**40.97**
DogDance	28.41	30.81	31.72	28.27	30.00	30.72	**31.13**	31.05
Grove2	23.28	30.67	25.40	22.10	26.99	29.90	32.88	**34.04**
Grove3	21.31	27.12	23.75	21.23	24.87	26.99	28.65	**29.37**
Hydrangea	27.66	35.38	32.62	25.81	33.35	34.97	**36.26**	35.54
MiniCooper	24.71	30.24	30.59	27.28	28.03	29.16	30.16	**30.38**
RubberWhale	38.70	39.25	36.51	36.31	40.18	39.70	42.27	**43.26**
Urban2	26.68	37.52	33.36	28.11	27.16	30.59	35.92	**36.23**
Urban3	27.33	35.31	32.11	28.65	28.07	29.19	**34.09**	33.42
Venus	25.07	32.83	28.64	25.41	29.04	30.21	34.22	**35.21**
Walking	33.93	31.73	33.53	31.82	**32.42**	32.39	32.38	31.89
Mean (12)	28.02	33.34	30.90	27.67	30.30	31.56	33.83	34.23
GMAC per 1080p frame	n/a	76.7	38.5	33.6	9.4	12.7	44.3	76.7

Table S59: Frame extrapolation (tau = 1.5) with engine data, PSNR (dB), not-gated samples. RIFE 4.26 takes no engine input, and its kept samples differ from the plain table because the gate depends on the input mode. Bold marks the best deployable tier per row

Suite (kept of samples)	Repeat	Avg.	MV	RIFE	igpu	low	mid	high
Synthetic (187 of 192)	29.79	27.25	32.88	28.26	32.20	33.56	34.14	**34.38**
Vimeo-1080p, 544 lines (11 of 12)	29.50	27.53	32.03	27.07	31.77	32.11	32.57	**32.82**
Vimeo-1080p, native (6 of 6)	28.58	26.75	29.86	27.08	29.88	29.86	30.60	**30.63**
Vid4 (4 of 4)	21.41	19.73	25.89	21.95	25.81	24.99	25.63	**25.92**
TartanAir (10 of 10)	18.88	18.71	26.91	19.02	24.79	24.77	**25.22**	25.12
Sintel (8 of 10)	21.90	21.42	24.20	21.97	**24.19**	24.08	24.15	24.14

Table S60: Frame extrapolation (tau = 1.5) on plain video, PSNR (dB), not-gated samples (kept of all samples in brackets). Bold marks the best deployable tier per row

Suite (kept of samples)	Repeat	Average	RIFE 4.26	igpu	low	mid	high
Synthetic (174 of 192)	30.68	28.01	29.03	29.64	32.65	34.86	**35.20**
Vimeo-1080p, 544 lines (9 of 12)	30.15	28.15	27.27	29.92	31.31	33.08	**33.38**
Vimeo-1080p, native (5 of 6)	28.79	26.82	27.01	27.86	28.94	**31.01**	30.65
Vid4 (4 of 4)	21.41	19.73	21.95	21.86	23.93	25.31	**25.67**
TartanAir (9 of 10)	19.37	19.15	19.51	19.55	22.28	24.82	**24.94**
Sintel (8 of 10)	21.90	21.42	21.97	22.14	22.94	23.85	**24.02**

between 0.15 dB and 2.91 dB. RIFE 4.26 is at most 0.54 dB above repetition, on Vid4, and it falls below repetition on both Vimeo-1080p suites, by 2.89 dB at 544 lines.

[Observation] RIFE 4.26 is an interpolation network [5], so at $\tau = 1.5$ its blend lies outside the range it was built for, and its output tends to stay near the newest frame. The one network that the run trained for extrapolation as well is the `high` tier. This is a mechanism we infer from the design, and the run did not isolate it.

S7.3.2 Engine data

[Preliminary] With engine data the `high` tier is the best tier on the synthetic suite and on both Vimeo-1080p suites, although on native Vimeo-1080p its lead over the `mid` tier is only 0.03 dB. On TartanAir, where the motion vectors are exact, motion-vector reprojection is still better by 1.79 dB, and on Sintel and Vid4 the two are within 0.1 dB of each other. Reprojection is therefore a strong baseline for extrapolation when its input is exact, and a learned network does not clearly beat it on every suite.

S7.4 Engine hints against none

The engine hints are the motion vectors, the depth, the camera prior and the availability flags of the engine. The network receives them through zero-initialised channels (Section S6), so a network trained with hints can be run without them. Table S61 compares the `high` tier with and without hints on the same samples, using all samples so that the two modes share the sample set.

Table S61: Engine hints against none for the `high` tier, PSNR (dB) over all samples, so both input modes share the same samples. The difference is engine minus plain

	Interpolation			Extrapolation		
Suite	Plain	Engine	Diff.	Plain	Engine	Diff.
Synthetic	38.79	39.34	+0.55	33.48	33.85	+0.37
Vimeo-90K triplet	34.31	35.03	+0.72	n/a	n/a	n/a
Vimeo-1080p, 544 lines	33.40	34.05	+0.64	31.88	32.50	+0.62
Vimeo-1080p, native	32.85	32.89	+0.05	30.13	30.63	+0.50
Vid4	29.05	28.94	−0.11	25.67	25.92	+0.25
TartanAir	28.57	29.98	+1.42	23.89	25.12	+1.23
Sintel	24.70	24.48	−0.23	22.64	22.74	+0.09

[Preliminary] For extrapolation the hints never lower the score on any suite. They raise it by between 0.09 dB on Sintel and 1.23 dB on TartanAir, where the vectors are exact. For interpolation the effect is mixed. The hints raise the score on five of the seven suites, from 0.05 dB on native Vimeo-1080p to 1.42 dB on TartanAir, and they lower it on Vid4 and Sintel, by 0.11 and 0.23 dB.

[Observation] The gain from hints on interpolation is largest on TartanAir, where the vectors are exact, and close to zero on the native Vimeo-1080p clips. The hints are not a uniform improvement on interpolation, and a single number for their effect would hide this. On the synthetic suite RIFE 4.26 stays ahead of the `high` tier in both modes, by 1.14 dB with engine data.

S7.5 User interface

The synthetic HUD suite draws HUD tiles, a crosshair and a subtitle that changes between the real frames. Its target keeps the UI of the newest real frame, so a method that copies that UI is correct there. Table S62 gives the UI measures, which are the PSNR over the whole frame, the PSNR inside the UI pixels and the UI ghosting rate. The rate is the share of UI pixels that are more than 10 percent wrong in at least one colour channel, so it counts the UI pixels that a blend has visibly corrupted.

Table S62: Synthetic HUD and subtitle suite (48 samples per mode, all samples): PSNR over the frame, PSNR inside UI pixels and the UI ghosting rate (share of UI pixels off by more than 10 percent), per mode and input mode

Method	PSNR	UI PSNR	UI ghosting rate
Interpolation, engine data			
Repeat	26.39	47.86	0.025
Average	26.12	31.67	0.300
RIFE 4.26	27.47	22.18	0.228
MV reprojection	25.48	16.66	0.302
`igpu`	31.21	31.53	0.062
`low`	32.40	33.23	0.047
`mid`	33.02	34.38	0.048
`high`	33.40	32.63	0.049
`igpu` + UI layer	32.83	52.58	0.011
`low` + UI layer	34.64	53.79	0.010
Ablation, no UI training	25.76	18.52	0.362
Interpolation, plain			
Repeat	26.39	47.86	0.025
Average	26.12	31.67	0.300
RIFE 4.26	28.50	23.37	0.206
`igpu`	30.47	32.08	0.049
`low`	32.03	34.21	0.040
`mid`	33.03	34.25	0.049
`high`	33.66	32.92	0.048
Ablation, no UI training	26.39	20.06	0.341
Extrapolation, engine data			
Repeat	26.96	46.69	0.020
Average	23.04	33.01	0.225
RIFE 4.26	26.19	32.05	0.048
MV reprojection	24.37	18.50	0.363
`igpu`	29.00	32.59	0.050
`low`	29.78	34.13	0.042
`mid`	30.45	35.90	0.033
`high`	30.32	33.46	0.054
`igpu` + UI layer	29.92	48.83	0.015
`low` + UI layer	30.75	49.46	0.013
Ablation, no UI training	24.97	21.15	0.260
Extrapolation, plain			
Repeat	26.96	46.69	0.020
Average	23.04	33.01	0.225
RIFE 4.26	26.21	30.41	0.051
`igpu`	27.00	34.91	0.025
`low`	29.13	33.94	0.035
`mid`	30.73	34.70	0.033
`high`	31.02	32.66	0.047
Ablation, no UI training	25.58	24.15	0.139

[Preliminary] With engine data the `low` tier has a UI ghosting rate of 0.047 for interpolation and the `high` tier 0.049. RIFE 4.26 has 0.228 and motion-vector reprojection 0.302. The predicted UI mask gives the network a region to copy from the newest frame, and the learned tiers stay far below the blends. Without UI training the rate rises to 0.362, so the UI training is what keeps the UI region clean. With the engine's separate UI layer composited after generation, the rate falls to 0.010 and the UI PSNR rises to 53.79 dB.

[Observation] Repetition has the lowest ghosting rate of all, at 0.025, because the target UI equals the newest frame's UI and repetition copies it exactly. Repetition is therefore the reference for the UI region on this suite, and a low ghosting rate does not by itself show that a network is better than repetition overall. The plain-input rates in Table S62 show the same ordering: the `low` tier has 0.040 and RIFE 4.26 0.206.

Figure S38 shows one HUD sample with every method, and Fig. S39 shows the four special-case measures together.



Fig. S38: HUD and subtitle sample from the test split: target, the baselines, RIFE 4.26, the `low` tier with and without engine data, the `low` tier with the UI layer, and the predicted UI mask, blend weight and flow

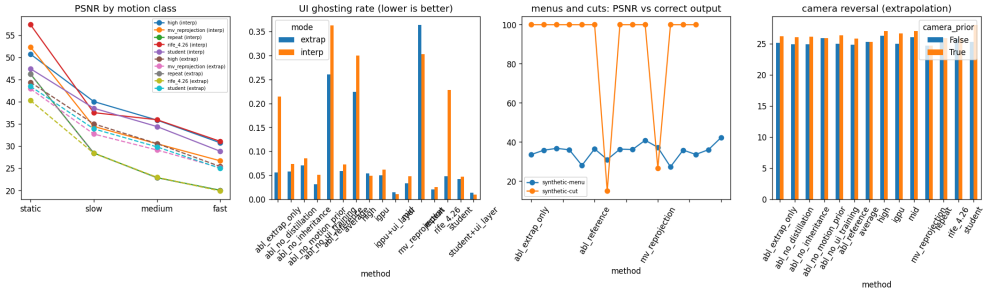


Fig. S39: Special cases with engine data. Left: PSNR by motion class for interpolation (solid) and extrapolation (dashed). Second: UI ghosting rate for interpolation and extrapolation. Third: PSNR against the newest real frame on the menu and cut suites. Right: extrapolation on the reversal suite without and with the late-latched camera prior

S7.5.1 Menus and scene cuts

A menu is a full-screen panel that replaces the scene for a sample, and a scene cut is a sample whose second frame comes from a different scene. In both suites the correct output is the newest real frame, so the gate and the repetition baseline are the references. Table S63 gives the results.

Table S63: Menus and scene cuts with engine data, PSNR (dB) against the correct output (the newest real frame), all samples. The gate repeats the newest real frame, so a gated sample scores the 100 dB cap of the metric

Method	Menu, interp.	Menu, extrap.	Cut, interp.	Cut, extrap.
Repeat	34.82	36.96	100.00	100.00
Average	32.25	29.80	15.01	14.96
RIFE 4.26	33.36	33.95	100.00	100.00
MV reprojection	26.69	27.97	26.87	26.20
`igpu`	35.62	36.79	100.00	100.00
`low`	35.12	37.04	100.00	100.00
`mid`	36.42	38.12	100.00	100.00
`high`	35.26	37.49	100.00	100.00

[Preliminary] On menus the learned tiers are a little above repetition. With engine data the `low` tier beats repetition by 0.30 dB on interpolation and the `high` tier by 0.43 dB, and the `mid` tier by 1.60 dB. Motion-vector reprojection and frame averaging fall well below repetition, because they blend the menu with the scene.

[Observation] On the cut suite every method that the gate covers scores the 100 dB cap, because the gate outputs the newest real frame, which is the target. The averaging and reprojection baselines are not gated and score far lower. The 100 dB values therefore show that the gate fires on every cut and does not show that any network generates well across a cut. The cut suite contributes no evidence about generation quality.

S7.6 Camera reversal and the late-latched prior

A camera reversal is a sample in which the camera turns sharply in the last real frame, so that the motion before the reversal says little about the motion after it. The late-latched prior is a camera-only flow, sampled from the newest camera input and given to the extrapolation network as an engine hint (Section S6). The reversal suite has 48 samples, and each is run twice, once with the prior and once with the camera input switched off.

[Preliminary] Repetition, frame averaging and RIFE 4.26 do not use the camera, so their scores are the same in both settings. Repetition scores 25.91 dB in both. The right panel of Fig. S39 shows the networks and the ablations with and without the prior. **[Observation]** For those networks the bars with the prior are higher than the bars without it, so the prior helps where the camera reverses. Figure S39 is the only result of this section for the reversal suite. The generated results average the two camera settings, so the size of the prior's effect is not quoted here.

S7.7 Scene cuts and the cut gate

The cut gate decides, for each sample, whether to generate or to repeat the newest real frame. It scores the two real frames with the distance between their 32-bin luma histograms, the half-L1 distance of the normalised histograms. When the motion vector of the newer frame is available, the score adds twice the mean difference between the newer frame and the older frame warped by that motion vector, so that fast motion is not read as a cut. Each score is compared with its own threshold, which was set on synthetic validation pairs to give at most one percent false alarms on continuous pairs. Figure S40 shows the score distributions of those pairs and of the synthetic cut pairs, and Table S64 gives the thresholds.

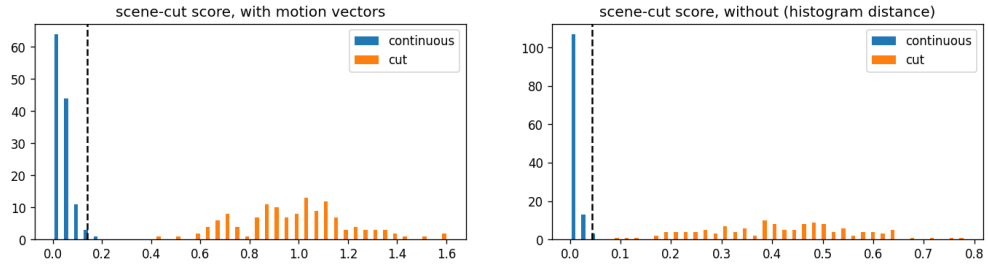


Fig. S40: Scene-cut scores of the calibration pairs. Left: with motion vectors. Right: histogram distance alone. Continuous pairs in blue, cut pairs in orange, and the dashed line is the threshold of each score

Table S64: Scene-cut detector thresholds calibrated on the synthetic validation pairs (one percent false alarms on continuous pairs) and the detection rate on the synthetic cut pairs

Score	Threshold	Cuts detected (of 48)	False-alarm target
With motion vectors (motion-compensated)	0.1417	1.000	0.01
Without motion vectors (histogram only)	0.04312	1.000	0.01

[Preliminary] Both gates detect every calibration cut, with detection rates of 1.000 with motion vectors and 1.000 without them, and the synthetic cut suite confirms this on the test split, with 48 of 48 cuts gated with motion vectors and 48 of 48 without. **[Observation]** The cut scores separate cleanly from continuous motion in Fig. S40, and the separation looks wider with motion vectors. The histogram-only threshold lies close to the continuous mass, which suggests that small changes in brightness or motion could cross it. We did not test this.

The real test clips contain no cuts, so every gated real sample is a false alarm. Table S65 lists the gate by suite, mode and input. **[Under revision]** With motion vectors the gate fired on 2 of 10 Sintel samples and on 1 of 12 Vimeo-1080p samples, and on none of the TartanAir and Vimeo-90K triplet samples. Without motion vectors the gate fired on 3 of 48 triplets, on 3 of 12 Vimeo-1080p samples and on 2 of 10 Sintel samples. On the synthetic plain suite the share is 0.094 against a target of 0.01. A threshold calibrated on adjacent synthetic pairs therefore does not transfer to real content, and to wider frame gaps, without recalibration.

Table S65: Cut gate by suite, mode and input mode: samples the gate replaced by the newest real frame, out of all samples, the true cuts in the suite and the gated share. On the real suites every gated sample is a false alarm

Suite	Mode	Input	Gated	Samples	True cuts	Gated share
sintel	extrap	engine	2	10	0	0.200
sintel	extrap	plain	2	10	0	0.200
sintel	interp	engine	2	10	0	0.200
sintel	interp	plain	2	10	0	0.200
sintel	mfg3x	plain	10	20	0	0.500
synthetic	extrap	engine	5	192	0	0.026
synthetic	extrap	plain	18	192	0	0.094
synthetic	interp	engine	5	192	0	0.026
synthetic	interp	plain	18	192	0	0.094
synthetic-cut	extrap	engine	48	48	48	1.000
synthetic-cut	extrap	plain	48	48	48	1.000
synthetic-cut	interp	engine	48	48	48	1.000
synthetic-cut	interp	plain	48	48	48	1.000
synthetic-menu	extrap	engine	1	48	0	0.021
synthetic-menu	extrap	plain	5	48	0	0.104
synthetic-menu	interp	engine	2	48	0	0.042
synthetic-menu	interp	plain	7	48	0	0.146
synthetic-reversal	extrap	engine	2	96	0	0.021
synthetic-ui	extrap	engine	11	48	0	0.229
synthetic-ui	extrap	plain	6	48	0	0.125
synthetic-ui	interp	engine	7	48	0	0.146
synthetic-ui	interp	plain	8	48	0	0.167
tartanair	extrap	engine	0	10	0	0.000
tartanair	extrap	plain	1	10	0	0.100
tartanair	interp	engine	0	10	0	0.000
tartanair	interp	plain	1	10	0	0.100
tartanair	mfg3x	plain	6	20	0	0.300
vid4	extrap	engine	0	4	0	0.000
vid4	extrap	plain	0	4	0	0.000
vid4	interp	engine	0	4	0	0.000
vid4	interp	plain	0	4	0	0.000
vid4	mfg3x	plain	0	8	0	0.000
vimeo1080p	extrap	engine	1	12	0	0.083
vimeo1080p	extrap	plain	3	12	0	0.250
vimeo1080p	interp	engine	1	12	0	0.083
vimeo1080p	interp	plain	3	12	0	0.250
vimeo1080p	mfg3x	plain	8	24	0	0.333
vimeo1080p_native	extrap	engine	0	6	0	0.000
vimeo1080p_native	extrap	plain	1	6	0	0.167
vimeo1080p_native	interp	engine	0	6	0	0.000
vimeo1080p_native	interp	plain	1	6	0	0.167
vimeo1080p_native	mfg3x	plain	4	12	0	0.333
vimeo_triplet	interp	engine	0	48	0	0.000
vimeo_triplet	interp	plain	3	48	0	0.062

S7.8 Temporal consistency

Generated frames sit between real frames in the displayed sequence, so the motion has to stay smooth across the real, generated and real frames. The temporal measure tOF compares the optical flow of the displayed sequence with the flow of the true sequence, in pixels, and lower is better. It is computed for interpolation on plain input, because the displayed sequence is then the ordinary-video case. Table S66 gives the mean and maximum over clips of each suite, since a suite mean can be set by a few clips with large motion.

Table S66: Temporal consistency of the displayed sequence (tOF, pixels, lower is better) without engine data: mean and maximum over the clips of each suite (12 triplets, 10 1080p clips and 10 TartanAir clips, all real clips that passed the clip checks)

Method	Triplet mean	Triplet max	1080p mean	1080p max	TartanAir mean	TartanAir max
Average	1.774	14.33	6.369	58.2	8.282	18.34
MV reprojection	0.3057	0.741	11.48	123.1	0.7497	1.427
RIFE 4.26	0.2838	0.5204	8.738	90.92	0.6722	1.121
`igpu`	1.653	13.69	9.139	92.46	7.798	18.12
`low`	1.095	9.746	10.08	105.9	5.349	16.74
`mid`	0.4191	1.44	9.92	104.5	1.806	7.194
`high`	0.3121	0.5327	10.04	102.6	0.8344	2.357

[Preliminary] On the Vimeo-90K triplets the `high` tier scores 0.3121 against 0.2838 for RIFE 4.26, and the `low` tier scores 1.095. On TartanAir the `low` tier scores 5.349 against 0.8344 for the `high` tier. Frame averaging on the triplets has a mean of 1.774 and a maximum over clips of 14.33, so one clip dominates the suite mean.

[Under revision] The measure does not rank the methods consistently across suites. On the Vimeo-1080p clips frame averaging has the lowest mean, 6.369, below the 8.739 of RIFE 4.26, although a frame average is a blend of two real frames. A flow measure may reward an output whose motion is near the mean of its neighbours, and a frame average is such an output. This is a plausible mechanism that the run did not test. We do not use tOF to rank the methods in this section.

S7.9 Real clips overview

Figure S41 summarises the real suites in one chart per mode and input. It shows the mean PSNR of each method on each real suite, and it is the figure behind the tables above.

[Observation] In plain interpolation the `high` tier and RIFE 4.26 are the two tallest bars on most suites, and repetition and averaging are the shortest. In plain extrapolation the learned tiers rise above repetition on every suite, and RIFE 4.26 falls below it on the Vimeo-1080p suites. With engine data, reprojection is tall on TartanAir and short on the Vimeo-90K triplets, which matches the table in Section S7.2.3. The chart shows the ordering but not the margins, so the tables carry the numbers.

S7.10 Ablations

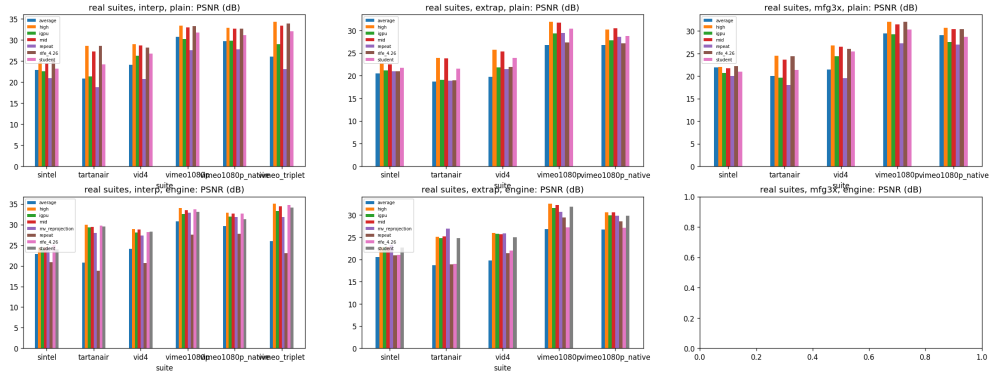


Fig. S41: Mean PSNR (dB) of every method on the real suites, for interpolation, extrapolation and three-frame generation with plain input, and for interpolation and extrapolation with engine data. The three-frame panel has no engine variant

Each ablation changes one part of the `low` recipe and trains it for 30 minutes. Table S67 gives the six runs on six real and synthetic columns, with the difference to the reference in brackets, and Fig. S42 plots the same differences. The ablations use all samples, which keeps their means comparable with the tables of the reference run.

Table S67: The six 30-minute ablations of the `low` recipe, PSNR (dB) over all samples. Each run after the reference shows its difference to the reference in brackets below its value. The last column is the UI ghosting rate on the synthetic HUD suite

Run	Triplet, interp., plain	Triplet, interp., engine	TartanAir, extrap., engine	Synthetic, interp., engine	UI ghosting, interp., engine
Reference	28.01	32.73	24.24	34.60	0.073
No UI training	30.00 (+1.99)	33.64 (+0.90)	24.93 (+0.69)	36.54 (+1.94)	0.362
No motion prior	29.19 (+1.18)	29.36 (−3.38)	19.04 (−5.20)	32.39 (−2.21)	0.051
Extrapolation only	21.02 (−6.99)	21.29 (−11.44)	24.72 (+0.47)	21.77 (−12.83)	0.215
No distillation	27.71 (−0.30)	32.53 (−0.21)	22.97 (−1.27)	34.22 (−0.38)	0.074
No RIFE inheritance	25.47 (−2.55)	31.94 (−0.80)	24.55 (+0.30)	33.93 (−0.67)	0.085

[Preliminary] Removing RIFE inheritance lowers plain interpolation on the Vimeo-90K triplets by 2.55 dB, from 28.01 to 25.47 dB. Removing the motion prior lowers TartanAir with engine data by 5.20 dB, the largest drop of any ablation on that column. On plain triplets the same ablation scores 1.18 dB above the reference. The extrapolation-only run fails at interpolation, scoring 21.29 dB on the triplets with engine data against 32.73 dB for the reference.

[Preliminary] Distillation from the `high` tier adds between 0.17 and 1.27 dB across the columns of the table, with the smallest gain on the synthetic extrapolation suite and the largest on TartanAir.

[Observation] Removing UI training raises PSNR in every column of the table, by between 0.69 dB on TartanAir with engine data and 1.94 dB on the synthetic interpolation suite. It also raises the UI ghosting rate in interpolation with engine data from 0.073 to 0.362. The triplets, the real clips and the synthetic suite of Table S67 contain no HUD overlay, so their PSNR does not measure UI handling. The trade-off between PSNR and the UI rate is therefore visible at this budget, and we did not test whether a longer run changes it.

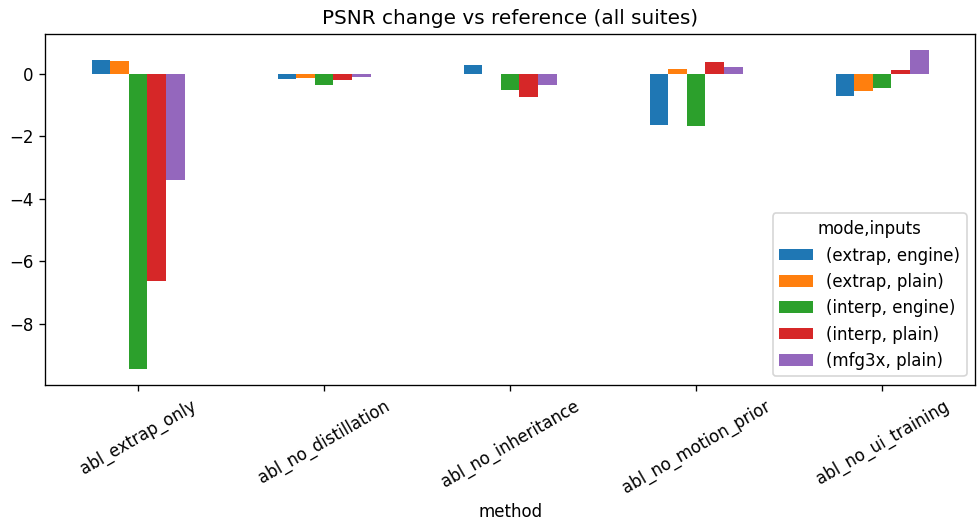


Fig. S42: PSNR change (dB) of each ablation against the reference, for five groups of suites and modes. The reference is zero and is not drawn

Each ablation is one 30-minute run and the reference is one run as well, so every difference in this section is a measured result for this budget and not an established effect. Differences of a few tenths of a decibel may lie within run-to-run variation, which this configuration has not measured, so the distillation effect is given as a range across the columns.

S7.11 Model tiers and latency

Table S68 lists the four tiers that the run trained, with their parameters and their cost per generated 1080p frame. The flow scale is the factor applied to a 1080p frame before flow estimation, which follows from each tier's flow limit. The tier `igpu` and `low` estimate flow at half resolution on a 1080p frame, and `mid` and `high` at full resolution. The feature encoder runs once per real frame in a running pipeline, so its share of the cost is reported separately. **[Confirmed]** The fused network reproduces the unfused network exactly on the random input used for the check, so fusion is a copy of the weights.

Table S68: Frame-generation tiers: parameters, the flow scale applied to a 1080p frame, multiply-accumulates per generated 1080p frame (GMAC) and the largest difference between the training and the fused network on a random input

Tier	Parameters	Flow scale at 1080p	GMAC per 1080p frame	of which feature encoder	Fusion error
`igpu`	454K	0.5	9.4	1.6	0
`low`	953K	0.5	12.7	1.6	0
`mid`	2.29M	1	44.3	6.3	0
`high`	5.71M	1	76.7	6.3	0

[Preliminary] Cost in multiply-accumulates does not predict measured time on the T4. Table S69 gives the latency of each tier at 1080p and 1440p, with the arithmetic estimate that the notebook derives at an assumed 4 TFLOPS of sustained integrated-GPU throughput. The `igpu` tier at 1080p is 46.8 ms on the T4, which is 9.95 times its arithmetic estimate. The notebook counts only multiply-accumulates, so the gap

is most likely the overhead of eager execution and the memory traffic of the feature maps. We have not profiled it, and the gap is a property of the eager runtime that the run used.

Table S69: Measured latency of the fused FP16 generators on one Tesla T4 in eager PyTorch with cuDNN autotuning (100 timed passes at batch 1 after 10 warm-up passes) and the arithmetic estimate used by the notebook

Tier and output	Mean (ms)	95th percentile (ms)	GMAC	Arithmetic estimate at 4 TFLOPS (ms)
`igpu`, 1080p	46.8	47.3	9.4	4.7
`igpu`, 1440p	50.0	50.4	16.7	8.4
`low`, 1080p	48.1	48.6	12.7	6.4
`low`, 1440p	51.9	52.6	22.6	11.3
`mid`, 1080p	106.6	107.3	44.3	22.1
`mid`, 1440p	85.7	87.2	78.8	39.4
`high`, 1080p	120.4	121.5	76.7	38.4
`high`, 1440p	91.2	91.7	136.4	68.2

[Under revision] The measured times are from eager PyTorch with cuDNN autotuning on one Tesla T4, which is a data-centre card and not the integrated GPU the tiers are designed for. Every tier exceeds the 16.7 ms budget of a 60 frames per second display, and the smallest tier, `igpu`, takes 46.8 ms at 1080p. The `mid` and `high` tiers take less time at 1440p than at 1080p, at 85.7 and 91.2 ms against 106.6 and 120.4 ms. **[Observation]** Their operations per frame rise with the pixel count, but their flow limit is 1152 lines, so the 1440p frame has its flow halved and the flow cost falls by about four times. The rest of the network still grows with the pixel count, so the total time falls only because the flow is the dominant cost at 1080p. This is an explanation from the design, and the run did not separate the two costs.

S7.12 Input-to-photon pacing model

Measured times do not show what a player sees, so the run also simulated a game loop with the measured generation time. The model runs 2000 frames of a GPU-bound loop. Each render takes its base time with a multiplicative log-normal jitter of 0.08, each frame needs 4 ms of CPU work that starts just in time so the GPU never queues frames, and each frame is presented 1 ms after it is ready. The generation time is the measured `low` time at 1080p, 48.1 ms, applied at every render rate, and the texture super-resolution time is zero, so the model covers frame generation alone. It separates three quantities. Game input latency is the time from an input to the first real frame that shows it. Camera latency is the time from an input to the first displayed frame, real or generated, whose camera reflects it. Content age is the time since the newest real input in each displayed frame. Table S70 gives the four modes at three base render rates, and Fig. S43 plots the displayed rate against the two latencies.

[Under revision] At a base render of 30 frames per second the native loop displays 30.01 fps with a game latency of 38.3 ms. Interpolation displays 24.56 fps, and its game latency rises to 127.1 ms, because a real frame is held until after the generated one. Extrapolation keeps the game latency at 38.3 ms but its camera latency is 58.7 ms. Late-latching the camera before generation brings the camera latency to 43.7 ms. The same pattern holds at the other base rates, and interpolation displays 27.35 fps at a base of 40 frames per second and 30.86 fps at 60.

[Preliminary] In this model, frame generation at the measured cost lowers the displayed rate in every mode. The displayed rate of the interpolation and extrapolation modes is below that of the native loop at the same base render rate, because the generation time is longer than the render time it saves, and each period then shows two frames but lasts longer. The model therefore says that this network cannot raise the frame rate of a game at these base rates on this runtime. A runtime that brings the generation time below the render time would change that result, and the model reports the threshold without measuring such a runtime.

Table S70: Input-to-photon pacing model with the measured `low` time of 1080p generation (48.1 ms, Table S69) for every mode: displayed frame rate, game input latency, camera latency and content age, means and 95th percentiles

Mode	Base (fps)	Shown (fps)	Game (ms)	Game p95 (ms)	Camera (ms)	Camera p95 (ms)	Age (ms)
Native	30	30.01	38.3	42.8	38.3	42.8	38.3
	40	39.97	30.0	33.4	30.0	33.4	30.0
	60	59.84	21.7	24.0	21.7	24.0	21.7
Interpolation	30	24.56	127.1	133.9	86.4	90.9	106.8
	40	27.35	114.7	119.8	78.1	81.5	96.4
	60	30.86	102.2	105.6	69.8	72.1	86.0
Extrapolation	30	24.56	38.3	42.8	58.7	84.3	58.7
	40	27.35	30.0	33.4	48.3	70.6	48.3
	60	30.86	21.7	24.0	37.9	56.8	37.9
Late-latched	30	24.56	38.3	42.8	43.7	49.1	58.7
	40	27.35	30.0	33.4	39.6	49.1	48.3
	60	30.86	21.7	24.0	35.4	49.1	37.9

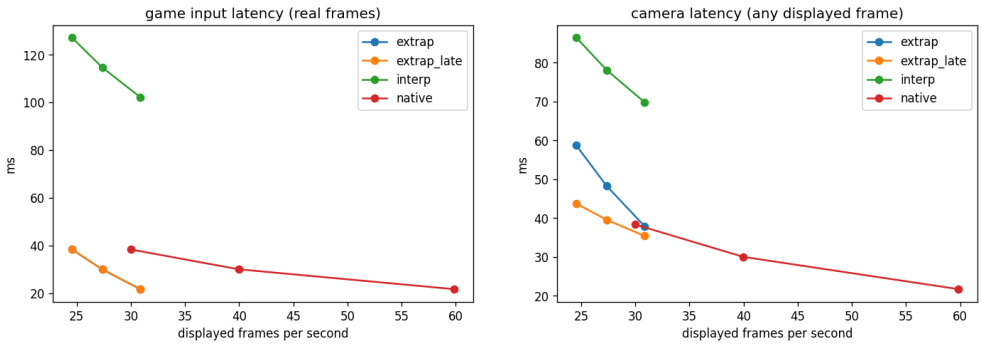


Fig. S43: Pacing model for the four modes at the three base render rates: game input latency (left) and camera latency (right) against the displayed frame rate

S7.13 Visual comparisons

The notebook drew a preview of the synthetic training samples for every special case (Fig. S44). Each row is one case, and the columns show the first real frame, the second real frame, the target, the predicted UI mask and the exact flow from the target to the second frame, coloured by direction with brightness giving the magnitude. **[Observation]** The interpolation row has a smooth flow field and an empty UI mask. The extrapolation row with a camera reversal has a flow field that differs on the moving object and on the background. The HUD row has its UI mask on the crosshair and the subtitle text. The menu row has a uniform grey UI mask over the whole frame, with the menu text in it, and the scene-cut row has a second frame from a different scene, with the target equal to that frame.

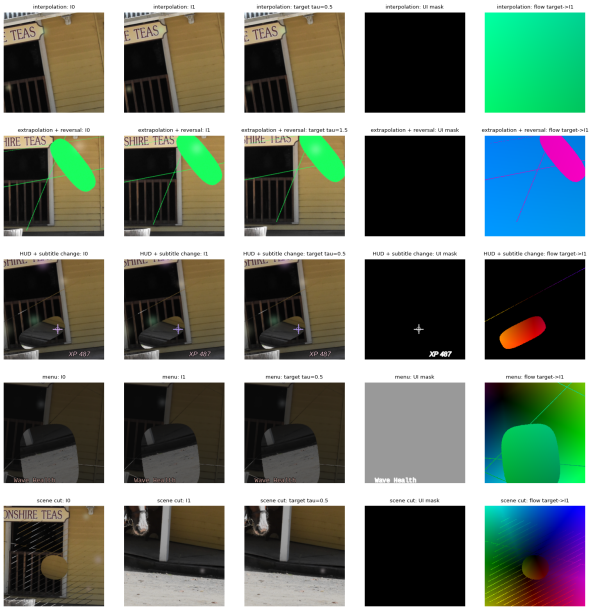
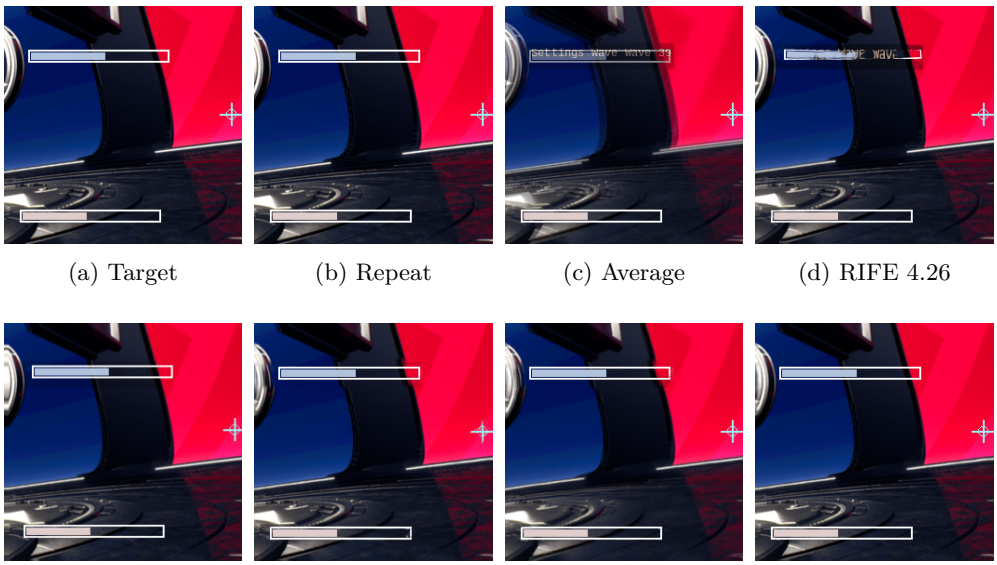


Fig. S44: Synthetic training samples for five special cases, one per row (interpolation, extrapolation with a camera reversal, HUD with a subtitle change, menu, scene cut). Columns: first frame, second frame, target, UI mask and the exact flow from target to second frame, coloured by direction

Fig. S38 shows one HUD and subtitle sample from the test split, with every method and the predicted UI mask, blend weight and flow. **[Observation]** The target shows the bars and the crosshair without subtitle text. Repetition and the learned networks, with and without the UI layer, show the bars without ghost text. Frame averaging and RIFE 4.26 show the subtitle as a blend over the upper bar, which is the ghosting that Table S62 counts. The predicted UI mask is bright on the two bars and on the crosshair, so those pixels are taken from the newest real frame instead of being blended.

Figure S45 shows the same eight panels at their saved resolution, with the target first. **[Observation]** At this size the panels differ mainly in the subtitle: frame averaging and RIFE 4.26 show it blended, and the other panels show the bars alone.



(a) Target (b) Repeat (c) Average (d) RIFE 4.26

(e) MV reprojection (f) `low`, engine (g) `low`, no engine (h) `low` with UI layer

Fig. S45: Panels of the HUD sample in Fig. S38 as saved, one image per method, interpolation at 256 by 256 pixels

Figure S46 shows one fast-motion extrapolation sample, an olive ellipse that moves across a dark, curved background. **[Observation]** At this scale the target, repetition, RIFE 4.26 and the `low` tier without engine data place the ellipse in similar positions. Frame averaging shows two overlapping copies of the ellipse. Motion-vector reprojection draws a light outline along the top of the ellipse, and the `low` tier with engine data has a fainter version of that edge. The differences between the network panels are small at this resolution, and the PSNR and LPIPS tables carry the quantitative comparison.

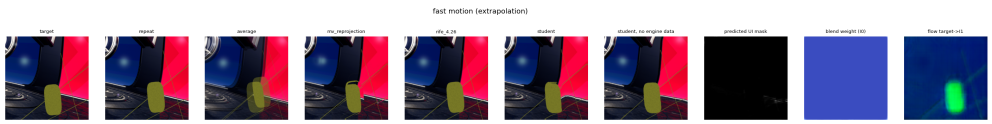
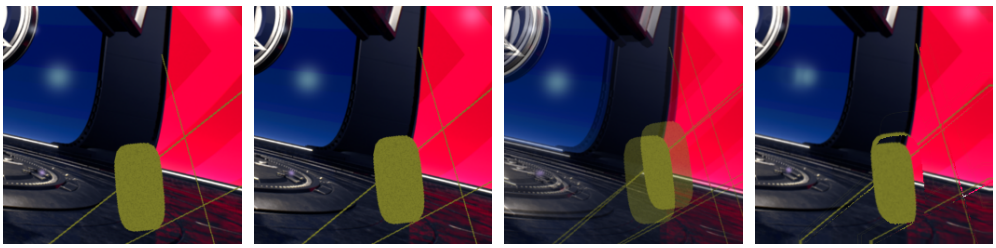
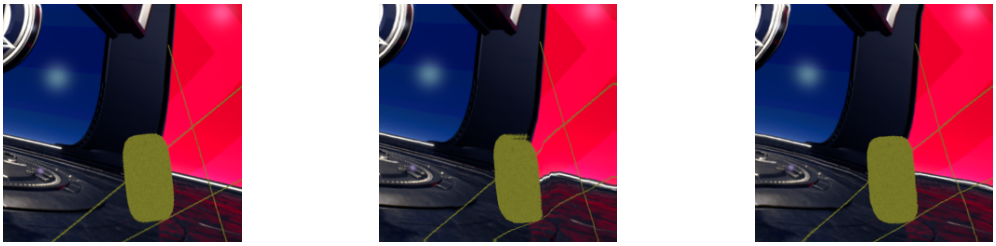


Fig. S46: Fast-motion extrapolation sample from the test split: target, the baselines, RIFE 4.26, the `low` tier with and without engine data, and the predicted UI mask, blend weight and flow

Figure S47 gives the seven panels of the same sample at their saved resolution.



(a) Target (b) Repeat (c) Average (d) MV reprojection



(e) RIFE 4.26 (f) `low`, engine (g) `low`, no engine

Fig. S47: Panels of the fast-motion sample in Fig. S46 as saved, extrapolation at 256 by 256 pixels

S7.14 ONNX export

The notebook exported the `low` and `igpu` tiers to ONNX, using opset 17 and example inputs of 128 by 128 pixels, and compared ONNX Runtime on the CPU with PyTorch on the same inputs. Table S71 gives the files and the largest differences.

Table S71: ONNX export of the fused frame generators (opset 17, exported at 128 by 128 pixels) and the largest difference to PyTorch on the same inputs, run with ONNX Runtime on the CPU

Tier	File size (MB)	Largest difference, ONNX Runtime against PyTorch
`low`	3.998	0.00000754
`igpu`	1.997	0.000005782

[Confirmed] The `low` graph agrees with PyTorch to within 0.00000754, and the `igpu` graph to within 0.000005782, values far below the step of one 8-bit grey level in an output frame. The export was not timed, so the ONNX graphs carry no latency result in this section.

S7.15 Open items of this section

[Under revision] Three comparisons quoted in the article are not part of the generated results of this run, and this section does not quote them. They are the split of the reversal suite by camera setting, the PSNR by motion class of the synthetic suite, and the timings of the earlier run. Each needs its own generated values before it can be stated here with a status.

S8 Status of findings, limitations and planned revisions

This section collects the findings of Sections S3, S5 and S7 with their status labels (Section S1.4), lists the defects found in the code that produced the reported runs, states the limitations of the evidence, and names the revisions planned for the next version of the pipeline. Values are given in the cited sections, so that this register does not need to change when only the numbers change.

S8.1 Register of findings

Table S72 lists each finding with its status, its evidence and the section that shows it.

Table S72: Findings of the reported runs, with their status and where they are shown

Finding	Status	Evidence
Data		
Copy detection with geometric verification left no duplicate between the training split and the validation or test splits in the second audit round.	[Confirmed]	Section S3.8
The calibrated detector separates synthetic copies from random and hard negatives with high recall at near-perfect precision.	[Preliminary]	Section S3.6
Temporal super-resolution		
With engine data the `low` 3x student is far above the analytic TAA upscaler on every test suite.	[Confirmed]	Section S5.3
On plain video the `low` 3x student is above bicubic upscaling on every test suite.	[Confirmed]	Section S5.3
Training on plain-video degradations is what makes the plain mode work, and it costs the engine mode nothing measurable.	[Preliminary]	Section S5.8
History and engine G-buffers are the largest engine-mode effects in the ablations.	[Preliminary]	Section S5.8
The built-in motion estimator adds a few tenths of a dB on plain video.	[Preliminary]	Section S5.8
Sixteen-frame unrolls are needed for static scenes to keep converging over many frames.	[Preliminary]	Section S5.7
The temporal-change error of the student is lower than that of TAA in engine mode but not lower than bicubic's on every suite in plain mode.	[Preliminary]	Section S5.3
Frame generation		
Fine-tuning RIFE 4.26 with zero-initialised hint channels gives a `high` tier at or above unchanged RIFE 4.26 on most interpolation suites, including Middlebury.	[Preliminary]	Section S7
The tiers small enough for integrated GPUs stay well below RIFE 4.26 in interpolation.	[Confirmed]	Section S7
In extrapolation the trained tiers are above frame repetition on plain video, while RIFE 4.26 is not built for it.	[Confirmed]	Section S7
With exact engine motion vectors, plain motion-vector reprojection remains as good as or better than learned extrapolation on some suites.	[Preliminary]	Section S7
Training with UI scenes and the UI mask sharply reduces UI ghosting on synthetic HUD scenes.	[Preliminary]	Section S7
The late-latched camera prior helps on rapid camera reversals in extrapolation.	[Preliminary]	Section S7
The histogram cut gate detects every synthetic cut but false-alarms on real content without motion vectors and on multi-frame gaps.	[Under revision]	Section S7
Inheriting RIFE 4.26 weights matters most on plain video in the ablations.	[Preliminary]	Section S7
Real-time cost		
In eager PyTorch on a T4 every measured tier except the smallest TSR tier exceeds a 16.7 ms frame budget at 1080p.	[Under revision]	Sections S5.9 and S7
Measured time is several times the arithmetic estimate, which points to per-kernel overhead and memory traffic. This is inferred from timings, not profiled.	[Observation]	Section S5.9
Interpolation adds a held frame of game latency, extrapolation does not, in the pacing model.	[Confirmed]	Section S7

Finding	Status	Evidence	
The fused networks export to ONNX and match PyTorch in ONNX Runtime.	[Confirmed]	Sections and S7	S5.9

S8.2 Defects found in the code of the reported runs

Writing this supplement meant checking every statement against the code, and that check found defects in the code that produced the reported runs. They are fixed in the repository for the next run. The reported numbers come from the code as it ran, so the affected results are labelled [Under revision] where they appear.

1. *Region PSNR on empty masks.* The TSR metric code tested the number of pixels of a region mask (disocclusions, particles) over the whole batch rather than per sample, so a sample with no pixels in the region scored the 100 dB cap. The region-restricted PSNR values of the reported runs are therefore biased upwards where such samples occur. The corrected code tests each sample and leaves out of the mean any sample whose region has 50 pixels or fewer. The whole-frame metrics are unaffected.
2. *Ablation inputs at test time.* Two ablations were evaluated with inputs that differ from their training inputs: the TSR model without the motion estimator received engine data in engine mode, and the FG model without the motion prior had depth removed although it was trained with depth. Their plain-mode rows and all main-model rows are unaffected.
3. *Strided render clips.* The native CARLA render clips are stored every tenth rendered frame, but TSR training treated them as consecutive frames. They are now excluded from temporal training windows, as FG already did, and kept for the native 2x test.
4. *ONNX reset.* The exported TSR graph had no reset input, so its first frame blended a caller-supplied history. A reset input now makes the exported graph start afresh exactly as the first frame of a clip does in PyTorch.
5. *Fallback limits.* The per-source limits used when the data-pipeline manifest is unavailable differed from those of the saved data-pipeline run. They now match it.
6. *Descriptions.* Several docstrings and notebook texts described behaviour the code does not have (for example that a fresh FG model outputs RIFE's frame exactly, or that one ONNX graph is exported per resolution). They now match the code, and the article was corrected where it repeated them.

S8.3 Limitations of the evidence

1. *One run per configuration.* Each configuration ran once with one seed. An earlier repeat of the TSR configuration with shorter budgets put run-to-run differences at a few tenths of a dB, so differences of that size are not distinguished from noise.
2. *Few samples on some suites.* Several FG means rest on few samples after removing those replaced by the cut gate (Section S7), and Middlebury has 12 sequences.
3. *Simulated inputs.* Low-resolution inputs are made from finished high-resolution frames, so level-of-detail, shading-rate and lighting effects of real renderers (specular aliasing, reflections) are absent. The plain-video protocol uses clean downscaling without codec artefacts.

4. *Little exact motion.* Real training windows are dominated by Vimeo-90K, and only a small share of TSR windows have exact motion vectors (Section S3).
5. *Latency on a data-centre GPU.* Times are eager PyTorch on a T4, not a deployment runtime on integrated or entry-level GPUs. The pacing model is a simulation, not a measurement on a display.
6. *No commercial comparison.* Commercial upscalers and frame generators need engine captures, which this study does not have.

S8.4 Planned revisions

The next version of the pipeline is expected to change the following, and the corresponding parts of this supplement and of the article will be revised with it.

1. Rerun all three notebooks with the corrected code of Section S8.2, and repeat key configurations with further seeds to measure run-to-run variation.
2. Calibrate the cut gate on real clips, separately for each frame gap, instead of on adjacent synthetic pairs.
3. Widen the small FG tiers with half-resolution flow, since capacity now limits them.
4. Time fused models in deployment runtimes (compute shaders, TensorRT, DirectML) on integrated GPUs, and wire the per-real-frame reuse of the FG feature encoder.
5. Add engine captures with exact motion and real renderer effects, which would also allow a comparison with commercial systems.

S9 Runs, hardware and reproducibility

This section records what produced each saved result, on which hardware and with which time budget, and how to regenerate every number and both documents. Three notebooks produced the outputs. The data pipeline (`neural-data-pipeline`) builds the leakage-safe dataset manifest. The temporal super-resolution notebook (TSR, `neural-supersampling`) and the frame-generation notebook (FG, `neural-frame-generation`) read that manifest, train their networks and evaluate them. The saved outputs of each notebook come from one run, the run listed in Table S73.

S9.1 Run provenance

Each notebook keeps its own clock. The clock starts when the notebook creates its run object in the setup cell, and every timed stage prints the elapsed time on its line of the log. Table S73 gives the dates of each run and two durations. The summary value is written to `summary.json` at the end of the notebook. The log value is the total printed on the last timed stage, which the generator stores as `run.<notebook>.hours`.

Table S73: Dates and wall-clock hours of the three runs. The difference is the summary value minus the log value.

Notebook	Start	End	Summary (h)	Log (h)	Difference (h)
Data pipeline	18 October 2026	18 October 2026	0.63	0.63	0.0000
TSR	3 November 2026	4 November 2026	10.13	10.09	+0.04
FG	8 November 2026	8 November 2026	10.07	10.02	+0.05

The two durations differ because of the code. The summary cell reads the clock after the last timed stage, and the cells between that stage and the summary are untimed. In TSR these cells aggregate the results,

compute the perceptual and static-scene metrics, draw the visual comparisons, tabulate the ablations, time the latency of the fused tiers and export the ONNX models. In FG they aggregate the results, compute the UI, temporal-consistency, visual, ablation, latency and pacing results, and export the ONNX models. The untimed time accounts for the differences in Table S73. The data pipeline writes its summary straight after its last timed stage, so its two values agree. For all three runs the log value agrees to within 0.01 h with the span from the first to the last log line, and the generator takes the run dates from the same log.

The configuration cells set a time budget of 10.8 h for TSR and 10.8 h for FG (`RUN_BUDGET_H`). The notebooks describe the budget as Kaggle's 12-hour session limit minus a 72-minute margin. That limit is the notebooks' own assumption, which this supplement does not check. Both runs finished below their budgets on either measure. The data pipeline has no budget and finished in under one hour. Neither duration includes the cells that delete the training caches and package the outputs, which run after the summary.

The executed notebooks show which code ran. The data pipeline and TSR copies have the same cell sources as a fresh build of the current `notebooks` and `src/neuralss` files, so the library code of those runs is the current code. The FG copy differs in one plotting cell, the figure of the real-suite results. That cell was changed after the run, so that it hides a panel with no data and rotates the axis labels by 20 degrees. The change affects only `real_results.png`, and no metric file depends on it.

S9.2 Hardware and software

Table S74 gives the hardware that each notebook detected at run time. Each run found two identical Tesla T4 GPUs. The training notebooks use both GPUs together for the main stages and one GPU per process for the parallel pairs and the ablations.

Table S74: Hardware and software recorded by each run (`metrics/hardware.json`)

Item	Data pipeline	TSR	FG
GPUs	2 Tesla T4	2 Tesla T4	2 Tesla T4
Memory per GPU (GB)	14.56	14.56	14.56
Compute capability	7.5	7.5	7.5
Multiprocessors per GPU	40	40	40
CPU cores	4	4	4
RAM (GB)	31.3	31.3	31.3
Free disk in the working folder (GB)	19.5	19.5	19.5
Python	3.10.12	3.13.16	3.13.16
PyTorch	2.0.0	2.11.0+cu130	2.11.0+cu130
CUDA	11.8	13.0	13.0

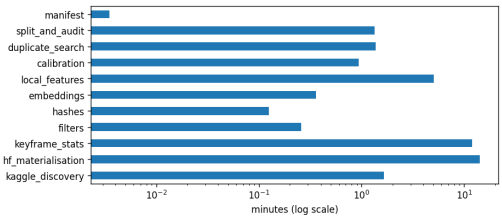
The data pipeline ran on an older software image than the two training notebooks. It used PyTorch 2.0.0 with CUDA 11.8, while TSR and FG used PyTorch 2.11.0+cu130 with CUDA 13.0. The training notebook states that the library supports PyTorch 2.0 and later.

The training notebooks install, when they are missing, three optional packages at run time: `lpips`, `flip-evaluator` and `onnxruntime`. Their logs record only whether each package could be imported afterwards. All three succeeded in both training runs, but the outputs do not record package versions. The versions of the other libraries, such as NumPy and pandas, are not recorded either.

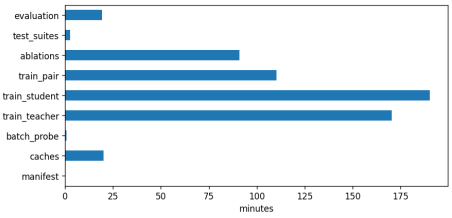
S9.3 Stage times

Each notebook times its stages with a context manager, which records the duration of each stage in `metrics/stage_times.csv` and draws the bar chart in `plots/stage_times.png`. Figure S48 shows the three charts, panel (a) for the data pipeline, panel (b) for TSR and panel (c) for FG, and Table S75 lists the values. The setup, configuration and packaging cells are not timed.

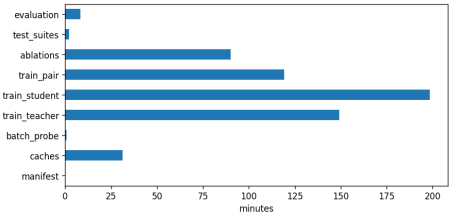
Fig. S48: Minutes per timed stage, as drawn by each notebook. Panel (a) uses a logarithmic axis, panels (b) and (c) a linear one.



(a) Data pipeline



(b) TSR



(c) FG

Table S75: Minutes per timed stage of each run (`metrics/stage_times.csv`). A dash means that the notebook has no such stage.

Stage	Data pipeline	TSR	FG
kaggle_discovery	1.659	–	–
hf_materialisation	14.24	–	–
keyframe_stats	11.95	–	–
filters	0.2582	–	–
hashes	0.1251	–	–
embeddings	0.3602	–	–
local_features	5.086	–	–
calibration	0.9402	–	–
duplicate_search	1.379	–	–
split_and_audit	1.341	–	–
manifest	0.003482	0.1221	0.2327
caches	–	19.97	31.41
batch_probe	–	0.6127	0.7802

train_teacher	–	170.5	149.2
train_student	–	190.3	198.5
train_pair	–	110.3	119.3
ablations	–	90.88	90.23
test_suites	–	2.682	2.283
evaluation	–	19.21	8.442

In Fig. S48a the Hugging Face transfer and the keyframe decoding are the two longest bars, and the manifest stage is too short to see. In Fig. S48b and Fig. S48c the four training stages are the four longest bars. The student stage is the longest bar in both training runs, and the caches stage is the longest stage that is not a training stage.

Training dominates the time budget of both training runs. The four training stages take longer than all the other stages together, and the student stage is the single largest stage. The caches and the evaluation are short by comparison, and the manifest is negligible in both. Table S76 compares each budgeted stage with its plan. The FG run had a caches stage longer than its planned cap. The notebook then shrank every training budget slightly, before training started, so that the run stayed within its ceiling. The data pipeline has no training, and its Hugging Face transfer and keyframe decoding together take longer than its other nine stages combined.

Table S76: Planned and measured minutes of the budgeted stages. The plan is the configuration value. The FG plan is scaled slightly down before training, so its measured times are close to the scaled plan, not equal to the configuration value.

Stage	TSR plan	TSR measured	FG plan	FG measured
Caches (at most)	30	19.97	30	31.41
Teacher	170	170.5	150	149.2
Student	190	190.3	200	198.5
Pair of parallel runs	110	110.3	120	119.3
Ablations (three pairs)	3×30	90.88	3×30	90.23

S9.4 Configuration

Each notebook writes its configuration to `metrics/config.json`. The file holds the configuration cell's dictionary, the run budget and, for the training notebooks, the ablation overrides. The tables below cover every key that affects a run. Their values are generated keys, so a new run updates them. Keys that exist only in the training configuration files (`code/cfg_*.json`) are typed, because no metric file holds them.

S9.4.1 Data pipeline

Table S77 lists the keys of the data pipeline configuration, which sets the caps of each source, the copy-detection thresholds and the split.

Table S77: Data pipeline configuration. Keys are under `cfg` in `metrics/config.json`.

Key	Value	Meaning
Discovery and seeds		
`seed`	7	seed of the randomised discovery walk and of the split hash
`list_cap_files`	40,000	most files listed per Kaggle dataset (a source may set its own cap)

Continued on the next page

Key	Value	Meaning
`workers`	4	worker processes for image decoding and SIFT features
Hugging Face subsets		
`hf_caps.game_stills`	1,200 / 250	game still images for training and for test
`hf_caps.tartanair`	120	frames per TartanAir environment
`hf_caps.tartanair_seq`	24	frames per TartanAir sequence
`hf_caps.gameir`	40 / 30	GameIR clips for training and for test
`hf_caps.vimeo1080p`	160 / 40	Vimeo-1080p videos for training, and the videos of the pinned validation shard, which become test items
Copy detection and split		
`calibration_pairs`	1,500	synthetic copy pairs for calibrating the duplicate thresholds
`dedup.k`	16	embedding neighbours per keyframe that become candidate copies
`dedup.retrieval_recall`	0.98	share of synthetic copies that the embedding threshold must retrieve
`dedup.sift_features`	400	SIFT keypoints per image in geometric verification
`dedup.min_std_hash`	15	grey-level standard deviation a keyframe needs before pHash matching
`dedup.max_hash_candidates_per_kf`	20	the pHash threshold is tightened until at most this many candidate pairs per keyframe remain, or until the exact threshold is reached
`dedup.audit_k`	5	nearest reference keyframes checked for each validation or test keyframe
`dedup.audit_rounds`	3	most audit rounds, each adding the duplicates it finds as edges and splitting again
`val_fraction`	0.05	share of train and validation components assigned to validation, by a hash of the component root

The sources are the datasets that the manifest draws on. Table S78 lists them as `config.json` records them. Every source was reachable in both training runs.

Table S78: Sources of the manifest. A host of `kaggle` means a Kaggle dataset and `hf` a Hugging Face repository. The kinds are single images (still), videos, rendered sequences (render_seq), rendered image pairs (render_pairs) and video files (video_file).

Source	Host	Kind	Identifier
DIV2K	Kaggle	still	`soumikrakshit/div2k-high-resolution-images`
Flickr2K	Kaggle	still	`daehoyang/flickr2k`
REDS	Kaggle	video	`amithkesavmrajagiri/reds-dataset`
Vimeo-90K septuplets	Kaggle	video	`wangsally/vimeo-90k-7`
Vimeo-90K triplets	Kaggle	video	`chenshu123/vimeo-triplet`
MPI Sintel	Kaggle	render_seq	`artemmmtry/mpi-sintel-dataset`
Vid4	Kaggle	video	`uom200647r/vid4-dataset`
Game stills	Hugging Face	still	`ericphann/video-game-super-resolution`
TartanAir	Hugging Face	render_seq	`theairlabcmu/tartanair`
GameIR	Hugging Face	render_pairs	`LLLebin/GameIR`
Vimeo-1080p	Hugging Face	video_file	`danjacobellis/vimeo1080p`

S9.4.2 Temporal super-resolution

The TSR training runs use AdamW [10] and FP16 autocast with loss scaling [9]. Table S80 lists the keys of the TSR configuration. The last group holds the keys that appear only in the training configuration files, and their values are typed.

The TSR tiers differ in the width and depth of the trunk and in the built-in motion estimator. Table S79 lists them. The trunk is a stack of RepConv blocks [32] and the output is unfolded with pixel shuffle [7]. The GMAC column gives billions of multiply-accumulate operations (MAC) per 1080p output frame, counted on the fused network.

Table S79: TSR tiers. Width, blocks and U-Net blocks describe the trunk, and hidden is the number of channels of the recurrent state. The motion estimator is given as its widths and its residual blocks, and the refinement is the width of the output-resolution head (0 means none). The parameters are the training-time counts, and the cost is per 1080p output frame at 3x.

Tier	Width	Blocks	U-Net	Hidden	Motion	Refine	Params	GMAC
`igpu`	16	3	0	8	32/24/16, 2	0	145K	6.2
`low`	24	4	2	8	48/32/24, 3	8	362K	16.1
`mid`	32	4	3	16	64/48/32, 4	8	728K	28.1
`high`	48	6	4	16	96/64/48, 4	12	1.62M	69.0
`teacher`	64	8	6	32	128/96/64, 6	16	3.94M	156.1

Table S80: TSR configuration. Keys are under `cfg` in `metrics/config.json` unless shown otherwise.

Key	Value	Meaning
Budget, model and data		
`budget_h` (top level)	10.8	ceiling of the run in hours
`seed`	11	seed of the training run
`scale`	3	primary super-resolution factor, from 360p to 1080p
`scale_alt`	2	secondary factor, used by the 2x student
`patch`	192	output patch in pixels. At 3x, the low-resolution patch has one third of this side
`seq_len`	6	frames per training sequence, the length of back-propagation through time
`still_size`	512	side in pixels of the texture crops from which synthetic scenes are rendered
`student_tier`, `small_tier`, `teacher_tier`	low, igpu, teacher	tiers of the student, of the integrated-GPU run and of the teacher (Table S79)
Caches (requested counts)		
`still_train`, `still_val`	6,500 / 256	still-image crops for training and for validation
`win_train`, `win_val`	3,600 / 192	sequence windows of `seq_len` frames for training and for validation
`build_min`	30	total minutes allowed for decoding all caches
Sampling		
`p_real`	0.4	share of each batch drawn from real sequences, the rest being synthetic
`p_video_synth`, `p_video_real`	0.3 / 0.6	share of synthetic and of real samples turned into plain video (no jitter, depth or motion vectors, with 8-bit quantisation and mild degradation)
`p_video_mv`	0.15	share of those plain-video samples that keep estimated motion vectors
`p_drop_mv`, `p_drop_depth`	0.25 / 0.3	share of render-like samples whose motion vectors or depth are withheld, so that the network also learns without them
`long_every`	4	every this many steps trains on a long unroll of synthetic frames
`long_T`, `long_patch`	16 / 96	frames and patch size of a long unroll
`mv_warmup_frac`	0.25	share of the run during which samples without motion vectors warp the history with the label motion, with a probability that falls from one to zero

Continued on the next page

Key	Value	Meaning
Optimisation		
`lr`, `lr_teacher`	0.002 / 0.001	peak learning rate of the students and of the teacher, followed by warm-up and a cosine decay over the wall-clock budget
`min_lr`	0.000001	floor of the cosine decay
`warmup_frac`	0.03	share of the budget spent warming the learning rate up
`weight_decay`	0.0001	AdamW weight decay
`ema_decay`	0.999	decay of the exponential moving average of the weights, which is the copy that is evaluated
`grad_clip`	1	gradient norm above which the gradients are scaled down
Loss weights (per-frame loss terms)		
`w_grad`, `w_fft`	0.1 / 0.05	weights of the gradient loss and of the loss on Fourier magnitudes
`w_temporal`	0.5	weight of the temporal-change loss along known motion
`w_flow`	0.1	weight of the motion-estimation loss, per low-resolution pixel
`w_lpips`	0.05	weight of the LPIPS perceptual loss, set to zero when the package is missing
`w_distill`	0	weight of distillation from the teacher, zero because no teacher is used
Validation and budgets		
`val_bs`, `val_batches`	8 / 4	samples per validation batch and batches per validation round
`val_interval_min`	8	longest time between validation rounds, in minutes
`minutes.teacher`, `minutes.student`	170 / 190	wall-clock training budget of the teacher and of the student, in minutes
`minutes.pair`, `minutes.ablation`	110 / 30	budget of each parallel pair, and of each ablation pair, in minutes
Evaluation		
`synthetic_per_class`	6	synthetic test crops per motion class, of four classes
`synthetic_T`, `synthetic_P`	16 / 384	frames and output size of each synthetic test sequence
`real_clips`, `real_T`	8 / 16	real test clips per suite, and frames per clip
`eval_reserve_h`	1	hours kept back for the test suites, the evaluation and the export before the training budgets are set
Pipeline caps (unused, the manifest was not built inline)		
`list_cap_files`	20,000	most files listed per Kaggle dataset
`seed`, `workers`	7 / 4	seed of the inline pipeline and its worker count
`hf_caps.game_stills`	600 / 120	game stills for training and for test
`hf_caps.tartanair`, `hf_caps.tartanair_seq`	80 / 24	frames per TartanAir environment, and frames per sequence
`hf_caps.gameir`	20 / 12	GameIR clips for training and for test
`hf_caps.vimeo1080p`	60 / 16	Vimeo-1080p videos for training and for the pinned validation shard
`calibration_pairs`, `val_fraction`	300 / 0.05	calibration pairs for the duplicate thresholds, and the validation share
Training configuration files (typed)		
`val_interval_s`	varies	seconds between validation rounds: 480 for the main runs, and one sixth of the budget for the ablations
`sync_every`	10	steps between the synchronised checks of the clock, of the loss flag and of the learning rate
`progress_interval_s`	15	seconds between progress lines
`select_metric`	psnr_y	validation score that selects the best checkpoint
`rife_weights`	path	RIFE 4.26 weights loaded into the built-in motion estimator, read from `/tmp/nss_rife/rife426.pt`
`teacher_ckpt`	null	checkpoint of the teacher for distillation, none in TSR

Continued on the next page

Key	Value	Meaning
`workers_per_rank`	2	data-loading worker processes per GPU process, half the CPU cores
`weights_dir`, `metrics_dir`	folders	output folders of the run, under `/kaggle/working`

Table S81 lists the TSR training runs as the notebook's launcher builds them. The 2x student is the `low` tier at scale 2, and its cost is 30.7 GMAC per 1080p frame. Every run used a batch of 48 sequences per GPU, the largest size of the batch probe that fitted for both the teacher and the student.

Table S81: TSR training runs. Each main run used both GPUs, and each pair ran one run per GPU in parallel. The six ablation runs form three pairs of 30-minute runs, and all use the `low` tier at scale 3.

Run	Tier	Scale	GPUs	Batch	Budget (min)	Peak lr
`tsr_teacher_s3`	`teacher`	3	both	48	170	0.001
`tsr_low_s3`	`low`	3	both	48	190	0.002
`tsr_igpu_s3`	`igpu`	3	GPU 0	48	110	0.002
`tsr_low_s2`	`low`	2	GPU 1	48	110	0.002
`abl_*` (six runs)	`low`	3	one each	48	30 each	0.002

Table S82 lists the ablations. Each changes only the keys listed, so all six share the reference's other keys and its budget.

Table S82: TSR ablations, from the notebook's `ABLATIONS` dictionary. Each row lists the keys that differ from the reference run.

Ablation	Keys that differ	What the run tests
`reference`	none	the full recipe at the same short budget
`no_history`	`use_history` false, `find_unused` true	no recurrent history, so each frame is upscaled alone. The unused-parameter flag of distributed training is on, because the history branch receives no gradient
`no_gbuffers`	`ablate_gbuffers` true	colour-only input, so the motion comes from the built-in estimator and no engine motion vectors or depth are used
`no_motion_net`	`motion_net` false, `ablate_gbuffers` true	colour-only input and no built-in estimator, so the motion is zero
`no_video_training`	`p_video_synth` and `p_video_real` 0	only render-like samples, with no plain video in training
`no_long_unroll`	`long_every` 0	only the six-frame sequences, with no long unroll

S9.4.3 Frame generation

The FG training runs use the same optimiser, schedule and training loop as TSR, with their own loss terms, and their distillation term is zero when no teacher checkpoint is given. Table S85 lists the keys of the FG configuration. The keys that differ between the four training runs are in Table S83.

Table S84 lists the FG tiers. The teacher has the architecture of the `high` tier, which is the RIFE 4.26 network with the engine-hint channels and the residual and UI head added. The smaller tiers inherit the leading channel slices of the RIFE weights, and the `teacher` and `high` rows are the same network.

Table S83: FG training runs, from `code/cfg_fg_*.json`. The four runs share every other key of Table S85, including the training-only keys. The budgets are the planned budgets, scaled slightly down before training.

Key	`high` (`fg_teacher`)	`low` (`fg_low`)	`mid` (`fg_mid`)	`igpu` (`fg_igpu`)
`name`	`fg_teacher`	`fg_low`	`fg_mid`	`fg_igpu`
`tier`	teacher	low	mid	igpu
`batch_per_gpu`	48	48	24	48
`lr`	0.0001	0.0004	0.0004	0.0004
`teacher_ckpt`	none	`fg_teacher_best.pt`	`fg_teacher_best.pt`	`fg_teacher_best.pt`
`train_seconds`	scaled teacher	scaled student	scaled pair	scaled pair

Table S84: FG tiers. Flow widths and residual blocks describe the IFNet, refinement is the width of the half-resolution residual and UI head, and flow lines is the largest image height at which the flow is estimated, taller frames being halved. The flow scale is the factor applied to a 1080p frame, and the cost is per 1080p output frame.

Tier	Flow widths	Blocks	Refine	Flow lines	Flow scale	Params	GMAC
`igpu`	64/48/32/24/16	3	12	576	0.5	454K	9.4
`low`	96/64/48/32/16	4	16	576	0.5	953K	12.7
`mid`	128/96/64/48/24	6	24	1152	1	2.29M	44.3
`high`	192/128/96/64/32	8	32	1152	1	5.71M	76.7

Table S85: FG configuration. Keys are under `cfg` in `metrics/config.json` unless shown otherwise.

Key	Value	Meaning
Budget, model and data		
`budget_h` (top level)	10.8	ceiling of the run in hours
`seed`	21	seed of the training run
`patch`	256	training crop in pixels. The coarsest RIFE level works at one sixteenth of the crop
`still_size`	512	side in pixels of the textures from which synthetic scenes are rendered
`student_tier`, `small_tier`, `mid_tier`	low, igpu, mid	tiers of the student, of the integrated-GPU run and of the mid-size run (Table S84). The teacher is the `high` tier.
Caches (requested counts)		
`still_train`, `still_val`	6,000 / 256	still-image crops for training and for validation
`quad_train`, `quad_val`	6,000 / 256	quads, meaning four consecutive frames of a video, for training and for validation
`build_min`	30	total minutes allowed for decoding all caches
Sampling		
`p_real`	0.4	share of each batch drawn from real quads, the rest being synthetic
`p_extrap`	0.4	share of samples that extrapolate a future frame from the two latest frames
`p_tau`	0.5	share of synthetic interpolation samples whose time is drawn at random between 0.15 and 0.85
`p_mfg`	0.25	share of real interpolation samples taken as 3x multi-frame generation
`p_drop_mv`, `p_drop_mv_real`	0.3 / 0.5	share of synthetic and of real samples whose motion vectors are withheld

Continued on the next page

Key	Value	Meaning
p_ui, p_menu	0.5 / 0.03	share of synthetic samples with HUD and subtitle overlays, and with full-screen menus
p_cut, p_flip	0.03 / 0.2	share of synthetic samples with a scene cut, and with a rapid camera reversal
Optimisation		
lr, lr_teacher	0.0004 / 0.0001	peak learning rate of the students and of the teacher, followed by warm-up and a cosine decay over the wall-clock budget
min_lr, warmup_frac	0.000001 / 0.03	floor of the cosine decay, and share of the budget spent warming up
weight_decay, ema_decay	0.0001 / 0.999	AdamW weight decay, and decay of the evaluated moving average of the weights
grad_clip	1	gradient norm above which the gradients are scaled down
Loss weights (per-sample loss terms)		
w_census	0.5	weight of the census loss, a soft ternary distance that tolerates brightness changes
w_lpips	0.1	weight of the LPIPS perceptual loss, set to zero when the package is missing
w_flow	0.01	weight of the flow supervision, per pixel of flow error, with RAFT labels on video at a quarter of this weight
w_ui	0.2	weight of the cross-entropy on the UI mask
w_distill	0.5	weight of distillation of the flows and output from the teacher, zero without a teacher checkpoint
Validation and budgets		
val_bs, val_batches	16 / 4	samples per validation batch and batches per validation round
val_interval_min	8	longest time between validation rounds, in minutes
minutes.teacher, minutes.student	150 / 200	planned wall-clock budget of the teacher and of the student, in minutes
minutes.pair, minutes.ablation	120 / 30	planned budget of each parallel pair, and of each ablation pair, in minutes
Evaluation		
synthetic_per_suite	48	synthetic test crops per synthetic suite
real_clips, native_clips	12 / 6	real clips per real suite, and clips of the native 1080p suite
eval_reserve_h	1	hours kept back for the test suites, the evaluation and the export before the training budgets are set
Pipeline caps (unused, the manifest was not built inline)		
list_cap_files	20,000	most files listed per Kaggle dataset
seed, workers	7 / 4	seed of the inline pipeline and its worker count
hf_caps.game_stills	600 / 120	game stills for training and for test
hf_caps.tartanair, hf_caps.tartanair_seq	80 / 24	frames per TartanAir environment, and frames per sequence
hf_caps.gameir	0 / 0	GameIR clips, both zero, so GameIR was not used
hf_caps.vimeo1080p	60 / 16	Vimeo-1080p videos for training and for the pinned validation shard
calibration_pairs, val_fraction	300 / 0.05	calibration pairs for the duplicate thresholds, and the validation share
Training configuration files (typed)		
val_interval_s	varies	seconds between validation rounds: 480 for the four main runs, and one sixth of the budget for the ablations
sync_every	10	steps between the synchronised checks of the clock, of the loss flag and of the learning rate
progress_interval_s	15	seconds between progress lines
select_metric	psnr	validation score that selects the best checkpoint
rife_weights	path	RIFE 4.26 weights loaded into the IFNet, unless the run is the no-inheritance ablation, read from /tmp/nss_rife/rife426.pt

Continued on the next page

Key	Value	Meaning
`ui_training`	true	whether the UI head is trained, false only in the no-UI-training ablation
`workers_per_rank`	2	data-loading worker processes per GPU process, half the CPU cores
`weights_dir`, `metrics_dir`	folders	output folders of the run, under `/kaggle/working`

Table S86 lists the FG ablations. Each changes only the keys listed. The ablations run for the ablation budget, and their validation interval is one sixth of that budget.

Table S86: FG ablations, from `code/cfg_abl_*.json`. Each row lists the keys that differ from the reference run, which is the `low` tier with the full recipe.

Ablation	Keys that differ	What the run tests
`reference`	none	the full recipe at the same short budget
`no_ui_training`	`ui_training` false, `w_ui` 0	no HUD, subtitle or menu overlays in training, and no UI loss
`no_motion_prior`	`ablate_prior` true	no engine motion vectors and no camera prior
`extrap_only`	`mode` extrap	every sample extrapolates, with no interpolation
`no_distillation`	`teacher_ckpt` null	no teacher, so the distillation term is zero
`no_inheritance`	`rife_weights` null	random initialisation instead of the RIFE 4.26 channel slices

S9.5 Reproducibility

The steps below are in the order that a new run needs them. The notebooks are built from the repository's sources and run on Kaggle. Their saved outputs are read by two scripts, which write the values that the text quotes and both documents.

S9.5.1 Building the notebooks

Each `notebooks/<name>.py` file is a percent-format script, and its cells become the cells of the notebook. A cell whose only line is `# WRITEFILE` followed by a file name of `src/neuralss` becomes a cell that writes that file into `/kaggle/working/code`. Every notebook therefore carries its own copy of the library, and the library has a single source. `tools/build_notebooks.py` writes the three `.ipynb` files with stable cell identifiers, and it rewrites a notebook only when its sources changed, which removes that notebook's outputs. Run it from the repository root with the repository's Python in isolated mode (`python -I tools/build_notebooks.py`). Its `--check` option exits with code 1 when a notebook does not match its sources, and ignores outputs. The build needs `nbformat`.

S9.5.2 Rerunning on Kaggle

Each notebook runs on Kaggle with the accelerator set to GPU T4 x2 and Internet switched on. The notebooks download the RIFE 4.26 weights, checked against SHA-256 hashes, and the pre-trained RAFT weights, and they read Hugging Face data for any source that is not attached. The optional secret `HF_TOKEN` raises the Hugging Face download limits. The hardware check stops the notebook unless it sees two GPUs.

1. Create the data pipeline notebook from `neural-data-pipeline/neural-data-pipeline.ipynb`, attach the seven Kaggle datasets listed in its header (DIV2K, Flickr2K, REDS, Vimeo-90K septuplets and triplets, MPI Sintel and Vid4), and choose *Save & Run All*.

2. Create the TSR and FG notebooks from their `.ipynb` files, attach the same datasets, attach the output of the data pipeline run with *Add Input*, and choose *Save & Run All*. Both read the manifest from that output.
3. A full run takes about the budget of its notebook. To check a change first, set `QUICK_RUN = True` in the configuration cell, which the notebooks estimate at about 40 minutes.
4. If Kaggle reports `ERRORED_MOUNTING_DATASET`, the session never starts and shows no log. Remove or re-add the dataset that failed, and save again.

S9.5.3 Where the outputs go

Each notebook writes to `/kaggle/working`. The training notebooks zip everything in that folder except the training caches, which they delete first, and the data pipeline zips its metrics, plots, logs, code and manifest. The `outputs` folder of each notebook in the repository has the same layout: `code` holds the library and the run configurations as written on Kaggle, `logs` holds the notebook log and one log per training run, `metrics` holds every CSV and JSON result, `plots` holds every figure and, for the training notebooks, `predictions` and `weights` hold the sample frames and the checkpoints. To update a notebook, extract its output into `<notebook>/outputs`, and save the Kaggle copy of the notebook, with its outputs, as `outputs/<name>.executed.ipynb`. The generator reads each run's dates from that copy if it exists and otherwise from the notebook file itself. The data pipeline has no such copy in `outputs`, because its executed notebook is the notebook file, so that file must keep its outputs.

S9.5.4 Regenerating the numbers and the documents

`tools/build_paper_results.py` reads the metric files of the three `outputs/metrics` folders, the run logs, the executed notebooks and the Middlebury comparison file. It writes `paper/results-values.tex`, which holds one `\SetResult` line per key. A metric file that has no rule stops the script, so that no number is left out without notice. The script scans the `paper` sources for `\Res` uses and prints a change report of the added, removed and changed keys, with the sentences that use each removed or changed key. A use of a key that does not exist is reported with its file and line, and the script exits with code 1 after writing the file. The script also copies the five `fig-tsr-*.png` figures from their notebook outputs when their bytes differ. The six `fig-fg-ui-*.png` figures are not regenerated, because each is a resampled copy of a notebook image and the resampling cannot be reproduced exactly.

`tools/build_paper.py` runs the generator and then compiles the six documents with Tectonic in `paper`. The article and its supplement are built in the order `main`, `supplementary`, `main`, so that each one's cross-references resolve against the other. The anonymised pair follows, then the title page and the copy without references. It prints the page count of each PDF. The option `--no-generate` skips the generator, and `--supplement-only` compiles the supplement alone. Tectonic is `paper/tectonic.exe` when that file exists, and otherwise `tectonic` on the `PATH`.

S9.5.5 The Middlebury comparison

The Middlebury comparison of the FG tiers is not produced by the notebooks. `tools/compare_fg_tiers_middlebury.py` runs on a CPU and needs three inputs, each of which can be given on the command line. The first is the `other-color-allframes` and `other-gt-interp` archives of the Middlebury optical-flow benchmark [16], extracted into one folder. The second is the RIFE 4.26 `flownet.pkl` from the `RIFEv4.26_0921.zip` archive of `huggingface.co/hzwer/RIFE`, whose SHA-256 hash the notebooks check when they download it. The third is the four `fg_*_best.pt` checkpoints of the FG run. The script defaults expect the downloads under `.agent-local/downloads`, a folder that git does not track, so pass the paths explicitly. It evaluates frame averaging, RIFE 4.26 at full, half and quarter flow resolution, and the four FG tiers, on the twelve sequences. It writes `neural-frame-generation/local-evaluation/middlebury_psnr.csv`, with one row per sequence and method and the mean of each method with its cost in GMAC per 1080p frame. The generator reads that file as the `mb` keys, so rerun the generator after the script.

S9.5.6 Weights and exported models

Each training run saves three PyTorch checkpoints in its `weights` folder. The file `<run>_best.pt` holds the exponential-moving-average weights of the best validation round. The file `<run>_last.pt` holds the weights of the last round, and `<run>_fused.pt` holds the network with its RepConv branches folded into plain convolutions. Table S87 lists the models that the two training runs exported. The checkpoints are research weights, trained on research-use data, and the repository's `RESEARCH.md` records the data licences. The

notebooks load RIFE 4.26 [5] into the motion estimators and the FG network, and they log how many tensors were copied exactly, copied by slicing or left missing. No run left a tensor missing.

Table S87: Weights and exported models of the two training runs. Each name stands for the three checkpoints of its run (`_best`, `_last` and `_fused`) unless it names an ONNX file. File sizes are in the repository's README.

Files	Notebook	Model
`tsr_low_s3_*.pt`	TSR	main 3x student, `low` tier
`tsr_igpu_s3_*.pt`	TSR	integrated-GPU 3x student
`tsr_low_s2_*.pt`	TSR	2x student, `low` tier
`tsr_teacher_s3_*.pt`	TSR	3x teacher, the quality ceiling of the architecture
`abl_*_*.pt`	TSR	the six ablation runs
`tsr_student_s3_fused.onnx`, `tsr_igpu_s3_fused.onnx`	TSR	one recurrent step of the 3x student and of the integrated-GPU student, with static shapes
`fg_teacher_*.pt`	FG	`high` tier, RIFE 4.26 fine-tuned, the strongest FG model
`fg_mid_*.pt`, `fg_low_*.pt`, `fg_igpu_*.pt`	FG	`mid`, `low` (the student) and `igpu` tiers
`abl_*_*.pt`	FG	the six ablation runs
`fg_student_fused.onnx`, `fg_igpu_fused.onnx`	FG	`low` (student) and `igpu` tiers at 128 by 128 pixels, with static shapes

Each ONNX file was exported with the TorchScript exporter at opset 17 and checked against PyTorch with ONNX Runtime on the CPU. The maximum difference of each check is in `onnx_export.json`. The ONNX files have static shapes, so deployment at another resolution needs a new export with `nss_models.export_onnx`.

S9.6 Revising the supplement after a new run

A new run changes the numbers, and some of the sentences that quote them. The following procedure keeps the text consistent with the saved outputs.

1. Edit the notebook sources in `notebooks` or the library in `src/neuralss`, never the `.ipynb` files. Rebuild the notebooks, check them with `--check`, and run the lint and format checks that the repository's README lists.
2. Store the executed copy of each notebook that is being replaced, with its outputs, as described in Section S9.5.3, before the rebuild removes its outputs.
3. Run the notebook on Kaggle as in Section S9.5.2, and replace its `outputs` folder with the new output.
4. Rerun `tools/build_paper_results.py` and read its change report. Each removed or changed key lists the sentences that use it, and each of them needs checking against the new value and its meaning.
5. Revisit each listed sentence and its status label, as defined in the first section of this supplement. A finding whose value moves by more than the run-to-run variation described there needs its label checked, and its sentence must describe the new value, not only quote its number.
6. Update the run descriptions in the notebook headers and in the repository's README, which quote earlier runs. Update the typed values of Tables S81 and S83

and of the training-only rows from the new `code/cfg_*.json` files and notebook sources, because the generator does not read them.
7. If the checkpoints changed, rerun `tools/compare_fg_tiers_middlebury.py`, then the generator, since the Middlebury values are not produced on Kaggle.
8. Rebuild the documents with `tools/build_paper.py`, read the page counts it prints, and check the article against its twelve-page limit before the supplement is finalised. Then view every rendered page.

References

[1] Shannon, C.E.: Communication in the presence of noise. Proceedings of the IRE **37**(1), 10–21 (1949) https://doi.org/10.1109/JRPROC.1949.232969

[2] Halton, J.H.: On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals. Numerische Mathematik **2**(1), 84–90 (1960) https://doi.org/10.1007/BF01386213

[3] Xiao, L., Nouri, S., Chapman, M., Fix, A., Lanman, D., Kaplanyan, A.: Neural supersampling for real-time rendering. ACM Transactions on Graphics **39**(4) (2020) https://doi.org/10.1145/3386569.3392376

[4] Teed, Z., Deng, J.: RAFT: Recurrent all-pairs field transforms for optical flow. In: Computer Vision – ECCV 2020, vol. 12347, pp. 402–419 (2020). https://doi.org/10.1007/978-3-030-58536-5_24

[5] Huang, Z., Zhang, T., Heng, W., Shi, B., Zhou, S.: Real-time intermediate flow estimation for video frame interpolation. In: Computer Vision – ECCV 2022, vol. 13674, pp. 624–642 (2022). https://doi.org/10.1007/978-3-031-19781-9_36

[6] Yang, L., Liu, S., Salvi, M.: A survey of temporal antialiasing techniques. Computer Graphics Forum **39**(2), 607–621 (2020) https://doi.org/10.1111/cgf.14018

[7] Shi, W., Caballero, J., Huszár, F., Totz, J., Aitken, A.P., Bishop, R., Rueckert, D., Wang, Z.: Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1874–1883 (2016). https://doi.org/10.1109/CVPR.2016.207

[8] Ronneberger, O., Fischer, P., Brox, T.: U-Net: Convolutional networks for biomedical image segmentation. In: Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015. Lecture Notes in Computer Science, pp. 234–241 (2015). https://doi.org/10.1007/978-3-319-24574-4_28

[9] Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E.K., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., Wu, H.: Mixed precision

training. In: International Conference on Learning Representations (2018). https://openreview.net/forum?id=r1gs9JgRZ

[10] Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: International Conference on Learning Representations (2019). https://openreview.net/forum?id=Bkg6RiCqY7

[11] Loshchilov, I., Hutter, F.: SGDR: Stochastic gradient descent with warm restarts. In: International Conference on Learning Representations (ICLR) (2017). https://openreview.net/forum?id=Skq89Scxx

[12] Charbonnier, P., Blanc-Féraud, L., Aubert, G., Barlaud, M.: Deterministic edge-preserving regularization in computed imaging. IEEE Transactions on Image Processing **6**(2), 298–311 (1997) https://doi.org/10.1109/83.551699

[13] Wang, Z., Bovik, A.C., Sheikh, H.R., Simoncelli, E.P.: Image quality assessment: From error visibility to structural similarity. IEEE Transactions on Image Processing **13**(4), 600–612 (2004) https://doi.org/10.1109/TIP.2003.819861

[14] Zhang, R., Isola, P., Efros, A.A., Shechtman, E., Wang, O.: The unreasonable effectiveness of deep features as a perceptual metric. In: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 586–595 (2018). https://doi.org/10.1109/CVPR.2018.00068

[15] Andersson, P., Nilsson, J., Akenine-Möller, T., Oskarsson, M., Åström, K., Fairchild, M.D.: FLIP: A difference evaluator for alternating images. Proceedings of the ACM on Computer Graphics and Interactive Techniques **3**(2), 1–23 (2020) https://doi.org/10.1145/3406183

[16] Baker, S., Scharstein, D., Lewis, J.P., Roth, S., Black, M.J., Szeliski, R.: A database and evaluation methodology for optical flow. International Journal of Computer Vision **92**(1), 1–31 (2011) https://doi.org/10.1007/s11263-010-0390-2

[17] Chu, M., Xie, Y., Mayer, J., Leal-Taixé, L., Thuerey, N.: Learning temporal coherence via self-supervision for GAN-based video generation. ACM Transactions on Graphics **39**(4) (2020) https://doi.org/10.1145/3386569.3392457

[18] Agustsson, E., Timofte, R.: NTIRE 2017 challenge on single image super-resolution: Dataset and study. In: 2017 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pp. 1122–1131 (2017). https://doi.org/10.1109/CVPRW.2017.150

[19] Nah, S., Baik, S., Hong, S., Moon, G., Son, S., Timofte, R., Lee, K.M.: NTIRE 2019 challenge on video deblurring and super-resolution: Dataset and study. In: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pp. 1996–2005 (2019). https://doi.org/10.1109/CVPRW.2019

.00251

[20] Xue, T., Chen, B., Wu, J., Wei, D., Freeman, W.T.: Video enhancement with task-oriented flow. International Journal of Computer Vision **127**(8), 1106–1125 (2019) https://doi.org/10.1007/s11263-018-01144-2

[21] Butler, D.J., Wulff, J., Stanley, G.B., Black, M.J.: A naturalistic open source movie for optical flow evaluation. In: Computer Vision – ECCV 2012. Lecture Notes in Computer Science, pp. 611–625 (2012). https://doi.org/10.1007/978-3-642-33783-3_44

[22] Wang, W., Zhu, D., Wang, X., Hu, Y., Qiu, Y., Wang, C., Hu, Y., Kapoor, A., Scherer, S.: TartanAir: A dataset to push the limits of visual SLAM. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 4909–4916 (2020). https://doi.org/10.1109/IROS45743.2020.9341801

[23] Zhou, L., Han, K., Ling, N., Wang, W., Jiang, W.: GameIR: A large-scale synthesized ground-truth dataset for image restoration over gaming content. In: Computer Vision – ACCV 2024 Workshops. Lecture Notes in Computer Science, pp. 179–196 (2025). https://doi.org/10.1007/978-981-96-2644-1_13

[24] Liu, C., Sun, D.: On Bayesian adaptive video super resolution. IEEE Transactions on Pattern Analysis and Machine Intelligence **36**(2), 346–360 (2014) https://doi.org/10.1109/TPAMI.2013.127

[25] Dosovitskiy, A., Ros, G., Codevilla, F., López, A., Koltun, V.: CARLA: An open urban driving simulator. In: Proceedings of the 1st Annual Conference on Robot Learning. Proceedings of Machine Learning Research, vol. 78, pp. 1–16 (2017). http://proceedings.mlr.press/v78/dosovitskiy17a.html

[26] Hasler, D., Süsstrunk, S.E.: Measuring colorfulness in natural images. In: Human Vision and Electronic Imaging VIII, vol. 5007, pp. 87–95 (2003). https://doi.org/10.1117/12.477378

[27] Venkatesan, R., Koon, S.-M., Jakubowski, M.H., Moulin, P.: Robust image hashing. In: Proceedings 2000 International Conference on Image Processing, pp. 664–666 (2000). https://doi.org/10.1109/ICIP.2000.899541

[28] Oquab, M., Darcet, T., Moutakanni, T., Vo, H.V., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.-Y., Li, S.-W., Misra, I., Rabbat, M., Sharma, V., Synnaeve, G., Xu, H., Jégou, H., Mairal, J., Labatut, P., Joulin, A., Bojanowski, P.: DINOv2: Learning Robust Visual Features without Supervision. Transactions on Machine Learning Research (2024). https://openreview.net/forum?id=a68SUt6zFt

[29] Lowe, D.G.: Distinctive image features from scale-invariant keypoints. International Journal of Computer Vision **60**(2), 91–110 (2004) https://doi.org/10.1023/B:VISI.0000029664.99615.94

[30] Fischler, M.A., Bolles, R.C.: Random sample consensus. Communications of the ACM **24**(6), 381–395 (1981) https://doi.org/10.1145/358669.358692

[31] Keys, R.G.: Cubic convolution interpolation for digital image processing. IEEE Transactions on Acoustics, Speech, and Signal Processing **29**(6), 1153–1160 (1981) https://doi.org/10.1109/TASSP.1981.1163711

[32] Ding, X., Zhang, X., Ma, N., Han, J., Ding, G., Sun, J.: RepVGG: Making VGG-style convnets great again. In: 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 13728–13737 (2021). https://doi.org/10.1109/CVPR46437.2021.01352

[33] Zhang, X., Zeng, H., Zhang, L.: Edge-oriented convolution block for real-time super resolution on mobile devices. In: Proceedings of the 29th ACM International Conference on Multimedia, pp. 4034–4043 (2021). https://doi.org/10.1145/3474085.3475291

[34] Williams, R.J., Peng, J.: An efficient gradient-based algorithm for on-line training of recurrent network trajectories. Neural Computation **2**(4), 490–501 (1990) https://doi.org/10.1162/neco.1990.2.4.490

[35] Kong, L., Jiang, B., Luo, D., Chu, W., Huang, X., Tai, Y., Wang, C., Yang, J.: IFRNet: Intermediate feature refine network for efficient frame interpolation. In: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1959–1968 (2022). https://doi.org/10.1109/CVPR52688.2022.00201

[36] Reda, F., Kontkanen, J., Tabellion, E., Sun, D., Pantofaru, C., Curless, B.: FILM: Frame interpolation for large motion. In: Computer Vision – ECCV 2022, vol. 13667, pp. 250–266 (2022). https://doi.org/10.1007/978-3-031-20071-7_15

[37] Zhang, G., Zhu, Y., Wang, H., Chen, Y., Wu, G., Wang, L.: Extracting motion and appearance via inter-frame attention for efficient video frame interpolation. In: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 5682–5692 (2023). https://doi.org/10.1109/CVPR52729.2023.00550

[38] Lee, S., Lee, H., Shin, C., Son, H., Lee, S.: Exploring discontinuity for video frame interpolation. In: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9791–9800 (2023). https://doi.org/10.1109/CVPR52729.2023.00944

[39] Briedis, K.M., Djelouah, A., Meyer, M., McGonigal, I., Gross, M., Schroers, C.:

Neural frame interpolation for rendered content. ACM Transactions on Graphics **40**(6), 1–13 (2021) https://doi.org/10.1145/3478513.3480553

[40] Guo, J., Fu, X., Lin, L., Ma, H., Guo, Y., Liu, S., Yan, L.-Q.: ExtraNet: Real-time extrapolated rendering for low-latency temporal supersampling. ACM Transactions on Graphics **40**(6), 1–16 (2021) https://doi.org/10.1145/3478513.3480531

[41] Wu, S., Kim, S., Zeng, Z., Vembar, D., Jha, S., Kaplanyan, A., Yan, L.-Q.: ExtraSS: A framework for joint spatial super sampling and frame extrapolation. In: SIGGRAPH Asia 2023 Conference Papers, pp. 1–11 (2023). https://doi.org/10.1145/3610548.3618224

[42] He, R., Zhou, S., Sun, Y., Cheng, R., Tan, W., Yan, B.: Low-latency space-time supersampling for real-time rendering. Proceedings of the AAAI Conference on Artificial Intelligence **38**(3), 2103–2111 (2024) https://doi.org/10.1609/aaai.v38i3.27982

[43] Wu, S., Vembar, D., Sochenov, A., Panneer, S., Kim, S., Kaplanyan, A., Yan, L.-Q.: GFFE: G-buffer free frame extrapolation for low-latency real-time rendering. ACM Transactions on Graphics **43**(6), 1–15 (2024) https://doi.org/10.1145/3687923

[44] Meister, S., Hur, J., Roth, S.: UnFlow: Unsupervised learning of optical flow with a bidirectional census loss. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32, pp. 7251–7259 (2018). https://doi.org/10.1609/aaai.v32i1.12276

[45] Gou, J., Yu, B., Maybank, S.J., Tao, D.: Knowledge distillation: A survey. International Journal of Computer Vision **129**(6), 1789–1819 (2021) https://doi.org/10.1007/s11263-021-01453-z